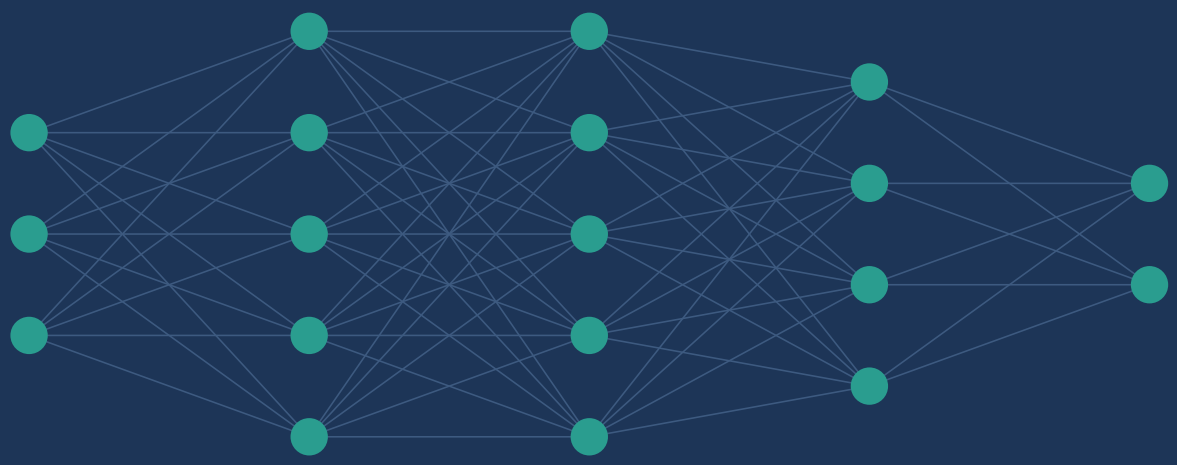
Fortran ile Sıfırdan LLM

Sinir ağlarından GPT'ye, kuantum kimyasından
graf sinir ağlarına: Fortran + PyTorch ile uygulamalı rehber



Kitaptaki her Fortran ve PyTorch/Python örneği gerçekten çalıştırılmıştır;
listelenen çıktılar gerçek çalıştırma sonuçlarıdır. Grafikler matematik ve çıktılardan üretilmiştir.

İçindekiler

1. Fortran ve Matematik Hazırlığı 7

Neden Fortran? 7

Fortran hızlı başlangıç kuralları 7

Türev nedir? Eğimin sayısı 10

Zincir kuralı: geri yayılımın kalbi 12

Gradyan: çok değişkenli türev 13

Olasılık, logaritma ve çapraz entropi 17

PyTorch hızlı başlangıç ve Fortran ↔ PyTorch sözlüğü 19

2. Sinir Ağları ve Geri Yayılım 20

Perceptron: en küçük öğrenen birim 20

Aktivasyon fonksiyonları 21

İleri yayılım ve geri yayılım 23

Kayıp fonksiyonları 25

Softmax ve çapraz entropi: en güzel gradyan 26

Çok katmanlı algılayıcı (MLP) ve XOR 28

Katman matrisleri: toplu (batch) hesap 31

Ağırlık başlatma ve normalizasyon 32

Normalizasyon: BatchNorm ve LayerNorm 34

3. Optimizasyon: SGD, Momentum ve Adam 37

Gradyan inişi neden çalışır? 37

SGD: rastgele küçük gruplarla öğrenmek 37

Momentum: ivme ekle 39

Adam: her parametre için ayrı öğrenme oranı 42

Ağırlık sönümü: L2 ve AdamW 44

4. Transformer Mimarisi 46

Büyük resim 46

Token embedding ve konum kodlaması 46

Self-attention: Query, Key, Value 48

Causal mask: geleceğe bakmak yasak 50

Öğrenilebilir Q, K, V projeksiyonları 52

Multi-head attention 53

Normalizasyon: LayerNorm ve RMSNorm 54

Feed-forward katmanı ve residual bağlantılar 56

Transformer bloğu 58

Logits, softmax ve sonraki token tahmini 60

Attention'ın geri yayılımı (kısa bakış) . . . 61

5. Donanım ve Performans . . . 64

Hesabın iki sınırı: işlemci hızı ve bellek hızı . . . 64
Fortran'da bellek düzeni: sütun-ana . . . 64
Matris çarpımı: sinir ağlarının işçi atı . . . 65
Bloklama (tiling): önbelleği akıllı kullan . . . 67
Paralellik: OpenMP ile çok çekirdek . . . 69
Bellek bant genişliği ve roofline . . . 70
Mixed precision: daha az bit, daha fazla hız . . . 72
Gradient accumulation: belleğe sığmayan batch . . . 74
Gradient checkpointing: bellek karşılığı hesap . . . 75
GPU programlama: bir bakış . . . 77

6. Tokenizer ve Metin İşleme . . . 79

Neden tokenizer? . . . 79
Unicode ve UTF-8 . . . 79
Metin normalizasyonu: Türkçe'nin tuzağı . . . 82
Byte Pair Encoding (BPE) . . . 84

7. Veri Mühendisliği . . . 91

Veri hattı (pipeline) genel görünümü . . . 91
HTML ayıklama ve metin temizliği . . . 91
Büyük dosyaları belleğe yüklemeden okuma . . . 93
Tekilleştirme (deduplication) . . . 94
Eğitim / doğrulama / test ve veri sızıntısı . . . 97
Token tabanlı veri paketleme ve ikili dosya . . . 99
Gizlilik, telif ve lisans . . . 101

8. Model Eğitimi . . . 103

Next-token prediction: tek bir hedef, her şey . . . 103
Batch boyutu ve dizi uzunluğu . . . 105
Öğrenme oranı çizelgesi: warmup ve cosine . . . 106
Gradient clipping . . . 108
Doğrulama kaybı, aşırı öğrenme ve perplexity . . . 109
Checkpoint: eğitimi kaldığı yerden sürdür . . . 111
Eğitim bütçesini hesaplamak . . . 114

9. Büyük Ölçekli Model Mühendisliği . . . 116

Parametre sayısı: bir Transformer'ın muhasebesi . . . 116
Bellek hesabı . . . 117

Eğitim FLOP hesabı . . . 119
Veri paralelliği (data parallelism) . . . 120
Tensör paralelliği (tensor parallelism) . . . 122
Boru hattı paralelliği (pipeline parallelism) . . . 123
FlashAttention'ın mantığı: online softmax . . . 125
Quantization: ağırlıkları az bitle saklamak . . . 126

10. Metin Üretimi ve Model Sunumu . . . 129

Üretim döngüsü . . . 129
Greedy decoding . . . 129
Temperature sampling . . . 130
Top-k ve top-p (nucleus) sampling . . . 132
KV cache: tekrar hesaplamayı önle . . . 134
Context window ve KV cache belleği . . . 136
Batching, gecikme ve throughput . . . 138
Modeli API olarak sunmak . . . 140

11. Bitirme Projesi: Kendi Dil Modellerimizi Eğitiyoruz . . . 143

Proje planı . . . 143
Veri ve ortak modül . . . 143
Model 2: bağlam penceresi olan MLP dil modeli . . . 147
Model 3: causal self-attention dil modeli (mini GPT) . . . 150
Sonuçların karşılaştırması ve dürüst yorum . . . 154

12. Kuantum Mekaniği: Schrödinger Denklemini Fortran ile Çözmek . . . 156

Neden kuantum kimyası? Yapay zekanın hedefi . . . 156
Schrödinger denklemi ve atomik birimler . . . 156
Hidrojen atomu: Coulomb potansiyelinde radyal denklem . . . 160
Varyasyonel ilke: kuantum mekaniğinin gradyan inişi . . . 161

13. Moleküler Orbitaller: Hückel Teorisi ve Hartree–Fock . . . 164

Born–Oppenheimer yaklaşımı ve potansiyel enerji yüzeyi . . . 164
Hückel teorisi: kimyasal graf = matris . . . 164
Hartree–Fock: çok elektronlu molekülün ilk ilkelerden çözümü . . . 166
Gauss tipi orbitaller ve integraller . . . 167
Potansiyel enerji eğrisi: bağ oluşumu ve HF'nin sınırı . . . 170
Korelasyon enerjisi: HF'nin eksiği . . . 172

14. Yoğunluk Fonksiyoneli Teorisi (DFT) . . . 173

Neden yoğunluk? . . . 173
Hohenberg–Kohn teoremleri ve Kohn–Sham yöntemi . . . 173

Helyum atomu için Kohn–Sham: kendi DFT programımız . . . 175
Jacob'un merdiveni: fonksiyonellerin hiyerarşisi . . . 178
Fonksiyonel türev = gradyan: DFT ve otomatik türev . . . 178

15. Graf Sinir Ağları ve Geometrik Derin Öğrenme . . . 180

Moleküller graftır . . . 180
Mesaj geçirmeli sinir ağı (MPNN) . . . 181
Moleküler özellik tahmini: GNN ve geleneksel parmak izi . . . 183
Geometrik derin öğrenme: simetrilere saygı . . . 185

16. Molekül Üretimi, Özellik Tahmini ve İlaç Keşfi . . . 189

Büyük resim: ilaç keşfi ve hesaplamalı kimyada yapay zekâ . . . 189
Üretken model: SMILES üzerinde Transformer . . . 190
İlaç benzerliği ve sanal tarama . . . 192
Kuantum kimyası verisiyle makine öğrenmesi potansiyeli . . . 193
DFT + makine öğrenmesi: öğrenilebilir fonksiyoneller . . . 195
Sınırlar ve sorumlu kullanım . . . 197

Ek A. Ortak Modüller . . . 198

Ek B. Derleme, Hata Ayıklama ve Hız İpuçları . . . 199

Sık karşılaşılan Fortran hataları . . . 199

Ek C. Formül Kartı . . . 200

Ek D. Sonraki Adımlar ve Küçük Sözlük . . . 201

Bu kitaptan gerçek bir LLM'e giden yol . . . 201
Küçük sözlük . . . 201

BÖLÜM 1

1. Fortran ve Matematik Hazırlığı

Bu kitapta her şeyi Fortran ile, hazır derin öğrenme kütüphanesi olmadan yazacağız. Önce araçlarımızı keskinleştirelim.

Bu bölümde

- Fortran 90+ ile program, modül, dizi ve fonksiyon yazmak
- Türev, gradyan ve zincir kuralını sayısal olarak doğrulamak
- Gradyan inişinin (gradient descent) neden çalıştığını ve öğrenme oranının etkisini görmek
- Olasılık, logaritma ve çapraz entropiyi (cross-entropy) kavramak

Hedef: Sonraki bölümlerde kullanacağımız matematiksel ve programlama araç kutusunu hazırlamak.

Neden Fortran?

Fortran, bilimsel hesaplamanın en eski ve hâlâ en hızlı dillerinden biridir. Diziler ve matrisler dilin **birinci sınıf** vatandaşıdır: `matmul`, `sum`, `maxval`, `spread` gibi işlevler hazır gelir, derleyici bu işlemleri çok iyi optimize eder. Bir sinir ağı özünde matris çarpımı ve eleman bazlı fonksiyonlardan ibaret olduğu için Fortran bu iş için şaşırtıcı derecede uygundur. Üstelik "framework" arkasına saklanmadan, her satırın ne yaptığını görürsün.

Bu kitapta modern Fortran (90/2003/2008) kullanıyoruz. Bütün örnekleri **gfortran** ile şöyle derleyip çalıştırabilirsin:

```
gfortran -O2 ornek.f90 -o ornek
./ornek
```

İpucu

Kitaptaki her örnek, tek başına derlenebilen bir `.f90` dosyasıdır. Bazı örnekler önceki örneklerde tanımlanan modülleri kullanır; bunların kaynağı örneğin altında belirtilir. Kitapla birlikte verilen *ornekler* klasöründe her örneğin tam ve çalışır hâli vardır.

Fortran hızlı başlangıç kuralları

- **Her program** `implicit none` ile başlasın: tanımlamadığın değişken kullanılamaz, yazım hataları derleyicide yakalanır.
- **Kesinlik:** `integer, parameter :: dp = kind(1.0d0)` ile çift duyarlıklı (64 bit) sayı türünü `dp` diye adlandırırız. Sabitler `2.0_dp` biçiminde yazılır.
- **Dizi indisleri 1'den başlar** (varsayılan) ve **sütun-ana (column-major)** sıralıdır: `A(i,j)` içinde `i` en hızlı değişen indekstir. Bu, performans bölümünde çok önemli olacak.
- **Dizi işlemleri:** `A + B`, `A * 2`, `sum(A)`, `matmul(A,B)`, `transpose(A)`, dilimler `A(:,2)`, `A(1:3,:)`.
- **Alt programlar:** `function` bir değer döndürür; `subroutine` argümanlarını değiştirir. İkisini de `module` içine koyarız.
- **Çıktı:** `print '(A,F8.4)', 'x = ', x` gibi biçim tanımlı yazdırma kullanırız (A: metin, F: sabit noktalı, ES: bilimsel gösterim, I: tamsayı).

Örnek 1.1 — Ortalama ve standart sapma

PROBLEM

x = [2, 4, 4, 4, 6] dizisinin ortalamasını, standart sapmasını ve en büyük elemanını hesapla.

ÇÖZÜM (FORTRAN)

```
program istatistik
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x(5), mu, sd
  x = [2.0_dp, 4.0_dp, 4.0_dp, 4.0_dp, 6.0_dp]
  mu = sum(x) / size(x)
  sd = sqrt(sum((x - mu)**2) / size(x))
  print '(A,F8.4)', 'ortalama   = ', mu
  print '(A,F8.4)', 'std sapma  = ', sd
  print '(A,F8.4)', 'en buyuk   = ', maxval(x)
end program istatistik
```

ÇIKTI

```
ortalama   =   4.0000
std sapma  =   1.2649
en buyuk   =   6.0000
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor([2., 4., 4., 4., 6.], dtype=torch.float64)
print(f"ortalama   = {x.mean():8.4f}")
print(f"std sapma  = {torch.std(x, correction=0):8.4f}")   # Fortran'daki gibi N'e bolunur
print(f"en buyuk   = {x.max():8.4f}")
```

PYTORCH ÇIKTISI

```
ortalama   =   4.0000
std sapma  =   1.2649
en buyuk   =   6.0000
```

NASIL ÇALIŞIYOR?

Ortalama (2+4+4+4+6)/5 = 4. Farkların kareleri 4, 0, 0, 0, 4; toplamları 8; 8/5 = 1.6 varyans; karekökü 1.2649. Dikkat: `(x - mu)**2` bir *dizi ifadesidir*; döngü yazmadan her eleman için ayrı hesaplanır, `sum` ile toplanır. Bu "vektörleştirilmiş" yazım tarzı kitap boyunca karşımıza çıkacak.

Örnek 1.2 — Modül ve eleman bazlı fonksiyon: sigmoid

PROBLEM

Sigmoid fonksiyonunu `σ(x) = 1/(1+e^(−x))` bir modül içinde tanımla ve hem tek değere hem de bütün diziye uygula.

ÇÖZÜM (FORTRAN)

```
module aktivasyon
  implicit none
  integer, parameter :: dp = kind(1.0d0)
contains
  elemental function sigmoid(x) result(y)
    real(dp), intent(in) :: x
    real(dp) :: y
    y = 1.0_dp / (1.0_dp + exp(-x))
  end function sigmoid
end module aktivasyon

program ornek
  use aktivasyon
  implicit none
  real(dp) :: x(5)
```

```
  integer :: i
  x = [-2.0_dp, -1.0_dp, 0.0_dp, 1.0_dp, 2.0_dp]
  do i = 1, size(x)
    print '(A,F5.1,A,F8.4)', 'sigmoid(', x(i), ') = ', sigmoid(x(i))
  end do
  print '(A,5F8.4)', 'hepsi birden: ', sigmoid(x)
end program ornek
```

ÇIKTI

```
sigmoid( -2.0) =   0.1192
sigmoid( -1.0) =   0.2689
sigmoid(  0.0) =   0.5000
sigmoid(  1.0) =   0.7311
sigmoid(  2.0) =   0.8808
hepsi birden:   0.1192  0.2689  0.5000  0.7311  0.8808
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor([-2., -1., 0., 1., 2.], dtype=torch.float64)
for v in x:
    print(f"sigmoid({v:5.1f}) = {torch.sigmoid(v):8.4f}")      # torch.sigmoid zaten elemental
print("hepsi birden:", " ".join(f"{v:.4f}" for v in torch.sigmoid(x)))
```

PYTORCH ÇIKTISI

```
sigmoid( -2.0) =   0.1192
sigmoid( -1.0) =   0.2689
sigmoid(  0.0) =   0.5000
sigmoid(  1.0) =   0.7311
sigmoid(  2.0) =   0.8808
hepsi birden: 0.1192 0.2689 0.5000 0.7311 0.8808
```

NASIL ÇALIŞIYOR?

`elemental` anahtar sözcüğü, skaler yazılmış fonksiyonun diziye *otomatik* uygulanmasını sağlar. Böylece `sigmoid(x)` yazınca her eleman için çağrılır. Beklenen: $\sigma(0) = 0.5$, $\sigma(1) = 0.7311$ ve $\sigma(-x) = 1 - \sigma(x)$ simetrisi ($\sigma(-2) = 0.1192$, $\sigma(2) = 0.8808$).

Örnek 1.3 — Matris çarpımı: elle ve matmul ile

PROBLEM

A (2×3) ve B (3×2) matrislerinin çarpımını önce üç iç içe döngüyle, sonra `matmul` ile hesapla; sonuçları karşılaştır.

ÇÖZÜM (FORTRAN)

```
program matris
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: A(2,3), B(3,2), C(2,2), D(2,2)
  integer :: i, j, k
  ! order=[2,1] : degerleri satir satir yaz
  A = reshape([1,2,3, 4,5,6] * 1.0_dp, [2,3], order=[2,1])
  B = reshape([7,8, 9,10, 11,12] * 1.0_dp, [3,2], order=[2,1])
  C = 0.0_dp
  do i = 1, 2
    do j = 1, 2
      do k = 1, 3
        C(i,j) = C(i,j) + A(i,k) * B(k,j)
      end do
    end do
  end do
  D = matmul(A, B)
  do i = 1, 2
    print '(A,2F8.1)', 'C satiri: ', C(i,:)
  end do
  print '(A,ES10.2)', 'max |C - D| = ', maxval(abs(C - D))
end program matris
```

ÇIKTI

```
C satiri:     58.0    64.0
C satiri:    139.0   154.0
max |C - D| =   0.00E+00
```

PYTORCH KARŞILIĞI

```
import torch
A = torch.tensor([[1., 2, 3], [4, 5, 6]], dtype=torch.float64)
B = torch.tensor([[7., 8], [9, 10], [11, 12]], dtype=torch.float64)
C = torch.zeros(2, 2, dtype=torch.float64)
for i in range(2):
    for j in range(2):
        for k in range(3):
            C[i, j] += A[i, k] * B[k, j]
D = A @ B                                   # matmul / torch.mm ile ayni
for i in range(2):
    print("C satiri:", " ".join(f"{v:8.1f}" for v in C[i]))
print("max |C - D| =", (C - D).abs().max().item())
print("einsum ile  :", torch.einsum('ik,kj->ij', A, B).tolist())
```

PYTORCH ÇIKTISI

```
C satiri:     58.0     64.0
C satiri:    139.0    154.0
max |C - D| = 0.0
einsum ile  : [[58.0, 64.0], [139.0, 154.0]]
```

NASIL ÇALIŞIYOR?

Kural: `C(i,j) = Σ_k A(i,k)·B(k,j)`. Birinci satır için 1·7+2·9+3·11 = 58 ve 1·8+2·10+3·12 = 64. İkinci satır 4·7+5·9+6·11 = 139 ve 4·8+5·10+6·12 = 154. Sinir ağlarının hesabının neredeyse tamamı bu işlemdir; ileride bu tek satırı (`matmul`) hızlandırmanın neden bu kadar önemli olduğunu göreceğiz.

Türev nedir? Eğimin sayısı

Basit matematik

Bir f(x) fonksiyonunun türevi, x'i çok az oynattığımızda f'nin ne kadar değiştiğini söyler:

f'(x) ≈ [f(x+h) − f(x−h)] / (2h)

Örnek: f(x) = x² için f'(3) = 6. Gerçekten de x 3'ten 3.001'e çıkınca f 9'dan 9.006001'e çıkar; değişim ≈ 6 × 0.001. Türev, "bu yöne küçük bir adım atarsam çıktı kaç kat değişir?" sorusunun cevabıdır. Öğrenme dediğimiz şey, kaybı (loss) küçülten yöne küçük adımlar atmaktır.

Örnek 1.4 — Sayısal türev ve adım boyutu h

PROBLEM

f(x) = x³ fonksiyonunun x = 2 noktasındaki türevini (gerçek değer 3x² = 12) merkezî farkla hesapla. h'yi küçülttükçe hata nasıl değişiyor?

ÇÖZÜM (FORTRAN)

```
program turev
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x, h, num, ana
  integer :: k
  x = 2.0_dp
  ana = 3.0_dp * x**2
  do k = 1, 12, 1
    h = 10.0_dp**(-k)
    num = ((x+h)**3 - (x-h)**3) / (2.0_dp*h)
    print '(A,ES8.1,A,F16.10,A,ES9.2)', 'h=', h, '  turev=', num, '  hata=', abs(num-ana)
  end do
end program turev
```

ÇIKTI

```
h= 1.0E-01  turev=   12.0100000000  hata= 1.00E-02
h= 1.0E-02  turev=   12.0001000000  hata= 1.00E-04
h= 1.0E-03  turev=   12.0000010000  hata= 1.00E-06
h= 1.0E-04  turev=   12.0000000100  hata= 1.00E-08
h= 1.0E-05  turev=   12.0000000002  hata= 2.12E-10
h= 1.0E-06  turev=   12.0000000003  hata= 3.45E-10
h= 1.0E-07  turev=   11.9999999937  hata= 6.32E-09
h= 1.0E-08  turev=   11.9999999271  hata= 7.29E-08
h= 1.0E-09  turev=   12.0000009929  hata= 9.93E-07
h= 1.0E-10  turev=   12.0000009929  hata= 9.93E-07
h= 1.0E-11  turev=   12.0000009929  hata= 9.93E-07
h= 1.0E-12  turev=   12.0010668070  hata= 1.07E-03
```

PYTORCH KARŞILIĞI

```

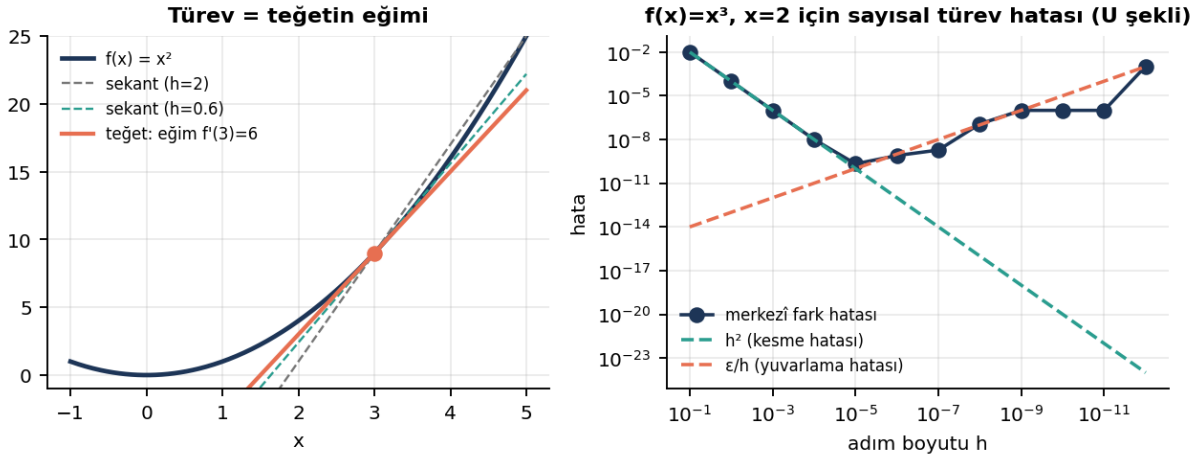
import torch
x = torch.tensor(2.0, dtype=torch.float64, requires_grad=True)
y = x ** 3
y.backward()                                # otomatik turev (autograd)
print("autograd turevi  :", x.grad.item(), "  (gercek 3x^2 = 12)")
for k in range(1, 13):                      # Fortran'daki sayisal turev
    h = 10.0 ** (-k)
    num = ((2 + h) ** 3 - (2 - h) ** 3) / (2 * h)
    print(f"h={h:8.1e}  turev={num:16.10f}  hata={abs(num - 12):9.2e}")
```

PYTORCH ÇIKTISI

```
autograd turevi  : 12.0   (gercek 3x^2 = 12)
h= 1.0e-01  turev=   12.0100000000  hata= 1.00e-02
h= 1.0e-02  turev=   12.0001000000  hata= 1.00e-04
h= 1.0e-03  turev=   12.0000010000  hata= 1.00e-06
h= 1.0e-04  turev=   12.0000000100  hata= 1.00e-08
h= 1.0e-05  turev=   12.0000000002  hata= 2.12e-10
h= 1.0e-06  turev=   12.0000000008  hata= 7.89e-10
h= 1.0e-07  turev=   11.9999999937  hata= 6.32e-09
h= 1.0e-08  turev=   11.9999998827  hata= 1.17e-07
h= 1.0e-09  turev=   12.0000009929  hata= 9.93e-07
h= 1.0e-10  turev=   12.0000009929  hata= 9.93e-07
h= 1.0e-11  turev=   12.0000009929  hata= 9.93e-07
h= 1.0e-12  turev=   12.0010668070  hata= 1.07e-03
```

NASIL ÇALIŞIYOR?

Merkezî farkın hatası kabaca h^2 ile orantılı küçülür (h=0.1 → hata 0.01, h=0.01 → 0.0001). Ama h çok küçülünce bilgisayarın yuvarlama hatası baskın olur: iki neredeyse eşit sayıyı çıkarırız ve hassasiyet kaybolur. Çıktıda hatanın önce azalıp sonra tekrar büyüdüğünü görürsün. Pratik kural: çift duyarlıkta h ≈ 1e-5 civarı iyi bir seçimdir. Bu yöntem, ileride **gradyan kontrolü (gradient check)** için bize yardımcı olacak: elle yazdığımız geri yayılımı sayısal türevle doğrulayacağız.



Şekil 1.1 — Solda: teğet, sekantların limitidir; türev = teğetin eğimi. Sağda: sayısal türev hatasının adım boyutu h'ye göre U şekli (kesme hatası h^2, yuvarlama hatası ε/h).

Türetme: sayısal türevin hatası

Merkezî farkın hatasını Taylor serisiyle görelim: $f(x\pm h) = f \pm h\,f' + (h^2/2)\,f'' \pm (h^3/6)\,f''' + \dots$

$[f(x+h) - f(x-h)] / (2h) = f'(x) + (h^2/6)\,f'''(x) + O(h^4)$

Kesme hatası h^2 ile orantılıdır ($f(x) = x^3$ için tam olarak h^2, çünkü $f''' = 6$). Ama bilgisayar f'yi $\varepsilon \approx 10^{-16}$ bağıl hassasiyetle saklar; çıkarma işlemindeki yuvarlama hatası $\approx \varepsilon|f|/h$ kadar büyür. Toplam hata $\approx h^2 + \varepsilon/h$, $h^* \approx \varepsilon^{1/3} \approx 10^{-5}$ civarında minimum yapar: şekildeki U şeklinin yeri.

Zincir kuralı: geri yayılımın kalbi

Basit matematik

Eğer $h(x) = f(g(x))$ ise, türev iç içe türevlerin çarpımıdır:

$dh/dx = (df/dg) \cdot (dg/dx)$

Örnek: $h(x) = \sin(x^2)$. İç fonksiyon $g = x^2$, dış fonksiyon $f = \sin(g)$. $dg/dx = 2x$ ve $df/dg = \cos(g)$ olduğundan $dh/dx = \cos(x^2)\cdot 2x$.

Bir sinir ağı yüzlerce iç içe fonksiyondur; geri yayılım (backpropagation) sadece bu çarpımı **sondan başa doğru** sistemli biçimde yapmaktır.

Örnek 1.5 — Zincir kuralını sayısal olarak doğrula

PROBLEM

$h(x) = \sin(x^2)$ için $x = 1.5$ noktasındaki türevi zincir kuralıyla ve sayısal yöntemle hesapla.

ÇÖZÜM (FORTRAN)

```
program zincir
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x, g, dgdx, dfdg, ana, num, h
  x = 1.5_dp
  h = 1.0e-6_dp
  g    = x**2              ! ic fonksiyon
  dgdx = 2.0_dp * x        ! dg/dx
  dfdg = cos(g)            ! d sin(g) / dg
  ana  = dfdg * dgdx       ! zincir kurali
  num  = (sin((x+h)**2) - sin((x-h)**2)) / (2.0_dp*h)
  print '(A,F12.8)', 'zincir kurali : ', ana
  print '(A,F12.8)', 'sayisal turev : ', num
  print '(A,ES10.2)', 'fark          : ', abs(ana - num)
end program zincir
```

ÇIKTI

```
zincir kurali :  -1.88452087
sayisal turev :  -1.88452087
fark          :   1.45E-10
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor(1.5, dtype=torch.float64, requires_grad=True)
h = torch.sin(x ** 2)
h.backward()
g = x ** 2
zincir = torch.cos(g) * 2 * x              # dh/dg * dg/dx
print(f"autograd      : {x.grad.item():12.8f}")
print(f"zincir kurali : {zincir.item():12.8f}")
eps = 1e-6
num = (torch.sin((x + eps) ** 2) - torch.sin((x - eps) ** 2)) / (2 * eps)
print(f"sayisal turev : {num.item():12.8f}")
```

PYTORCH ÇIKTISI

```
autograd      :  -1.88452087
zincir kurali :  -1.88452087
sayisal turev :  -1.88452087
```

NASIL ÇALIŞIYOR?

İki sonuç 8-9 basamağa kadar aynı çıkar. Programda yaptığımız "önce g, sonra türevleri, sonra çarp" adımı, geri yayılımın minyatürüdür: ileri geçişte *ara değerleri* (burada g) saklarız, geri geçişte yerel türevleri (dgdx, dfdg) çarparız.

Gradyan: çok değişkenli türev

Basit matematik

Birden fazla girdi olduğunda her girdi için ayrı bir kısmi türev vardır. Hepsini bir vektörde toplarsak **gradyan** elde ederiz:

$\nabla f = (\partial f/\partial x, \partial f/\partial y)$

Örnek: $f(x,y) = x^2 + 3xy + y^2$. $\partial f/\partial x = 2x + 3y$ ve $\partial f/\partial y = 3x + 2y$. $(x,y) = (1,2)$ noktasında $\nabla f = (8, 7)$.

Gradyan, fonksiyonun **en hızlı arttığı yönü** gösterir. Kaybı azaltmak istediğimiz için gradyanın tersine yürürüz: $\theta \leftarrow \theta - \eta \cdot \nabla L$. Buradaki η (eta) **öğrenme oranıdır**.

Örnek 1.6 — Kısmi türevler ve gradyanın yönü

PROBLEM

$f(x,y) = x^2 + 3xy + y^2$ için (1,2) noktasında gradyanı hem formülle hem sayısal olarak bul. Gradyan yönünde ve başka bir yönde birim adım başına artışı karşılaştır.

ÇÖZÜM (FORTRAN)

```
program gradyan
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x, y, h, gx, gy, nx, ny, u(2), gn, d1, d2
  x = 1.0_dp; y = 2.0_dp; h = 1.0e-6_dp
  print '(A,2F9.4)', 'formul   grad = ', 2*x+3*y, 3*x+2*y
  nx = (f(x+h,y) - f(x-h,y)) / (2*h)
  ny = (f(x,y+h) - f(x,y-h)) / (2*h)
  print '(A,2F9.4)', 'sayisal  grad = ', nx, ny
  gx = 2*x+3*y; gy = 3*x+2*y
  gn = sqrt(gx**2 + gy**2)
  u = [gx, gy] / gn                        ! gradyan yonunde birim vektor
  d1 = (f(x+h*u(1), y+h*u(2)) - f(x,y)) / h
  print '(A,F9.4)', '|grad|                    = ', gn
  print '(A,F9.4)', 'gradyan yonundeki egim   = ', d1
```

```
  u = [1.0_dp, 0.0_dp]                    ! x ekseni yonu
  d2 = (f(x+h*u(1), y+h*u(2)) - f(x,y)) / h
  print '(A,F9.4)', 'x ekseni yonundeki egim = ', d2
contains
  function f(a, b) result(r)
    real(dp), intent(in) :: a, b
    real(dp) :: r
    r = a**2 + 3*a*b + b**2
  end function f
end program gradyan
```

ÇIKTI

```
formul   grad =    8.0000   7.0000
sayisal  grad =    8.0000   7.0000
|grad|                    =   10.6301
gradyan yonundeki egim   =   10.6301
x ekseni yonundeki egim =    8.0000
```

PYTORCH KARŞILIĞI

```
import torch
xy = torch.tensor([1.0, 2.0], dtype=torch.float64, requires_grad=True)
f = lambda v: v[0]**2 + 3*v[0]*v[1] + v[1]**2
f(xy).backward()
g = xy.grad
print("autograd grad =", [round(v, 4) for v in g.tolist()])
u = g / g.norm()                          # gradyan yonunde birim vektor
h = 1e-6
d1 = (f(xy.detach() + h*u) - f(xy.detach())) / h
d2 = (f(xy.detach() + h*torch.tensor([1., 0.], dtype=torch.float64)) - f(xy.detach())) / h
print(f"|grad|                  = {g.norm():.4f}")
print(f"gradyan yonundeki egim  = {d1:.4f}")
print(f"x ekseni yonundeki egim = {d2:.4f}")
```

PYTORCH ÇIKTISI

```
autograd grad = [8.0, 7.0]
|grad|                  = 10.6301
gradyan yonundeki egim  = 10.6301
x ekseni yonundeki egim = 8.0000
```

NASIL ÇALIŞIYOR?

Gradyan yönündeki eğim tam olarak gradyanın uzunluğuna ($\sqrt{64+49} = 10.63$) eşittir; başka hiçbir yönde bu kadar dik artış yoktur. x ekseni yönünde ise yalnızca $\partial f/\partial x = 8$ elde ederiz. İşte bu yüzden "en dik iniş" için gradyanın tersine gideriz.

Örnek 1.7 — Gradyan inişi: tek boyutta

PROBLEM

$f(x) = (x-3)^2 + 1$ fonksiyonunun minimumunu ($x = 3$) gradyan inişiyle bul. Başlangıç $x = 0$, öğrenme oranı $\eta = 0.1$.

ÇÖZÜM (FORTRAN)

```
program gd1d
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x, lr, grad
  integer :: k
  x = 0.0_dp
  lr = 0.1_dp
  do k = 0, 10
    grad = 2.0_dp * (x - 3.0_dp)
    print '(A,I3,A,F9.5,A,F9.5)', 'adim ', k, '  x = ', x, '  f(x) = ', (x-3.0_dp)**2 + 1.0_dp
    x = x - lr * grad
  end do
end program gd1d
```

ÇIKTI

```
adim   0  x =   0.00000  f(x) =  10.00000
adim   1  x =   0.60000  f(x) =   6.76000
adim   2  x =   1.08000  f(x) =   4.68640
adim   3  x =   1.46400  f(x) =   3.35930
adim   4  x =   1.77120  f(x) =   2.50995
adim   5  x =   2.01696  f(x) =   1.96637
adim   6  x =   2.21357  f(x) =   1.61848
adim   7  x =   2.37085  f(x) =   1.39582
adim   8  x =   2.49668  f(x) =   1.25333
adim   9  x =   2.59735  f(x) =   1.16213
adim  10  x =   2.67788  f(x) =   1.10376
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
opt = torch.optim.SGD([x], lr=0.1)         # theta <- theta - lr * grad
for k in range(11):
    loss = (x - 3) ** 2 + 1
    print(f"adim {k:2d}  x = {x.item():9.5f}  f(x) = {loss.item():9.5f}")
    opt.zero_grad(); loss.backward(); opt.step()
```

PYTORCH ÇIKTISI

```
adim  0  x =   0.00000  f(x) =  10.00000
adim  1  x =   0.60000  f(x) =   6.76000
adim  2  x =   1.08000  f(x) =   4.68640
adim  3  x =   1.46400  f(x) =   3.35930
adim  4  x =   1.77120  f(x) =   2.50995
adim  5  x =   2.01696  f(x) =   1.96637
adim  6  x =   2.21357  f(x) =   1.61848
adim  7  x =   2.37085  f(x) =   1.39582
adim  8  x =   2.49668  f(x) =   1.25333
adim  9  x =   2.59735  f(x) =   1.16213
adim 10  x =   2.67788  f(x) =   1.10376
```

NASIL ÇALIŞIYOR?

Güncelleme $x \leftarrow x - \eta \cdot 2(x-3)$ şeklindedir. Her iki tarafa 3 çıkarırsak $(x-3) \leftarrow (1-2\eta)(x-3)$ elde ederiz: minimuma olan uzaklık her adımda **0.8 ile çarpılır** $(1 - 0.2)$. Yani 10 adımdan sonra uzaklık $3 \times 0.8^{10} \approx 0.32$ olur. Bu "geometrik yakınsama" gradyan inişinin temel davranışıdır.

Örnek 1.8 — Öğrenme oranı çok küçükse, çok büyükse?

PROBLEM

Aynı fonksiyonda farklı η değerleri dene ve 20 adım sonra x değerine bak.

ÇÖZÜM (FORTRAN)

```
program lrdene
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: lrs(6), x
  integer :: i, k
  lrs = [0.01_dp, 0.1_dp, 0.5_dp, 0.9_dp, 1.0_dp, 1.1_dp]
  do i = 1, 6
    x = 0.0_dp
    do k = 1, 20
      x = x - lrs(i) * 2.0_dp * (x - 3.0_dp)
    end do
    print '(A,F5.2,A,ES13.5,A,F8.4)', 'lr=', lrs(i), ' x(20)=', x, &
          ' carpan=', 1.0_dp - 2.0_dp*lrs(i)
  end do
end program lrdene
```

ÇIKTI

```
lr= 0.01 x(20)=  9.97176E-01 carpan=  0.9800
lr= 0.10 x(20)=  2.96541E+00 carpan=  0.8000
lr= 0.50 x(20)=  3.00000E+00 carpan=  0.0000
lr= 0.90 x(20)=  2.96541E+00 carpan= -0.8000
lr= 1.00 x(20)=  0.00000E+00 carpan= -1.0000
lr= 1.10 x(20)= -1.12013E+02 carpan= -1.2000
```

PYTORCH KARŞILIĞI

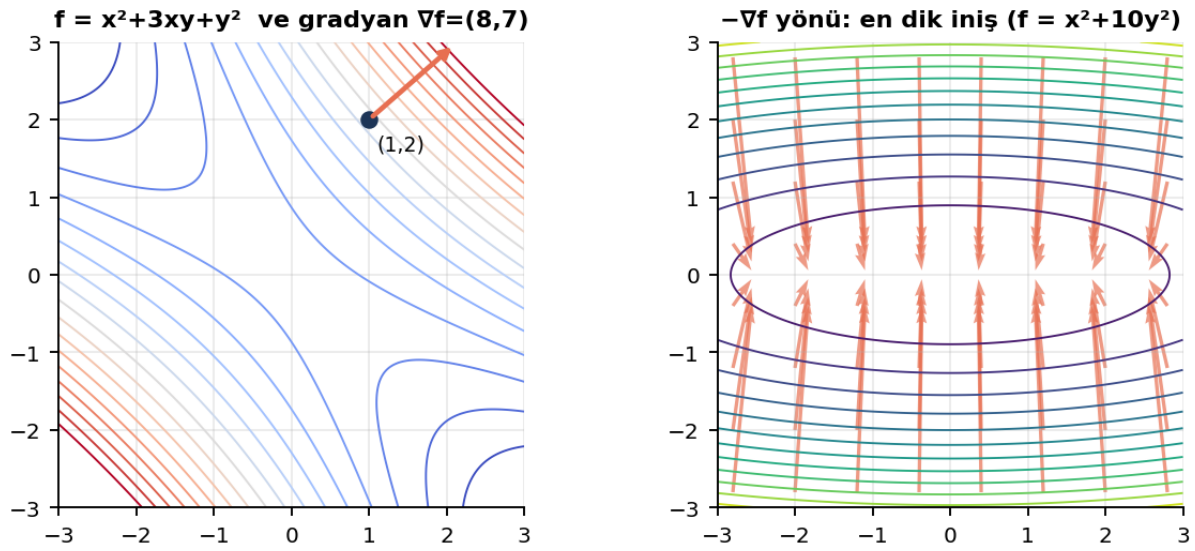
```
import torch
for lr in [0.01, 0.1, 0.5, 0.9, 1.0, 1.1]:
    x = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
    opt = torch.optim.SGD([x], lr=lr)
    for _ in range(20):
        opt.zero_grad(); ((x - 3) ** 2 + 1).backward(); opt.step()
    print(f"lr={lr:5.2f}  x(20)={x.item():13.5e}  carpan={1 - 2*lr:8.4f}")
```

PYTORCH ÇIKTISI

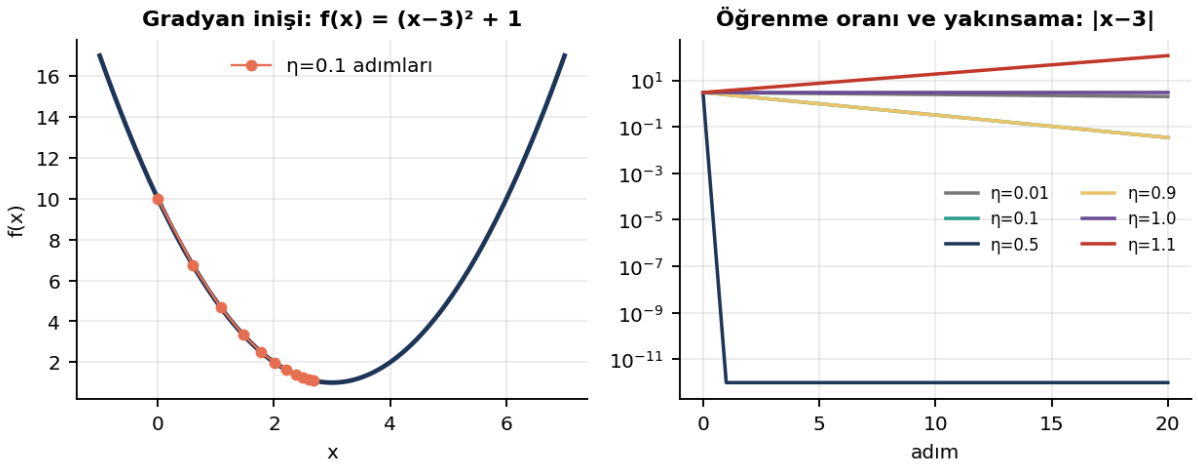
```
lr= 0.01  x(20)=  9.97176e-01  carpan=  0.9800
lr= 0.10  x(20)=  2.96541e+00  carpan=  0.8000
lr= 0.50  x(20)=  3.00000e+00  carpan=  0.0000
lr= 0.90  x(20)=  2.96541e+00  carpan= -0.8000
lr= 1.00  x(20)=  0.00000e+00  carpan= -1.0000
lr= 1.10  x(20)= -1.12013e+02  carpan= -1.2000
```

NASIL ÇALIŞIYOR?

Uzaklığın her adımdaki çarpanı $(1-2\eta)$. $|\text{çarpan}| < 1$ ise yakınsarız. $\eta = 0.01$ için çarpan 0.98 (çok yavaş), $\eta = 0.5$ için 0 (tek adımda mükemmel — ama sadece bu özel fonksiyonda!), $\eta = 0.9$ için -0.8 (ileri geri sıçrayarak yakınsar), $\eta = 1.0$ için -1 (sonsuza kadar sıçrar), $\eta = 1.1$ için -1.2 (**ıraksar**, patlar). Gerçek ağlarda da öğrenme oranı en önemli ayardır: çok küçük = yavaş, çok büyük = kayıp patlar (NaN).



Şekil 1.2 — Solda: f = x² + 3xy + y² konturları ve (1,2) noktasında gradyan oku (en dik çıkış). Sağda: −∇f vektör alanı, kontur çizgilerine dik ve vadinin dibine doğru.



Şekil 1.3 — Solda: gradyan inişi parabol üzerinde. Sağda: farklı öğrenme oranlarında |x−3| uzaklığı: η=0.5 tek adımda biter, η=1 sonsuza dek sıçrar, η=1.1 ıraksar.

Türetme: gradyan inişinin yakınsama hızı

Kuadratik bir kayıp $L(\theta) = \frac{1}{2}(\theta-\theta^*)^T H (\theta-\theta^*)$ alalım (H simetrik pozitif tanımlı, öz-değerleri $\lambda_1 \le \ldots \le \lambda_d$). Gradyan $\nabla L = H(\theta-\theta^*)$, gradyan inişi:

$\theta_{t+1} - \theta^* = (I - \eta H)(\theta_t - \theta^*) \Rightarrow \theta_t - \theta^* = (I - \eta H)^t (\theta_0 - \theta^*)$

H'nin öz-vektör doğrultusu k boyunca hata her adım $(1 - \eta\lambda_k)$ ile çarpılır. Yakınsama için tüm k'lar için $|1 - \eta\lambda_k| < 1$, yani $\boldsymbol{\eta < 2/\lambda_{max}}$. En iyi $\eta^* = 2/(\lambda_{min} + \lambda_{max})$ ile hız $(\kappa-1)/(\kappa+1)$, $\kappa = \lambda_{max}/\lambda_{min}$ **koşul sayısıdır**. Bizim 1B örneğimizde $L = (x-3)^2$ olduğundan $\lambda = 2$, çarpan $(1 - 2\eta)$: Örnek 1.7–1.8'deki sayılar. κ büyükse (uzun dar vadi) yakınsama çok yavaştır; Bölüm 3'ün konusu budur.

Olasılık, logaritma ve çapraz entropi

Basit matematik

Dil modeli, bir sonraki token için bir **olasılık dağılımı** üretir: $p = (p_1, \ldots, p_V)$, her $p_i \ge 0$ ve toplam 1.

Şaşırma (surprisal): olasılığı p olan bir olay gerçekleşirse şaşkınlığımız $-\log p$'dir. $p = 1 \to 0$ (hiç şaşırmadık), $p = 0.5 \to 0.693$, $p = 0.01 \to 4.605$.

Entropi: $H(p) = -\Sigma\, p_i \log p_i$, dağılımın ortalama belirsizliğidir.

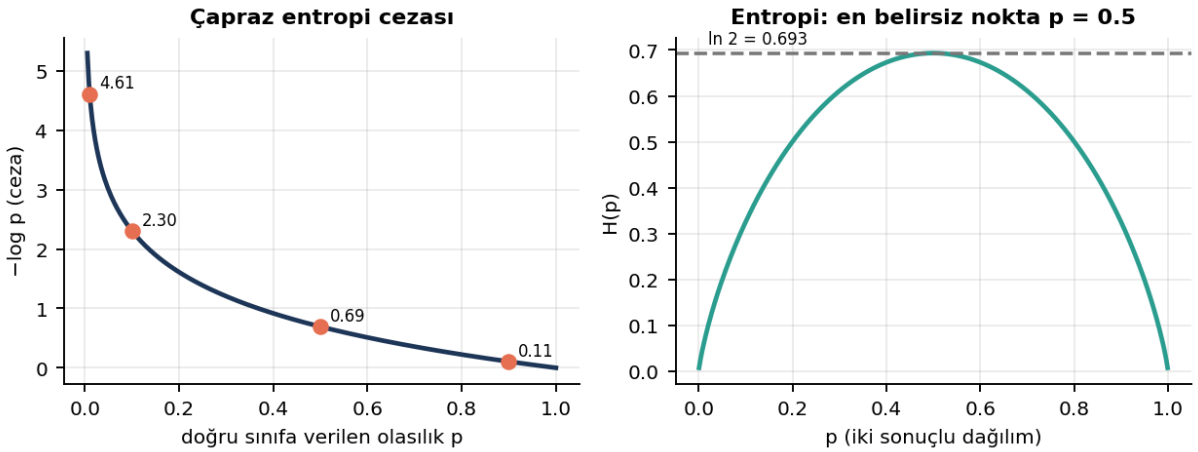
Çapraz entropi: $H(p,q) = -\Sigma\, p_i \log q_i$. Gerçek dağılım p, modelin tahmini q ise bu, modelin ortalama şaşkınlığıdır. LLM eğitiminin kaybı tam olarak budur. Doğru token'a düşük olasılık verirsen ceza büyük olur.

Türetme: çapraz entropi = entropi + KL

Çapraz entropi ile KL ıraksaması arasındaki ilişki, kaybı yorumlamanın anahtarıdır:

$H(p, q) = -\Sigma\, p_i \log q_i = H(p) + D_{KL}(p \,\|\, q)$, $D_{KL}(p \,\|\, q) = \Sigma\, p_i \log(p_i/q_i) \ge 0$

$D_{KL} \ge 0$ (Jensen eşitsizliği: −log içbükey) ve yalnızca p = q iken 0'dır. H(p) modelden bağımsız sabit olduğundan, **çapraz entropiyi minimize etmek KL ıraksamasını, yani modelin dağılımını gerçek dağılıma yaklaştırmayı minimize eder**. Doğru sınıf tek ve kesin ise (p = one-hot) $H(p) = 0$ ve kayıp doğrudan $-\log q_{doğru}$ olur.



Şekil 1.4 — Solda: −log p cezası (p küçüldükçe hızla büyür). Sağda: iki sonuçlu dağılımın entropisi, p = 0.5'te maksimum (ln 2).

Örnek 1.9 — Entropi ve çapraz entropi

PROBLEM

Üç dağılımın entropisini hesapla. Sonra gerçek dağılım p = (1,0,0) iken (yani doğru sınıf birinci) iki farklı model tahmininin çapraz entropisini bul.

ÇÖZÜM (FORTRAN)

```
program entropi
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: p1(2), p2(2), p3(4), gercek(3), q1(3), q2(3)
  p1 = [0.5_dp, 0.5_dp]
  p2 = [0.9_dp, 0.1_dp]
  p3 = [0.25_dp, 0.25_dp, 0.25_dp, 0.25_dp]
  print '(A,F8.4)', 'H(0.5,0.5)         = ', -sum(p1*log(p1))
  print '(A,F8.4)', 'H(0.9,0.1)         = ', -sum(p2*log(p2))
  print '(A,F8.4)', 'H(4 esit olasilik) = ', -sum(p3*log(p3))
  gercek = [1.0_dp, 0.0_dp, 0.0_dp]
  q1 = [0.7_dp, 0.2_dp, 0.1_dp]
  q2 = [0.1_dp, 0.6_dp, 0.3_dp]
  print '(A,F8.4)', 'CE(iyi model)  = ', -log(q1(1))
  print '(A,F8.4)', 'CE(kotu model) = ', -log(q2(1))
  print '(A,F8.4)', 'ln(3) (rastgele tahmin) = ', log(3.0_dp)
end program entropi
```

ÇIKTI

```
H(0.5,0.5)         =   0.6931
H(0.9,0.1)         =   0.3251
H(4 esit olasilik) =   1.3863
CE(iyi model)  =   0.3567
CE(kotu model) =   2.3026
ln(3) (rastgele tahmin) =   1.0986
```

PYTORCH KARŞILIĞI

```
import torch
import torch.nn.functional as F
ent = lambda p: -(p * p.log()).sum()
print("H(0.5,0.5)         =", round(ent(torch.tensor([.5, .5])).item(), 4))
print("H(0.9,0.1)         =", round(ent(torch.tensor([.9, .1])).item(), 4))
print("H(4 esit olasilik) =", round(ent(torch.tensor([.25]*4)).item(), 4))
q1 = torch.tensor([0.7, 0.2, 0.1]); q2 = torch.tensor([0.1, 0.6, 0.3])
# dogru sinif 0 iken capraz entropi = nll_loss(log q)
ce = lambda q: F.nll_loss(q.log()[None, :], torch.tensor([0]))
print("CE(iyi model)  =", round(ce(q1).item(), 4))
print("CE(kotu model) =", round(ce(q2).item(), 4))
print("ln(3)          =", round(torch.log(torch.tensor(3.0)).item(), 4))
```

PYTORCH ÇIKTISI

```
H(0.5,0.5)         = 0.6931
H(0.9,0.1)         = 0.3251
H(4 esit olasilik) = 1.3863
CE(iyi model)  = 0.3567
CE(kotu model) = 2.3026
ln(3)          = 1.0986
```

NASIL ÇALIŞIYOR?

Adil yazı-tura en belirsiz iki-sonuçlu dağılımdır: ln 2 = 0.6931. (0.9, 0.1) daha az belirsizdir (0.325). Dört eşit olasılık ln 4 = 1.386 verir. Gerçek sınıf birinci olduğundan çapraz entropi sadece `−log q(1)`'e iner: iyi model 0.357, kötü model 2.303 ceza alır. Hiçbir şey bilmeyen bir modelin (tekdüze tahmin) kaybı ln V'dir; **eğitime başlarken kaybın ≈ ln V olması bir sağlık kontrolüdür.** Bunu Transformer bölümünde kullanacağız.

PyTorch hızlı başlangıç ve Fortran ↔ PyTorch sözlüğü

Bu kitapta her Fortran örneğinin yanında **PyTorch karşılığı** da verilir. Fortran ile hesabın her adımını elle yazarak mekanizmayı öğrenirsin; PyTorch ile aynı işi üretim ortamlarında kullanılan araçla yaparsın. İkisinin sonuçları aynıdır (kitaptaki çıktılar gerçekten çalıştırılmıştır; rastgele sayı üreteçleri farklı olduğu için rastgele veri kullanan örneklerde yalnızca davranış aynıdır). PyTorch'un üç temel fikri:

- **Tensör**: Fortran dizisinin (matris/vektör) karşılığı; `torch.tensor`, GPU'ya taşınabilir, `dtype` ile kesinlik seçilir (`float64` = Fortran `real(dp)`).
- **Autograd (otomatik türev)**: `requires_grad=True` olan tensörler üzerindeki işlemleri bir hesap grafına kaydeder; `loss.backward()` zincir kuralını sondan başa otomatik uygular. Bölüm 2'de elle yazdığımız geri yayılımın aynısıdır.
- **nn.Module ve optim**: katmanlar (`nn.Linear`), kayıplar (`F.cross_entropy`) ve optimize ediciler (`SGD, Adam, AdamW`) hazırdır; Bölüm 3'te yazdığımız güncelleme kurallarının kütüphane sürümleridir.

Fortran (bu kitap)	PyTorch
real(dp) :: A(n,m)	torch.zeros(n, m, dtype=torch.float64)
matmul(A, B), transpose(A)	A @ B, A.T
sum(x), maxval(x), x**2	x.sum(), x.max(), x ** 2
elemental function f(x)	her torch fonksiyonu zaten elemental
dizi dilimi A(1:3,:), indeks 1'den	A[:3, :], indeks 0'dan
sütun-ana (column-major) bellek	satır-ana (row-major) bellek
elle yazılmış gradyan ve gradyan kontrolü	loss.backward(), torch.autograd.gradcheck
adam_step(p, g, m, s, t, lr)	torch.optim.Adam(params, lr)
write(u) x (ikili dosya)	torch.save / np.fromfile
OpenMP, CUDA Fortran	torch.set_num_threads, tensor.to('cuda')

İpucu

PyTorch bu kitapta **CPU** üzerinde ve tek iş parçacığıyla çalıştırılmıştır; GPU'lu bir makinede kodlar `.to("cuda")` ile çok daha hızlı çalışır.

BÖLÜM 2

2. Sinir Ağları ve Geri Yayılım

Derin öğrenmenin temeli: nöronlardan katmanlara, ileri geçişten gradyanların elle hesaplanmasına.

Bu bölümde

- Perceptron ve çok katmanlı algılayıcı (MLP)
- Aktivasyon fonksiyonları ve türevleri
- İleri yayılım (forward pass) ve kayıp fonksiyonları
- Geri yayılım (backpropagation): zincir kuralı ile tüm gradyanlar
- Ağırlık başlatma (Xavier/He) ve normalizasyon (BatchNorm/LayerNorm)

Hedef: Otomatik türev kütüphanesi olmadan bir ağın gradyanlarını hesaplayabilmek.

Perceptron: en küçük öğrenen birim

Perceptron, girdilerin ağırlıklı toplamını alıp bir eşikten geçiren tek bir "nöron"dur:

$$y = 1 \text{ eğer } w \cdot x + b > 0, \text{ aksi halde } 0$$

Ağırlıklar **w** girdinin önemini, **b** (bias) eşiği belirler. Öğrenme kuralı çok sezgiseldir: tahmin yanlışsa ağırlıkları hatanın yönünde kaydır. $w \leftarrow w + \eta\,(t - y)\,x$. Hedef $t = 1$ iken tahmin $y = 0$ ise w, x yönünde artar; $t = 0$ iken $y = 1$ ise azalır.

Örnek 2.1 — Perceptron ile AND kapısını öğren

PROBLEM

AND fonksiyonunu (yalnızca 1,1 girdisinde 1 üretir) perceptron öğrenme kuralıyla öğret. Ağırlıklar sıfırdan başlasın, $\eta = 0.5$.

ÇÖZÜM (FORTRAN)

```
program perceptron
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: X(4,2), t(4), w(2), b, lr, y
  integer :: ep, i, hata
  X = reshape([0,0,1,1, 0,1,0,1] * 1.0_dp, [4,2])   ! satirlar: 00 01 10 11
  t = [0,0,0,1] * 1.0_dp
  w = 0.0_dp; b = 0.0_dp; lr = 0.5_dp
  do ep = 1, 20
    hata = 0
    do i = 1, 4
      if (dot_product(w, X(i,:)) + b > 0.0_dp) then
        y = 1.0_dp
      else
        y = 0.0_dp
      end if
      if (y /= t(i)) hata = hata + 1
      w = w + lr * (t(i) - y) * X(i,:)
      b = b + lr * (t(i) - y)
    end do
    print '(A,I2,A,I2,A,2F6.2,A,F6.2)', 'epoch ', ep, '  hata=', hata, '  w=', w, '  b=', b
    if (hata == 0) exit
  end do
  do i = 1, 4
    y = merge(1.0_dp, 0.0_dp, dot_product(w, X(i,:)) + b > 0.0_dp)
    print '(A,2F4.0,A,F4.0)', 'girdi', X(i,:), '  -> ', y
  end do
end program perceptron
```

ÇIKTI

```
epoch  1  hata= 1  w=  0.50  0.50  b=  0.50
epoch  2  hata= 3  w=  1.00  0.50  b=  0.00
epoch  3  hata= 3  w=  1.00  0.50  b= -0.50
epoch  4  hata= 2  w=  1.00  1.00  b= -0.50
epoch  5  hata= 1  w=  1.00  0.50  b= -1.00
epoch  6  hata= 0  w=  1.00  0.50  b= -1.00
girdi  0.  0.  ->   0.
girdi  0.  1.  ->   0.
girdi  1.  0.  ->   0.
girdi  1.  1.  ->   1.
```

PYTORCH KARŞILIĞI

```
import torch
X = torch.tensor([[0., 0], [0, 1], [1, 0], [1, 1]])
t = torch.tensor([0., 0, 0, 1])
w = torch.zeros(2); b = torch.tensor(0.0); lr = 0.5
for ep in range(1, 21):
    hata = 0
    for i in range(4):
        y = float((X[i] @ w + b) > 0)
        hata += int(y != t[i])
        w += lr * (t[i] - y) * X[i]
        b += lr * (t[i] - y)
    print(f"epoch {ep:2d}  hata={hata:2d}  w={w.tolist()}  b={b.item():.2f}")
    if hata == 0: break
for i in range(4):
    print("girdi", X[i].tolist(), "->", float((X[i] @ w + b) > 0))
```

PYTORCH ÇIKTISI

```
epoch  1  hata= 1  w=[0.5, 0.5]  b=0.50
epoch  2  hata= 3  w=[1.0, 0.5]  b=0.00
epoch  3  hata= 3  w=[1.0, 0.5]  b=-0.50
epoch  4  hata= 2  w=[1.0, 1.0]  b=-0.50
epoch  5  hata= 1  w=[1.0, 0.5]  b=-1.00
epoch  6  hata= 0  w=[1.0, 0.5]  b=-1.00
girdi [0.0, 0.0] -> 0.0
girdi [0.0, 1.0] -> 0.0
girdi [1.0, 0.0] -> 0.0
girdi [1.0, 1.0] -> 1.0
```

NASIL ÇALIŞIYOR?

Ağırlıklar yalnızca yanlış tahmin yapılan örneklerde değişir. Birkaç epoch sonra hata 0 olur ve program durur. Perceptron *doğrusal olarak ayrılabilen* problemleri (AND, OR) öğrenir; ama XOR gibi ayrılamayan problemleri öğrenemez. Çözüm: perceptron'ları katmanlara dizip aralarına doğrusal olmayan aktivasyon koymak, yani MLP. Bunu Örnek 2.6'da XOR ile göreceğiz.

Aktivasyon fonksiyonları

Katmanlar arasında doğrusal olmayan bir fonksiyon olmazsa, kaç katman koyarsan koy tüm ağ tek bir matris çarpımına indirgenir (doğrusal × doğrusal = doğrusal). Aktivasyon, ağa "eğri" çizebilme gücü verir.

Fonksiyon	Formül	Türev	Not
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$\sigma(1-\sigma)$	(0,1) aralığı; uçlarda gradyan kaybolur
Tanh	$\tanh(x)$	$1 - \tanh^2$	(−1,1) aralığı; sıfır merkezli
ReLU	$\max(0,x)$	1 (x>0), 0 (x<0)	Hızlı ve basit; "ölü nöron" riski
GELU	$x\cdot\Phi(x) \approx 0.5x(1+\tanh(\sqrt{(2/\pi)}(x+0.044715x^3)))$	türev yumuşak	Transformer'larda yaygın

Örnek 2.2 — Aktivasyon fonksiyonları tablosu ve türev kontrolü

PROBLEM

Sigmoid, tanh, ReLU ve GELU değerlerini x = −3…3 için yazdır. Sigmoid ve tanh türevlerinin formüllerini sayısal türevle doğrula.

ÇÖZÜM (FORTRAN)

```
module act
  implicit none
  integer, parameter :: dp = kind(1.0d0)
contains
  elemental function sigmoid(x) result(y)
    real(dp), intent(in) :: x
    real(dp) :: y
    y = 1.0_dp / (1.0_dp + exp(-x))
  end function sigmoid
  elemental function relu(x) result(y)
    real(dp), intent(in) :: x
    real(dp) :: y
    y = max(0.0_dp, x)
  end function relu
  elemental function gelu(x) result(y)
    real(dp), intent(in) :: x
    real(dp) :: y
    y = 0.5_dp * x * (1.0_dp + tanh(0.7978845608028654_dp * (x + 0.044715_dp * x**3)))
  end function gelu
end module act

program aktivasyon
  use act
  implicit none
  integer :: i
  real(dp) :: x0, h, ds, dt, ds_n, dt_n, xi
  print '(A)', '     x   sigmoid     tanh     relu     gelu'
  do i = -3, 3
    xi = real(i, dp)
    print '(F6.1,4F9.4)', xi, sigmoid(xi), tanh(xi), relu(xi), gelu(xi)
  end do
  x0 = 0.7_dp; h = 1.0e-6_dp
  ds = sigmoid(x0) * (1.0_dp - sigmoid(x0))
  dt = 1.0_dp - tanh(x0)**2
  ds_n = (sigmoid(x0+h) - sigmoid(x0-h)) / (2*h)
  dt_n = (tanh(x0+h) - tanh(x0-h)) / (2*h)
  print '(A,2F12.8)', 'sigmoid turev (formul, sayisal): ', ds, ds_n
  print '(A,2F12.8)', 'tanh    turev (formul, sayisal): ', dt, dt_n
end program aktivasyon
```

ÇIKTI

```
     x   sigmoid     tanh     relu     gelu
  -3.0   0.0474  -0.9951   0.0000  -0.0036
  -2.0   0.1192  -0.9640   0.0000  -0.0454
  -1.0   0.2689  -0.7616   0.0000  -0.1588
   0.0   0.5000   0.0000   0.0000   0.0000
   1.0   0.7311   0.7616   1.0000   0.8412
   2.0   0.8808   0.9640   2.0000   1.9546
   3.0   0.9526   0.9951   3.0000   2.9964
sigmoid turev (formul, sayisal):   0.22171287  0.22171287
tanh    turev (formul, sayisal):   0.63473959  0.63473959
```

PYTORCH KARŞILIĞI

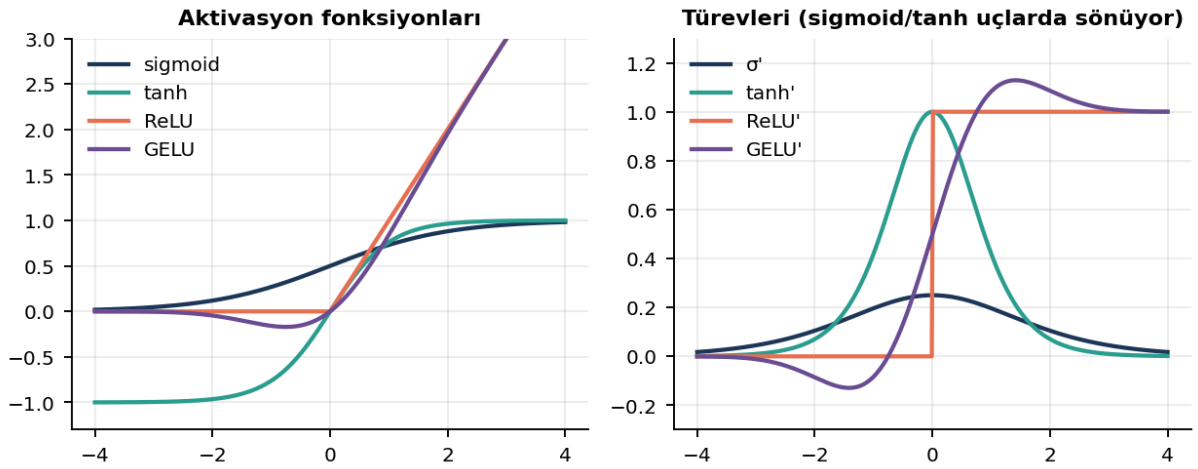
```
import torch
import torch.nn.functional as F
x = torch.arange(-3., 4., dtype=torch.float64)
print("     x   sigmoid     tanh     relu     gelu")
for v in x:
    print(f"{v:6.1f}{torch.sigmoid(v):9.4f}{torch.tanh(v):9.4f}{F.relu(v):9.4f}{F.gelu(v,
          approximate='tanh'):9.4f}")
x0 = torch.tensor(0.7, dtype=torch.float64, requires_grad=True)
ds, = torch.autograd.grad(torch.sigmoid(x0), x0)     # autograd turevi
dt, = torch.autograd.grad(torch.tanh(x0), x0)
s = torch.sigmoid(x0.detach())
print("sigmoid turev (formul, autograd):", f"{(s*(1-s)).item():.8f} {ds.item():.8f}")
print("tanh    turev (formul, autograd):", f"{(1-torch.tanh(x0.detach())**2).item():.8f} {dt.item():.8f}")
```

PYTORCH ÇIKTISI

```
     x   sigmoid     tanh     relu     gelu
  -3.0   0.0474  -0.9951   0.0000  -0.0036
  -2.0   0.1192  -0.9640   0.0000  -0.0454
  -1.0   0.2689  -0.7616   0.0000  -0.1588
   0.0   0.5000   0.0000   0.0000   0.0000
   1.0   0.7311   0.7616   1.0000   0.8412
   2.0   0.8808   0.9640   2.0000   1.9546
   3.0   0.9526   0.9951   3.0000   2.9964
sigmoid turev (formul, autograd): 0.22171287 0.22171287
tanh    turev (formul, autograd): 0.63473959 0.63473959
```

NASIL ÇALIŞIYOR?

Tabloda sigmoid'in sıfırda 0.5, tanh'ın sıfırda 0 verdiğini; ReLU'nun negatifleri kestiğini; GELU'nun ReLU'ya benzediği ama sıfır civarında yumuşak geçiş yaptığını görürsün. Sigmoid için türev formülü $\sigma(1-\sigma)$ kuralı çok kullanışlıdır: türevi hesaplamak için fonksiyonu *yeniden* hesaplamaya gerek yok, ileri geçişteki çıktıyı kullanırız. Tanh için aynı şekilde $1 - y^2$. Sayısal ve formül sonuçları örtüşür.



Şekil 2.1 — Aktivasyon fonksiyonları (solda) ve türevleri (sağda). Sigmoid ve tanh uçlarda doyar (türev ≈ 0): gradyan kaybolur; ReLU ve GELU pozitif tarafta sönmez.

İleri yayılım ve geri yayılım

İleri yayılım: girdiyi katman katman ilerletip tahmine ve kayba ulaşmaktır. Katman l için $z_l = W_l \cdot a_{l-1} + b_l$, $a_l = \varphi(z_l)$. **Geri yayılım**, kayıp L'nin her parametreye göre türevini zincir kuralıyla sondan başa doğru hesaplar.

Basit matematik

$\delta_l = \partial L/\partial z_l$ diyelim ("katmanın hatası"). Son katmandan başlayarak:

$\delta_l = (W_{l+1}^T \delta_{l+1}) \odot \varphi'(z_l)$

$\partial L/\partial W_l = \delta_l \cdot a_{l-1}^T$, $\partial L/\partial b_l = \delta_l$

İfade ağın tüm katmanları için aynıdır. Hata sinyali δ ağın sonundan başa doğru W^T ile geri taşınır ve her katmanda yerel türev φ' ile çarpılır. Böylece *tek bir geri geçişte* tüm parametrelerin gradyanı bulunur. Maliyet kabaca ileri geçişin 2 katıdır.

Örnek 2.3 — Tek nöronda ileri ve geri geçiş (elle)

PROBLEM

$y = \sigma(w \cdot x + b)$, kayıp $L = \frac{1}{2}(y - t)^2$. $x = 1.5$, $w = 0.8$, $b = -0.3$, hedef $t = 1$ için $\partial L/\partial w$ ve $\partial L/\partial b$ değerlerini zincir kuralıyla hesapla ve sayısal türevle doğrula.

ÇÖZÜM (FORTRAN)

```
program noron
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x, w, b, t, z, y, dLdy, dydz, dLdz, dw, db, h, dw_n, db_n
  x = 1.5_dp; w = 0.8_dp; b = -0.3_dp; t = 1.0_dp; h = 1.0e-6_dp
  ! ileri gecis
  z = w*x + b
  y = 1.0_dp / (1.0_dp + exp(-z))
  print '(A,F10.6,A,F10.6,A,F10.6)', 'z=', z, '  y=', y, '  L=', 0.5_dp*(y-t)**2
  ! geri gecis
  dLdy = y - t
  dydz = y * (1.0_dp - y)
  dLdz = dLdy * dydz
  dw = dLdz * x
  db = dLdz
  ! sayisal kontrol
  dw_n = (loss(w+h, b) - loss(w-h, b)) / (2*h)
  db_n = (loss(w, b+h) - loss(w, b-h)) / (2*h)
  print '(A,2F14.9)', 'dL/dw (geri yayilim, sayisal): ', dw, dw_n
  print '(A,2F14.9)', 'dL/db (geri yayilim, sayisal): ', db, db_n
contains
  function loss(ww, bb) result(l)
    real(dp), intent(in) :: ww, bb
    real(dp) :: l, yy
    yy = 1.0_dp / (1.0_dp + exp(-(ww*x + bb)))
    l = 0.5_dp * (yy - t)**2
  end function loss
end program noron
```

ÇIKTI

```
z=  0.900000  y=  0.710950  L=  0.041775
dL/dw (geri yayilim, sayisal):   -0.089099949  -0.089099949
dL/db (geri yayilim, sayisal):   -0.059399966  -0.059399966
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor(1.5, dtype=torch.float64)
t = torch.tensor(1.0, dtype=torch.float64)
w = torch.tensor(0.8, dtype=torch.float64, requires_grad=True)
b = torch.tensor(-0.3, dtype=torch.float64, requires_grad=True)
z = w * x + b
y = torch.sigmoid(z)
L = 0.5 * (y - t) ** 2
print(f"z={z.item():.6f}  y={y.item():.6f}  L={L.item():.6f}")
L.backward()                              # geri gecis
dLdz = (y - t) * y * (1 - y)              # Fortran'daki elle formul
print("dL/dw (autograd, elle):", f"{w.grad.item():.9f} {(dLdz * x).item():.9f}")
print("dL/db (autograd, elle):", f"{b.grad.item():.9f} {dLdz.item():.9f}")
```

PYTORCH ÇIKTISI

```
z=0.900000  y=0.710950  L=0.041775
dL/dw (autograd, elle): -0.089099949 -0.089099949
dL/db (autograd, elle): -0.059399966 -0.059399966
```

NASIL ÇALIŞIYOR?

Zincir: L → y → z → (w, b). Önce ∂L/∂y = y − t, sonra ∂y/∂z = y(1−y), çarpınca ∂L/∂z. Parametrelere gelince z = w·x + b olduğu için ∂z/∂w = x ve ∂z/∂b = 1. Bu yüzden dw = dLdz·x, db = dLdz. İlk satırdaki sayılar: z = 0.9, y = σ(0.9) ≈ 0.7109. Geri yayılım sonuçları sayısal türevle yaklaşık 9 basamağa kadar uyuşur.

Kayıp fonksiyonları

Kayıp	Formül	Ne zaman?
MSE	$(1/n)\ \Sigma\ (\hat{y} - y)^2$	Regresyon (sayı tahmini)
MAE	$(1/n)\ \Sigma\ \lvert\hat{y} - y\rvert$	Aykırı değerlere dayanıklı regresyon
İkili çapraz entropi (BCE)	$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$	İki sınıflı sınıflandırma (sigmoid çıkışı)
Çapraz entropi (CE)	$-\log p_{\text{doğru}}$	Çok sınıflı (softmax) → dil modeli

Örnek 2.4 — Dört kayıp fonksiyonunu hesapla

PROBLEM

Gerçek y = [1,0,1,1] ve tahmin ŷ = [0.9, 0.2, 0.6, 0.99] için MSE, MAE ve BCE'yi bul. Ayrıca "kendinden emin ama yanlış" bir tahminin cezasına bak.

ÇÖZÜM (FORTRAN)

```
program kayiplar
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: y(4), yh(4)
  y  = [1.0_dp, 0.0_dp, 1.0_dp, 1.0_dp]
  yh = [0.9_dp, 0.2_dp, 0.6_dp, 0.99_dp]
  print '(A,F8.5)', 'MSE = ', sum((yh - y)**2) / 4
  print '(A,F8.5)', 'MAE = ', sum(abs(yh - y)) / 4
  print '(A,F8.5)', 'BCE = ', -sum(y*log(yh) + (1-y)*log(1-yh)) / 4
  print '(A)', 'Gercek sinif 1 iken tahmin olasiligi p icin -log(p):'
  print '(A,F8.4)', '  p=0.99 : ', -log(0.99_dp)
  print '(A,F8.4)', '  p=0.50 : ', -log(0.50_dp)
  print '(A,F8.4)', '  p=0.01 : ', -log(0.01_dp)
end program kayiplar
```

ÇIKTI

```
MSE =  0.05253
MAE =  0.17750
BCE =  0.21235
Gercek sinif 1 iken tahmin olasiligi p icin -log(p):
  p=0.99 :   0.0101
  p=0.50 :   0.6931
  p=0.01 :   4.6052
```

PYTORCH KARŞILIĞI

```
import torch
import torch.nn.functional as F
y = torch.tensor([1., 0., 1., 1.]); yh = torch.tensor([0.9, 0.2, 0.6, 0.99])
print(f"MSE = {F.mse_loss(yh, y):.5f}")
print(f"MAE = {F.l1_loss(yh, y):.5f}")
print(f"BCE = {F.binary_cross_entropy(yh, y):.5f}")
print("Gercek sinif 1 iken -log(p):")
for p in [0.99, 0.5, 0.01]:
    print(f"  p={p:.2f} : {-torch.log(torch.tensor(p)):.4f}")
```

PYTORCH ÇIKTISI

```
MSE = 0.05252
MAE = 0.17750
BCE = 0.21235
Gercek sinif 1 iken -log(p):
  p=0.99 : 0.0101
  p=0.50 : 0.6931
  p=0.01 : 4.6052
```

NASIL ÇALIŞIYOR?

MSE: hatalar 0.1, 0.2, 0.4, 0.01 → kareleri 0.01, 0.04, 0.16, 0.0001 → ortalama 0.0525. MAE: (0.1+0.2+0.4+0.01)/4 = 0.1775. BCE logaritmik olduğu için **güvenle yapılan yanlışı** ağır cezalandırır: doğru sınıfa 0.01 olasılık veren model 4.6 ceza alırken 0.99 veren sadece 0.01 alır. Sınıflandırmada ve dil modellemede CE kullanmamızın nedeni budur.

Softmax ve çapraz entropi: en güzel gradyan

Basit matematik

Softmax, ham skorları (logits) olasılığa çevirir: $p_i = \exp(z_i) / \Sigma_j \exp(z_j)$.

Çapraz entropi ile birleştirilince ($L = -\log p_k$, k doğru sınıf) türev inanılmaz sadeleşir:

$\partial L/\partial z_i = p_i - 1[i = k]$

Yani gradyan = "tahmin edilen olasılıklar − gerçek one-hot vektörü". Doğru sınıf için negatif (z_k artmalı), yanlışlar için pozitif (azalmalı). LLM'in son katmanındaki geri yayılım başlangıcı tam olarak bu tek satırdır.

Örnek 2.5 — Softmax + çapraz entropi gradyanı ve sayısal kararlılık

PROBLEM

z = [2.0, 1.0, 0.1] logitleri için, doğru sınıf 2 iken softmax, kayıp ve gradyan (p − onehot) değerlerini hesapla ve sayısal türevle doğrula. Ardından z = [1000, 1001, 1002] için saf softmax ile "max çıkarma" hilesini karşılaştır.

ÇÖZÜM (FORTRAN)

```
program smax
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: z(3), p(3), g(3), onehot(3), zz(3), pp(3), lp, lm, gn(3), h
  real(dp) :: big(3), naive(3), stable(3)
  integer :: k, i
  z = [2.0_dp, 1.0_dp, 0.1_dp]; k = 2; h = 1.0e-6_dp
  call softmax(z, p)
  onehot = 0.0_dp; onehot(k) = 1.0_dp
  g = p - onehot
  print '(A,3F10.6)', 'p            = ', p
  print '(A,F10.6)',  'kayip -log p2= ', -log(p(k))
  do i = 1, 3
    zz = z; zz(i) = z(i) + h; call softmax(zz, pp); lp = -log(pp(k))
    zz = z; zz(i) = z(i) - h; call softmax(zz, pp); lm = -log(pp(k))
    gn(i) = (lp - lm) / (2*h)
  end do
  print '(A,3F10.6)', 'grad p-onehot= ', g
  print '(A,3F10.6)', 'sayisal grad = ', gn
```

```
  big = [1000.0_dp, 1001.0_dp, 1002.0_dp]
  naive = exp(big) / sum(exp(big))          ! exp(1000) tasar -> Inf/Inf = NaN
  call softmax(big, stable)
  print '(A,3F10.6)', 'saf softmax  = ', naive
  print '(A,3F10.6)', 'kararli      = ', stable
contains
  subroutine softmax(a, q)
    real(dp), intent(in)  :: a(:)
    real(dp), intent(out) :: q(:)
    q = exp(a - maxval(a))                  ! en buyuk deger 0 olur, tasma yok
    q = q / sum(q)
  end subroutine softmax
end program smax
```

ÇIKTI

```
p            =   0.659001  0.242433  0.098566
kayip -log p2=   1.417030
grad p-onehot=   0.659001 -0.757567  0.098566
sayisal grad =   0.659001 -0.757567  0.098566
saf softmax  =        NaN       NaN       NaN
kararli      =   0.090031  0.244728  0.665241
```

PYTORCH KARŞILIĞI

```
import torch
import torch.nn.functional as F
z = torch.tensor([2.0, 1.0, 0.1], dtype=torch.float64, requires_grad=True)
k = 1                                       # dogru sinif (0 tabanli)
loss = F.cross_entropy(z[None, :], torch.tensor([k]))   # softmax + CE tek fonksiyonda
loss.backward()
p = torch.softmax(z.detach(), 0)
print("p            =", [round(v, 6) for v in p.tolist()])
print("kayip        =", round(loss.item(), 6))
print("autograd grad=", [round(v, 6) for v in z.grad.tolist()])
print("p - onehot   =", [round(v, 6) for v in (p - F.one_hot(torch.tensor(k), 3)).tolist()])
big = torch.tensor([1000., 1001., 1002.], dtype=torch.float64)
print("saf softmax  =", (big.exp() / big.exp().sum()).tolist())   # inf/inf = nan
print("torch.softmax=", [round(v, 6) for v in torch.softmax(big, 0).tolist()])
```

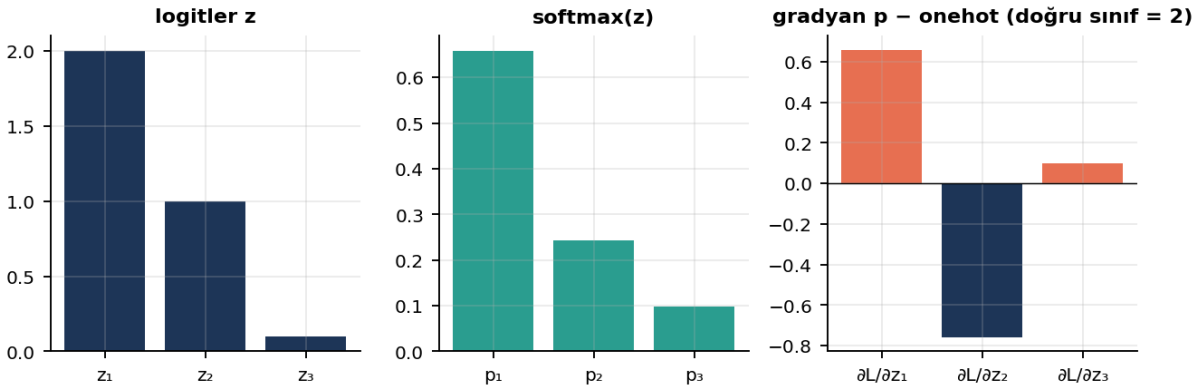
PYTORCH ÇIKTISI

```
p            = [0.659001, 0.242433, 0.098566]
kayip        = 1.41703
autograd grad= [0.659001, -0.757567, 0.098566]
p - onehot   = [0.659001, -0.757567, 0.098566]
saf softmax  = [nan, nan, nan]
torch.softmax= [0.090031, 0.244728, 0.665241]
```

NASIL ÇALIŞIYOR?

Softmax değişmezdir: tüm girdilerden aynı sabiti çıkarmak sonucu değiştirmez (pay ve paydada e^{-c} sadeleşir). Bu yüzden en büyük logit'i çıkarırız; en büyük üs $e^0 = 1$ olur ve taşma imkânsızdır. Saf yöntem 1000'in üssünü almaya çalışıp `NaN` üretir. **Gerçek kodda softmax her zaman max çıkarılarak yazılır.** Gradyan satırı p − onehot ile sayısal türev aynı çıkar.

> **Türetme: softmax + çapraz entropi gradyanı**
> Önce softmax'ın türevi. $p_i = e^{z_i}/S$, $S = \Sigma_j e^{z_j}$. Bölüm kuralıyla:
> $\partial p_i/\partial z_j = [\delta_{ij}\, e^{z_i}\, S - e^{z_i}\, e^{z_j}] / S^2 = p_i\,(\delta_{ij} - p_j)$
> Kayıp $L = -\log p_k$ (k doğru sınıf). Zincir kuralı:
> $\partial L/\partial z_j = -(1/p_k)\, \partial p_k/\partial z_j = -(1/p_k)\, p_k(\delta_{kj} - p_j) = p_j - \delta_{kj}$
> Mükemmel bir sadeleşme: gradyan "tahmin − hedef"tir. Aynı yapı, sigmoid + ikili çapraz entropide $\hat{y} - t$ olarak ve doğrusal çıkış + MSE'de $(\hat{y} - y)$ olarak görünür; bunlar üstel ailenin **kanonik bağlantı fonksiyonları**dır ve bu tesadüf değil.



Şekil 2.2 — Logitler z, softmax olasılıkları p ve çapraz entropi gradyanı p − onehot. Doğru sınıfın gradyanı negatif (logit artmalı), yanlışlarınki pozitif.

Çok katmanlı algılayıcı (MLP) ve XOR

XOR, tek perceptron'ın çözemediği klasik problemdir. Gizli katmanı olan küçük bir MLP bunu çözer. Aşağıdaki program kitabın **ilk tam sinir ağıdır**: ileri geçiş, elle yazılmış geri geçiş, gradyan kontrolü ve eğitim döngüsü. Parametrelerin hepsini tek bir düz vektörde (`th`) tutup matris/vektör "görünümlerini" *pointer remapping* ile çıkarıyoruz. Bu hile gradyan kontrolünü ve optimizasyonu çok kolaylaştırır; kitap boyunca kullanacağız.

Örnek 2.6 — XOR için 2-4-1 MLP: geri yayılım + gradyan kontrolü + eğitim

PROBLEM

2 girdili, 4 tanh gizli nöronlu, 1 sigmoid çıkışlı ağı BCE kaybıyla XOR üzerinde eğit. Eğitimden önce elle yazılmış gradyanları sayısal türevle karşılaştır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module mlp
  use rng_mod
  implicit none
  integer, parameter :: nh = 4, nth = 4*nh + 1
  real(dp), target, save :: th(nth), g(nth)
  real(dp), pointer :: W1(:,:), b1(:), W2(:), b2
  real(dp), pointer :: gW1(:,:), gb1(:), gW2(:), gb2
  real(dp), save :: X(4,2), t(4)
contains
  subroutine baglan()                  ! duz vektore matris "gorunumleri" ver
    W1(1:2,1:nh) => th(1:2*nh)
    b1 => th(2*nh+1:3*nh)
    W2 => th(3*nh+1:4*nh)
    b2 => th(4*nh+1)
    gW1(1:2,1:nh) => g(1:2*nh)
    gb1 => g(2*nh+1:3*nh)
    gW2 => g(3*nh+1:4*nh)
    gb2 => g(4*nh+1)
  end subroutine baglan

  subroutine fwd(x1, h, yhat)          ! ileri gecis (tek ornek)
    real(dp), intent(in)  :: x1(2)
    real(dp), intent(out) :: h(nh), yhat
    h = tanh(matmul(x1, W1) + b1)
    yhat = 1.0_dp / (1.0_dp + exp(-(dot_product(h, W2) + b2)))
  end subroutine fwd

  function loss() result(l)            ! ortalama BCE
    real(dp) :: l, h(nh), yh
    integer :: i
    l = 0.0_dp
    do i = 1, 4
```

```
      call fwd(X(i,:), h, yh)
      l = l - t(i)*log(yh) - (1.0_dp - t(i))*log(1.0_dp - yh)
    end do
    l = l / 4
  end function loss

  subroutine backward()                ! geri gecis: g vektorunu doldurur
    real(dp) :: h(nh), yh, dz2, dh(nh), dz1(nh)
    integer :: i
    g = 0.0_dp
    do i = 1, 4
      call fwd(X(i,:), h, yh)
      dz2 = yh - t(i)                  ! sigmoid + BCE -> p - hedef
      gW2 = gW2 + dz2 * h
      gb2 = gb2 + dz2
      dh  = dz2 * W2
      dz1 = dh * (1.0_dp - h**2)       ! tanh turevi
      gW1 = gW1 + spread(X(i,:), 2, nh) * spread(dz1, 1, 2)
      gb1 = gb1 + dz1
    end do
    g = g / 4
  end subroutine backward
end module mlp

program xor_egitim
  use mlp
  implicit none
  integer :: i, ep
  real(dp) :: eps, old, lp, lm, num, maxrel, h(nh), yh
  call baglan()
  X = reshape([0,0,1,1, 0,1,0,1] * 1.0_dp, [4,2])
  t = [0.0_dp, 1.0_dp, 1.0_dp, 0.0_dp]
  call rng_seed(7)
  call randn1(th, 0.8_dp)
  ! --- gradyan kontrolu ---
  call backward()
  eps = 1.0e-6_dp; maxrel = 0.0_dp
  do i = 1, nth
    old = th(i)
    th(i) = old + eps; lp = loss()
    th(i) = old - eps; lm = loss()
    th(i) = old
    num = (lp - lm) / (2*eps)
    maxrel = max(maxrel, abs(num - g(i)) / max(1.0e-8_dp, abs(num) + abs(g(i))))
  end do
  print '(A,ES10.2)', 'gradyan kontrolu, en buyuk bagil hata = ', maxrel
  ! --- egitim ---
  do ep = 0, 6000
    if (mod(ep, 1000) == 0) print '(A,I5,A,F10.6)', 'adim ', ep, '  kayip = ', loss()
    call backward()
    th = th - 0.5_dp * g
  end do
  do i = 1, 4
    call fwd(X(i,:), h, yh)
    print '(A,2F4.0,A,F8.4)', 'girdi', X(i,:), '  tahmin = ', yh
  end do
end program xor_egitim
```

ÇIKTI

```
gradyan kontrolu, en buyuk bagil hata =   5.19E-07
adim     0  kayip =   0.708526
adim  1000  kayip =   0.005387
adim  2000  kayip =   0.002512
adim  3000  kayip =   0.001635
adim  4000  kayip =   0.001212
adim  5000  kayip =   0.000962
adim  6000  kayip =   0.000798
girdi  0.  0.  tahmin =   0.0001
girdi  0.  1.  tahmin =   0.9991
girdi  1.  0.  tahmin =   0.9991
girdi  1.  1.  tahmin =   0.0013
```

PYTORCH KARŞILIĞI

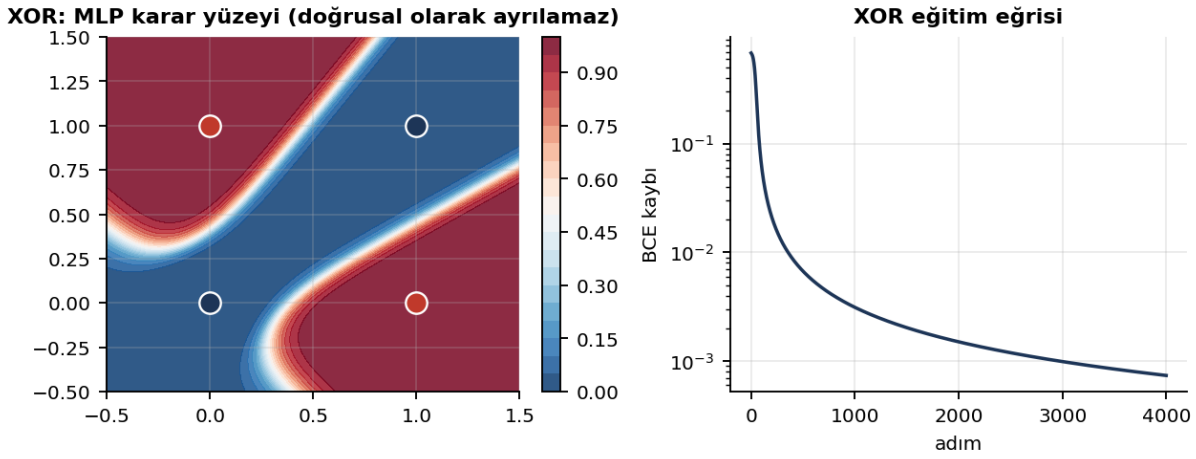
```
import torch
import torch.nn.functional as F
torch.set_default_dtype(torch.float64)
torch.manual_seed(7)
X = torch.tensor([[0., 0], [0, 1], [1, 0], [1, 1]]); t = torch.tensor([0., 1, 1, 0])
nh = 4
W1 = (torch.randn(2, nh) * 0.8).requires_grad_(); b1 = torch.zeros(nh, requires_grad=True)
W2 = (torch.randn(nh) * 0.8).requires_grad_();    b2 = torch.zeros((), requires_grad=True)
def forward(W1, b1, W2, b2):
    return torch.tanh(X @ W1 + b1) @ W2 + b2
def loss_fn(W1, b1, W2, b2):
    return F.binary_cross_entropy_with_logits(forward(W1, b1, W2, b2), t)
ok = torch.autograd.gradcheck(loss_fn, (W1, b1, W2, b2), eps=1e-6, atol=1e-6)
print("gradcheck (autograd vs sayisal turev):", ok)
opt = torch.optim.SGD([W1, b1, W2, b2], lr=0.5)
for ep in range(6001):
    loss = loss_fn(W1, b1, W2, b2)
    if ep % 1000 == 0: print(f"adim {ep:5d}  kayip = {loss.item():10.6f}")
    opt.zero_grad(); loss.backward(); opt.step()
for i in range(4):
    print("girdi", X[i].tolist(), " tahmin =", round(torch.sigmoid(forward(W1, b1, W2, b2))[i].item(), 4))
```

PYTORCH ÇIKTISI

```
gradcheck (autograd vs sayisal turev): True
adim     0  kayip =   0.746221
adim  1000  kayip =   0.004749
adim  2000  kayip =   0.002225
adim  3000  kayip =   0.001446
adim  4000  kayip =   0.001070
adim  5000  kayip =   0.000848
adim  6000  kayip =   0.000702
girdi [0.0, 0.0]  tahmin = 0.0001
girdi [0.0, 1.0]  tahmin = 0.999
girdi [1.0, 0.0]  tahmin = 0.9993
girdi [1.0, 1.0]  tahmin = 0.001
```

NASIL ÇALIŞIYOR?

Geri geçiş, "Zincir kuralı" bölümündeki formüllerin birebir kodudur: `dz2 = ŷ − t` (sigmoid+BCE sadeleşmesi), çıkış katmanı gradyanları `dz2·h`, geriye taşınan hata `dz2·W2`, tanh türeviyle çarpım `(1−h²)` ve ilk katman gradyanı girdi ile dış çarpım. `spread(x,2,nh) * spread(dz1,1,2)` iki vektörün dış çarpımını (2×nh matris) tek satırda üretir. Gradyan kontrolünde bağıl hata ~1e-9 mertebesinde çıkıyorsa geri yayılımı doğru yazmışsındır; 1e-4'ten büyükse bir yerde hata vardır. Eğitimin sonunda kayıp sıfıra yaklaşır ve XOR (0,1,1,0) öğrenilir. **Hiçbir otomatik türev kütüphanesi kullanmadan** bir ağı eğittin.



Şekil 2.3 — Solda: 2-4-1 MLP'nin XOR için öğrendiği karar yüzeyi: iki sınıf doğrusal olarak ayrılamaz ama tanh gizli katmanı yüzeyi büker. Sağda: eğitim kaybı.

Türetme: iki katmanlı ağda geri yayılım

Gizli katmanlı bir ağda geri yayılımı iki katmanlı bir örnek üzerinde türetelim: $z_1 = W_1x + b_1$, $a = \varphi(z_1)$, $z_2 = W_2a + b_2$, $L = \ell(z_2, t)$. Zincir kuralıyla:

$\delta_2 = \partial L/\partial z_2 = \ell'(z_2, t)$, $\partial L/\partial W_2 = \delta_2\, a^T$, $\partial L/\partial b_2 = \delta_2$

$\partial L/\partial a = W_2^T\, \delta_2$, $\delta_1 = \partial L/\partial z_1 = (W_2^T\, \delta_2) \odot \varphi'(z_1)$, $\partial L/\partial W_1 = \delta_1\, x^T$, $\partial L/\partial b_1 = \delta_1$

Her W için gradyan, o katmana *giren* aktivasyon ile çıkıştan gelen hata sinyalinin **dış çarpımıdır**. Bu yüzden ileri geçişte aktivasyonları saklamak zorundayız (Bölüm 5'te bellek maliyeti). Derin ağda $\delta_l = (W_{l+1}^T\, \delta_{l+1}) \odot \varphi'$ çarpımı L kez tekrarlanır; her çarpanın büyüklüğü 1'den küçükse gradyan *kaybolur*, büyükse *patlar*: başlatma ve normalizasyonun gerekçesi.

Katman matrisleri: toplu (batch) hesap

Gerçekte örnekleri tek tek değil, **B** örneklik yığınlar hâlinde işleriz. Bir yoğun (dense) katman `Y = X·W + b` olur; X boyutu (B × girdi). Geri yayılım da matris çarpımına dönüşür:

$$dW = X^T \cdot dY,\; db = \Sigma_{\text{satırlar}}\, dY,\; dX = dY \cdot W^T$$

Örnek 2.7 — Yoğun katmanın toplu geri geçişi

PROBLEM

B=3 örnek, 4 girdi, 2 çıktılı bir katmanda $L = \tfrac{1}{2}\Sigma Y^2$ kaybı için dW, db ve dX'i matris formülleriyle hesapla ve sayısal türevle kontrol et.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program dense
  use rng_mod
  implicit none
  integer, parameter :: B = 3, nin = 4, nout = 2
  real(dp) :: X(B,nin), W(nin,nout), bias(nout), Y(B,nout), dY(B,nout)
  real(dp) :: dW(nin,nout), db(nout), dX(B,nin), eps, old, num
  call rng_seed(1)
  call randn2(X, 1.0_dp); call randn2(W, 0.5_dp)
  bias = [0.1_dp, -0.2_dp]; eps = 1.0e-6_dp
  Y  = matmul(X, W) + spread(bias, 1, B)
  dY = Y                                   ! L = 0.5*sum(Y^2) icin dL/dY = Y
  dW = matmul(transpose(X), dY)
  db = sum(dY, dim=1)
  dX = matmul(dY, transpose(W))
  old = W(2,1)
  W(2,1) = old + eps; num = L()
  W(2,1) = old - eps; num = (num - L()) / (2*eps)
  W(2,1) = old
  print '(A,2F12.7)', 'dW(2,1): geri yayilim, sayisal = ', dW(2,1), num
  old = bias(2)
  bias(2) = old + eps; num = L()
  bias(2) = old - eps; num = (num - L()) / (2*eps)
  bias(2) = old
  print '(A,2F12.7)', 'db(2)  : geri yayilim, sayisal = ', db(2), num
  old = X(3,4)
  X(3,4) = old + eps; num = L()
  X(3,4) = old - eps; num = (num - L()) / (2*eps)
  X(3,4) = old
  print '(A,2F12.7)', 'dX(3,4): geri yayilim, sayisal = ', dX(3,4), num
contains
  function L() result(r)
    real(dp) :: r
    r = 0.5_dp * sum((matmul(X, W) + spread(bias, 1, B))**2)
  end function L
end program dense
```

ÇIKTI

```
dW(2,1): geri yayilim, sayisal =   -2.5173745  -2.5173745
db(2)  : geri yayilim, sayisal =    0.5114069   0.5114069
dX(3,4): geri yayilim, sayisal =    0.2667316   0.2667316
```

PYTORCH KARŞILIĞI

```
import torch
torch.set_default_dtype(torch.float64)
torch.manual_seed(1)
X = torch.randn(3, 4, requires_grad=True)
W = (torch.randn(4, 2) * 0.5).requires_grad_()
b = torch.tensor([0.1, -0.2], requires_grad=True)
Y = X @ W + b                              # = torch.nn.functional.linear(X, W.T, b)
L = 0.5 * (Y ** 2).sum()
L.backward()
dY = Y.detach()
print("dW(2,1) autograd vs X^T dY :", f"{W.grad[1,0]:.7f} {(X.detach().T @ dY)[1,0]:.7f}")
print("db(2)   autograd vs sum dY :", f"{b.grad[1]:.7f} {dY.sum(0)[1]:.7f}")
print("dX(3,4) autograd vs dY W^T :", f"{X.grad[2,3]:.7f} {(dY @ W.detach().T)[2,3]:.7f}")
```

PYTORCH ÇIKTISI

```
dW(2,1) autograd vs X^T dY : -0.3084336 -0.3084336
db(2)   autograd vs sum dY : 0.2759555 0.2759555
dX(3,4) autograd vs dY W^T : -0.2757662 -0.2757662
```

NASIL ÇALIŞIYOR?

Boyut kontrolü gradyanı hatasız yazmanın en iyi aracıdır: dW, W ile aynı boyutta (4×2) olmalı. X^T (4×3) · dY (3×2) = (4×2) □. dX, X ile aynı boyutta (3×4): dY (3×2) · W^T (2×4) □. db, bias ile aynı boyutta ve tüm örneklerin gradyanlarının toplamıdır (her örnek aynı bias'ı kullandı). Üç sayısal kontrol de eşleşir.

Ağırlık başlatma ve normalizasyon

Ağırlıkları rastgele ama **doğru ölçekte** başlatmak zorundayız. Çok küçük başlatırsan sinyal katmanlardan geçtikçe söner (vanishing), çok büyük başlatırsan patlar ya da tanh/sigmoid doyuma gider ve gradyan sıfırlanır.

> **Basit matematik**
> Bir katmanda n girdi varsa ve ağırlıklar bağımsız, varyansı σ^2_w ise, çıktının varyansı yaklaşık $n \cdot \sigma^2_w \cdot \mathrm{Var}(\text{girdi})$ olur. Sinyal boyutunu korumak için $n \cdot \sigma^2_w \approx 1$ istenir:
>
> **Xavier/Glorot** (tanh, sigmoid): $\sigma_w = 1/\sqrt{n}$
>
> **He/Kaiming** (ReLU): ReLU girdilerin yarısını sıfırladığı için $\sigma_w = \sqrt{(2/n)}$

Örnek 2.8 — Başlatma ölçeği: 20 katman boyunca sinyal

PROBLEM

64 genişlikli, 20 katmanlı bir ağdan standart normal bir girdiyi geçir. tanh için üç farklı ağırlık ölçeğinde, ReLU için Xavier ve He başlatmada katmanlardaki aktivasyonların RMS değerini yazdır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program baslatma
  use rng_mod
  implicit none
  integer, parameter :: n = 64, nl = 20
  integer, parameter :: show(5) = [1, 2, 5, 10, 20]
  real(dp) :: W(n,n), x(n), sc(3), rec(5,3), rr(5,2)
```

```
  integer :: k, l, j
  call rng_seed(3)
  sc = [0.02_dp, 1.0_dp/sqrt(real(n,dp)), 0.5_dp]
  do k = 1, 3                                        ! tanh
    call randn1(x, 1.0_dp)
    do l = 1, nl
      call randn2(W, sc(k))
      x = tanh(matmul(x, W))
      do j = 1, 5
        if (l == show(j)) rec(j,k) = sqrt(sum(x**2) / n)
      end do
    end do
  end do
  print '(A)', 'TANH  katman   std=0.02   std=1/sqrt(n)   std=0.5'
  do j = 1, 5
    print '(I11,3ES14.3)', show(j), rec(j,:)
  end do
  do k = 1, 2                                        ! ReLU
    call randn1(x, 1.0_dp)
    do l = 1, nl
      call randn2(W, merge(1.0_dp, sqrt(2.0_dp), k == 1) / sqrt(real(n,dp)))
      x = max(0.0_dp, matmul(x, W))
      do j = 1, 5
        if (l == show(j)) rr(j,k) = sqrt(sum(x**2) / n)
      end do
    end do
  end do
  print '(A)', 'RELU  katman   Xavier    He'
  do j = 1, 5
    print '(I11,2ES14.3)', show(j), rr(j,:)
  end do
end program baslatma
```

ÇIKTI

```
TANH  katman   std=0.02   std=1/sqrt(n)   std=0.5
          1     2.047E-01     5.235E-01     8.982E-01
          2     3.137E-02     4.238E-01     9.013E-01
          5     1.408E-04     3.296E-01     8.757E-01
         10     1.605E-08     2.565E-01     8.954E-01
         20     1.530E-16     1.828E-01     8.667E-01
RELU  katman   Xavier    He
          1     5.634E-01     8.802E-01
          2     4.061E-01     1.050E+00
          5     1.333E-01     7.490E-01
         10     1.981E-02     8.129E-01
         20     9.815E-04     5.692E-01
```

PYTORCH KARŞILIĞI

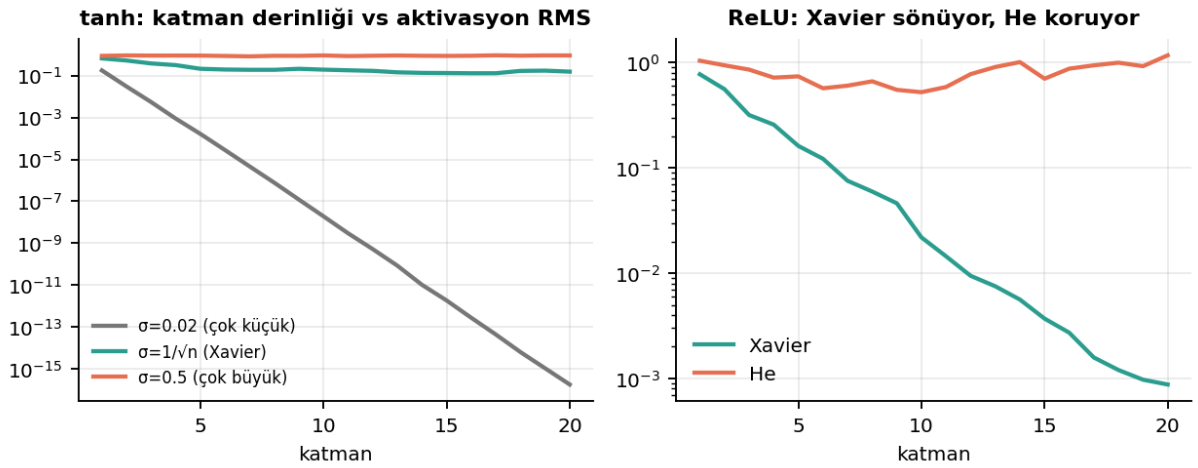
```
import torch, math
torch.manual_seed(3)
n, nl = 64, 20
show = [1, 2, 5, 10, 20]
def rms(act, std):
    x = torch.randn(n); out = []
    for l in range(1, nl + 1):
        x = act(x @ (torch.randn(n, n) * std))
        if l in show: out.append(x.pow(2).mean().sqrt().item())
    return out
tanh_r = [rms(torch.tanh, s) for s in [0.02, 1 / math.sqrt(n), 0.5]]
print("TANH  katman   std=0.02   std=1/sqrt(n)   std=0.5")
for j, l in enumerate(show):
    print(f"{l:11d}" + "".join(f"{tanh_r[k][j]:14.3e}" for k in range(3)))
relu = torch.relu
rr = [rms(relu, 1 / math.sqrt(n)), rms(relu, math.sqrt(2 / n))]
print("RELU  katman   Xavier    He")
for j, l in enumerate(show):
    print(f"{l:11d}" + "".join(f"{rr[k][j]:14.3e}" for k in range(2)))
# hazir baslaticilar: nn.init.xavier_normal_, nn.init.kaiming_normal_
w = torch.empty(n, n); torch.nn.init.kaiming_normal_(w, nonlinearity='relu')
print("kaiming_normal_ std =", round(w.std().item(), 4), " (sqrt(2/n) =", round(math.sqrt(2 / n), 4), ")")
```

PYTORCH ÇIKTISI

```
TANH  katman   std=0.02   std=1/sqrt(n)   std=0.5
          1     1.341e-01     7.233e-01     8.868e-01
          2     2.063e-02     4.993e-01     8.960e-01
          5     6.854e-05     3.376e-01     8.531e-01
         10     6.564e-09     2.152e-01     8.800e-01
         20     6.471e-17     2.166e-01     8.980e-01
RELU  katman   Xavier    He
          1     6.419e-01     8.650e-01
          2     4.358e-01     7.840e-01
          5     1.416e-01     8.529e-01
         10     1.382e-02     1.036e+00
         20     4.787e-04     4.808e-01
kaiming_normal_ std = 0.1792  (sqrt(2/n) = 0.1768 )
```

NASIL ÇALIŞIYOR?

tanh ile std = 0.02 alırsak aktivasyonlar her katmanda küçülür ve 20. katmanda neredeyse sıfır olur: ağ hiçbir şey öğrenemez. std = 0.5 ise ön-aktivasyonlar büyür, tanh doyuma gider (çıktılar ±1 civarına yapışır, türev ≈ 0). Xavier ölçeği ise sinyali çok daha yavaş söndürür (0.52 → 0.18) ve eğitilebilir kalır. ReLU'da Xavier her katmanda sinyali yaklaşık √2 oranında zayıflatır (20 katmanda ≈ 1e-3); He başlatma bunu düzeltir ve RMS ≈ 0.5–1 aralığında kalır. Bu yüzden "doğru başlatma" bir ayrıntı değil, eğitimin başlayıp başlamayacağını belirler.



Şekil 2.4 — Katman derinliği boyunca aktivasyon RMS'si. Solda (tanh): çok küçük σ → sönüm, çok büyük σ → doyum. Sağda (ReLU): Xavier sönüyor, He sinyali koruyor.

Türetme: Xavier ve He başlatma

Bir katmanda $y = \Sigma_{i=1}^{n} w_i x_i$ olsun; w_i ve x_i bağımsız, E[w] = 0 olsun. Varyans bağımsız terimlerde toplanır: $\mathrm{Var}(y) = \Sigma_i \mathrm{Var}(w_i x_i) = n \cdot \mathrm{Var}(w) \cdot E[x^2]$ (E[w] = 0 olduğundan Var(wx) = Var(w)E[x²])

Sinyalin her katmanda aynı ölçekte kalması için n Var(w) = 1, yani $\boldsymbol{\sigma_w = 1/\sqrt{n}}$ (Xavier). ReLU girdinin yarısını sıfırlar: E[x²] = ½ Var(ön-aktivasyon); denge için n Var(w)/2 = 1, $\boldsymbol{\sigma_w = \sqrt{(2/n)}}$ (He). L katman sonra sinyalin ölçeği $(n\,\mathrm{Var}(w))^L$ ile çarpılır: n Var(w) = 0.5 ise 20 katmanda 10^{-6}, 2 ise 10^{6}: şekildeki sönüm ve patlama.

Normalizasyon: BatchNorm ve LayerNorm

Aktivasyonların ölçeğini eğitim boyunca kontrol altında tutmak için normalizasyon katmanları kullanılır. İkisi de aynı formülü uygular: $\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$, sonra öğrenilebilir ölçek ve kayma: $y = \gamma \hat{x} + \beta$. Fark, μ ve σ²'nin **hangi eksende** hesaplandığıdır:

- **BatchNorm**: her özellik için *batch boyunca* (örnekler arası). Batch boyutuna bağımlıdır; diziler ve LLM'lerde sorunludur.
- **LayerNorm**: her örnek için *özellikler boyunca*. Batch'ten bağımsızdır; Transformer'larda standarttır.

Örnek 2.9 — BatchNorm ile LayerNorm: eksen farkı

PROBLEM

X = [[1,2],[3,6],[5,10]] (3 örnek, 2 özellik) matrisine BatchNorm ve LayerNorm uygula; her birinin hangi eksenden ortalama/varyans aldığını sonuçlardan göster.

ÇÖZÜM (FORTRAN)

```
program normalizasyon
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: X(3,2), Xb(3,2), Xl(3,2), mu, v, eps
  integer :: i, j
  X = reshape([1,3,5, 2,6,10] * 1.0_dp, [3,2])      ! sutunlar = ozellikler
  eps = 1.0e-5_dp
  do j = 1, 2                                       ! BatchNorm: sutun bazli
    mu = sum(X(:,j)) / 3
    v  = sum((X(:,j) - mu)**2) / 3
    Xb(:,j) = (X(:,j) - mu) / sqrt(v + eps)
  end do
  do i = 1, 3                                       ! LayerNorm: satir bazli
    mu = sum(X(i,:)) / 2
    v  = sum((X(i,:) - mu)**2) / 2
    Xl(i,:) = (X(i,:) - mu) / sqrt(v + eps)
  end do
  print '(A)', 'BatchNorm (her sutun ortalama 0, varyans 1):'
  do i = 1, 3
    print '(2F10.4)', Xb(i,:)
  end do
  print '(A)', 'LayerNorm (her satir ortalama 0, varyans 1):'
  do i = 1, 3
    print '(2F10.4)', Xl(i,:)
  end do
end program normalizasyon
```

ÇIKTI

```
BatchNorm (her sutun ortalama 0, varyans 1):
   -1.2247   -1.2247
    0.0000    0.0000
    1.2247    1.2247
LayerNorm (her satir ortalama 0, varyans 1):
   -1.0000    1.0000
   -1.0000    1.0000
   -1.0000    1.0000
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
X = torch.tensor([[1., 2], [3, 6], [5, 10]])
bn = nn.BatchNorm1d(2, affine=False, eps=1e-5)    # batch ekseni (sutun bazli)
ln = nn.LayerNorm(2, elementwise_affine=False, eps=1e-5)   # ozellik ekseni (satir bazli)
bn.train()
print("BatchNorm:"); print(bn(X))
print("LayerNorm:"); print(ln(X))
```

PYTORCH ÇIKTISI

```
BatchNorm:
tensor([[-1.2247e+00, -1.2247e+00],
        [-5.9605e-08,  0.0000e+00],
        [ 1.2247e+00,  1.2247e+00]])
LayerNorm:
tensor([[-1.0000,  1.0000],
        [-1.0000,  1.0000],
        [-1.0000,  1.0000]])
```

NASIL ÇALIŞIYOR?

Birinci özellik [1,3,5]: ortalama 3, varyans 8/3, standart sapma 1.633 → [−1.2247, 0, 1.2247]. İkinci özellik birincinin 2 katı olduğu için normalleşince aynı olur. LayerNorm'da her satır tek başına

normalleşir: $[1,2] \to [-1, 1]$, $[3,6] \to [-1, 1]$, $[5,10] \to [-1, 1]$ (ε yüzünden çok az eksik). Dikkat: LayerNorm bir örneğin sonucunu başka örneklere bağlamaz. Bu, metin üretirken tek token işlerken de aynı hesabı yapabilmemizi sağlar.

BÖLÜM 3

3. Optimizasyon: SGD, Momentum ve Adam

Gradyanı bulduk; şimdi onunla ne yapacağız? Basit matematikle, adım adım optimizasyon algoritmalarını kuruyoruz.

Bu bölümde

- Gradyan inişinin neden işe yaradığı (Taylor yaklaşımı)
- Mini-batch SGD ve gürültünün rolü
- Momentum: top yuvarlanıyor gibi düşün
- Adam: her parametre için kendi adım boyutu
- AdamW ve ağırlık sönümü (weight decay)

Hedef: SGD, Momentum ve Adam'ı sıfırdan yazıp aralarındaki farkı sayılarla görmek.

Gradyan inişi neden çalışır?

Basit matematik

Küçük bir adım atarsak fonksiyon yaklaşık doğrusal davranır (Taylor):

$L(\theta + \Delta) \approx L(\theta) + \nabla L \cdot \Delta$

Adımı $\Delta = -\eta\, \nabla L$ seçersek: $L(\theta - \eta \nabla L) \approx L(\theta) - \eta\, |\nabla L|^2$. Kare her zaman pozitif olduğundan, η yeterince küçükse kayıp **mutlaka azalır**.

Sınır nerede? Yaklaşım yalnızca küçük adımlarda geçerlidir; η büyürse eğriliği hesaba katmadığımız için fonksiyon yukarı sıçrayabilir (Bölüm 1'deki ıraksama).

SGD: rastgele küçük gruplarla öğrenmek

Gerçek veri kümelerinde kaybı *bütün* örnekler üzerinden hesaplamak çok pahalıdır. **Stochastic Gradient Descent (SGD)** her adımda yalnızca küçük, rastgele bir örnek grubunun (mini-batch) gradyanını kullanır. Bu gradyan gerçeğinin *gürültülü ama yansız* bir tahminidir. Gürültü bir dezavantaj gibi görünse de, ucuz adımlar sayesinde toplamda çok daha hızlı ilerleriz ve gürültü dar çukurlardan kaçmaya yardım eder.

Örnek 3.1 — Mini-batch SGD ile doğru uydurma

PROBLEM

$y = 2x + 1$ + gürültü verisi üret (100 nokta). Doğrusal modeli $\hat{y} = w \cdot x + b$, MSE kaybı ve batch boyutu 10 olan SGD ile eğit. Sonucu analitik en küçük kareler çözümüyle karşılaştır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program sgd_dogru
  use rng_mod
  implicit none
  integer, parameter :: n = 100, bs = 10
  real(dp) :: x(n), y(n), w, b, lr, gw, gb, r, xm, ym, wols, bols, loss
  integer :: i, ep, s
  call rng_seed(11)
  do i = 1, n
    x(i) = 2.0_dp * uniform() - 1.0_dp
```

```
    y(i) = 2.0_dp * x(i) + 1.0_dp + 0.1_dp * gauss()
  end do
  w = 0.0_dp; b = 0.0_dp; lr = 0.1_dp
  do ep = 0, 50
    if (mod(ep, 10) == 0) then
      loss = sum((w*x + b - y)**2) / n
      print '(A,I3,A,F8.4,A,F8.4,A,F9.5)', 'epoch ', ep, '  w=', w, '  b=', b, '  kayip=', loss
    end if
    do s = 1, n, bs                       ! her epoch'ta 10 mini-batch
      r  = 0.0_dp
      gw = 0.0_dp; gb = 0.0_dp
      do i = s, s + bs - 1
        r  = w*x(i) + b - y(i)            ! hata
        gw = gw + 2.0_dp * r * x(i)
        gb = gb + 2.0_dp * r
      end do
      w = w - lr * gw / bs
      b = b - lr * gb / bs
    end do
  end do
  xm = sum(x)/n; ym = sum(y)/n           ! analitik cozum
  wols = sum((x-xm)*(y-ym)) / sum((x-xm)**2)
  bols = ym - wols*xm
  print '(A,F8.4,A,F8.4)', 'en kucuk kareler: w=', wols, '  b=', bols
end program sgd_dogru
```

ÇIKTI

```
epoch   0  w=  0.0000  b=  0.0000  kayip=  2.23360
epoch  10  w=  1.9919  b=  1.0080  kayip=  0.00971
epoch  20  w=  1.9928  b=  1.0081  kayip=  0.00971
epoch  30  w=  1.9928  b=  1.0081  kayip=  0.00971
epoch  40  w=  1.9928  b=  1.0081  kayip=  0.00971
epoch  50  w=  1.9928  b=  1.0081  kayip=  0.00971
en kucuk kareler: w=  1.9978  b=  1.0038
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(11)
n, bs = 100, 10
x = torch.rand(n, 1) * 2 - 1
y = 2 * x + 1 + 0.1 * torch.randn(n, 1)
model = torch.nn.Linear(1, 1)
torch.nn.init.zeros_(model.weight); torch.nn.init.zeros_(model.bias)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for ep in range(51):
    if ep % 10 == 0:
        loss = torch.nn.functional.mse_loss(model(x), y)
        print(f"epoch {ep:3d}  w={model.weight.item():7.4f}  b={model.bias.item():7.4f}
              kayip={loss.item():8.5f}")
    for s in range(0, n, bs):                    # mini-batch'ler
        xb, yb = x[s:s+bs], y[s:s+bs]
        loss = torch.nn.functional.mse_loss(model(xb), yb)
        opt.zero_grad(); loss.backward(); opt.step()
A = torch.cat([x, torch.ones(n, 1)], 1)
sol = torch.linalg.lstsq(A, y).solution.flatten()
print(f"en kucuk kareler: w={sol[0]:.4f}  b={sol[1]:.4f}")
```

PYTORCH ÇIKTISI

```
epoch   0  w= 0.0000  b= 0.0000  kayip= 2.40489
epoch  10  w= 2.0165  b= 0.9930  kayip= 0.00923
epoch  20  w= 2.0193  b= 0.9928  kayip= 0.00923
epoch  30  w= 2.0193  b= 0.9928  kayip= 0.00923
epoch  40  w= 2.0193  b= 0.9928  kayip= 0.00923
epoch  50  w= 2.0193  b= 0.9928  kayip= 0.00923
en kucuk kareler: w=2.0210  b=0.9889
```

NASIL ÇALIŞIYOR?

MSE için gradyan: $\partial L/\partial w = (2/B)\ \Sigma\ (\hat{y}-y)\cdot x$ ve $\partial L/\partial b = (2/B)\ \Sigma\ (\hat{y}-y)$. Her mini-batch'te bunları hesaplayıp parametreleri güncelliyoruz; bir epoch 10 güncelleme demektir. w ve b kısa sürede gerçek değerlere (2 ve 1) yaklaşır; küçük sapma veri gürültüsündendir ve analitik çözümle aynı

bölgededir. SGD'nin tam bir geçişi beklemeden her 10 örnekte ilerlediğine dikkat et.

Türetme: momentum neden √κ hızlanması verir?
Kuadratik $L = ½ \lambda \theta^2$ (tek öz-doğrultu) üzerinde momentum yinelemesi: $v_{t+1} = \beta v_t + \lambda\theta_t$, $\theta_{t+1} = \theta_t - \eta v_{t+1}$. Bu doğrusal bir sistemdir:

$[\theta_{t+1}; v_{t+1}] = M [\theta_t; v_t]$, $M = [[1 - \eta\lambda, -\eta\beta], [\lambda, \beta]]$

Yakınsama hızı M'nin spektral yarıçapıyla belirlenir. Optimal ayarda ($\beta = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$, $\eta = 4/(\sqrt{\lambda_{max}} + \sqrt{\lambda_{min}})^2$) hız $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ olur; sade gradyan inişinde $(\kappa - 1)/(\kappa + 1)$ idi. $\kappa = 100$ için GD ≈ 0.98, momentum ≈ 0.82 çarpanı: adım sayısı kabaca κ'dan $\sqrt{\kappa}$'ya düşer (**100 → 10 kat hızlanma**). Bu, Örnek 3.3'te momentumun GD'den belirgin hızlı olmasının nedenidir.

Momentum: ivme ekle

Dar ve uzun bir vadide (bir yönde dik, diğer yönde yatay) düz gradyan inişi duvarlar arasında zikzak çizer ve vadi boyunca çok yavaş ilerir. **Momentum** geçmiş gradyanların hareketli ortalamasını tutar: duvarlar arası sıçramalar birbirini götürür, vadi yönündeki sürekli bileşenler ise birikir.

Basit matematik
$v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$ ($\beta \approx 0.9$)

Eğer gradyan sabit g ise v büyür ve sonunda $v \to g/(1-\beta)$ olur. $\beta = 0.9$ için bu 10 katıdır: etkin adım boyutu $\eta/(1-\beta)$ olur.

Örnek 3.2 — Momentum'un etkin adımı

PROBLEM

Sabit gradyan $g = 1$ ve $\beta = 0.9$ iken hız v'nin adım adım nasıl arttığını izle.

ÇÖZÜM (FORTRAN)

```
program momentum_hiz
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: v, beta, g
  integer :: k
  beta = 0.9_dp; g = 1.0_dp; v = 0.0_dp
  do k = 1, 30
    v = beta * v + g
    if (k <= 5 .or. mod(k, 5) == 0) print '(A,I3,A,F9.5)', 'adim ', k, '   v = ', v
  end do
  print '(A,F9.5)', 'teorik sinir g/(1-beta) = ', g / (1.0_dp - beta)
end program momentum_hiz
```

ÇIKTI

```
adim   1   v =   1.00000
adim   2   v =   1.90000
adim   3   v =   2.71000
adim   4   v =   3.43900
adim   5   v =   4.09510
adim  10   v =   6.51322
adim  15   v =   7.94109
adim  20   v =   8.78423
adim  25   v =   9.28210
adim  30   v =   9.57609
teorik sinir g/(1-beta) =  10.00000
```

PYTORCH KARŞILIĞI

```
import torch
p = torch.tensor(0.0, requires_grad=True)
opt = torch.optim.SGD([p], lr=1.0, momentum=0.9)   # v = 0.9 v + g ; p -= lr*v
for k in range(1, 31):
    p.grad = torch.tensor(1.0)                      # sabit gradyan g = 1
    opt.step()
```

```
    v = opt.state[p]['momentum_buffer'].item()
    if k <= 5 or k % 5 == 0: print(f"adim {k:3d}   v = {v:9.5f}")
print("teorik sinir g/(1-beta) =", 1 / (1 - 0.9))
```

PYTORCH ÇIKTISI

```
adim   1   v =   1.00000
adim   2   v =   1.90000
adim   3   v =   2.71000
adim   4   v =   3.43900
adim   5   v =   4.09510
adim  10   v =   6.51321
adim  15   v =   7.94109
adim  20   v =   8.78423
adim  25   v =   9.28210
adim  30   v =   9.57609
teorik sinir g/(1-beta) = 10.000000000000002
```

NASIL ÇALIŞIYOR?

$v_k = 1 + 0.9 + 0.9^2 + \ldots$ geometrik serisidir ve $1/(1-0.9) = 10$'a yaklaşır. Yani momentumla sürekli aynı yöne giden parametreler yaklaşık **10 kat hızlı** ilerler; sürekli işaret değiştiren gradyanlar ise birbirini sönümler.

Örnek 3.3 — Kötü koşullanmış yüzeyde GD, Momentum ve Adam

PROBLEM

$f(x,y) = x^2 + 100y^2$ fonksiyonunu (x yönü yatay, y yönü dik bir vadi) (5,5) noktasından başlayarak üç yöntemle küçült. Belirli adımlarda f değerini karşılaştır.

ÇÖZÜM (FORTRAN)

```
program karsilastir
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer, parameter :: show(6) = [1, 10, 20, 50, 100, 200]
  real(dp) :: p(2), g(2), v(2), m(2), s(2), lr, beta, b1, b2, eps
  real(dp) :: fgd(6), fmo(6), fad(6), mh(2), sh(2)
  integer :: k, j
  ! ---- duz gradyan inisi
  p = [5.0_dp, 5.0_dp]; lr = 0.0099_dp
  do k = 1, 200
    g = [2.0_dp*p(1), 200.0_dp*p(2)]
    p = p - lr * g
    do j = 1, 6
      if (k == show(j)) fgd(j) = p(1)**2 + 100.0_dp*p(2)**2
    end do
  end do
  ! ---- momentum
  p = [5.0_dp, 5.0_dp]; v = 0.0_dp; lr = 0.01_dp; beta = 0.9_dp
  do k = 1, 200
    g = [2.0_dp*p(1), 200.0_dp*p(2)]
    v = beta * v + g
    p = p - lr * v
    do j = 1, 6
      if (k == show(j)) fmo(j) = p(1)**2 + 100.0_dp*p(2)**2
    end do
  end do
  ! ---- Adam
  p = [5.0_dp, 5.0_dp]; m = 0.0_dp; s = 0.0_dp
  lr = 0.3_dp; b1 = 0.9_dp; b2 = 0.999_dp; eps = 1.0e-8_dp
  do k = 1, 200
    g = [2.0_dp*p(1), 200.0_dp*p(2)]
    m = b1*m + (1.0_dp - b1)*g
    s = b2*s + (1.0_dp - b2)*g**2
    mh = m / (1.0_dp - b1**k)
    sh = s / (1.0_dp - b2**k)
    p = p - lr * mh / (sqrt(sh) + eps)
    do j = 1, 6
      if (k == show(j)) fad(j) = p(1)**2 + 100.0_dp*p(2)**2
    end do
  end do
```

```
  print '(A)', '  adim          GD      Momentum          Adam'
  do j = 1, 6
    print '(I6,3ES14.3)', show(j), fgd(j), fmo(j), fad(j)
  end do
end program karsilastir
```

ÇIKTI

```
  adim          GD      Momentum          Adam
     1     2.425E+03     2.524E+03     2.231E+03
    10     1.686E+03     1.151E+02     4.440E+02
    20     1.125E+03     4.538E+01     8.637E-01
    50     3.349E+02     2.398E+01     1.253E+00
   100     4.443E+01     1.271E-01     4.689E-02
   200     7.817E-01     3.869E-07     1.563E-06
```

PYTORCH KARŞILIĞI

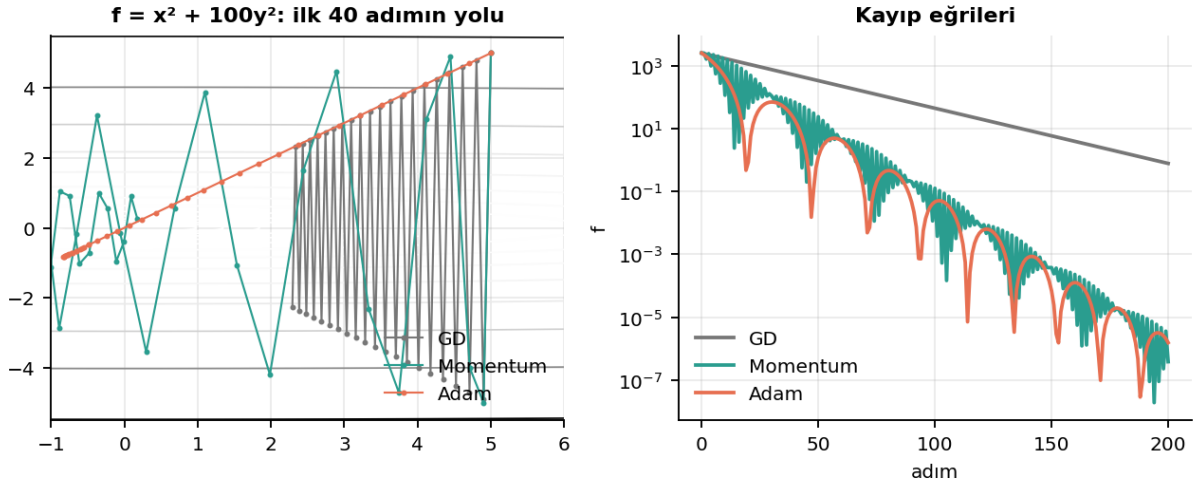
```
import torch
def run(make_opt, steps=200):
    p = torch.tensor([5.0, 5.0], dtype=torch.float64, requires_grad=True)
    opt = make_opt([p]); hist = {}
    for k in range(1, steps + 1):
        f = p[0] ** 2 + 100 * p[1] ** 2
        opt.zero_grad(); f.backward(); opt.step()
        if k in (1, 10, 20, 50, 100, 200):
            hist[k] = (p[0] ** 2 + 100 * p[1] ** 2).item()
    return hist
gd  = run(lambda ps: torch.optim.SGD(ps, lr=0.0099))
mo  = run(lambda ps: torch.optim.SGD(ps, lr=0.01, momentum=0.9))
ad  = run(lambda ps: torch.optim.Adam(ps, lr=0.3))
print("  adim          GD      Momentum          Adam")
for k in gd: print(f"{k:6d}{gd[k]:14.3e}{mo[k]:14.3e}{ad[k]:14.3e}")
```

PYTORCH ÇIKTISI

```
  adim          GD      Momentum          Adam
     1     2.425e+03     2.524e+03     2.231e+03
    10     1.686e+03     1.151e+02     4.440e+02
    20     1.125e+03     4.538e+01     8.637e-01
    50     3.349e+02     2.398e+01     1.253e+00
   100     4.443e+01     1.271e-01     4.689e-02
   200     7.817e-01     3.869e-07     1.563e-06
```

NASIL ÇALIŞIYOR?

GD için öğrenme oranı en dik yönün keskinliğiyle sınırlıdır: y yönünde eğrilik 200 olduğundan $\eta < 0.01$ olmak zorunda (aksi halde y yönü ıraksar). Bu yüzden $\eta = 0.0099$ seçtik; ama x yönünde eğrilik yalnızca 2 olduğundan o yönde adım başına çarpan $1-2\eta \approx 0.98$ olur: çok yavaş ilerler (200 adımda ancak $0.98^{200} \approx 0.02$ kat). Momentum, vadi yönündeki ilerlemeyi biriktirerek (etkin adım $\eta/(1-\beta) = 10$ kat) ve y yönündeki zikzağı sönümleyerek belirgin biçimde daha hızlı iner. Adam her koordinatı kendi gradyan büyüklüğüne bölerek *adım boyutlarını eşitler*, bu yüzden x ve y yönlerinde benzer hızda ilerler; sabit öğrenme oranıyla minimum çevresinde küçük salınımlar yapabilir. Kötü koşullanmış yüzeyler gerçek ağlarda çok yaygındır; adaptif yöntemlerin popülerliğinin nedeni budur.



Şekil 3.1 — f = x² + 100y² üzerinde GD, Momentum ve Adam: solda ilk 60 adımın yolu (momentum y ekseninde salınır ama vadi yönünde hızlıdır), sağda kayıp eğrileri (log ölçek).

Adam: her parametre için ayrı öğrenme oranı

Adam iki hareketli ortalama tutar: gradyanın kendisinin (**m**, 1. moment) ve karesinin (**s**, 2. moment).

Basit matematik

$m \leftarrow \beta_1 m + (1-\beta_1)\, g$

$s \leftarrow \beta_2 s + (1-\beta_2)\, g^2$

$\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$ (bias düzeltmesi)

$\theta \leftarrow \theta - \eta \cdot \hat{m} \,/\, (\sqrt{\hat{s}} + \varepsilon)$

Sezgi: $\hat{m}$ yön ve ortalama büyüklüğü verir, $\sqrt{\hat{s}}$ tipik gradyan büyüklüğünü. Oran yaklaşık ±1 olur, yani her parametre adım başına yaklaşık η kadar hareket eder. Gradyanı büyük parametre yavaşlar, küçük olan hızlanır. Varsayılanlar: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Örnek 3.4 — Adam'ın ilk adımları: bias düzeltmesi neden gerekli?

PROBLEM

Sabit gradyan g = 0.5 ve η = 0.1 için Adam'ın ilk 4 adımındaki m, s, m̂, ŝ ve adım büyüklüğünü yazdır.

ÇÖZÜM (FORTRAN)

```
program adam_elle
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: m, s, mh, sh, g, lr, b1, b2, eps, step, theta
  integer :: t
  g = 0.5_dp; lr = 0.1_dp; b1 = 0.9_dp; b2 = 0.999_dp; eps = 1.0e-8_dp
  m = 0.0_dp; s = 0.0_dp; theta = 1.0_dp
  print '(A)', ' t        m          s          mhat       shat       adim    theta'
  do t = 1, 4
    m = b1*m + (1.0_dp - b1)*g
    s = b2*s + (1.0_dp - b2)*g**2
    mh = m / (1.0_dp - b1**t)
    sh = s / (1.0_dp - b2**t)
    step = lr * mh / (sqrt(sh) + eps)
    theta = theta - step
    print '(I2,6F11.6)', t, m, s, mh, sh, step, theta
  end do
end program adam_elle
```

ÇIKTI

```
 t        m          s          mhat       shat       adim    theta
 1   0.050000   0.000250   0.500000   0.250000   0.100000   0.900000
 2   0.095000   0.000500   0.500000   0.250000   0.100000   0.800000
 3   0.135500   0.000749   0.500000   0.250000   0.100000   0.700000
 4   0.171950   0.000999   0.500000   0.250000   0.100000   0.600000
```

PYTORCH KARŞILIĞI

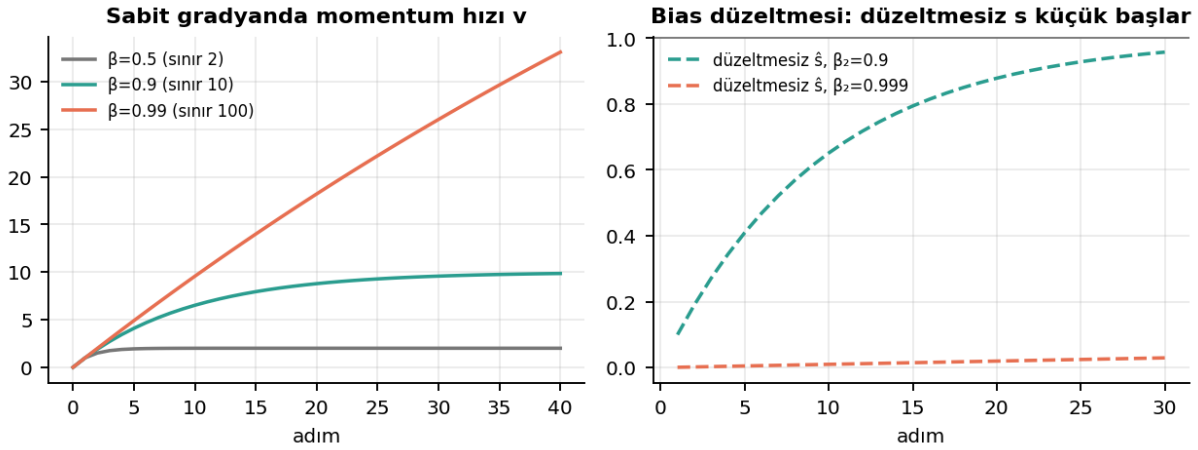
```
import torch
theta = torch.tensor(1.0, dtype=torch.float64, requires_grad=True)
opt = torch.optim.Adam([theta], lr=0.1, betas=(0.9, 0.999), eps=1e-8)
print(" t        m          s          mhat       shat       adim    theta")
for t in range(1, 5):
    theta.grad = torch.tensor(0.5, dtype=torch.float64)    # sabit gradyan
    before = theta.item()
    opt.step()
    st = opt.state[theta]
    m, s = st['exp_avg'].item(), st['exp_avg_sq'].item()
    mh, sh = m / (1 - 0.9 ** t), s / (1 - 0.999 ** t)
    print(f"{t:2d}{m:11.6f}{s:11.6f}{mh:11.6f}{sh:11.6f}{before -
          theta.item():11.6f}{theta.item():11.6f}")
```

PYTORCH ÇIKTISI

```
 t        m          s          mhat       shat       adim    theta
 1   0.050000   0.000250   0.500000   0.250000   0.100000   0.900000
 2   0.095000   0.000500   0.500000   0.250000   0.100000   0.800000
 3   0.135500   0.000749   0.500000   0.250000   0.100000   0.700000
 4   0.171950   0.000999   0.500000   0.250000   0.100000   0.600000
```

NASIL ÇALIŞIYOR?

m ve s sıfırdan başladığı için ilk adımlarda gerçek değerlerinin çok altındadır: t = 1'de m = 0.05 (gerçek 0.5'in onda biri), s = 0.00025 (gerçek 0.25'in binde biri). Bias düzeltmesi $(1-\beta^t)$ ile bölünce $\hat{m}$ = 0.5 ve $\hat{s}$ = 0.25 olur. Böylece adım = $\eta \cdot 0.5/\sqrt{0.25} = \eta = 0.1$: **gradyan ne kadar büyük olursa olsun ilk adım yaklaşık η kadardır**. Düzeltme olmasaydı ilk adımlar çok küçük (m) ve orantısız (s) olurdu.



Şekil 3.2 — Solda: sabit gradyanda momentum hızı 1/(1−β) sınırına yaklaşır. Sağda: Adam'ın ikinci moment tahmini (düzeltmesiz) başlangıçta 1'in çok altından başlar, bias düzeltmesi bunu telafi eder.

Türetme: Adam bias düzeltmesi ve adım boyu

Adam'ın bias düzeltmesinin kaynağı. $s_t = \beta_2\, s_{t-1} + (1-\beta_2)\, g_t^2$, $s_0 = 0$. Açarsak: $s_t = (1-\beta_2) \sum_{k=1}^{t} \beta_2^{t-k} g_k^2$. Gradyan karesi sabit ve g^2 ise: $E[s_t] = g^2 (1-\beta_2) \sum \beta_2^{t-k} = g^2 (1 - \beta_2^t)$. Yani ilk adımlarda s gerçek değerin yalnızca $(1 - \beta_2^t)$ kesri kadardır (t=1'de %0.1). $1 - \beta_2^t$ ile bölmek tahmini yansız yapar. Aynı mantık m için $(1 - \beta_1^t)$.

Sabit rejimde Adam adımı $\eta\, \hat{m}/(\sqrt{\hat{s}}) \approx \eta\, g/|g| = \eta \cdot \text{işaret}(g)$: **adım boyu gradyan büyüklüğünden bağımsız**, her parametre yaklaşık η kadar hareket eder. Bu, kötü koşullanmış yüzeylerde (Örnek 3.3) Adam'ın neden x ve y eksenlerinde benzer hızda ilerlediğini açıklar.

Ağırlık sönümü: L2 ve AdamW

Ağırlıkların büyümesini engellemek (aşırı öğrenmeyi azaltmak) için her adımda onları biraz sıfıra doğru çekeriz. Klasik yöntem kayba $\lambda/2 \cdot |\theta|^2$ eklemektir (L2 düzenlemesi); gradyana $\lambda\theta$ eklenir. SGD'de bu "ağırlık sönümü" ile aynıdır. Ama Adam'da gradyan $\sqrt{\hat{s}}$ ile bölündüğü için bu ek terim de normalize olur ve istenmeyen sonuçlar doğurur. **AdamW** sönümü gradyandan ayırır: $\theta \leftarrow \theta - \eta\, (\hat{m}/(\sqrt{\hat{s}}+\varepsilon) + \lambda\theta)$.

Örnek 3.5 — L2 düzenlemesi ile AdamW sönümü

PROBLEM

Gradyanı (kayıptan gelen) sıfır olan bir parametre w = 1 düşün; yalnızca sönüm etkilesin. η = 0.1, λ = 0.1 ile 10 adımda iki yöntemi karşılaştır.

ÇÖZÜM (FORTRAN)

```
program adamw
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: w1, w2, m, s, mh, sh, g, lr, wd, b1, b2, eps
  integer :: t
  lr = 0.1_dp; wd = 0.1_dp; b1 = 0.9_dp; b2 = 0.999_dp; eps = 1.0e-8_dp
  w1 = 1.0_dp; w2 = 1.0_dp
  m = 0.0_dp; s = 0.0_dp
  print '(A)', ' adim   Adam+L2(w)   AdamW(w)'
  do t = 1, 10
    ! Adam + L2: sonum gradyanin icine giriyor, sonra normalize ediliyor
    g = wd * w1
    m = b1*m + (1.0_dp - b1)*g
    s = b2*s + (1.0_dp - b2)*g**2
    mh = m / (1.0_dp - b1**t); sh = s / (1.0_dp - b2**t)
    w1 = w1 - lr * mh / (sqrt(sh) + eps)
    ! AdamW: kayip gradyani 0 oldugundan adim 0; sonum dogrudan agirliga
    w2 = w2 - lr * wd * w2
    print '(I5,2F13.6)', t, w1, w2
  end do
end program adamw
```

ÇIKTI

```
 adim   Adam+L2(w)   AdamW(w)
    1     0.900000     0.990000
    2     0.800412     0.980100
    3     0.701586     0.970299
    4     0.603939     0.960596
    5     0.507964     0.950990
    6     0.414237     0.941480
    7     0.323421     0.932065
    8     0.236264     0.922745
    9     0.153585     0.913517
   10     0.076249     0.904382
```

PYTORCH KARŞILIĞI

```
import torch
w1 = torch.tensor(1.0, dtype=torch.float64, requires_grad=True)   # Adam + L2 (weight_decay gradyana
      eklenir)
w2 = torch.tensor(1.0, dtype=torch.float64, requires_grad=True)   # AdamW (ayrik sonum)
o1 = torch.optim.Adam([w1], lr=0.1, weight_decay=0.1)
o2 = torch.optim.AdamW([w2], lr=0.1, weight_decay=0.1)
print(" adim   Adam+L2(w)   AdamW(w)")
for t in range(1, 11):
    w1.grad = torch.zeros((), dtype=torch.float64)    # kayip gradyani 0: yalniz sonum etkili
    w2.grad = torch.zeros((), dtype=torch.float64)
    o1.step(); o2.step()
    print(f"{t:5d}{w1.item():13.6f}{w2.item():13.6f}")
```

PYTORCH ÇIKTISI

```
 adim   Adam+L2(w)   AdamW(w)
    1     0.900000     0.990000
    2     0.800412     0.980100
    3     0.701586     0.970299
    4     0.603939     0.960596
    5     0.507964     0.950990
    6     0.414237     0.941480
    7     0.323421     0.932065
    8     0.236264     0.922745
    9     0.153585     0.913517
   10     0.076249     0.904382
```

NASIL ÇALIŞIYOR?

AdamW'da her adımda ağırlık $(1 - \eta\lambda) = 0.99$ ile çarpılır: 10 adımda $0.99^{10} \approx 0.904$. Yumuşak, ağırlığın büyüklüğüyle orantılı bir çekimdir. Adam+L2'de ise sönüm gradyanın içine girip $\sqrt{\hat{s}}$ ile normalize olduğundan adım $\approx \eta \cdot$işaret olur: ağırlık küçük de olsa adım başına yaklaşık 0.1 azalır ve çok hızlı sıfıra (hatta sıfırın ötesine) çarpılır. Modern LLM eğitimi bu yüzden AdamW kullanır (tipik $\lambda \approx 0.1$; yalnızca matris ağırlıklarına uygulanır, bias ve norm parametrelerine uygulanmaz).

BÖLÜM 4

4. Transformer Mimarisi

Modern LLM'lerin çekirdeği. GPT benzeri bir modelin ileri geçişini parça parça, her parçayı küçük bir örnekle kuruyoruz.

Bu bölümde

- Token embedding ve konum kodlaması (positional encoding)
- Self-attention: Query, Key, Value ve causal mask
- Multi-head attention
- Feed-forward katmanı, LayerNorm/RMSNorm ve residual bağlantılar
- Logits, softmax, çapraz entropi ve sonraki token tahmini
- Parçaları birleştirip minik bir GPT'nin ileri geçişini yazmak

Hedef: GPT benzeri bir Transformer'ı sıfırdan kurabilmek.

Büyük resim

GPT tipi bir model şu boru hattından ibarettir:

token kimlikleri → embedding + konum → [N × Transformer bloğu] → son norm → logits → softmax → sonraki token olasılığı

Her **blok** iki alt katmandan oluşur ve ikisinde de bir **residual (atlama) bağlantısı** vardır: (1) causal multi-head self-attention, (2) pozisyon bazlı feed-forward ağ. Modern modeller her alt katmandan *önce* normalizasyon uygular (pre-norm):

$$x \leftarrow x + \text{Attention}(\text{Norm}(x))$$

$$x \leftarrow x + \text{FFN}(\text{Norm}(x))$$

Tüm tensörleri **(T × d)** matrisleri olarak tutacağız: T dizideki token sayısı, d model boyutu. Her satır bir token'ın vektörüdür.

Token embedding ve konum kodlaması

Model tamsayı token kimliklerini doğrudan işleyemez. **Embedding tablosu** E (V × d) matrisidir: token i'nin vektörü E'nin i. satırıdır. Tablo öğrenilen parametrelerdir. Ama self-attention sıralamayı bilmez (token'ları bir *küme* gibi görür); bu yüzden her token vektörüne bir **konum vektörü** ekleriz. Orijinal Transformer sinüs/kosinüs kullanır:

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}),\ PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

Düşük indeksli boyutlar hızlı, yüksek indeksli boyutlar yavaş dalgalanır; böylece her konum benzersiz bir "parmak izi" alır. GPT-2 gibi modeller bunun yerine konum vektörlerini de öğrenir.

Örnek 4.1 — Embedding araması ve sinüs konum kodlaması

PROBLEM

V = 5 token'lık, d = 4 boyutlu bir embedding tablosu oluştur. Konum kodlamasını (pos = 0…3) hesapla. Token dizisi [3, 5, 2] için embedding + konum toplamını bul.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program embed_pe
  use rng_mod
  implicit none
  integer, parameter :: V = 5, d = 4, T = 3
  real(dp) :: E(V,d), PE(0:3,d), X(T,d), freq
  integer :: ids(T), pos, i, t1
  call rng_seed(5)
  call randn2(E, 1.0_dp)
  do pos = 0, 3
    do i = 0, d/2 - 1
      freq = 1.0_dp / 10000.0_dp**(2.0_dp*i/d)
      PE(pos, 2*i+1) = sin(pos * freq)        ! 1 tabanli: 2i+1 -> sin
      PE(pos, 2*i+2) = cos(pos * freq)        !            2i+2 -> cos
    end do
  end do
  print '(A)', 'Konum kodlamasi PE(pos, :) :'
  do pos = 0, 3
    print '(A,I1,A,4F9.5)', 'pos=', pos, '  ', PE(pos,:)
  end do
  ids = [3, 5, 2]
  do t1 = 1, T
    X(t1,:) = E(ids(t1),:) + PE(t1-1,:)       ! embedding arama + konum
  end do
  print '(A)', 'Girdi matrisi X = E[ids] + PE (T x d):'
  do t1 = 1, T
    print '(4F10.5)', X(t1,:)
  end do
end program embed_pe
```

ÇIKTI

```
Konum kodlamasi PE(pos, :) :
pos=0    0.00000  1.00000  0.00000  1.00000
pos=1    0.84147  0.54030  0.01000  0.99995
pos=2    0.90930 -0.41615  0.02000  0.99980
pos=3    0.14112 -0.98999  0.03000  0.99955
Girdi matrisi X = E[ids] + PE (T x d):
   0.40218  -0.45303  -0.07957   0.73642
   0.75185   0.53403   0.21182  -0.53502
   2.38881  -0.92117  -0.37552   2.87831
```

PYTORCH KARŞILIĞI

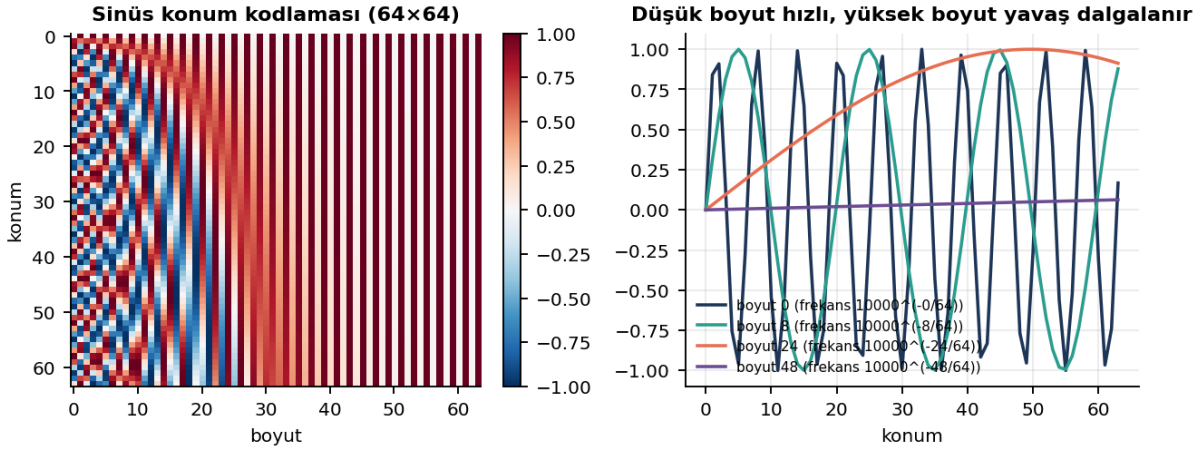
```
import torch, torch.nn as nn
torch.manual_seed(5)
V, d = 5, 4
E = nn.Embedding(V, d)                      # embedding tablosu (V x d)
pos = torch.arange(4).unsqueeze(1).double()
i = torch.arange(d // 2).double()
freq = 1.0 / 10000.0 ** (2 * i / d)
PE = torch.zeros(4, d, dtype=torch.float64)
PE[:, 0::2] = torch.sin(pos * freq)
PE[:, 1::2] = torch.cos(pos * freq)
print("Konum kodlamasi PE(pos, :) :")
for p in range(4):
    print(f"pos={p}  ", " ".join(f"{v:9.5f}" for v in PE[p]))
ids = torch.tensor([2, 4, 1])               # 0 tabanli kimlikler
X = E(ids) + PE[:3].float()                 # embedding arama + konum
print("Girdi matrisi X = E[ids] + PE (T x d):")
print(X.detach())
```

PYTORCH ÇIKTISI

```
Konum kodlamasi PE(pos, :) :
pos=0     0.00000   1.00000   0.00000   1.00000
pos=1     0.84147   0.54030   0.01000   0.99995
pos=2     0.90930  -0.41615   0.02000   0.99980
pos=3     0.14112  -0.98999   0.03000   0.99955
Girdi matrisi X = E[ids] + PE (T x d):
tensor([[ 2.6146,  0.5660,  0.3523,  0.9354],
        [ 1.5862,  0.5720, -1.1624, -0.5070],
        [ 0.7796, -1.0179,  0.1650,  0.8500]])
```

NASIL ÇALIŞIYOR?

Embedding aramak, E matrisinden satır seçmektir (`E(ids(t),:)`); bir one-hot vektörü ile E'nin çarpımına eşdeğerdir ama çok daha ucuzdur. Konum kodlamasında pos = 0 için sin 0 = 0 ve cos 0 = 1 olduğundan satır [0, 1, 0, 1] çıkar. pos = 1'de ilk iki boyut [sin 1, cos 1] = [0.8415, 0.5403], sonraki ikisi çok yavaş dalgalandığı için [sin 0.01, cos 0.01] = [0.0100, 0.9999] olur. Toplama işlemi (X = E + PE) iki bilgiyi aynı vektörde birleştirir: "hangi token" ve "hangi konumda".



Şekil 4.1 — Solda: 64 konum × 64 boyutlu sinüs konum kodlaması ısı haritası; her satır benzersiz bir parmak izi. Sağda: farklı boyutların dalga boyları: düşük boyutlar hızlı, yüksek boyutlar yavaş.

Self-attention: Query, Key, Value

Attention, her token'ın diğer token'lara bakıp ilgili olanlardan bilgi toplamasıdır. Her token üç vektör üretir:

- **Query (Q)**: "Ben ne arıyorum?"
- **Key (K)**: "Bende ne var?" (aranabilirlik etiketi)
- **Value (V)**: "Seçilirsem ne bilgi veririm?"

Basit matematik

Token i, token j'ye ne kadar dikkat eder? $Q_i \cdot K_j$ iç çarpımıyla ölçülür (benzer vektörler büyük skor verir). Skorlar softmax ile ağırlıklara çevrilir ve Value'ların ağırlıklı ortalaması alınır:

$$\text{Attention}(Q,K,V) = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

$\sqrt{d_k}$ ile bölme neden? İki d_k boyutlu rastgele vektörün iç çarpımının standart sapması $\sqrt{d_k}$ olur; bölmezsek skorlar büyür, softmax doyar (bir token ≈ 1, diğerleri ≈ 0) ve gradyanlar kaybolur.

Örnek 4.2 — Tek bir query için attention (elle hesap)

PROBLEM

q = [1, 0], anahtarlar K = [[1,0],[0,1],[1,1]], değerler V = [[1,0],[0,1],[1,1]], d_k = 2. Skorları, softmax ağırlıklarını ve çıktıyı hesapla.

ÇÖZÜM (FORTRAN)

```
program attn_tek
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: q(2), K(3,2), V(3,2), s(3), w(3), o(2)
  integer :: j
  q = [1.0_dp, 0.0_dp]
  K = reshape([1,0, 0,1, 1,1] * 1.0_dp, [3,2], order=[2,1])
  V = K
  do j = 1, 3
    s(j) = dot_product(q, K(j,:)) / sqrt(2.0_dp)      ! olcekli skor
  end do
  w = exp(s - maxval(s)); w = w / sum(w)               ! softmax
  o = matmul(w, V)                                    ! agirlikli ortalama
  print '(A,3F9.4)', 'skorlar   = ', s
  print '(A,3F9.4)', 'agirliklar= ', w
  print '(A,2F9.4)', 'cikti     = ', o
end program attn_tek
```

ÇIKTI

```
skorlar   =    0.7071   0.0000   0.7071
agirliklar=    0.4011   0.1978   0.4011
cikti     =    0.8022   0.5989
```

PYTORCH KARŞILIĞI

```
import torch, math
import torch.nn.functional as F
q = torch.tensor([1., 0.])
K = torch.tensor([[1., 0], [0, 1], [1, 1]]); V = K.clone()
s = K @ q / math.sqrt(2)                    # olcekli skorlar
w = torch.softmax(s, 0); o = w @ V
print("skorlar    =", [round(v, 4) for v in s.tolist()])
print("agirliklar =", [round(v, 4) for v in w.tolist()])
print("cikti      =", [round(v, 4) for v in o.tolist()])
ref = F.scaled_dot_product_attention(q[None, None, :], K[None], V[None])   # hazir fonksiyon
print("F.scaled_dot_product_attention =", [round(v, 4) for v in ref.flatten().tolist()])
```

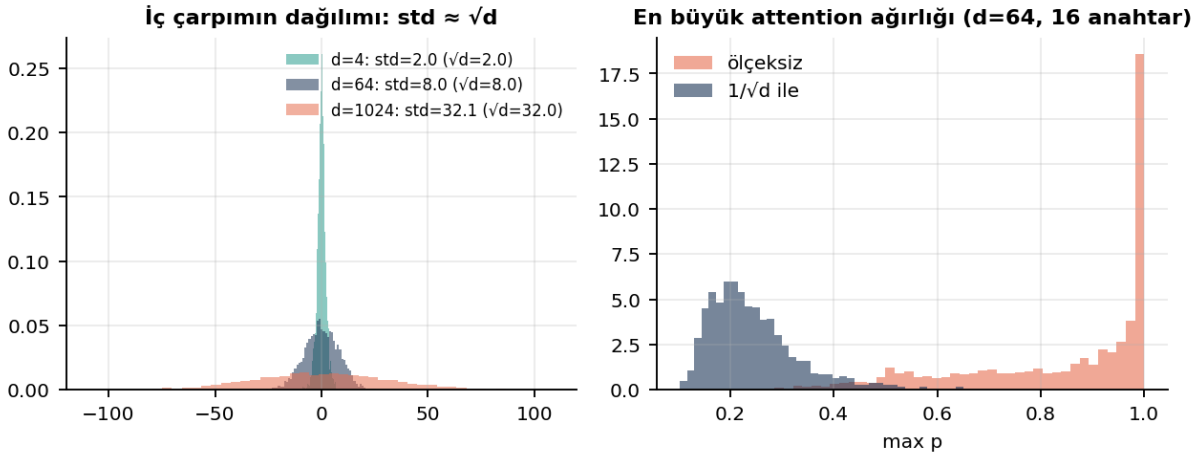
PYTORCH ÇIKTISI

```
skorlar    = [0.7071, 0.0, 0.7071]
agirliklar = [0.4011, 0.1978, 0.4011]
cikti      = [0.8022, 0.5989]
F.scaled_dot_product_attention = [0.8022, 0.5989]
```

NASIL ÇALIŞIYOR?

Skorlar: $q \cdot k_1 = 1$, $q \cdot k_2 = 0$, $q \cdot k_3 = 1$; $\sqrt{2}$'ye bölünce [0.7071, 0, 0.7071]. Softmax: $e^{0.7071} = 2.028$, $e^0 = 1$; toplam 5.056; ağırlıklar [0.401, 0.198, 0.401]. Çıktı = 0.401·[1,0] + 0.198·[0,1] + 0.401·[1,1] = [0.802, 0.599]. Yani query'ye benzeyen 1. ve 3. anahtarların değerleri baskın olarak karışır. Attention "yumuşak bir sözlük aramasıdır": sert bir eşleşme yerine ağırlıklı bir karışım döndürür.

Türetme: attention neden $\sqrt{d_k}$ ile bölünür?
q ve k, bağımsız, sıfır ortalamalı, birim varyanslı bileşenlere sahip d_k boyutlu vektörler olsun. İç çarpım $q \cdot k = \Sigma_i q_i k_i$:
$E[q \cdot k] = 0$, $\mathrm{Var}(q \cdot k) = \Sigma_i \mathrm{Var}(q_i k_i) = \Sigma_i E[q_i^2] E[k_i^2] = d_k$
Yani skorların standart sapması $\sqrt{d_k}$. $d_k = 64$ için skorlar ± 8 civarındadır; softmax $e^8 / e^0 \approx 3000$ oranında bir ağırlık farkı üretir ve neredeyse one-hot olur. Doymuş softmax'ın Jacobian'ı $p_i(\delta_{ij} - p_j) \approx 0$ olduğundan **gradyan kaybolur**. $\sqrt{d_k}$ ile bölmek $\mathrm{Var}(\text{skor}) = 1$ yapar ve softmax'ı hassas bölgede tutar (aşağıdaki şekil).



Şekil 4.2 — Solda: iç çarpım dağılımının standart sapması √d ile büyür. Sağda: ölçeksiz skorlarla (d=64) en büyük attention ağırlığı neredeyse hep ≈ 1 (doymuş); 1/√d ile dağılım yumuşak kalır.

Causal mask: geleceğe bakmak yasak

Dil modelinde token t, yalnızca kendisinden önceki (ve kendisi dahil) token'ları görebilmelidir; yoksa cevabı doğrudan kopyalar. Bunu softmax'tan **önce** skor matrisinin üst üçgenine $-\infty$ (pratikte -10^{30}) koyarak sağlarız: $e^{-\infty} = 0$ olduğundan o ağırlıklar sıfırlanır.

Örnek 4.3 — Causal self-attention matrisi

PROBLEM

T = 4 token'lık, d = 2 boyutlu bir X için Q = K = V = X alarak causal attention ağırlık matrisini ve çıktıyı hesapla. Ağırlık matrisinin alt üçgen olduğunu ve her satırın toplamının 1 olduğunu göster.

Bu örnek şu modülleri kullanır: `prec_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module attn_mod
  use prec_mod
  implicit none
contains
  subroutine softmax_rows(S)               ! her satiri ayri softmax'la
    real(dp), intent(inout) :: S(:,:)
    integer :: i
    do i = 1, size(S,1)
      S(i,:) = exp(S(i,:) - maxval(S(i,:)))
      S(i,:) = S(i,:) / sum(S(i,:))
    end do
  end subroutine softmax_rows

  subroutine causal_attention(Q, K, V, A, O)
    real(dp), intent(in)  :: Q(:,:), K(:,:), V(:,:)    ! (T,dk), (T,dk), (T,dv)
    real(dp), intent(out) :: A(:,:), O(:,:)            ! (T,T), (T,dv)
    integer :: T, i, j
    T = size(Q,1)
    A = matmul(Q, transpose(K)) / sqrt(real(size(Q,2), dp))
    do i = 1, T
```

```
      do j = i+1, T
        A(i,j) = -1.0e30_dp                              ! gelecegi maskele
      end do
    end do
    call softmax_rows(A)
    O = matmul(A, V)
  end subroutine causal_attention
end module attn_mod

program causal
  use attn_mod
  implicit none
  integer, parameter :: T = 4, d = 2
  real(dp) :: X(T,d), A(T,T), O(T,d)
  integer :: i
  X = reshape([1.0_dp, 0.0_dp,  0.0_dp, 1.0_dp,  1.0_dp, 1.0_dp,  0.5_dp, -0.5_dp], &
              [T,d], order=[2,1])
  call causal_attention(X, X, X, A, O)
  print '(A)', 'Attention agirliklari A (satir = sorgu token, sutun = bakilan token):'
  do i = 1, T
    print '(4F9.4,A,F8.4)', A(i,:), '   satir toplami =', sum(A(i,:))
  end do
  print '(A)', 'Cikti O = A V :'
  do i = 1, T
    print '(2F10.4)', O(i,:)
  end do
end program causal
```

ÇIKTI

```
Attention agirliklari A (satir = sorgu token, sutun = bakilan token):
   1.0000   0.0000   0.0000   0.0000   satir toplami =  1.0000
   0.3302   0.6698   0.0000   0.0000   satir toplami =  1.0000
   0.2483   0.2483   0.5035   0.0000   satir toplami =  1.0000
   0.3130   0.1543   0.2198   0.3130   satir toplami =  1.0000
Cikti O = A V :
    1.0000    0.0000
    0.3302    0.6698
    0.7517    0.7517
    0.6892    0.2176
```

PYTORCH KARŞILIĞI

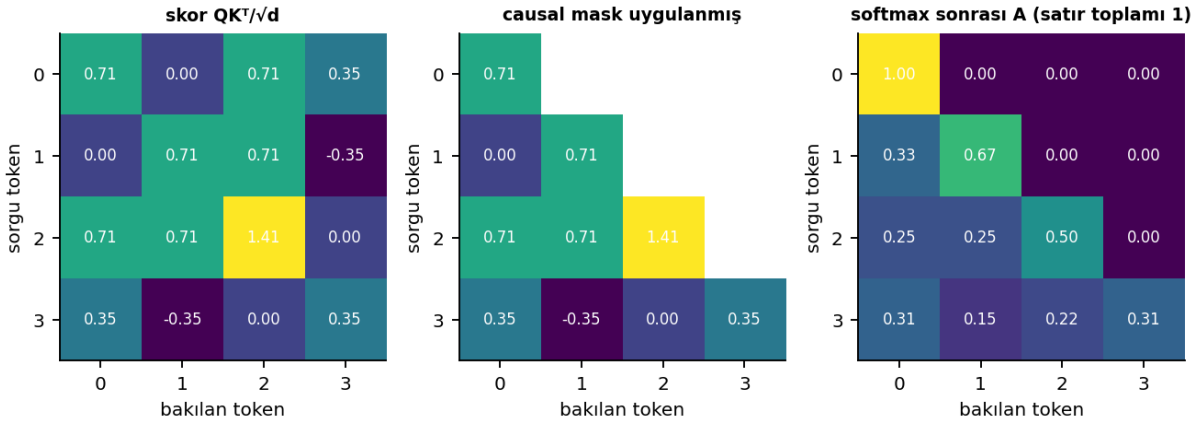
```
import torch, math
import torch.nn.functional as F
X = torch.tensor([[1., 0], [0, 1], [1, 1], [0.5, -0.5]])
T_, d = X.shape
S = X @ X.T / math.sqrt(d)
mask = torch.triu(torch.ones(T_, T_, dtype=torch.bool), diagonal=1)   # gelecegi maskele
A = torch.softmax(S.masked_fill(mask, float('-inf')), dim=-1)
O = A @ X
print("Attention agirliklari A:")
for i in range(T_):
    print(" ".join(f"{v:9.4f}" for v in A[i]), f"   satir toplami = {A[i].sum():.4f}")
print("Cikti O = A V:"); print(O)
ref = F.scaled_dot_product_attention(X[None], X[None], X[None], is_causal=True)[0]
print("SDPA(is_causal=True) ile fark:", (ref - O).abs().max().item())
```

PYTORCH ÇIKTISI

```
Attention agirliklari A:
   1.0000    0.0000    0.0000    0.0000    satir toplami = 1.0000
   0.3302    0.6698    0.0000    0.0000    satir toplami = 1.0000
   0.2483    0.2483    0.5035    0.0000    satir toplami = 1.0000
   0.3130    0.1543    0.2198    0.3130    satir toplami = 1.0000
Cikti O = A V:
tensor([[1.0000, 0.0000],
        [0.3302, 0.6698],
        [0.7517, 0.7517],
        [0.6892, 0.2176]])
SDPA(is_causal=True) ile fark: 0.0
```

NASIL ÇALIŞIYOR?

İlk satır her zaman [1, 0, 0, 0]'dır: ilk token yalnızca kendini görebilir. Her satırın toplamı 1, ve üst üçgen tam sıfırdır. İkinci token ilk iki token arasında, üçüncü ilk üç token arasında dikkatini paylaştırır. Bu `attn_mod` modülü sonraki örneklerde yeniden kullanılacak; `-1e30` hilesinin softmax ile birleşince tam sıfıra yol açtığına dikkat et.



Şekil 4.3 — Causal attention adım adım (T=4): ham skorlar $QK^T/\sqrt{d}$, causal mask sonrası alt üçgen, softmax sonrası satır toplamı 1 olan dikkat ağırlıkları.

Öğrenilebilir Q, K, V projeksiyonları

Gerçek bir katmanda Q, K ve V doğrudan X değildir; her biri X'in öğrenilen bir matrisle çarpımıdır: $Q = X W_q$, $K = X W_k$, $V = X W_v$. Böylece model "arama" ve "değer" uzaylarını görev için kendisi şekillendirir.

Örnek 4.4 — Projeksiyonlu attention ve nedensellik (causality) testi

PROBLEM

$T = 6$, $d = 8$ için rastgele W_q, W_k, W_v ile attention hesapla. Son token'ı değiştirdiğinde önceki token'ların çıktılarının *hiç* değişmediğini, yalnızca son satırın değiştiğini göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A),* `attn_mod` *(Örnek 4.3). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program proj_attn
  use rng_mod
  use attn_mod
  implicit none
  integer, parameter :: T = 6, d = 8
  real(dp) :: X(T,d), X2(T,d), Wq(d,d), Wk(d,d), Wv(d,d)
  real(dp) :: A(T,T), O1(T,d), O2(T,d)
  call rng_seed(21)
  call randn2(X, 1.0_dp)
  call randn2(Wq, 0.5_dp); call randn2(Wk, 0.5_dp); call randn2(Wv, 0.5_dp)
  call causal_attention(matmul(X,Wq), matmul(X,Wk), matmul(X,Wv), A, O1)
  X2 = X
  X2(T,:) = X2(T,:) + 5.0_dp                ! SON token'i degistir
  call causal_attention(matmul(X2,Wq), matmul(X2,Wk), matmul(X2,Wv), A, O2)
  print '(A,ES10.2)', 'ilk 5 satirin degisimi (max)  = ', maxval(abs(O1(1:T-1,:) - O2(1:T-1,:)))
  print '(A,ES10.2)', 'son satirin degisimi (max)    = ', maxval(abs(O1(T,:) - O2(T,:)))
end program proj_attn
```

ÇIKTI

```
ilk 5 satirin degisimi (max)  =   0.00E+00
son satirin degisimi (max)    =   5.62E-01
```

PYTORCH KARŞILIĞI

```
import torch, math
import torch.nn as nn
import torch.nn.functional as F
torch.manual_seed(21)
T_, d = 6, 8
Wq, Wk, Wv = (nn.Linear(d, d, bias=False) for _ in range(3))
def attn(X):
    return F.scaled_dot_product_attention(Wq(X), Wk(X), Wv(X), is_causal=True)
X = torch.randn(T_, d)
X2 = X.clone(); X2[-1] += 5.0              # SON token'i degistir
O1, O2 = attn(X), attn(X2)
print("ilk 5 satirin degisimi (max) =", (O1[:-1] - O2[:-1]).abs().max().item())
print("son satirin degisimi (max)   =", (O1[-1] - O2[-1]).abs().max().item())
```

PYTORCH ÇIKTISI

```
ilk 5 satirin degisimi (max) = 0.0
son satirin degisimi (max)   = 3.291703462600708
```

NASIL ÇALIŞIYOR?

Bu test, causal mask'ın doğru çalıştığının en basit kanıtıdır: geleceğe bakan bir model, son token değişince önceki çıktıları da değiştirirdi. Burada ilk 5 satırın değişimi **tam sıfır**, son satırınki ise belirgin biçimde sıfırdan büyüktür. Aynı özellik sayesinde metin üretirken önceki token'ların K ve V değerlerini saklayıp yeniden kullanabiliriz (KV cache, Bölüm 10).

Multi-head attention

Tek bir attention, her token için tek bir "bakış" üretir. **Multi-head** yaklaşımı d boyutlu uzayı h parçaya ($d_h = d/h$) böler ve her parça kendi attention'ını çalıştırır; sonuçlar yan yana birleştirilir ve bir çıkış matrisiyle (W_o) karıştırılır. Farklı head'ler farklı ilişkileri öğrenebilir (sözdizimi, yakın komşu, uzak referans gibi). Parametre sayısı değişmez: $4d^2$ (W_q, W_k, W_v, W_o).

Örnek 4.5 — Multi-head attention modülü

PROBLEM

d = 8, h = 2 head için çok başlı attention yaz; çıktı boyutunu, parametre sayısını ve nedensellik özelliğini kontrol et.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A),* `attn_mod` *(Örnek 4.3). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module mha_mod
  use attn_mod
  implicit none
contains
  subroutine mha(X, Wq, Wk, Wv, Wo, nh, Y)
    real(dp), intent(in)  :: X(:,:), Wq(:,:), Wk(:,:), Wv(:,:), Wo(:,:)
    integer, intent(in)   :: nh
    real(dp), intent(out) :: Y(:,:)
    integer :: T, d, dh, h, c1, c2
    real(dp), allocatable :: Q(:,:), K(:,:), V(:,:), A(:,:), Oh(:,:), O(:,:)
    T = size(X,1); d = size(X,2); dh = d / nh
    allocate(Q(T,d), K(T,d), V(T,d), A(T,T), Oh(T,dh), O(T,d))
    Q = matmul(X, Wq); K = matmul(X, Wk); V = matmul(X, Wv)
    do h = 1, nh                                  ! her head icin sutun dilimi
      c1 = (h-1)*dh + 1; c2 = h*dh
      call causal_attention(Q(:,c1:c2), K(:,c1:c2), V(:,c1:c2), A, Oh)
      O(:,c1:c2) = Oh                             ! head ciktilarini yan yana koy
    end do
    Y = matmul(O, Wo)
  end subroutine mha
end module mha_mod
```

```
program mha_test
  use rng_mod
  use mha_mod
  implicit none
  integer, parameter :: T = 5, d = 8, nh = 2
  real(dp) :: X(T,d), X2(T,d), Wq(d,d), Wk(d,d), Wv(d,d), Wo(d,d), Y1(T,d), Y2(T,d)
  call rng_seed(8)
  call randn2(X, 1.0_dp)
  call randn2(Wq, 0.4_dp); call randn2(Wk, 0.4_dp)
  call randn2(Wv, 0.4_dp); call randn2(Wo, 0.4_dp)
  call mha(X, Wq, Wk, Wv, Wo, nh, Y1)
  X2 = X; X2(T,:) = 0.0_dp
  call mha(X2, Wq, Wk, Wv, Wo, nh, Y2)
  print '(A,I0,A,I0)', 'cikti boyutu: ', size(Y1,1), ' x ', size(Y1,2)
  print '(A,I0)', 'parametre sayisi 4*d^2 = ', 4*d*d
  print '(A,ES10.2)', 'onceki satirlarin degisimi = ', maxval(abs(Y1(1:T-1,:) - Y2(1:T-1,:)))
end program mha_test
```

ÇIKTI

```
cikti boyutu: 5 x 8
parametre sayisi 4*d^2 = 256
onceki satirlarin degisimi =   0.00E+00
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
torch.manual_seed(8)
T_, d, nh = 5, 8, 2
mha = nn.MultiheadAttention(d, nh, batch_first=True)
causal = torch.triu(torch.ones(T_, T_, dtype=torch.bool), diagonal=1)
def run(X):
    return mha(X[None], X[None], X[None], attn_mask=causal, need_weights=False)[0][0]
X = torch.randn(T_, d)
X2 = X.clone(); X2[-1] = 0.0
Y1, Y2 = run(X), run(X2)
print("cikti boyutu:", tuple(Y1.shape))
print("parametre sayisi (agirlik) 4*d^2 =", 4 * d * d, "  | toplam (bias dahil) =", sum(p.numel() for p
      in mha.parameters()))
print("onceki satirlarin degisimi =", (Y1[:-1] - Y2[:-1]).abs().max().item())
```

PYTORCH ÇIKTISI

```
cikti boyutu: (5, 8)
parametre sayisi (agirlik) 4*d^2 = 256   | toplam (bias dahil) = 288
onceki satirlarin degisimi = 0.0
```

NASIL ÇALIŞIYOR?

Her head, Q, K, V matrislerinin kendi sütun dilimini (`c1:c2`) kullanır. Dilimleri doğrudan alt programa aktarabiliyoruz çünkü `assumed-shape` dizi argümanları bunu destekler. Çıkış boyutu girişle aynıdır (T × d), bu yüzden bloklar üst üste dizilebilir. Nedensellik özelliği çok başlı yapıda da korunur.

Normalizasyon: LayerNorm ve RMSNorm

Basit matematik

LayerNorm (bir token vektörü için): $\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$, $y = \gamma \odot \hat{x} + \beta$.

RMSNorm ortalamayı çıkarmayı bırakır, yalnızca ölçeği düzeltir: $y = \gamma \odot x / \sqrt{(\text{mean}(x^2) + \varepsilon)}$. Daha ucuz ve pratikte aynı kalitededir (LLaMA, Mistral, vb. kullanır).

Örnek: x = [1,2,3,4]. μ = 2.5, σ² = 1.25 → LayerNorm ≈ [−1.342, −0.447, 0.447, 1.342]. mean(x²) = 7.5 → RMSNorm ≈ x / 2.7386 = [0.365, 0.730, 1.095, 1.461].

Örnek 4.6 — LayerNorm ve RMSNorm modülü

PROBLEM

İki normalizasyonu satır bazlı (her token için) yaz. x = [1,2,3,4] üzerinde dene ve x'i 10 ile çarpınca çıktının nasıl davrandığına bak.

Bu örnek şu modülleri kullanır: `prec_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module norm_mod
  use prec_mod
  implicit none
contains
  subroutine layernorm_rows(X, gamma, beta, Y)
    real(dp), intent(in)  :: X(:,:), gamma(:), beta(:)
    real(dp), intent(out) :: Y(:,:)
    real(dp) :: mu, v
    integer :: i
    do i = 1, size(X,1)
      mu = sum(X(i,:)) / size(X,2)
      v  = sum((X(i,:) - mu)**2) / size(X,2)
      Y(i,:) = gamma * (X(i,:) - mu) / sqrt(v + 1.0e-5_dp) + beta
    end do
  end subroutine layernorm_rows

  subroutine rmsnorm_rows(X, gamma, Y)
    real(dp), intent(in)  :: X(:,:), gamma(:)
    real(dp), intent(out) :: Y(:,:)
    integer :: i
    do i = 1, size(X,1)
      Y(i,:) = gamma * X(i,:) / sqrt(sum(X(i,:)**2) / size(X,2) + 1.0e-5_dp)
    end do
  end subroutine rmsnorm_rows
end module norm_mod

program norm_test
  use norm_mod
  implicit none
  real(dp) :: X(1,4), Y(1,4), g(4), b(4)
  X(1,:) = [1.0_dp, 2.0_dp, 3.0_dp, 4.0_dp]
  g = 1.0_dp; b = 0.0_dp
  call layernorm_rows(X, g, b, Y);        print '(A,4F9.4)', 'LayerNorm(x)    = ', Y(1,:)
  call rmsnorm_rows(X, g, Y);             print '(A,4F9.4)', 'RMSNorm(x)      = ', Y(1,:)
  call layernorm_rows(10*X, g, b, Y);     print '(A,4F9.4)', 'LayerNorm(10x)  = ', Y(1,:)
  call rmsnorm_rows(10*X, g, Y);          print '(A,4F9.4)', 'RMSNorm(10x)    = ', Y(1,:)
  call layernorm_rows(X + 100, g, b, Y);  print '(A,4F9.4)', 'LayerNorm(x+100)= ', Y(1,:)
end program norm_test
```

ÇIKTI

```
LayerNorm(x)    =   -1.3416  -0.4472   0.4472   1.3416
RMSNorm(x)      =    0.3651   0.7303   1.0954   1.4606
LayerNorm(10x)  =   -1.3416  -0.4472   0.4472   1.3416
RMSNorm(10x)    =    0.3651   0.7303   1.0954   1.4606
LayerNorm(x+100)=   -1.3416  -0.4472   0.4472   1.3416
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
x = torch.tensor([[1., 2., 3., 4.]])
ln = nn.LayerNorm(4, eps=1e-5, elementwise_affine=False)
rms = nn.RMSNorm(4, eps=1e-5, elementwise_affine=False)
f = lambda t: " ".join(f"{v:9.4f}" for v in t.flatten())
print("LayerNorm(x)    =", f(ln(x)))
print("RMSNorm(x)      =", f(rms(x)))
print("LayerNorm(10x)  =", f(ln(10 * x)))
print("RMSNorm(10x)    =", f(rms(10 * x)))
print("LayerNorm(x+100)=", f(ln(x + 100)))
```

PYTORCH ÇIKTISI

```
LayerNorm(x)    =   -1.3416   -0.4472    0.4472    1.3416
RMSNorm(x)      =    0.3651    0.7303    1.0954    1.4606
LayerNorm(10x)  =   -1.3416   -0.4472    0.4472    1.3416
RMSNorm(10x)    =    0.3651    0.7303    1.0954    1.4606
LayerNorm(x+100)=   -1.3416   -0.4472    0.4472    1.3416
```

NASIL ÇALIŞIYOR?

Sonuçlar el hesabıyla örtüşür. Her iki normalizasyon da **ölçek değişmezdir**: girdiyi 10 ile çarpmak çıktıyı (neredeyse, ε yüzünden çok az fark) değiştirmez. LayerNorm ayrıca **kayma değişmezdir** (x+100 aynı sonucu verir); RMSNorm ortalamayı çıkarmadığı için kaymaya duyarlıdır. Normalizasyonun amacı budur: önceki katmanın çıktı ölçeği ne olursa olsun, sonraki katman sabit ölçekte girdi görür.

Feed-forward katmanı ve residual bağlantılar

Attention token'lar *arasında* bilgi taşır; **feed-forward ağ (FFN)** ise her token'ı *kendi başına* işler: d → 4d → d boyutlarında iki doğrusal katman ve aralarında GELU. Modelin parametrelerinin çoğu (≈ 2/3'ü) burada yaşar.

Basit matematik

FFN(x) = GELU(x W_1 + b_1) W_2 + b_2, W_1: (d × 4d), W_2: (4d × d)

Residual bağlantı: y = x + f(x). Türev: $\partial y/\partial x = 1 + \partial f/\partial x$. "1" terimi sayesinde gradyan, derin ağda bile f ne yaparsa yapsın başa doğru sönmeden akabilir. Residual yapıda ağ başlangıçta neredeyse bir "özdeşlik fonksiyonu"dur (f küçük başlatılırsa); eğitim yalnızca gerekli düzeltmeleri ekler.

Örnek 4.7 — FFN ve residual bağlantı

PROBLEM

d = 4 için FFN'i (4d = 16 gizli boyut) yaz. Küçük ağırlıklarla başlatınca x + FFN(x)'in x'e çok yakın olduğunu göster; parametre sayısını ($8d^2 + 5d$) doğrula.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module ffn_mod
  use prec_mod
  implicit none
contains
  elemental function gelu(x) result(y)
    real(dp), intent(in) :: x
    real(dp) :: y
    y = 0.5_dp * x * (1.0_dp + tanh(0.7978845608028654_dp * (x + 0.044715_dp * x**3)))
  end function gelu

  subroutine ffn(X, W1, b1, W2, b2, Y)
    real(dp), intent(in)  :: X(:,:), W1(:,:), b1(:), W2(:,:), b2(:)
    real(dp), intent(out) :: Y(:,:)
    real(dp), allocatable :: Hd(:,:)
    integer :: T
    T = size(X,1)
    allocate(Hd(T, size(W1,2)))
    Hd = gelu(matmul(X, W1) + spread(b1, 1, T))
    Y  = matmul(Hd, W2) + spread(b2, 1, T)
  end subroutine ffn
end module ffn_mod

program ffn_test
  use rng_mod
  use ffn_mod
  implicit none
  integer, parameter :: T = 3, d = 4
  real(dp) :: X(T,d), W1(d,4*d), b1(4*d), W2(4*d,d), b2(d), Y(T,d)
```

```
  call rng_seed(2)
  call randn2(X, 1.0_dp)
  call randn2(W1, 0.02_dp); call randn2(W2, 0.02_dp)
  b1 = 0.0_dp; b2 = 0.0_dp
  call ffn(X, W1, b1, W2, b2, Y)
  print '(A,ES10.2)', '|FFN(x)| en buyuk eleman        = ', maxval(abs(Y))
  print '(A,ES10.2)', '|x + FFN(x) - x| en buyuk fark   = ', maxval(abs((X + Y) - X))
  print '(A,I0)', 'parametre sayisi        = ', size(W1) + size(b1) + size(W2) + size(b2)
  print '(A,I0)', '8*d^2 + 5*d             = ', 8*d*d + 5*d
end program ffn_test
```

ÇIKTI

```
|FFN(x)| en buyuk eleman        =   7.31E-03
|x + FFN(x) - x| en buyuk fark   =   7.31E-03
parametre sayisi        = 148
8*d^2 + 5*d             = 148
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
torch.manual_seed(2)
T_, d = 3, 4
ffn = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(approximate='tanh'), nn.Linear(4 * d, d))
for p in ffn.parameters(): nn.init.normal_(p, std=0.02)
for p in [ffn[0].bias, ffn[2].bias]: nn.init.zeros_(p)
X = torch.randn(T_, d)
Y = ffn(X)
print("|FFN(x)| en buyuk eleman       =", f"{Y.abs().max().item():.2e}")
print("|x + FFN(x) - x| en buyuk fark =", f"{((X + Y) - X).abs().max().item():.2e}")
print("parametre sayisi:", sum(p.numel() for p in ffn.parameters()), " | 8d^2+5d =", 8 * d * d + 5 * d)
```

PYTORCH ÇIKTISI

```
|FFN(x)| en buyuk eleman       = 2.92e-03
|x + FFN(x) - x| en buyuk fark = 2.92e-03
parametre sayisi: 148  | 8d^2+5d = 148
```

NASIL ÇALIŞIYOR?

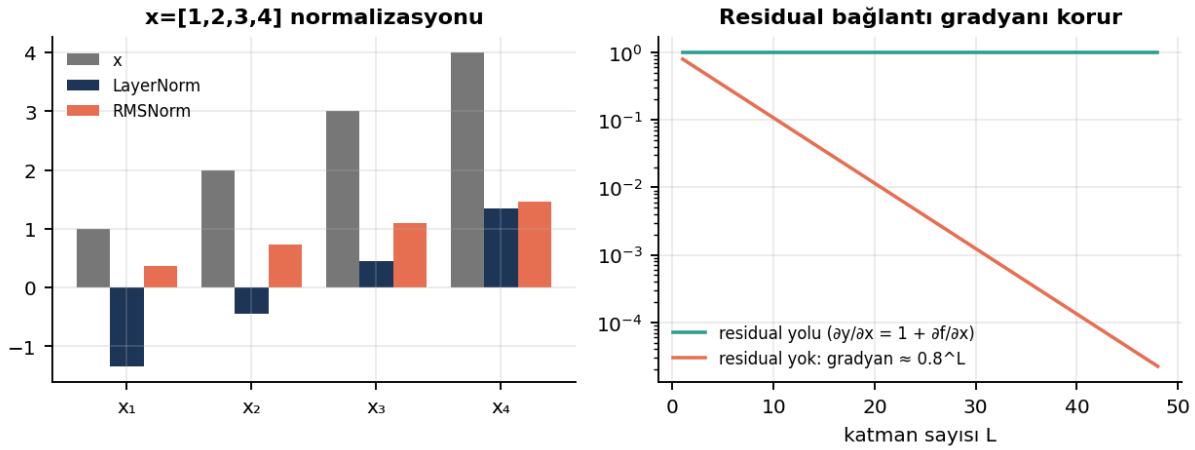
Küçük ağırlıklarla FFN çıktısı yaklaşık 10^{-2} mertebesindedir; residual toplamdan sonra x neredeyse aynen korunur. Bu, derin ağlarda eğitimin kararlı başlaması için temel hiledir. Parametre sayımı: W_1 için $d{\cdot}4d = 64$, b_1 için 16, W_2 için 64, b_2 için $4 \rightarrow 148 = 8{\cdot}16 + 5{\cdot}4$. GPT-2 small'da ($d = 768$) yalnızca FFN katmanı başına $8{\cdot}768^2 \approx 4.7$ milyon parametre vardır.

Türetme: residual bağlantı gradyanı korur

Residual bağlantıyla $y = x + f(x)$. L katman sonra çıktı için gradyan:

$\partial y_L/\partial x_0 = \Pi_{l=1}^{L} (I + J_l) = I + \Sigma_l J_l + \Sigma_{l<m} J_l J_m + \ldots$ ($J_l = \partial f_l/\partial x$)

Açılımda her zaman **özdeşlik terimi I** vardır; J_l'ler ne kadar küçük olursa olsun gradyan başa doğru en az bir doğrudan yoldan sönmeden akar. Residual yoksa $\partial y/\partial x = \Pi J_l$: ortalama $|J| < 1$ ise üstel sönüm (şekildeki 0.8^L eğrisi). Bu, 100+ katmanlı Transformer'ların eğitilebilmesinin ana nedenlerinden biridir. Pre-norm düzeni normalizasyonu residual yolun *dışına* koyar, böylece özdeşlik yolu hiç bozulmaz.



Şekil 4.4 — Solda: x = [1,2,3,4]'ün LayerNorm ve RMSNorm sonrası değerleri. Sağda: residual yolun gradyanı (sabit 1) ile residual olmayan derin zincirin gradyanı ($\approx 0.8^{L}$).

Transformer bloğu

Şimdi parçaları birleştiriyoruz. Pre-norm, RMSNorm'lu bir blok: önce normalize et, attention uygula, residual olarak ekle; sonra tekrar normalize et, FFN uygula, residual ekle.

Örnek 4.8 — Tam bir Transformer bloğu

PROBLEM

attn, norm ve ffn parçalarından bir blok oluştur (tür tanımı + başlatma + ileri geçiş). T = 6, d = 8, 2 head için nedensellik testini bloğun tamamında tekrarla.

Bu örnek şu modülleri kullanır: `mha_mod` *(Örnek 4.5),* `norm_mod` *(Örnek 4.6),* `ffn_mod` *(Örnek 4.7),* `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module blk_mod
  use mha_mod
  use norm_mod
  use ffn_mod
  use rng_mod
  implicit none
  type blk_t
    real(dp), allocatable :: g1(:), g2(:)
    real(dp), allocatable :: Wq(:,:), Wk(:,:), Wv(:,:), Wo(:,:)
    real(dp), allocatable :: W1(:,:), b1(:), W2(:,:), b2(:)
  end type blk_t
contains
  subroutine blk_init(p, d, std)
    type(blk_t), intent(out) :: p
    integer, intent(in) :: d
    real(dp), intent(in) :: std
    allocate(p%g1(d), p%g2(d), p%Wq(d,d), p%Wk(d,d), p%Wv(d,d), p%Wo(d,d))
    allocate(p%W1(d,4*d), p%b1(4*d), p%W2(4*d,d), p%b2(d))
    p%g1 = 1.0_dp; p%g2 = 1.0_dp; p%b1 = 0.0_dp; p%b2 = 0.0_dp
    call randn2(p%Wq, std); call randn2(p%Wk, std)
    call randn2(p%Wv, std); call randn2(p%Wo, std)
    call randn2(p%W1, std); call randn2(p%W2, std)
  end subroutine blk_init

  subroutine blk_forward(p, X, nh, Y)
    type(blk_t), intent(in) :: p
    real(dp), intent(in)  :: X(:,:)
    integer, intent(in)   :: nh
    real(dp), intent(out) :: Y(:,:)
    real(dp), allocatable :: Xn(:,:), Z(:,:)
    allocate(Xn(size(X,1),size(X,2)), Z(size(X,1),size(X,2)))
    call rmsnorm_rows(X, p%g1, Xn)                    ! 1) norm
    call mha(Xn, p%Wq, p%Wk, p%Wv, p%Wo, nh, Z)       !    attention
```

```
    Y = X + Z                                          !    residual
    call rmsnorm_rows(Y, p%g2, Xn)                     ! 2) norm
    call ffn(Xn, p%W1, p%b1, p%W2, p%b2, Z)            !    FFN
    Y = Y + Z                                          !    residual
  end subroutine blk_forward
end module blk_mod

program blok_test
  use blk_mod
  implicit none
  integer, parameter :: T = 6, d = 8, nh = 2
  type(blk_t) :: p
  real(dp) :: X(T,d), X2(T,d), Y1(T,d), Y2(T,d)
  call rng_seed(4)
  call blk_init(p, d, 0.3_dp)
  call randn2(X, 1.0_dp)
  call blk_forward(p, X, nh, Y1)
  X2 = X; X2(T,:) = X2(T,:) - 3.0_dp
  call blk_forward(p, X2, nh, Y2)
  print '(A,ES10.2)', 'onceki satirlarin degisimi = ', maxval(abs(Y1(1:T-1,:) - Y2(1:T-1,:)))
  print '(A,F8.4)',   'giris RMS = ', sqrt(sum(X**2)/size(X))
  print '(A,F8.4)',   'cikis RMS = ', sqrt(sum(Y1**2)/size(Y1))
end program blok_test
```

ÇIKTI

```
onceki satirlarin degisimi =   0.00E+00
giris RMS =   0.8865
cikis RMS =   1.3060
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
import torch.nn.functional as F

class Block(nn.Module):                     # pre-norm, RMSNorm'lu Transformer blogu
    def __init__(self, d, nh):
        super().__init__()
        self.nh = nh
        self.n1, self.n2 = nn.RMSNorm(d, eps=1e-5), nn.RMSNorm(d, eps=1e-5)
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.proj = nn.Linear(d, d, bias=False)
        self.ffn = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(approximate='tanh'), nn.Linear(4 * d, d))
    def forward(self, x):                   # x: (B, T, d)
        B, T, d = x.shape
        q, k, v = self.qkv(self.n1(x)).chunk(3, dim=-1)
        q, k, v = (t.view(B, T, self.nh, d // self.nh).transpose(1, 2) for t in (q, k, v))
        a = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # causal multi-head attention
        x = x + self.proj(a.transpose(1, 2).reshape(B, T, d))         # residual 1
        return x + self.ffn(self.n2(x))                               # residual 2

torch.manual_seed(4)
blk = Block(8, 2)
for p in blk.parameters():
    if p.dim() > 1: nn.init.normal_(p, std=0.3)
X = torch.randn(1, 6, 8)
X2 = X.clone(); X2[0, -1] -= 3.0
Y1, Y2 = blk(X), blk(X2)
print("onceki satirlarin degisimi =", f"{(Y1[0, :-1] - Y2[0, :-1]).abs().max().item():.2e}")
print("giris RMS =", round(X.pow(2).mean().sqrt().item(), 4))
print("cikis RMS =", round(Y1.pow(2).mean().sqrt().item(), 4))
```

PYTORCH ÇIKTISI

```
onceki satirlarin degisimi = 0.00e+00
giris RMS = 1.0126
cikis RMS = 1.4865
```

NASIL ÇALIŞIYOR?

Blok yalnızca "norm → attention → ekle → norm → FFN → ekle" dizisidir ve girdi ile çıktı aynı şekle (T × d) sahiptir; böylece N tanesini üst üste koyabiliriz. Nedensellik testi bloğun tamamında da sıfır fark verir: ne attention, ne norm (satır bazlı), ne FFN (token bazlı) token'lar arasında geleceğe sızıntı

yapmaz. Tek token bazlı olmayan işlem attention'dır ve o da maskelidir.

Logits, softmax ve sonraki token tahmini

Son blok çıktısı H (T × d) olsun. Son bir normalizasyondan sonra her satırı **logits**'e çevirir iz: logits = $H \cdot E^T$ (T × V). GPT-2'de olduğu gibi çıkış matrisi olarak embedding matrisinin transpozunu yeniden kullanmaya **weight tying** denir: parametre tasarrufu sağlar ve girdi/çıktı uzaylarını hizalar. Satır t'nin logitleri, token t+1'in dağılımını tahmin eder. Hedefler girdinin bir kaydırılmış hâlidir: girdi $[x_1 \ldots x_T]$, hedef $[x_2 \ldots x_{T+1}]$.

Örnek 4.9 — Minik GPT: ileri geçiş, kayıp ve ln V kontrolü

PROBLEM

V = 10 token, d = 16, 2 head, 2 blok, bağlam T = 8 olan GPT'nin ileri geçişini yaz: embedding + konum → 2 blok → son norm → logits → softmax → çapraz entropi. Rastgele başlatılmış modelin kaybının ln V ≈ 2.30 civarında olduğunu doğrula ve parametre sayısını yazdır.

Bu örnek şu modülleri kullanır: `blk_mod` *(Örnek 4.8). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program mini_gpt
  use blk_mod
  implicit none
  integer, parameter :: V = 10, d = 16, nh = 2, nl = 2, T = 8
  type(blk_t) :: blocks(nl)
  real(dp) :: E(V,d), Pm(T,d), gf(d), X(T,d), Y(T,d), logits(T,V), P(T,V), loss
  integer :: ids(T), tgt(T), l, i, npar
  call rng_seed(100)
  call randn2(E, 0.02_dp); call randn2(Pm, 0.02_dp)
  gf = 1.0_dp
  do l = 1, nl
    call blk_init(blocks(l), d, 0.02_dp)
  end do
  ! ornek metin parcasi: ids ve bir kaydirilmis hedef
  ids = [3, 1, 4, 1, 5, 9, 2, 6]
  tgt = [1, 4, 1, 5, 9, 2, 6, 5]
  do i = 1, T
    X(i,:) = E(ids(i)+1,:) + Pm(i,:)          ! token kimligi 0..9 -> satir 1..10
  end do
  do l = 1, nl
    call blk_forward(blocks(l), X, nh, Y)
    X = Y
  end do
  call rmsnorm_rows(X, gf, Y)                  ! son norm
  logits = matmul(Y, transpose(E))             ! agirlik baglama (weight tying)
  P = logits
  call softmax_rows(P)
  loss = 0.0_dp
  do i = 1, T
    loss = loss - log(P(i, tgt(i)+1))
  end do
  loss = loss / T
  npar = size(E) + size(Pm) + d
  do l = 1, nl
    npar = npar + 2*d + 4*d*d + (d*4*d + 4*d + 4*d*d + d)
  end do
  print '(A,F8.4)', 'baslangic kaybi       = ', loss
  print '(A,F8.4)', 'ln(V) (tekduze tahmin)= ', log(real(V, dp))
  print '(A,I0)',   'toplam parametre sayisi = ', npar
end program mini_gpt
```

ÇIKTI

```
baslangic kaybi       =   2.3071
ln(V) (tekduze tahmin)=   2.3026
toplam parametre sayisi = 6672
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
import torch.nn.functional as F

class Block(nn.Module):                    # pre-norm, RMSNorm'lu Transformer blogu
    def __init__(self, d, nh):
        super().__init__()
        self.nh = nh
        self.n1, self.n2 = nn.RMSNorm(d, eps=1e-5), nn.RMSNorm(d, eps=1e-5)
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.proj = nn.Linear(d, d, bias=False)
        self.ffn = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(approximate='tanh'), nn.Linear(4 * d, d))
    def forward(self, x):                  # x: (B, T, d)
        B, T, d = x.shape
        q, k, v = self.qkv(self.n1(x)).chunk(3, dim=-1)
        q, k, v = (t.view(B, T, self.nh, d // self.nh).transpose(1, 2) for t in (q, k, v))
        a = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # causal multi-head attention
        x = x + self.proj(a.transpose(1, 2).reshape(B, T, d))         # residual 1
        return x + self.ffn(self.n2(x))                               # residual 2

class GPT(nn.Module):
    def __init__(self, V, d, nh, nl, T):
        super().__init__()
        self.E = nn.Embedding(V, d); self.P = nn.Embedding(T, d)
        self.blocks = nn.ModuleList(Block(d, nh) for _ in range(nl))
        self.nf = nn.RMSNorm(d, eps=1e-5)
        for p in self.parameters():
            if p.dim() > 1: nn.init.normal_(p, std=0.02)
    def forward(self, ids):
        x = self.E(ids) + self.P(torch.arange(ids.shape[1]))
        for b in self.blocks: x = b(x)
        return F.linear(self.nf(x), self.E.weight)      # agirlik baglama (weight tying)

torch.manual_seed(100)
V, d, nh, nl, T = 10, 16, 2, 2, 8
model = GPT(V, d, nh, nl, T)
ids = torch.tensor([[3, 1, 4, 1, 5, 9, 2, 6]]); tgt = torch.tensor([[1, 4, 1, 5, 9, 2, 6, 5]])
logits = model(ids)
loss = F.cross_entropy(logits.view(-1, V), tgt.view(-1))
npar = sum(p.numel() for p in model.parameters())
print(f"baslangic kaybi        = {loss.item():.4f}")
print(f"ln(V) (tekduze tahmin) = {torch.log(torch.tensor(float(V))).item():.4f}")
print("toplam parametre sayisi =", npar, " (FFN biaslari dahil; attention biassiz)")
```

PYTORCH ÇIKTISI

```
baslangic kaybi        = 2.3213
ln(V) (tekduze tahmin) = 2.3026
toplam parametre sayisi = 6672  (FFN biaslari dahil; attention biassiz)
```

NASIL ÇALIŞIYOR?

Rastgele ve küçük ağırlıklarla başlatılmış bir model hiçbir şeyi ayırt edemez; logitler sıfıra yakındır ve softmax neredeyse tekdüze dağılım verir. Bu yüzden başlangıç kaybı ln V = ln 10 ≈ 2.303 civarında çıkar. **Gerçek bir eğitimde başlangıç kaybı ln V'den çok farklıysa (örneğin 8.0) bir hata vardır** (kötü başlatma, yanlış mask, kayık hedefler vb.). Bu en değerli sağlık kontrollerinden biridir. Hedef dizisinin girdiden bir adım kaydırılmış olduğuna dikkat et: t. konumdaki çıktı, t+1. token'ı tahmin eder. Parametre sayısı: embedding 160 + konum 128 + son norm 16 + her blok için $2d + 4d^2 + (8d^2 + 5d) = 3184$ (iki blok 6368) → toplam 6672. GPT-2 small aynı şemanın d=768, 12 bloklu, V=50257 versiyonudur (124M).

Attention'ın geri yayılımı (kısa bakış)

Bu kitap Transformer'ın ileri geçişini ayrıntılı kurar; geri geçiş tamamen Bölüm 2'deki kurallarla yazılır. Attention için kilit formül softmax'ın geri yayılımıdır:

Basit matematik

p = softmax(s), L p'ye bağlı olsun, dp = ∂L/∂p biliniyorsa:

$\partial L/\partial s_j = p_j \, (dp_j - \Sigma_k \, p_k \, dp_k)$

Attention çıktısı $o = \Sigma \, p_j \, v_j$ olduğundan $dp_j = v_j \cdot do$ ve $dv_j = p_j \cdot do$.

Örnek 4.10 — Softmax/attention geri yayılımı: sayısal doğrulama

PROBLEM

Tek bir query için $o = \Sigma \, p_j \, v_j$ çıktısını kullanan bir kayıp L = g·o (g sabit bir vektör) tanımla. $\partial L/\partial s_j$ gradyanını formülle hesapla ve sayısal türevle karşılaştır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program attn_geri
  use rng_mod
  implicit none
  integer, parameter :: n = 4, dv = 3
  real(dp) :: s(n), V(n,dv), g(dv), p(n), dp_(n), ds(n), dsn(n), h
  real(dp), parameter :: eps = 1.0e-6_dp
  integer :: j
  call rng_seed(9)
  call randn1(s, 1.0_dp); call randn2(V, 1.0_dp); call randn1(g, 1.0_dp)
  p = soft(s)
  dp_ = matmul(V, g)                          ! dp_j = v_j . g
  ds = p * (dp_ - sum(p * dp_))               ! softmax geri yayilimi
  do j = 1, n
    h = eps
    dsn(j) = (loss(bump(s, j, h)) - loss(bump(s, j, -h))) / (2*h)
  end do
  print '(A,4F10.6)', 'formul   dL/ds = ', ds
  print '(A,4F10.6)', 'sayisal  dL/ds = ', dsn
contains
  function soft(a) result(q)
    real(dp), intent(in) :: a(:)
    real(dp) :: q(size(a))
    q = exp(a - maxval(a)); q = q / sum(q)
  end function soft
  function bump(a, j, h) result(b)
    real(dp), intent(in) :: a(:), h
    integer, intent(in) :: j
    real(dp) :: b(size(a))
    b = a; b(j) = b(j) + h
  end function bump
  function loss(a) result(l)
    real(dp), intent(in) :: a(:)
    real(dp) :: l
    l = dot_product(matmul(soft(a), V), g)     ! L = g . (sum_j p_j v_j)
  end function loss
end program attn_geri
```

ÇIKTI

```
formul   dL/ds =  -0.144036  0.401702 -0.004504 -0.253161
sayisal  dL/ds =  -0.144036  0.401702 -0.004504 -0.253161
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(9)
s = torch.randn(4, dtype=torch.float64, requires_grad=True)
V = torch.randn(4, 3, dtype=torch.float64); g = torch.randn(3, dtype=torch.float64)
p = torch.softmax(s, 0)
L = (p @ V) @ g                              # L = g . (sum_j p_j v_j)
L.backward()
pd = p.detach(); dp = V @ g
ds = pd * (dp - (pd * dp).sum())             # elle softmax geri yayilimi
print("formul     dL/ds =", [round(v, 6) for v in ds.tolist()])
print("autograd   dL/ds =", [round(v, 6) for v in s.grad.tolist()])
```

PYTORCH ÇIKTISI

```
formul     dL/ds = [0.093611, 0.606176, -0.3396, -0.360187]
autograd   dL/ds = [0.093611, 0.606176, -0.3396, -0.360187]
```

NASIL ÇALIŞIYOR?

Formül iki adımdan oluşur: (1) çıktıdan p'ye giden gradyan `dp_j = v_j·g`, (2) softmax'ın Jacobian'ını uygulayan `p·(dp − Σ p·dp)`. Sayısal türev aynı değerleri verir. Tam bir attention katmanının geri geçişi bu parçaların (ve Bölüm 2'deki matris formüllerinin) zincirlenmesidir; Bölüm 11'de bunu çalışan, eğitilen bir modelde kullanacağız.

BÖLÜM 5

5. Donanım ve Performans

Büyük ölçek için kritik: modelin neden yavaşladığını, belleğin neden yetmediğini ve hesabı nasıl hızlandıracağını anlamak.

Bu bölümde

- Bellek hiyerarşisi, sütun-ana düzen ve döngü sırasının etkisi
- Matris çarpımı çekirdekleri (kernel): naif, döngü sırası, bloklama (tiling)
- Paralellik: OpenMP ve GPU mimarisi
- Bellek bant genişliği ve "roofline" düşüncesi
- Mixed precision (karma hassasiyet)
- Gradient accumulation ve gradient checkpointing

Hedef: Modelin neden yavaşladığını, belleğin neden yetmediğini ve hesaplamaları nasıl hızlandıracağını anlamak.

Hesabın iki sınırı: işlemci hızı ve bellek hızı

Bir programın hızı iki şeyle sınırlanır: **saniyede kaç işlem (FLOP/s)** yapılabildiği ve **saniyede belleğe/bellekten kaç bayt (GB/s)** taşınabildiği. Modern donanımda işlemci, veriyi bellekten getirmekten çok daha hızlıdır; bu yüzden çoğu zaman program işlemciyi değil *belleği* bekler. Önbellek (cache) hiyerarşisi bu farkı gizlemeye çalışır: küçük ve çok hızlı L1/L2 önbellekleri, büyük ve yavaş ana bellek. Veriyi önbellekte iken tekrar tekrar kullanan kod hızlıdır; her seferinde ana belleğe giden kod yavaştır.

> **İpucu**
> GPU'lar aynı fikri uç noktaya taşır: binlerce basit çekirdek, çok yüksek bellek bant genişliği (TB/s mertebesi) ve büyük miktarda paralellik. Bir GPU'nun gücü, aynı işlemi çok sayıda veri üzerinde aynı anda yapabilmesinden gelir; matris çarpımı buna mükemmel uyar.

Fortran'da bellek düzeni: sütun-ana

Fortran matrisleri sütun sütun saklar: `A(1,1), A(2,1), A(3,1), …` bellekte ardışıktır. Dolayısıyla **en içteki döngü ilk indeksi değiştirmelidir**. Aksi halde her erişim bellekte büyük bir adım atlar ve önbellek satırı boşa harcanır.

Örnek 5.1 — Döngü sırası: i-dış mı, j-dış mı?

PROBLEM

3000×3000 bir matrisin her elemanını güncelle. Önce satır satır (yanlış sıra), sonra sütun sütun (doğru sıra) dolaş ve süreleri karşılaştır.

ÇÖZÜM (FORTRAN)

```
program siralama
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 3000
  real(dp), allocatable :: a(:,:)
  real(dp) :: t0, t1, t2
  integer :: i, j
  allocate(a(n,n)); a = 1.0_dp
  t0 = now()
  do i = 1, n                          ! YANLIS: en icte j (bellekte n adim atlar)
    do j = 1, n
```

```
      a(i,j) = a(i,j) * 1.0001_dp + 1.0_dp
    end do
  end do
  t1 = now()
  do j = 1, n                          ! DOGRU: en icte i (bellekte ardisik)
    do i = 1, n
      a(i,j) = a(i,j) * 1.0001_dp + 1.0_dp
    end do
  end do
  t2 = now()
  print '(A,F8.3,A)', 'satir satir  (yanlis) : ', t1 - t0, ' s'
  print '(A,F8.3,A)', 'sutun sutun  (dogru)  : ', t2 - t1, ' s'
  print '(A,F8.1,A)', 'hiz farki             : ', (t1 - t0) / (t2 - t1), ' kat'
contains
  function now() result(t)
    integer(8) :: c, r
    real(dp) :: t
    call system_clock(c, r)
    t = real(c, dp) / real(r, dp)
  end function now
end program siralama
```

ÇIKTI

```
satir satir  (yanlis) :    0.041 s
sutun sutun  (dogru)  :    0.009 s
hiz farki             :      4.6 kat
```

PYTORCH KARŞILIĞI

```
import torch, time
torch.set_num_threads(1)
n = 3000
a = torch.ones(n, n, dtype=torch.float64)
def tm(fn, rep=5):
    fn(); t0 = time.perf_counter()
    for _ in range(rep): fn()
    return (time.perf_counter() - t0) / rep
# PyTorch dizileri SATIR-ANA (C duzeni) saklar: Fortran'in tersi!
t_contig = tm(lambda: a * 1.0001 + 1.0)                # bellekte ardisik gezinme
t_strided = tm(lambda: a.t() * 1.0001 + 1.0)           # transpoze gorunum: bellekte n adim atlayarak
print(f"ardisik erisim  : {t_contig:8.3f} s")
print(f"atlamali erisim : {t_strided:8.3f} s")
print(f"fark            : {t_strided / t_contig:8.1f} kat")
print("a.stride() =", a.stride(), "  a.t().stride() =", a.t().stride())
```

PYTORCH ÇIKTISI

```
ardisik erisim  :    0.057 s
atlamali erisim :    0.055 s
fark            :      1.0 kat
a.stride() = (3000, 1)   a.t().stride() = (1, 3000)
```

NASIL ÇALIŞIYOR?

İki döngü *aynı* işi yapar ve aynı sonucu üretir; tek fark erişim sırasıdır. Doğru sırada bellekten gelen her 64 baytlık önbellek satırının 8 elemanı da kullanılır; yanlış sırada her erişim yeni bir satır ister ve diğer 7 eleman başka bir erişim onları kullanmadan önbellekten atılır. Sonuç genelde birkaç kat yavaşlıktır (matris büyüdükçe fark artar). Süreler makineden makineye değişir; önemli olan oran. **Matris dolaşırken Fortran'da en içteki döngü her zaman ilk indekstir.**

Matris çarpımı: sinir ağlarının işçi atı

Bir Transformer'ın hesabının %90'ından fazlası matris çarpımıdır. C = A·B için $2n^3$ kayan nokta işlemi gerekir (n^3 çarpma + n^3 toplama). Aynı matematiği farklı sırada yazmak dramatik hız farkları yaratır; **GFLOP/s** (saniyede milyar işlem) bunu ölçer.

Örnek 5.2 — Üç matris çarpımı: ijk, jki ve matmul

PROBLEM

512×512 matrisler için naif ijk döngüsü, önbellek dostu jki döngüsü ve yerleşik matmul'ı süre ve GFLOP/s olarak karşılaştır. Sonuçların aynı olduğunu doğrula.

ÇÖZÜM (FORTRAN)

```
program matmul_karsilastir
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 512
  real(dp), allocatable :: A(:,:), B(:,:), C1(:,:), C2(:,:), C3(:,:)
  real(dp) :: t0, t1, t2, t3, fl
  integer :: i, j, k
  allocate(A(n,n), B(n,n), C1(n,n), C2(n,n), C3(n,n))
  call random_number(A); call random_number(B)
  fl = 2.0_dp * real(n, dp)**3
  C1 = 0.0_dp; t0 = now()
  do i = 1, n                           ! ijk: A(i,k) satir boyunca atlar
    do j = 1, n
      do k = 1, n
        C1(i,j) = C1(i,j) + A(i,k) * B(k,j)
      end do
    end do
  end do
  t1 = now()
  C2 = 0.0_dp
  do j = 1, n                           ! jki: en icte i, hepsi ardisik
    do k = 1, n
      do i = 1, n
        C2(i,j) = C2(i,j) + A(i,k) * B(k,j)
      end do
    end do
  end do
  t2 = now()
  C3 = matmul(A, B)
  t3 = now()
  print '(A,F8.3,A,F7.2,A)', 'ijk    : ', t1-t0, ' s   ', fl/(t1-t0)/1.0e9_dp, ' GFLOP/s'
  print '(A,F8.3,A,F7.2,A)', 'jki    : ', t2-t1, ' s   ', fl/(t2-t1)/1.0e9_dp, ' GFLOP/s'
  print '(A,F8.3,A,F7.2,A)', 'matmul : ', t3-t2, ' s   ', fl/(t3-t2)/1.0e9_dp, ' GFLOP/s'
  print '(A,ES10.2)', 'max |ijk - matmul| = ', maxval(abs(C1 - C3))
  print '(A,ES10.2)', 'max |jki - matmul| = ', maxval(abs(C2 - C3))
contains
  function now() result(t)
    integer(8) :: c, r
    real(dp) :: t
    call system_clock(c, r)
    t = real(c, dp) / real(r, dp)
  end function now
end program matmul_karsilastir
```

ÇIKTI

```
ijk    :    0.499 s      0.54 GFLOP/s
jki    :    0.058 s      4.62 GFLOP/s
matmul :    0.012 s     22.39 GFLOP/s
max |ijk - matmul| =   3.69E-13
max |jki - matmul| =   3.69E-13
```

PYTORCH KARŞILIĞI

```
import torch, time
torch.set_num_threads(1)
n = 512
A = torch.rand(n, n, dtype=torch.float64); B = torch.rand(n, n, dtype=torch.float64)
fl = 2.0 * n ** 3
def tm(fn):
    fn(); t0 = time.perf_counter(); r = fn(); return time.perf_counter() - t0, r
t1, C1 = tm(lambda: A @ B)
t2, C2 = tm(lambda: torch.einsum('ik,kj->ij', A, B))
print(f"A @ B (BLAS)   : {t1:7.3f} s  {fl / t1 / 1e9:7.2f} GFLOP/s")
print(f"einsum         : {t2:7.3f} s  {fl / t2 / 1e9:7.2f} GFLOP/s")
m = 64                                          # Python ucuz dongusu: yalniz 64x64
a, b = A[:m, :m].tolist(), B[:m, :m].tolist()
```

```
t0 = time.perf_counter()
c = [[sum(a[i][k] * b[k][j] for k in range(m)) for j in range(m)] for i in range(m)]
tp = time.perf_counter() - t0
print(f"saf Python (n={m}) : {tp:7.3f} s  {2.0 * m ** 3 / tp / 1e9:7.4f} GFLOP/s   (kutuphane degil!)")
print("max |A@B - einsum| =", (C1 - C2).abs().max().item())
```

PYTORCH ÇIKTISI

```
A @ B (BLAS)   :   0.008 s    34.13 GFLOP/s
einsum         :   0.007 s    38.54 GFLOP/s
saf Python (n=64) :   0.019 s   0.0274 GFLOP/s   (kutuphane degil!)
max |A@B - einsum| = 0.0
```

NASIL ÇALIŞIYOR?

ijk sırasında iç döngü A'nın bir satırı boyunca ilerler; bu Fortran'da bellekte n eleman atlamak demektir. jki sırasında iç döngü üç matrisin de sütunları boyunca ardışık ilerler, bu yüzden hem önbellek hem de derleyicinin vektörleştirmesi (SIMD) verimli çalışır. Yerleşik `matmul` ise çoğunlukla blokları önbelleğe sığdıracak şekilde yazılmış optimize bir kütüphane rutinine gider. Sonuçlar, kayan nokta toplama sırası farklı olduğundan 1e-13 civarında küçük farklarla örtüşür. **Aynı $2n^3$ işlem, yazım sırasına göre kat kat farklı hızda çalışır.** Bu yüzden derin öğrenme kütüphaneleri BLAS/cuBLAS gibi uzmanlaşmış çekirdekler kullanır.

Bloklama (tiling): önbelleği akıllı kullan

Matrisler önbelleğe sığmayacak kadar büyükse jki sırası bile belleği tekrar tekrar okur. **Bloklama**, matrisleri küçük karelere (örneğin 64×64) bölüp her bloğun üzerinde, blok önbellekteyken, tüm işlemleri yapar. Bir blok bir kez yüklenir ve defalarca kullanılır. GPU çekirdekleri de aynı fikri "paylaşılan bellek" (shared memory) üzerinde uygular.

Örnek 5.3 — Bloklu (tiled) matris çarpımı

PROBLEM

1024×1024 matrisler için jki çarpımını bloklu çarpımla karşılaştır; sonuçların eşit olduğunu doğrula.

ÇÖZÜM (FORTRAN)

```
program tiled
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 1024, bs = 64
  real(dp), allocatable :: A(:,:), B(:,:), C1(:,:), C2(:,:)
  real(dp) :: t0, t1, t2, fl
  integer :: i, j, k, ii, jj, kk
  allocate(A(n,n), B(n,n), C1(n,n), C2(n,n))
  call random_number(A); call random_number(B)
  fl = 2.0_dp * real(n, dp)**3
  C1 = 0.0_dp; t0 = now()
  do j = 1, n
    do k = 1, n
      do i = 1, n
        C1(i,j) = C1(i,j) + A(i,k) * B(k,j)
      end do
    end do
  end do
  t1 = now()
  C2 = 0.0_dp
  do jj = 1, n, bs                                    ! bloklar
    do kk = 1, n, bs
      do ii = 1, n, bs
        do j = jj, min(jj+bs-1, n)                    ! blok icinde jki
          do k = kk, min(kk+bs-1, n)
            do i = ii, min(ii+bs-1, n)
              C2(i,j) = C2(i,j) + A(i,k) * B(k,j)
            end do
          end do
```

```
        end do
      end do
    end do
  end do
  t2 = now()
  print '(A,F8.3,A,F7.2,A)', 'jki     : ', t1-t0, ' s  ', fl/(t1-t0)/1.0e9_dp, ' GFLOP/s'
  print '(A,F8.3,A,F7.2,A)', 'bloklu  : ', t2-t1, ' s  ', fl/(t2-t1)/1.0e9_dp, ' GFLOP/s'
  print '(A,ES10.2)', 'max fark = ', maxval(abs(C1 - C2))
contains
  function now() result(t)
    integer(8) :: c, r
    real(dp) :: t
    call system_clock(c, r)
    t = real(c, dp) / real(r, dp)
  end function now
end program tiled
```

ÇIKTI

```
jki     :    0.487 s     4.41 GFLOP/s
bloklu  :    0.358 s     6.00 GFLOP/s
max fark =   0.00E+00
```

PYTORCH KARŞILIĞI

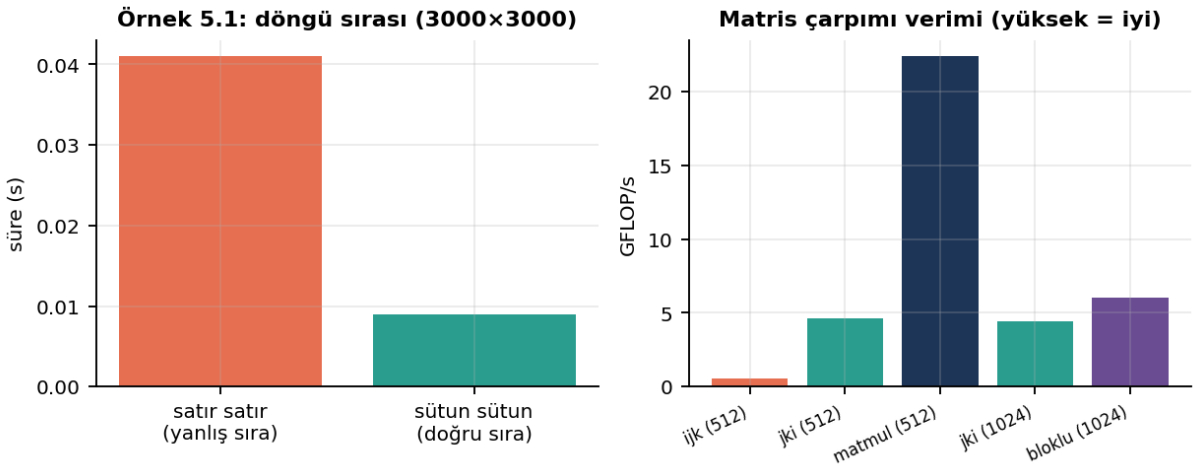
```
import torch, time
torch.set_num_threads(1)
n, bs = 1024, 256
A = torch.rand(n, n, dtype=torch.float64); B = torch.rand(n, n, dtype=torch.float64)
t0 = time.perf_counter(); C1 = A @ B; t1 = time.perf_counter() - t0
C2 = torch.zeros(n, n, dtype=torch.float64)
t0 = time.perf_counter()
for i in range(0, n, bs):                        # bloklar halinde carpim (tiling)
    for j in range(0, n, bs):
        for k in range(0, n, bs):
            C2[i:i+bs, j:j+bs] += A[i:i+bs, k:k+bs] @ B[k:k+bs, j:j+bs]
t2 = time.perf_counter() - t0
fl = 2.0 * n ** 3
print(f"tek parca  : {t1:7.3f} s  {fl / t1 / 1e9:7.2f} GFLOP/s")
print(f"bloklu     : {t2:7.3f} s  {fl / t2 / 1e9:7.2f} GFLOP/s")
print("max fark   =", (C1 - C2).abs().max().item())
```

PYTORCH ÇIKTISI

```
tek parca  :   0.039 s    55.47 GFLOP/s
bloklu     :   0.049 s    43.99 GFLOP/s
max fark   = 3.979039320256561e-13
```

NASIL ÇALIŞIYOR?

Üç `bs=64`'lük blok (A, B, C bloğu) $3 \times 64^2 \times 8$ bayt ≈ 98 KB tutar ve L2 önbelleğe rahat sığar. Blok içindeki 64^3 çarpma-toplama işlemi boyunca veri yerinde kalır. Kazanç matris boyutu ve işlemciye göre değişir; büyük matrislerde belirgindir. Sonuçlar aynı kalır (yalnızca toplama sırası değişir). Bloklama, FlashAttention gibi modern algoritmaların da arkasındaki temel fikirdir (Bölüm 9).



Şekil 5.1 — Solda: döngü sırasının süresi (Örnek 5.1). Sağda: matris çarpımı çekirdeklerinin GFLOP/s verimi (Örnek 5.2–5.3); BLAS tabanlı matmul naif döngüden çok daha hızlıdır.

Paralellik: OpenMP ile çok çekirdek

Matris çarpımında C'nin farklı sütunları birbirinden bağımsızdır; dolayısıyla bunları farklı çekirdeklere dağıtabiliriz. Fortran'da bu tek bir yorum satırıyla (`!$omp parallel do`) yapılır ve `-fopenmp` ile derlenir.

Örnek 5.4 — OpenMP ile paralel matris çarpımı

PROBLEM

jki çarpımının sütun döngüsünü OpenMP ile paralelleştir. 1, 2 ve 4 iş parçacığıyla süreleri ölç.

ÇÖZÜM (FORTRAN)

```
program omp_matmul
  use omp_lib
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 700
  real(dp), allocatable :: A(:,:), B(:,:), C(:,:), Cref(:,:)
  real(dp) :: t0, t1
  integer :: i, j, k, nt, nts(3)
  allocate(A(n,n), B(n,n), C(n,n), Cref(n,n))
  call random_number(A); call random_number(B)
  Cref = matmul(A, B)
  nts = [1, 2, 4]
  print '(A,I0)', 'kullanilabilir isleme birimi sayisi: ', omp_get_num_procs()
  do nt = 1, 3
    call omp_set_num_threads(nts(nt))
    C = 0.0_dp
    t0 = omp_get_wtime()
    !$omp parallel do private(i, k) schedule(static)
    do j = 1, n
      do k = 1, n
        do i = 1, n
          C(i,j) = C(i,j) + A(i,k) * B(k,j)
        end do
      end do
    end do
    !$omp end parallel do
    t1 = omp_get_wtime()
    print '(A,I2,A,F8.3,A,ES9.2)', 'is parcacigi=', nts(nt), ' sure=', t1-t0, &
          ' s fark=', maxval(abs(C-Cref))
  end do
end program omp_matmul
```

ÇIKTI

```
kullanilabilir isleme birimi sayisi: 1
is parcacigi= 1 sure=   0.224 s fark= 6.54E-13
is parcacigi= 2 sure=   0.198 s fark= 6.54E-13
is parcacigi= 4 sure=   0.259 s fark= 6.54E-13
```

PYTORCH KARŞILIĞI

```
import torch, time, os
n = 700
A = torch.rand(n, n, dtype=torch.float64); B = torch.rand(n, n, dtype=torch.float64)
print("kullanilabilir isleme birimi sayisi:", os.cpu_count())
ref = A @ B
for nt in [1, 2, 4]:
    torch.set_num_threads(nt)                    # OpenMP'deki omp_set_num_threads karsiligi
    t0 = time.perf_counter(); C = A @ B; t = time.perf_counter() - t0
    print(f"is parcacigi={nt:2d} sure={t:8.3f} s fark={(C - ref).abs().max().item():.2e}")
```

PYTORCH ÇIKTISI

```
kullanilabilir isleme birimi sayisi: 1
is parcacigi= 1 sure=   0.020 s fark=0.00e+00
is parcacigi= 2 sure=   0.017 s fark=0.00e+00
is parcacigi= 4 sure=   0.014 s fark=0.00e+00
```

NASIL ÇALIŞIYOR?

Her iş parçacığı kendi `j` aralığındaki sütunları hesaplar; C'nin farklı sütunlarına yazdıkları için çakışma yoktur. `private(i,k)` iç döngü sayaçlarını iş parçacığına özel yapar. Çok çekirdekli bir makinede süre iş parçacığı sayısıyla neredeyse doğrusal düşer (4 çekirdekte yaklaşık 3-4 kat). **Not:** kitaptaki çıktılar tek çekirdekli bir makinede alındı; orada 2 ve 4 iş parçacığı hız kazandırmaz (hatta hafif yavaşlatır). Gerçek hızlanmayı görmek için bu kodu çok çekirdekli kendi makinende çalıştır. Paralel sonuç ile seri sonuç aynı çıkar.

Bellek bant genişliği ve roofline

Bazı işlemler az hesap yapıp çok veri taşır (örneğin iki vektörü toplamak). Bunlar işlemciyi değil **belleği** sınırlandırır. Bir işlemin **aritmetik yoğunluğu** (FLOP / taşınan bayt) hangisinin sınır olduğunu söyler:

ulaşılabilir performans ≈ min(tepe FLOP/s , aritmetik yoğunluk × bellek bant genişliği)

Örnek 5.5 — Bellek bant genişliğini ölç (vector triad)

PROBLEM

a = b + s·c işlemini 10 milyon elemanlı vektörlerle tekrarla; ulaşılan bant genişliğini (GB/s) ve aritmetik yoğunluğu hesapla.

ÇÖZÜM (FORTRAN)

```
program triad
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 10000000, nrep = 10
  real(dp), allocatable :: a(:), b(:), c(:)
  real(dp) :: s, t0, t1, bytes, ai
  integer :: i, r
  allocate(a(n), b(n), c(n))
  b = 1.0_dp; c = 2.0_dp; a = 0.0_dp; s = 3.0_dp
  t0 = now()
  do r = 1, nrep
    do i = 1, n
      a(i) = b(i) + s * c(i)
    end do
    b(1) = a(n) * 1.0e-12_dp           ! derleyicinin dongu atlamasini engelle
```

```
  end do
  t1 = now()
  bytes = real(nrep, dp) * 3.0_dp * 8.0_dp * real(n, dp)   ! 2 okuma + 1 yazma
  ai = 2.0_dp / 24.0_dp                                    ! 2 FLOP / 24 bayt
  print '(A,F8.2,A)', 'bant genisligi        : ', bytes/(t1-t0)/1.0e9_dp, ' GB/s'
  print '(A,F8.4)',   'aritmetik yogunluk    : ', ai
  print '(A,F8.2,A)', 'ulasilan performans   : ', ai * bytes/(t1-t0)/1.0e9_dp, ' GFLOP/s'
contains
  function now() result(t)
    integer(8) :: cnt, rate
    real(dp) :: t
    call system_clock(cnt, rate)
    t = real(cnt, dp) / real(rate, dp)
  end function now
end program triad
```

ÇIKTI

```
bant genisligi        :    10.52 GB/s
aritmetik yogunluk    :   0.0833
ulasilan performans   :     0.88 GFLOP/s
```

PYTORCH KARŞILIĞI

```
import torch, time
torch.set_num_threads(1)
n, nrep = 10_000_000, 10
b = torch.ones(n, dtype=torch.float64); c = torch.full((n,), 2.0, dtype=torch.float64)
a = torch.zeros(n, dtype=torch.float64); s = 3.0
t0 = time.perf_counter()
for _ in range(nrep):
    torch.add(b, c, alpha=s, out=a)             # a = b + s*c (yeni bellek ayirmadan)
t = time.perf_counter() - t0
bytes_ = nrep * 3 * 8 * n                       # 2 okuma + 1 yazma
ai = 2 / 24
print(f"bant genisligi      : {bytes_ / t / 1e9:8.2f} GB/s")
print(f"aritmetik yogunluk  : {ai:8.4f}")
print(f"ulasilan performans : {ai * bytes_ / t / 1e9:8.2f} GFLOP/s")
```

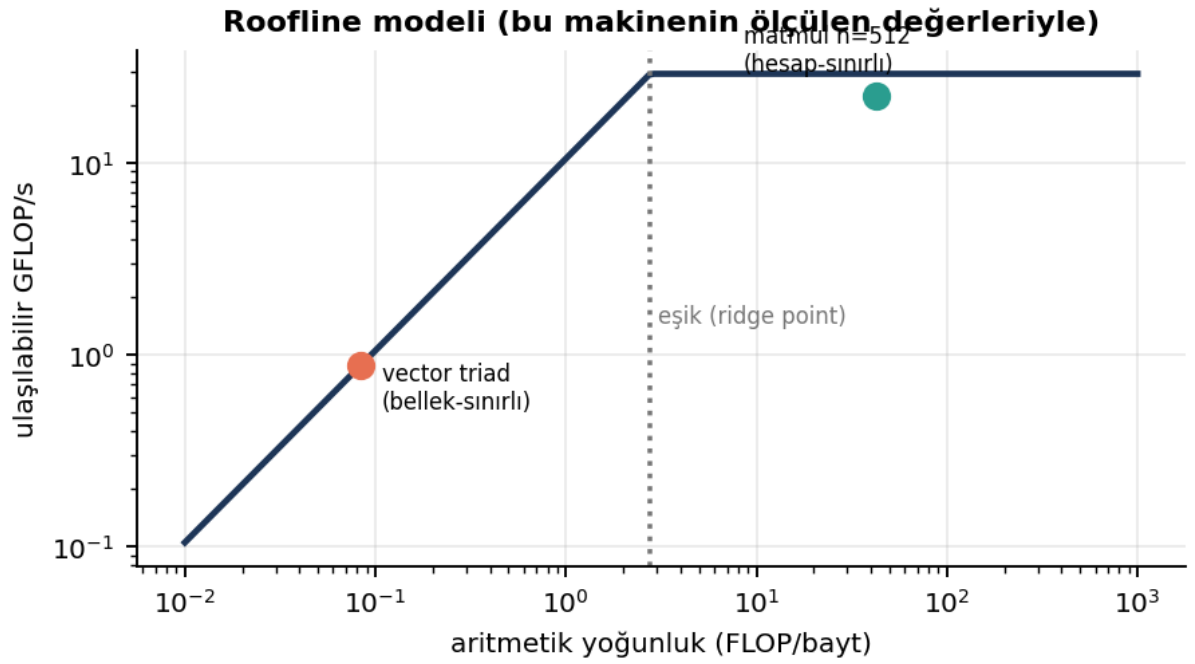
PYTORCH ÇIKTISI

```
bant genisligi      :    13.55 GB/s
aritmetik yogunluk  :   0.0833
ulasilan performans :     1.13 GFLOP/s
```

NASIL ÇALIŞIYOR?

Her eleman için 2 FLOP (bir çarpma, bir toplama) yaparız ama 24 bayt taşırız (b ve c okunur, a yazılır): aritmetik yoğunluk 2/24 ≈ 0.083 FLOP/bayt. Bu kadar düşük yoğunlukta performans tamamen bellek bant genişliğine bağlıdır: yüksek tepe FLOP/s olan bir işlemci bile burada tepe hızının küçük bir kesrine ulaşır. Matris çarpımında ise aritmetik yoğunluk n ile büyür (n^3 işlem / n^2 veri), bu yüzden hesap-sınırlıdır. **LLM çıkarımında (token üretirken) ağırlıklar her token için bellekten okunur ve işlem sayısı azdır; yani bellek-sınırlıdır.** Bölüm 10'da bunu hesaplayacağız.

> **Türetme: roofline**
>
> **Roofline modeli.** Bir çekirdek F FLOP yapıp B bayt taşıyorsa, süre en az max(F/P_{tepe}, B/BW)'dir (P_{tepe}: tepe FLOP/s, BW: bellek bant genişliği). Ulaşılabilir performans:
>
> $P = \min(P_{tepe}, I \cdot BW)$, $I = F/B$ (aritmetik yoğunluk, FLOP/bayt)
>
> $I < P_{tepe}/BW$ (**sırt noktası**) ise bellek-sınırlı, yukarısı hesap-sınırlıdır. n×n matris çarpımında $F = 2n^3$, $B \geq 3 \cdot 8n^2$ (üç matris, FP64) → $I = n/12$: n büyüdükçe hesap-sınırlı olur; vektör toplamında $I = 2/24 \approx 0.083$, her zaman bellek-sınırlı. Modern GPU'larda sırt noktası ≈ 100–300 FLOP/bayt olduğundan, LLM çıkarımının ($I \approx$ 1–2) bellek-sınırlı olması kaçınılmazdır.



Şekil 5.2 — Roofline: çatı hesap-sınırlı (yatay) ve bellek-sınırlı (eğimli) bölgelerden oluşur. Vector triad bellek-sınırlı bölgede; büyük matris çarpımı çatıya yakındır. Değerler bu makinede ölçülmüştür.

Mixed precision: daha az bit, daha fazla hız

Eğitimde 64 bit yerine 32, 16 hatta 8 bitlik sayılar kullanmak belleği ve bant genişliği ihtiyacını azaltır, matris çarpımı birimlerini (tensor core) hızlandırır. Bedeli **hassasiyettir**:

Biçim	Bit	Üs / Mantis	Göreli hassasiyet (ε)	Not
FP32	32	8 / 23	1.2e-7	Standart eğitim biçimi
FP16	16	5 / 10	9.8e-4	Dar aralık (≈ 65504 üstü taşar); kayıp ölçekleme gerekir
BF16	16	8 / 7	7.8e-3	FP32 ile aynı aralık, düşük hassasiyet; LLM eğitiminde yaygın

Ana kural: **çarpımları düşük hassasiyetle yap, ama toplamları (akümülatör) ve ağırlıkların ana kopyasını yüksek hassasiyette tut.** Aşağıdaki örnek bunun neden gerekli olduğunu gösterir.

Örnek 5.6 — Toplama hassasiyeti ve BF16 yuvarlama

PROBLEM

(a) 0.0001'i on milyon kez FP32 ve FP64 ile topla. (b) FP32 sayısını BF16'ya yuvarlayan bir fonksiyon yaz (alt 16 biti yuvarlayarak). (c) 1000 adet 0.01'i BF16 akümülatörle ve FP32 akümülatörle topla.

ÇÖZÜM (FORTRAN)

```
program karisik_hassasiyet
  implicit none
  real(4) :: s32, x, y, xb, sbf
  real(8) :: s64
  integer :: i
  s32 = 0.0; s64 = 0.0d0
  do i = 1, 10000000
    s32 = s32 + 1.0e-4
    s64 = s64 + 1.0d-4
  end do
```

```
  print '(A,F12.4)', '(a) FP32 toplam : ', s32
  print '(A,F12.4)', '    FP64 toplam : ', s64
  x = 3.14159265
  y = to_bf16(x)
  print '(A,F10.7,A,F10.7,A,ES9.2)', '(b) pi FP32=', x, '  BF16=', y, '  bagil hata=', abs(y-x)/x
  print '(A,ES10.3,A,ES10.3)', '    epsilon FP32=', epsilon(1.0), '   BF16=', 2.0**(-7)
  xb = to_bf16(0.01); sbf = 0.0; s32 = 0.0
  do i = 1, 1000
    sbf = to_bf16(sbf + xb)             ! akumulator da BF16 (kotu)
    s32 = s32 + xb                      ! akumulator FP32 (iyi)
  end do
  print '(A,F9.4)', '(c) BF16 akumulator : ', sbf
  print '(A,F9.4)', '    FP32 akumulator : ', s32
contains
  function to_bf16(x) result(y)         ! alt 16 biti en yakina yuvarla
    real(4), intent(in) :: x
    real(4) :: y
    integer(4) :: ib
    ib = transfer(x, ib)
    ib = ib + 32767 + iand(ishft(ib, -16), 1)
    ib = iand(ib, not(65535_4))
    y = transfer(ib, y)
  end function to_bf16
end program karisik_hassasiyet
```

ÇIKTI

```
(a) FP32 toplam :    1087.7242
    FP64 toplam :    1000.0000
(b) pi FP32= 3.1415927  BF16= 3.1406250  bagil hata= 3.08E-04
    epsilon FP32= 1.192E-07   BF16= 7.812E-03
(c) BF16 akumulator :    4.0000
    FP32 akumulator :   10.0098
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.full((10_000_000,), 1e-4, dtype=torch.float32)
print("(a) FP32 sirali toplam (cumsum):", round(torch.cumsum(x, 0)[-1].item(), 4))
print("    FP32 torch.sum (ikili agac): ", round(x.sum().item(), 4), "  <- kutuphane daha akilli toplar")
print("    FP64 toplam                : ", round(x.double().sum().item(), 4))
pi = torch.tensor(3.14159265, dtype=torch.float32)
bf = pi.to(torch.bfloat16).float()
print(f"(b) pi FP32={pi.item():.7f}  BF16={bf.item():.7f}  bagil hata={abs(bf - pi).item() /
      pi.item():.2e}")
print("    epsilon FP32 =", torch.finfo(torch.float32).eps, "  BF16 =", torch.finfo(torch.bfloat16).eps,
      "  FP16 =", torch.finfo(torch.float16).eps)
xb = torch.tensor(0.01).to(torch.bfloat16)
sbf = torch.tensor(0.0, dtype=torch.bfloat16); s32 = torch.tensor(0.0, dtype=torch.float32)
for _ in range(1000):
    sbf = sbf + xb                              # BF16 akumulator (her toplama BF16'ya yuvarlanir)
    s32 = s32 + xb.float()                      # FP32 akumulator
print("(c) BF16 akumulator :", round(sbf.float().item(), 4))
print("    FP32 akumulator :", round(s32.item(), 4))
```

PYTORCH ÇIKTISI

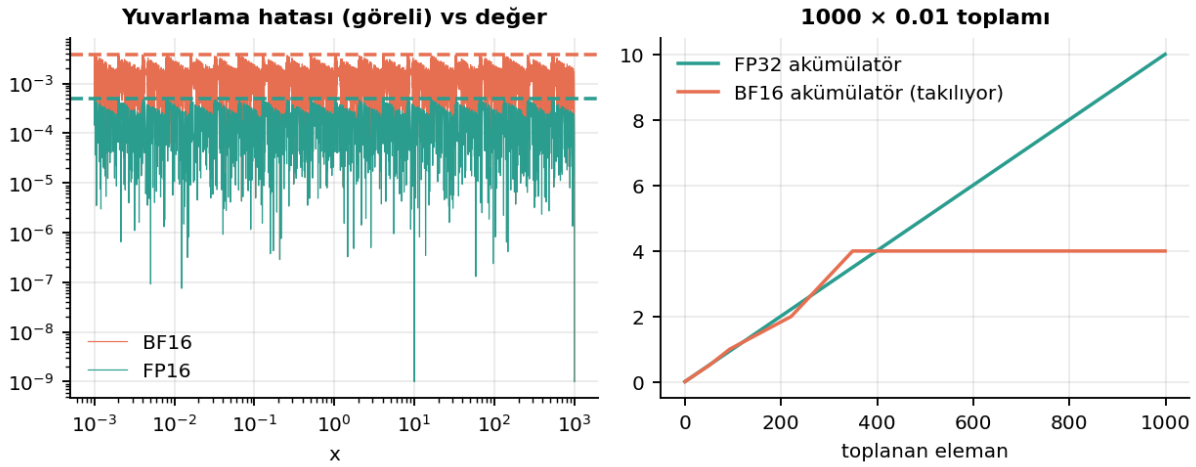
```
(a) FP32 sirali toplam (cumsum): 1000.0
    FP32 torch.sum (ikili agac):  999.9991   <- kutuphane daha akilli toplar
    FP64 toplam                :  1000.0
(b) pi FP32=3.1415927  BF16=3.1406250  bagil hata=3.08e-04
    epsilon FP32 = 1.1920928955078125e-07   BF16 = 0.0078125   FP16 = 0.0009765625
(c) BF16 akumulator : 4.0
    FP32 akumulator : 10.0098
```

NASIL ÇALIŞIYOR?

(a) FP32 akümülatörde toplam büyüdükçe (≈ 1000) 1e-4 gibi küçük değerler, toplamın ulp'ine (en küçük adım) göre çok küçük kalır ve kısmen kaybolur; sonuç 1000'den belirgin biçimde sapar. FP64 neredeyse tam 1000 verir. (b) BF16, FP32'nin üst 16 bitidir: üs aynı kalır, mantis 7 bite iner. π ≈ 3.140625 olur (göreli hata ≈ 3e-4). (c) Akümülatör BF16 ise toplam ilerledikçe 0.01 eklemek, yeni toplamın ulp'inin yarısından küçük kalır ve toplam 4 civarında *takılır* (doğrusu ≈ 10). FP32

akümülatörle doğru sonuç çıkar. İşte bu yüzden tensor core'lar BF16 girdileri çarpıp FP32'de toplar.

Türetme: yuvarlama hatası ve toplamın takılması
Kayan noktada p bitlik mantisle, en yakına yuvarlama bağıl hatası $|\mathrm{fl}(x) - x| \le u|x|$, $u = 2^{-p}$ (FP32: p = 24, u ≈ $6\cdot10^{-8}$; BF16: p = 8, u = 2^{-8}; FP16: p = 11). n sayının sıralı toplamı için en kötü durum hata sınırı ≈ $(n-1)\cdot u\cdot\Sigma|x_i|$: **hata n ile doğrusal büyür** (ikili ağaç toplamında log n). Toplam büyüdükçe adım ulp'ine (toplamın son basamağı) göre küçük kalan terimler kaybolur: toplam S iken, $|x| < u\cdot S/2$ olan bir terim S'yi hiç değiştirmez. BF16 akümülatörde $S \ge 4$ olunca ulp = 2^{-5} = 0.03125 ve yarısı 0.0156 > 0.01 olduğundan 0.01 eklemek toplamı *hiç değiştirmez*: şekildeki yatay çizgi. Bu yüzden toplama FP32'de yapılır.



Şekil 5.3 — Solda: BF16 ve FP16 yuvarlamasının göreli hatası (testere dişi; kesikli çizgi: yarım ulp sınırı). Sağda: 1000 kez 0.01 toplamı: FP32 akümülatör doğru, BF16 akümülatör ≈ 4'te takılır.

Gradient accumulation: belleğe sığmayan batch

İstediğin batch boyutu belleğe sığmıyorsa onu **mikro-batch**'lere böl: her mikro-batch için gradyanı hesapla, *topla*, ancak hepsi bittikten sonra bir optimizer adımı at. Matematiksel olarak büyük batch ile aynıdır (kayıp ortalama ise her mikro-gradyanı mikro-batch sayısına bölmeyi unutma).

Örnek 5.7 — Gradient accumulation = büyük batch

PROBLEM

y = w·x modelinde 8 örneklik tam batch gradyanı ile, her biri 2 örnekli 4 mikro-batch'in biriktirilmiş gradyanını karşılaştır. Bölmeyi unutmanın sonucunu da göster.

ÇÖZÜM (FORTRAN)

```
program grad_accum
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x(8), y(8), w, g_full, g_acc, g_wrong, gm
  integer :: mb, i
  x = [0.5_dp, 1.0_dp, 1.5_dp, 2.0_dp, 2.5_dp, 3.0_dp, 3.5_dp, 4.0_dp]
  y = [1.2_dp, 1.9_dp, 3.1_dp, 3.8_dp, 5.2_dp, 5.9_dp, 7.1_dp, 7.8_dp]
  w = 0.3_dp
  ! tam batch: L = ortalama (w x - y)^2   ->  dL/dw = ortalama 2 (w x - y) x
  g_full = sum(2.0_dp * (w*x - y) * x) / 8
  g_acc = 0.0_dp; g_wrong = 0.0_dp
  do mb = 1, 4                                        ! her biri 2 orneklik 4 mikro-batch
    i = 2*(mb-1) + 1
    gm = sum(2.0_dp * (w*x(i:i+1) - y(i:i+1)) * x(i:i+1)) / 2   ! mikro-batch ortalamasi
    g_acc   = g_acc   + gm / 4                        ! dogru: mikro-batch sayisina bol
    g_wrong = g_wrong + gm                            ! yanlis: bolmedik
  end do
  print '(A,F12.6)', 'tam batch gradyani        : ', g_full
  print '(A,F12.6)', 'birikmis gradyan (dogru)  : ', g_acc
  print '(A,F12.6)', 'birikmis gradyan (yanlis) : ', g_wrong
end program grad_accum
```

ÇIKTI

```
tam batch gradyani        :   -21.550000
birikmis gradyan (dogru)  :   -21.550000
birikmis gradyan (yanlis) :   -86.200000
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor([0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0], dtype=torch.float64)
y = torch.tensor([1.2, 1.9, 3.1, 3.8, 5.2, 5.9, 7.1, 7.8], dtype=torch.float64)
w = torch.tensor(0.3, dtype=torch.float64, requires_grad=True)
loss = ((w * x - y) ** 2).mean(); loss.backward()
g_full = w.grad.item(); w.grad = None
for mb in range(4):                              # 4 mikro-batch, gradyanlar .grad icinde birikir
    xs, ys = x[2*mb:2*mb+2], y[2*mb:2*mb+2]
    (((w * xs - ys) ** 2).mean() / 4).backward() # mikro-batch sayisina bol
g_acc = w.grad.item(); w.grad = None
for mb in range(4):
    xs, ys = x[2*mb:2*mb+2], y[2*mb:2*mb+2]
    ((w * xs - ys) ** 2).mean().backward()       # bolmeyi unutursak
g_wrong = w.grad.item()
print(f"tam batch gradyani        : {g_full:12.6f}")
print(f"birikmis gradyan (dogru)  : {g_acc:12.6f}")
print(f"birikmis gradyan (yanlis) : {g_wrong:12.6f}")
```

PYTORCH ÇIKTISI

```
tam batch gradyani        :   -21.550000
birikmis gradyan (dogru)  :   -21.550000
birikmis gradyan (yanlis) :   -86.200000
```

NASIL ÇALIŞIYOR?

Tam batch gradyanı 8 örneğin ortalamasıdır. Dört mikro-batch'in her biri kendi 2 örneğinin ortalamasını verir; bunların ortalaması (yani toplamı 4'e bölünmüş hâli) tam batch ortalamasına eşittir. Bölmeyi unutursak gradyan 4 kat büyük çıkar ve etkin öğrenme oranı 4 katına çıkar; eğitim patlayabilir. Gerçek eğitimde etkin batch = mikro-batch × birikim adımı × GPU sayısıdır.

Gradient checkpointing: bellek karşılığı hesap

Geri yayılım için ileri geçişteki tüm ara aktivasyonları saklamamız gerekir; derin ağlarda bu bellek ana darboğazdır (aktivasyon belleği katman sayısıyla doğrusal büyür). **Checkpointing** yalnızca her k katmanda bir aktivasyonu saklar; geri geçişte bir segmente gelince o segmentin ara değerlerini son kontrol noktasından yeniden hesaplar. L katman için segmentleri √L seçersen bellek O(L)'den O(√L)'e düşer; bedel yaklaşık *bir ek ileri geçiş* (≈ %33 daha fazla hesap).

Örnek 5.8 — Aktivasyon yeniden hesaplama ile aynı gradyan

PROBLEM

64 katmanlı bir zincir $y_k = \tanh(w_k \cdot y_{k-1})$ kur. Gradyanları (1) tüm aktivasyonları saklayarak, (2) her 8 katmanda bir kontrol noktası saklayıp yeniden hesaplayarak bul. Sonuçların aynı olduğunu ve saklanan değer sayısını göster.

ÇÖZÜM (FORTRAN)

```
program checkpointing
  implicit none
  integer, parameter :: dp = kind(1.0d0), L = 64, seg = 8
  real(dp) :: w(L), y(0:L), dw1(L), dw2(L), cp(0:L/seg), ys(0:seg)
  real(dp) :: dy, dz, yy
  integer :: k, s, j, k0, nrecomp
  do k = 1, L
    w(k) = 0.9_dp + 0.2_dp * sin(real(k, dp))
  end do
```

```
  ! (1) hepsini sakla
  y(0) = 0.5_dp
  do k = 1, L
    y(k) = tanh(w(k) * y(k-1))
  end do
  dy = 1.0_dp
  do k = L, 1, -1
    dz = dy * (1.0_dp - y(k)**2)
    dw1(k) = dz * y(k-1)
    dy = dz * w(k)
  end do
  ! (2) yalniz kontrol noktalari
  yy = 0.5_dp; cp(0) = yy
  do k = 1, L
    yy = tanh(w(k) * yy)
    if (mod(k, seg) == 0) cp(k/seg) = yy
  end do
  dy = 1.0_dp; nrecomp = 0
  do s = L/seg, 1, -1
    k0 = (s-1) * seg
    ys(0) = cp(s-1)
    do j = 1, seg                                ! segmenti yeniden hesapla
      ys(j) = tanh(w(k0+j) * ys(j-1)); nrecomp = nrecomp + 1
    end do
    do j = seg, 1, -1                            ! segmentin geri gecisi
      k = k0 + j
      dz = dy * (1.0_dp - ys(j)**2)
      dw2(k) = dz * ys(j-1)
      dy = dz * w(k)
    end do
  end do
  print '(A,ES10.2)', 'max |dw1 - dw2|            = ', maxval(abs(dw1 - dw2))
  print '(A,I0)', 'normal : saklanan aktivasyon = ', L + 1
  print '(A,I0)', 'ckpt   : saklanan (en fazla) = ', L/seg + 1 + seg + 1
  print '(A,I0,A,I0,A)', 'ekstra ileri hesap = ', nrecomp, ' adim (normal ileri gecis: ', L, ')'
end program checkpointing
```

ÇIKTI

```
max |dw1 - dw2|            =   0.00E+00
normal : saklanan aktivasyon = 65
ckpt   : saklanan (en fazla) = 18
ekstra ileri hesap = 64 adim (normal ileri gecis: 64)
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
from torch.utils.checkpoint import checkpoint_sequential
torch.set_default_dtype(torch.float64)
L, seg = 64, 8
class Layer(nn.Module):
    def __init__(self, w):
        super().__init__(); self.w = nn.Parameter(torch.tensor(w))
    def forward(self, y): return torch.tanh(self.w * y)
ws = [0.9 + 0.2 * torch.sin(torch.tensor(float(k))).item() for k in range(1, L + 1)]
net = nn.Sequential(*[Layer(w) for w in ws])
y0 = torch.tensor(0.5)
net(y0).backward()                               # normal: tum aktivasyonlar saklanir
g1 = torch.stack([l.w.grad for l in net]).clone()
for p in net.parameters(): p.grad = None
out = checkpoint_sequential(net, seg, y0, use_reentrant=False)   # 8 parca, aktivasyonlar yeniden
      hesaplanir
out.backward()
g2 = torch.stack([l.w.grad for l in net])
print("max |dw1 - dw2|            =", (g1 - g2).abs().max().item())
print("normal : saklanan aktivasyon =", L + 1)
print("ckpt   : saklanan (en fazla) =", L // seg + 1 + seg + 1)
```

PYTORCH ÇIKTISI

```
max |dw1 - dw2|            = 0.0
normal : saklanan aktivasyon = 65
ckpt   : saklanan (en fazla) = 18
```

NASIL ÇALIŞIYOR?

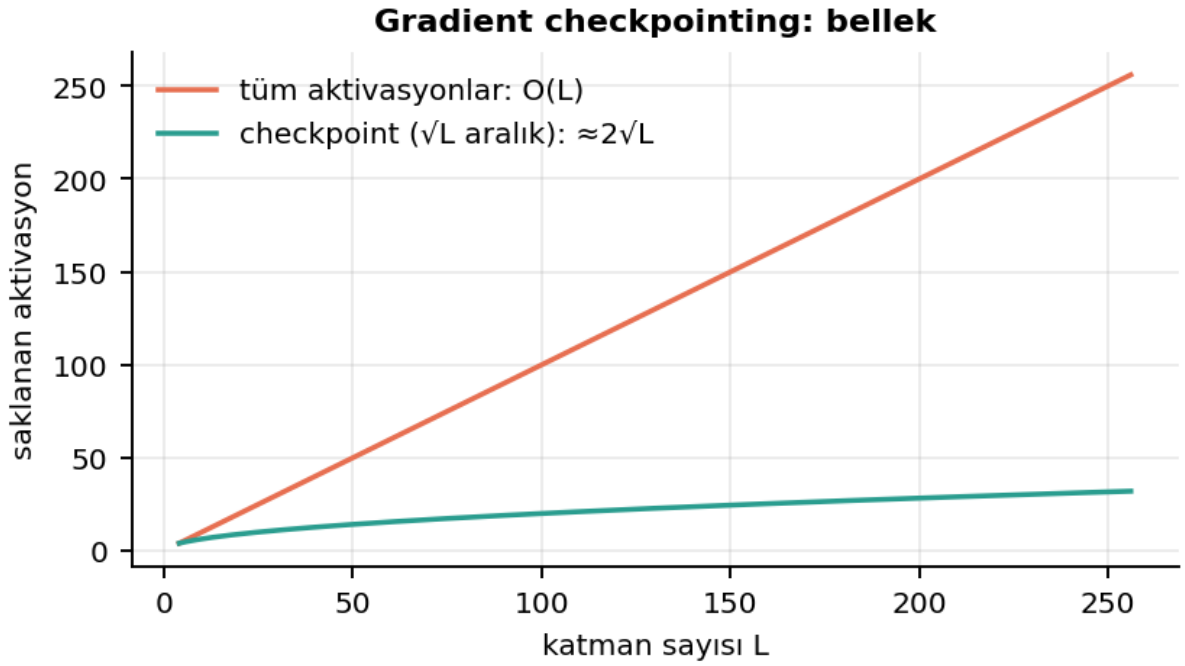
İki yöntemin gradyanları **birebir aynıdır**: checkpointing bir yaklaşıklık değil, aynı hesabı farklı sırayla yapmaktır. Normal yöntem 65 aktivasyon saklar; checkpoint yöntemi 9 kontrol noktası ile en fazla bir segmentin 9 aktivasyonunu aynı anda tutar (toplam 18). Bedel: 64 adımlık ileri hesabı bir kez daha yapmak. Bellek tasarrufu katman sayısıyla büyür: 100 katman için √100 = 10 ile tasarruf çok daha belirgindir. Büyük modellerde bu, "bellekte sığmıyor" ile "yavaş ama çalışıyor" arasındaki farktır.

Türetme: checkpoint için en iyi aralık

L katmanlı bir ağda her k katmanda bir kontrol noktası saklayalım. Bellek: L/k kontrol noktası + geri geçişte aynı anda tek bir segmentin k aktivasyonu:

$M(k) = L/k + k$, $dM/dk = -L/k^2 + 1 = 0 \Rightarrow k^* = \sqrt{L}$, $M^* = 2\sqrt{L}$

Hesap bedeli: her segment bir kez yeniden ileri hesaplanır, toplam bir ek ileri geçiş: ileri+geri maliyeti 3 birimden 4 birime çıkar (+%33). L = 64 için bellek 65 → 16, yani 4 kat tasarruf; L = 1000 için 1000 → 63 (16 kat).



Şekil 5.4 — Saklanan aktivasyon sayısı: tüm aktivasyonlar (L) ve √L aralıklı checkpointing (≈ 2√L). Fark katman sayısıyla büyür.

GPU programlama: bir bakış

GPU'da aynı matris çarpımı binlerce iş parçacığına dağıtılır: her iş parçacığı C'nin bir elemanını (veya küçük bir bloğunu) hesaplar. Fortran'da iki ana yol vardır: NVIDIA **CUDA Fortran** (nvfortran derleyicisi) veya standart `do concurrent` + OpenMP/OpenACC *offload*. Aşağıdaki kod yalnızca fikri gösterir ve bu kitabın örnekleri arasında GPU gerektiren **tek** koddur, bu yüzden çalıştırılmamıştır.

Örnek 5.9 — GPU çekirdeği: CUDA Fortran taslağı (çalıştırılmadı)

PROBLEM

C = A·B işlemini GPU'da, her iş parçacığı C'nin bir elemanını hesaplayacak şekilde yaz.

ÇÖZÜM (FORTRAN)

```
module gpu_kernels
  use cudafor
contains
  attributes(global) subroutine matmul_kernel(A, B, C, n)
    real(8), device :: A(n,n), B(n,n), C(n,n)
    integer, value :: n
    integer :: i, j, k
    real(8) :: s
    i = (blockIdx%x - 1) * blockDim%x + threadIdx%x      ! global satir
    j = (blockIdx%y - 1) * blockDim%y + threadIdx%y      ! global sutun
    if (i <= n .and. j <= n) then
      s = 0.0d0
      do k = 1, n
        s = s + A(i,k) * B(k,j)
      end do
      C(i,j) = s
    end if
  end subroutine matmul_kernel
end module gpu_kernels

! Ana programda (taslak):
!   real(8), device, allocatable :: dA(:,:), dB(:,:), dC(:,:)
!   dA = A; dB = B                                    ! ana bellek -> GPU
!   call matmul_kernel<<<dim3(n/16,n/16,1), dim3(16,16,1)>>>(dA, dB, dC, n)
!   C = dC                                            ! GPU -> ana bellek
```

PYTORCH KARŞILIĞI

```
import torch
dev = "cuda" if torch.cuda.is_available() else "cpu"
print("kullanilan cihaz:", dev)
n = 2048
A = torch.randn(n, n, device=dev); B = torch.randn(n, n, device=dev)
C = A @ B                                       # GPU'da cuBLAS, CPU'da BLAS cagrilir
if dev == "cuda":
    torch.cuda.synchronize()                    # asenkron calisma: olcmeden once bekle
print("C boyutu:", tuple(C.shape), " cihaz:", C.device)
# GPU icin tipik kullanim (GPU olmayan makinelerde calistirmayin):
#   model = model.to("cuda");  x = x.to("cuda")
#   with torch.autocast("cuda", dtype=torch.bfloat16): loss = model(x).mean()   # mixed precision
#   fast = torch.compile(model)                                                 # cekirdek birlestirme
```

PYTORCH ÇIKTISI

```
kullanilan cihaz: cpu
C boyutu: (2048, 2048)  cihaz: cpu
```

NASIL ÇALIŞIYOR?

Bu çekirdek 16×16 iş parçacığı bloklarıyla n×n iş parçacığı başlatır; her biri tek bir C(i,j) değerini hesaplar. Gerçek GPU çekirdekleri bunu paylaşılan bellek ve bloklama ile çok daha hızlı yapar. Önemli kavramlar: **host** (CPU) ve **device** (GPU) belleği ayrıdır ve aralarındaki veri aktarımı pahalıdır; hesabın aktarımı aşması gerekir. GPU'ya taşınabilir yazılım yazmak istiyorsan `do concurrent` ile derleyiciye "bu iterasyonlar bağımsız" demek en taşınabilir yoldur.

BÖLÜM 6

6. Tokenizer ve Metin İşleme

Model Türkçe kelimeleri değil, sayısal token kimliklerini işler. Metni sayıya, sayıyı tekrar metne çeviren boru hattını kuruyoruz.

Bu bölümde

- Unicode ve UTF-8: bayt nedir, karakter nedir?
- Metin normalizasyonu (ve Türkçe'nin I/ı, İ/i tuzağı)
- Byte Pair Encoding (BPE): vocabulary oluşturma, kodlama ve çözme
- Token ID'leri, embedding tabloları ve özel tokenlar (BOS, EOS, PAD)
- Türkçe eklemeli yapısının tokenizasyona etkisi

Hedef: Kendi BPE tokenizer'ını yazmak; Türkçe bir metni tokenlara ayırıp yeniden metne dönüştürmek.

Neden tokenizer?

Sinir ağı sayılarla çalışır. Metni sayıya çevirmenin üç yolu vardır: **karakter** düzeyi (kelime dağarcığı küçük ama diziler çok uzun), **kelime** düzeyi (diziler kısa ama dağarcık dev; bilinmeyen kelime sorunu var) ve ikisinin arası: **alt-kelime (subword)** düzeyi. Modern LLM'ler alt-kelime tokenizer'ları kullanır; sık kelimeler tek token, nadir kelimeler birkaç parça olur ve hiçbir metin "bilinmeyen" olmaz, çünkü en kötü ihtimalle bayt bayt kodlanır.

Unicode ve UTF-8

Bilgisayar metni **bayt** (0–255) dizisi olarak saklar. ASCII harfleri (a–z, 0–9) tek bayttır; ama Türkçe harfler (ç, ş, ğ, ı, ö, ü, İ) UTF-8'de **2 bayt** yer kaplar. Yani "karakter sayısı" ile "bayt sayısı" aynı değildir. UTF-8 kuralı basittir:

Kod noktası aralığı	Bayt sayısı	Bit kalıbı
U+0000 – U+007F	1	0xxxxxxx
U+0080 – U+07FF	2	110xxxxx 10xxxxxx
U+0800 – U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 – U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Devam baytları her zaman `10` ile başlar; bu sayede bir karakterin nerede başladığını kolayca anlarız.

Örnek 6.1 — Bir metnin baytları ve karakter sayısı

PROBLEM

"Türkçe şiir" metninin her baytını (onaltılık ve onluk) yazdır; bayt sayısını ve karakter sayısını hesapla.

ÇÖZÜM (FORTRAN)

```
program utf8_baytlar
  implicit none
  character(len=*), parameter :: s = 'Türkçe şiir'
  integer :: i, nb, nc, b
  nb = len(s)
  nc = 0
  print '(A,A)', 'metin : ', s
  do i = 1, nb
    b = ichar(s(i:i))
```

```
    if (iand(b, 192) /= 128) nc = nc + 1          ! devam bayti (10xxxxxx) sayma
    write(*,'(A,I2,A,Z2.2,A,I3)') 'bayt ', i, '   hex=', b, '   dec=', b
  end do
  print '(A,I0)', 'bayt sayisi     = ', nb
  print '(A,I0)', 'karakter sayisi = ', nc
end program utf8_baytlar
```

ÇIKTI

```
metin : Türkçe şiir
bayt  1   hex=54   dec= 84
bayt  2   hex=C3   dec=195
bayt  3   hex=BC   dec=188
bayt  4   hex=72   dec=114
bayt  5   hex=6B   dec=107
bayt  6   hex=C3   dec=195
bayt  7   hex=A7   dec=167
bayt  8   hex=65   dec=101
bayt  9   hex=20   dec= 32
bayt 10   hex=C5   dec=197
bayt 11   hex=9F   dec=159
bayt 12   hex=69   dec=105
bayt 13   hex=69   dec=105
bayt 14   hex=72   dec=114
bayt sayisi     = 14
karakter sayisi = 11
```

PYTORCH KARŞILIĞI

```
s = "Türkçe şiir"
b = s.encode("utf-8")                      # metin -> bayt dizisi
print("metin :", s)
for i, v in enumerate(b, 1):
    print(f"bayt {i:2d}   hex={v:02X}   dec={v:3d}")
print("bayt sayisi     =", len(b))
print("karakter sayisi =", len(s))
print("devam bayti olmayanlar =", sum(1 for v in b if (v & 0xC0) != 0x80))
```

PYTORCH ÇIKTISI

```
metin : Türkçe şiir
bayt  1   hex=54   dec= 84
bayt  2   hex=C3   dec=195
bayt  3   hex=BC   dec=188
bayt  4   hex=72   dec=114
bayt  5   hex=6B   dec=107
bayt  6   hex=C3   dec=195
bayt  7   hex=A7   dec=167
bayt  8   hex=65   dec=101
bayt  9   hex=20   dec= 32
bayt 10   hex=C5   dec=197
bayt 11   hex=9F   dec=159
bayt 12   hex=69   dec=105
bayt 13   hex=69   dec=105
bayt 14   hex=72   dec=114
bayt sayisi     = 14
karakter sayisi = 11
devam bayti olmayanlar = 11
```

NASIL ÇALIŞIYOR?

`len(s)` karakter değil **bayt** sayar. ü, ç, ş harflerinin her biri iki bayttır: örneğin ü = C3 BC. Karakter sayısını bulmak için "devam baytı" (üst iki biti 10 olan bayt, yani `iand(b,192) == 128`) olmayanları sayarız. Sonuç 11 karakter, 14 bayttır. Tokenizer'lar çoğunlukla *bayt* düzeyinde çalışır; böylece her olası metin kodlanabilir.

Örnek 6.2 — UTF-8 çözücü: baytlardan kod noktalarına

PROBLEM

Bir UTF-8 metnini elle çöz: her karakterin Unicode kod noktasını ve bayt uzunluğunu bul. Test metni: "İstanbul ığ" ve elle kurulan 4 baytlık bir dizi (U+1F600).

ÇÖZÜM (FORTRAN)

```
program utf8_coz
  implicit none
  character(len=:), allocatable :: s
  integer :: i, n, b, cp, nbytes, k
  s = 'İstanbul ığ ' // char(240) // char(159) // char(152) // char(128)
  n = len(s)
  i = 1
  do while (i <= n)
    b = ichar(s(i:i))
    if (b < 128) then
      cp = b;                                   nbytes = 1
    else if (b < 224) then
      cp = iand(b, 31);  nbytes = 2
    else if (b < 240) then
      cp = iand(b, 15);  nbytes = 3
    else
      cp = iand(b, 7);   nbytes = 4
    end if
    do k = 1, nbytes - 1                        ! devam baytlari: alt 6 biti ekle
      cp = cp * 64 + iand(ichar(s(i+k:i+k)), 63)
    end do
    write(*,'(A,Z6.6,A,I1,A)') 'U+', cp, '  (', nbytes, ' bayt)'
    i = i + nbytes
  end do
end program utf8_coz
```

ÇIKTI

```
U+000130  (2 bayt)
U+000073  (1 bayt)
U+000074  (1 bayt)
U+000061  (1 bayt)
U+00006E  (1 bayt)
U+000062  (1 bayt)
U+000075  (1 bayt)
U+00006C  (1 bayt)
U+000020  (1 bayt)
U+000131  (2 bayt)
U+00011F  (2 bayt)
U+000020  (1 bayt)
U+01F600  (4 bayt)
```

PYTORCH KARŞILIĞI

```
s = "İstanbul ığ " + chr(0x1F600)
b = s.encode("utf-8"); i = 0
while i < len(b):
    v = b[i]
    n = 1 if v < 128 else 2 if v < 224 else 3 if v < 240 else 4
    cp = v if n == 1 else v & (0x1F if n == 2 else 0x0F if n == 3 else 0x07)
    for k in range(1, n):
        cp = cp * 64 + (b[i + k] & 63)
    print(f"U+{cp:06X}  ({n} bayt)")
    i += n
print("ord() ile karsilastir:", [f"U+{ord(c):06X}" for c in s][:4], "...")
```

PYTORCH ÇIKTISI

```
U+000130  (2 bayt)
U+000073  (1 bayt)
U+000074  (1 bayt)
U+000061  (1 bayt)
U+00006E  (1 bayt)
U+000062  (1 bayt)
U+000075  (1 bayt)
U+00006C  (1 bayt)
U+000020  (1 bayt)
U+000131  (2 bayt)
U+00011F  (2 bayt)
U+000020  (1 bayt)
U+01F600  (4 bayt)
ord() ile karsilastir: ['U+000130', 'U+000073', 'U+000074', 'U+000061'] ...
```

NASIL ÇALIŞIYOR?

İlk baytın üst bitleri uzunluğu belirler: `0xxxxxxx` → 1 bayt, `110xxxxx` → 2 bayt, `1110xxxx` → 3 bayt, `11110xxx` → 4 bayt. Kod noktası, ilk baytın kalan bitleri ile devam baytlarının alt 6 bitlerinin art arda eklenmesiyle bulunur (her adımda 64 ile çarpıp ekleriz). "İ" için U+0130, "ı" için U+0131, "ğ" gibi harfler için 2 baytlık değerler çıkar; son satır 4 baytlık U+1F600'dür.



Şekil 6.1 — UTF-8 kodlama: ASCII harfler 1 bayt; Türkçe ü, ş, ç gibi harfler 2 bayt; € 3 bayt; emoji 4 bayt. Tokenizer'lar bu baytlar üzerinde çalışır.

Metin normalizasyonu: Türkçe'nin tuzağı

Tokenizer'dan önce metni tutarlı hâle getiririz: küçük harfe çevirme, fazla boşlukları temizleme, Unicode normalleştirme (NFC) vb. Burada Türkçe'ye özgü klasik bir hata vardır: İngilizce kurallarıyla `I → i` yaparsak "ISPARTA" "isparta" olur; oysa Türkçede **I → ı** ve **İ → i** olmalıdır.

Örnek 6.3 — Türkçeye uygun küçük harf dönüşümü

PROBLEM

"ISTANBUL İzmir ÇAY Şeker" metnini önce saf ASCII kuralıyla, sonra Türkçe kurallarıyla küçük harfe çevir ve fazla boşlukları tek boşluğa indir.

ÇÖZÜM (FORTRAN)

```
program normalizasyon
  implicit none
  character(len=:), allocatable :: s, naif, tr
  integer :: i, c
  s = 'ISTANBUL  İzmir   ÇAY  Şeker'
  naif = s
  do i = 1, len(naif)                                ! yalniz ASCII A-Z -> a-z
    c = ichar(naif(i:i))
    if (c >= 65 .and. c <= 90) naif(i:i) = char(c + 32)
  end do
  tr = tr_lower(s)
  do while (index(tr, '  ') > 0)                     ! bosluklari tekle
    tr = replace_all(tr, '  ', ' ')
  end do
```

```
  print '(A,A)', 'girdi          : ', s
  print '(A,A)', 'naif (ASCII)   : ', naif
  print '(A,A)', 'Turkce kurali  : ', tr
contains
  function tr_lower(x) result(r)
    character(len=*), intent(in) :: x
    character(len=:), allocatable :: r
    integer :: i, c
    r = replace_all(x, 'İ', 'i')
    r = replace_all(r, 'I', 'ı')
    r = replace_all(r, 'Ç', 'ç')
    r = replace_all(r, 'Ğ', 'ğ')
    r = replace_all(r, 'Ö', 'ö')
    r = replace_all(r, 'Ş', 'ş')
    r = replace_all(r, 'Ü', 'ü')
    do i = 1, len(r)                                  ! kalan ASCII buyuk harfler
      c = ichar(r(i:i))
      if (c >= 65 .and. c <= 90) r(i:i) = char(c + 32)
    end do
  end function tr_lower

  function replace_all(x, a, b) result(r)
    character(len=*), intent(in) :: x, a, b
    character(len=:), allocatable :: r
    integer :: p, i
    r = ''
    i = 1
    do
      p = index(x(i:), a)
      if (p == 0) then
        r = r // x(i:)
        exit
      end if
      r = r // x(i:i+p-2) // b
      i = i + p - 1 + len(a)
    end do
  end function replace_all
end program normalizasyon
```

ÇIKTI

```
girdi          : ISTANBUL  İzmir   ÇAY  Şeker
naif (ASCII)   : istanbul  İzmir   Çay  Şeker
Turkce kurali  : ıstanbul izmir çay şeker
```

PYTORCH KARŞILIĞI

```
import re, unicodedata
s = "ISTANBUL  İzmir   ÇAY  Şeker"
print("girdi          :", s)
print("naif .lower()  :", s.lower())          # Python 'I' -> 'i' yapar, 'İ' -> 'i̇' (birlesik nokta!)
def tr_lower(x):
    x = x.replace("İ", "i").replace("I", "ı")
    return x.lower()
t = re.sub(r"\s+", " ", tr_lower(s))
print("Turkce kurali  :", t)
print("NFC normalize  :", unicodedata.normalize("NFC", "i\u0307") == "\u0130".lower())
```

PYTORCH ÇIKTISI

```
girdi          : ISTANBUL  İzmir   ÇAY  Şeker
naif .lower()  : istanbul  i̇zmir   çay  şeker
Turkce kurali  : ıstanbul izmir çay şeker
NFC normalize  : True
```

NASIL ÇALIŞIYOR?

Saf ASCII yaklaşımı "ISTANBUL"u "istanbul" yapar (yanlış, olması gereken "ıstanbul"), "İzmir"deki İ'ye hiç dokunmaz ve Ç, Ş gibi harfleri büyük bırakır; aynı kelimenin farklı yazılışları farklı token'lara dönüşür. Türkçe kurallı sürüm önce Türkçe harfleri çiftler hâlinde değiştirir (UTF-8 bayt dizileri olarak), sonra kalan ASCII harfleri küçültür. `replace_all` işlevi `index` ile deseni bulup parçaları birleştirir; bu kalıbı veri temizliğinde de kullanacağız. Pratikte birçok modern tokenizer küçük harfe

hiç çevirmez; ama normalizasyon kararı *bilinçli* verilmelidir.

Byte Pair Encoding (BPE)

BPE basit ve güçlü bir fikirdir. Başlangıç vocabulary'si 256 bayttır. Sonra tekrar tekrar:

- Eğitim metninde **en sık yan yana gelen iki token** çiftini bul.
- Bu çifti **yeni bir token** olarak vocabulary'ye ekle ve metindeki tüm geçişlerini bu yeni token ile değiştir.
- İstediğin vocabulary boyutuna ulaşana kadar tekrarla. Birleştirme sırası kaydedilir.

Kodlama: yeni metne aynı birleştirmeleri *aynı sırayla* uygularsın. **Çözme**: her token'ın bayt dizisini art arda yazarsın. Böylece sık geçen parçalar (örneğin "ler", "lar", "den") tek token olur; nadir kelimeler birkaç parçaya bölünür.

Örnek 6.4 — Kendi BPE tokenizer'ım: eğitim, kodlama, çözme

PROBLEM

Küçük bir Türkçe kelime koleksiyonu üzerinde 24 birleştirme öğren. Her adımda birleşen çifti ve sıkıştırma oranını yazdır. Sonra yeni kelimeleri kodla, tekrar çöz ve orijinal metne eşit olduğunu doğrula.

ÇÖZÜM (FORTRAN)

```
program bpe_egitim
  implicit none
  integer, parameter :: maxv = 400, maxn = 2000, nmerge = 24
  character(len=*), parameter :: corpus = &
    'araba arabalar arabalarda kalem kalemler kalemlerden evde evden evlerden ' // &
    'gelmek gelecek gelenler gelenlerden kitap kitaplar kitaplarda kitaplardan '
  integer :: ids(maxn), n, n0, cnt(0:maxv-1, 0:maxv-1), mA(nmerge), mB(nmerge)
  integer :: nid, nm, m, i, j, ba, bb, best, tl(0:maxv-1), o(200), no, q
  character(len=24) :: tok(0:maxv-1)
  character(len=:), allocatable :: line, back
  do i = 0, 255
    tok(i) = char(i); tl(i) = 1
  end do
  n = len(corpus); n0 = n
  do i = 1, n
    ids(i) = ichar(corpus(i:i))
  end do
  nid = 256; nm = 0
  do m = 1, nmerge
    cnt = 0
    do i = 1, n - 1
      cnt(ids(i), ids(i+1)) = cnt(ids(i), ids(i+1)) + 1
    end do
    best = 0; ba = 0; bb = 0
    do i = 0, nid - 1
      do j = 0, nid - 1
        if (cnt(i,j) > best) then
          best = cnt(i,j); ba = i; bb = j
        end if
      end do
    end do
    if (best < 2) exit
    tok(nid) = tok(ba)(1:tl(ba)) // tok(bb)(1:tl(bb))
    tl(nid) = tl(ba) + tl(bb)
    j = 0; i = 1
    do while (i <= n)                                  ! cifti yeni tokenla degistir
      if (i < n) then
        if (ids(i) == ba .and. ids(i+1) == bb) then
          j = j + 1; ids(j) = nid; i = i + 2
          cycle
        end if
      end if
      j = j + 1; ids(j) = ids(i); i = i + 1
    end do
    n = j
```

```
    nm = nm + 1; mA(nm) = ba; mB(nm) = bb; nid = nid + 1
    line = '[' // show(ba) // '] + [' // show(bb) // '] -> [' // show(nid-1) // ']'
    write(*,'(I3,2X,A,T34,A,I2,A,I4,A,F5.2)') m, line, 'sayi=', best, '  dizi=', n, &
          '  bayt/token=', real(n0)/real(n)
  end do
  print '(A)', '--- yeni kelimeleri kodla ---'
  call pieces('kalemlerde')
  call pieces('arabalardan')
  call pieces('kitaplarda')
  call pieces('evlerde')
  call pieces('gelenlerde')
contains
  function show(id) result(r)
    integer, intent(in) :: id
    character(len=:), allocatable :: r
    integer :: p
    r = tok(id)(1:tl(id))
    do p = 1, len(r)
      if (r(p:p) == ' ') r(p:p) = '_'
    end do
  end function show

  subroutine encode(text, out, nout)
    character(len=*), intent(in) :: text
    integer, intent(out) :: out(:), nout
    integer :: p, mm, i2, j2
    nout = len(text)
    do p = 1, nout
      out(p) = ichar(text(p:p))
    end do
    do mm = 1, nm                                      ! birlestirmeleri sirayla uygula
      j2 = 0; i2 = 1
      do while (i2 <= nout)
        if (i2 < nout) then
          if (out(i2) == mA(mm) .and. out(i2+1) == mB(mm)) then
            j2 = j2 + 1; out(j2) = 255 + mm; i2 = i2 + 2
            cycle
          end if
        end if
        j2 = j2 + 1; out(j2) = out(i2); i2 = i2 + 1
      end do
      nout = j2
    end do
  end subroutine encode

  subroutine pieces(text)
    character(len=*), intent(in) :: text
    character(len=:), allocatable :: lin
    call encode(text, o, no)
    lin = ''; back = ''
    do q = 1, no
      lin = lin // show(o(q))
      if (q < no) lin = lin // '|'
      back = back // tok(o(q))(1:tl(o(q)))             ! cozme: baytlari birlestir
    end do
    write(*,'(A,A,A,A,I2,A,L1)') text, ' -> ', lin, '   token=', no, '   geri-cozme dogru=', &
          back == text
  end subroutine pieces
end program bpe_egitim
```

ÇIKTI

```
   1  [l] + [e] -> [le]           sayi=11  dizi= 136  bayt/token= 1.08
   2  [a] + [r] -> [ar]           sayi= 8  dizi= 128  bayt/token= 1.15
   3  [_] + [k] -> [_k]           sayi= 7  dizi= 121  bayt/token= 1.21
   4  [d] + [e] -> [de]           sayi= 5  dizi= 116  bayt/token= 1.27
   5  [l] + [ar] -> [lar]         sayi= 5  dizi= 111  bayt/token= 1.32
   6  [le] + [r] -> [ler]         sayi= 5  dizi= 106  bayt/token= 1.39
   7  [_] + [g] -> [_g]           sayi= 4  dizi= 102  bayt/token= 1.44
   8  [a] + [p] -> [ap]           sayi= 4  dizi=  98  bayt/token= 1.50
   9  [i] + [t] -> [it]           sayi= 4  dizi=  94  bayt/token= 1.56
  10  [_k] + [it] -> [_kit]       sayi= 4  dizi=  90  bayt/token= 1.63
  11  [de] + [n] -> [den]         sayi= 4  dizi=  86  bayt/token= 1.71
  12  [_g] + [e] -> [_ge]         sayi= 4  dizi=  82  bayt/token= 1.79
  13  [_kit] + [ap] -> [_kitap]   sayi= 4  dizi=  78  bayt/token= 1.88
  14  [_] + [e] -> [_e]           sayi= 3  dizi=  75  bayt/token= 1.96
  15  [a] + [b] -> [ab]           sayi= 3  dizi=  72  bayt/token= 2.04
  16  [a] + [le] -> [ale]         sayi= 3  dizi=  69  bayt/token= 2.13
  17  [d] + [a] -> [da]           sayi= 3  dizi=  66  bayt/token= 2.23
  18  [ar] + [ab] -> [arab]       sayi= 3  dizi=  63  bayt/token= 2.33
  19  [_k] + [ale] -> [_kale]     sayi= 3  dizi=  60  bayt/token= 2.45
  20  [lar] + [da] -> [larda]     sayi= 3  dizi=  57  bayt/token= 2.58
  21  [ler] + [den] -> [lerden]   sayi= 3  dizi=  54  bayt/token= 2.72
  22  [_ge] + [le] -> [_gele]     sayi= 3  dizi=  51  bayt/token= 2.88
  23  [_e] + [v] -> [_ev]         sayi= 3  dizi=  48  bayt/token= 3.06
  24  [arab] + [a] -> [araba]     sayi= 3  dizi=  45  bayt/token= 3.27
--- yeni kelimeleri kodla ---
kalemlerde -> k|ale|m|ler|de   token= 5   geri-cozme dogru=T
arabalardan -> araba|larda|n   token= 3   geri-cozme dogru=T
kitaplarda -> k|it|ap|larda   token= 4   geri-cozme dogru=T
evlerde -> e|v|ler|de   token= 4   geri-cozme dogru=T
gelenlerde -> g|e|le|n|ler|de   token= 6   geri-cozme dogru=T
```

PYTORCH KARŞILIĞI

```
from collections import Counter
corpus = ("araba arabalar arabalarda kalem kalemler kalemlerden evde evden evlerden "
          "gelmek gelecek gelenler gelenlerden kitap kitaplar kitaplarda kitaplardan ")
ids = list(corpus.encode()); tok = {i: bytes([i]) for i in range(256)}
merges = []; n0 = len(ids)
show = lambda t: tok[t].decode().replace(" ", "_")
for m in range(1, 25):
    pairs = Counter(zip(ids, ids[1:]))
    (a, b), cnt = max(sorted(pairs.items()), key=lambda kv: kv[1])   # esitlikte ilk (Fortran ile ayni)
    if cnt < 2: break
    new = 255 + m; tok[new] = tok[a] + tok[b]; merges.append((a, b, new))
    out, i = [], 0
    while i < len(ids):
        if i < len(ids) - 1 and (ids[i], ids[i + 1]) == (a, b): out.append(new); i += 2
        else: out.append(ids[i]); i += 1
    ids = out
    print(f"{m:3d}  [{show(a)}] + [{show(b)}] -> [{show(new)}]".ljust(34), f"sayi={cnt:2d}
          dizi={len(ids):4d}  bayt/token={n0/len(ids):5.2f}")
def encode(text):
    x = list(text.encode())
    for a, b, new in merges:
        out, i = [], 0
        while i < len(x):
            if i < len(x) - 1 and (x[i], x[i + 1]) == (a, b): out.append(new); i += 2
            else: out.append(x[i]); i += 1
        x = out
    return x
print("--- yeni kelimeleri kodla ---")
for w in ["kalemlerde", "arabalardan", "kitaplarda", "evlerde", "gelenlerde"]:
    t = encode(w); back = b"".join(tok[i] for i in t).decode()
    print(f"{w} -> {'|'.join(show(i) for i in t)}   token={len(t)}   geri-cozme dogru={back == w}")
```

PYTORCH ÇIKTISI

```
   1  [l] + [e] -> [le]             sayi=11  dizi= 136  bayt/token= 1.08
   2  [a] + [r] -> [ar]             sayi= 8  dizi= 128  bayt/token= 1.15
   3  [_] + [k] -> [_k]             sayi= 7  dizi= 121  bayt/token= 1.21
   4  [d] + [e] -> [de]             sayi= 5  dizi= 116  bayt/token= 1.27
   5  [l] + [ar] -> [lar]           sayi= 5  dizi= 111  bayt/token= 1.32
   6  [le] + [r] -> [ler]           sayi= 5  dizi= 106  bayt/token= 1.39
   7  [_] + [g] -> [_g]             sayi= 4  dizi= 102  bayt/token= 1.44
   8  [a] + [p] -> [ap]             sayi= 4  dizi=  98  bayt/token= 1.50
   9  [i] + [t] -> [it]             sayi= 4  dizi=  94  bayt/token= 1.56
  10  [_k] + [it] -> [_kit]         sayi= 4  dizi=  90  bayt/token= 1.63
  11  [de] + [n] -> [den]           sayi= 4  dizi=  86  bayt/token= 1.71
  12  [_g] + [e] -> [_ge]           sayi= 4  dizi=  82  bayt/token= 1.79
  13  [_kit] + [ap] -> [_kitap]     sayi= 4  dizi=  78  bayt/token= 1.88
  14  [_] + [e] -> [_e]             sayi= 3  dizi=  75  bayt/token= 1.96
  15  [a] + [b] -> [ab]             sayi= 3  dizi=  72  bayt/token= 2.04
  16  [a] + [le] -> [ale]           sayi= 3  dizi=  69  bayt/token= 2.13
  17  [d] + [a] -> [da]             sayi= 3  dizi=  66  bayt/token= 2.23
  18  [ar] + [ab] -> [arab]         sayi= 3  dizi=  63  bayt/token= 2.33
  19  [_k] + [ale] -> [_kale]       sayi= 3  dizi=  60  bayt/token= 2.45
  20  [lar] + [da] -> [larda]       sayi= 3  dizi=  57  bayt/token= 2.58
  21  [ler] + [den] -> [lerden]     sayi= 3  dizi=  54  bayt/token= 2.72
  22  [_ge] + [le] -> [_gele]       sayi= 3  dizi=  51  bayt/token= 2.88
  23  [_e] + [v] -> [_ev]           sayi= 3  dizi=  48  bayt/token= 3.06
  24  [arab] + [a] -> [araba]       sayi= 3  dizi=  45  bayt/token= 3.27
--- yeni kelimeleri kodla ---
kalemlerde -> k|ale|m|ler|de   token=5   geri-cozme dogru=True
arabalardan -> araba|larda|n   token=3   geri-cozme dogru=True
kitaplarda -> k|it|ap|larda   token=4   geri-cozme dogru=True
evlerde -> e|v|ler|de   token=4   geri-cozme dogru=True
gelenlerde -> g|e|le|n|ler|de   token=6   geri-cozme dogru=True
```

NASIL ÇALIŞIYOR?

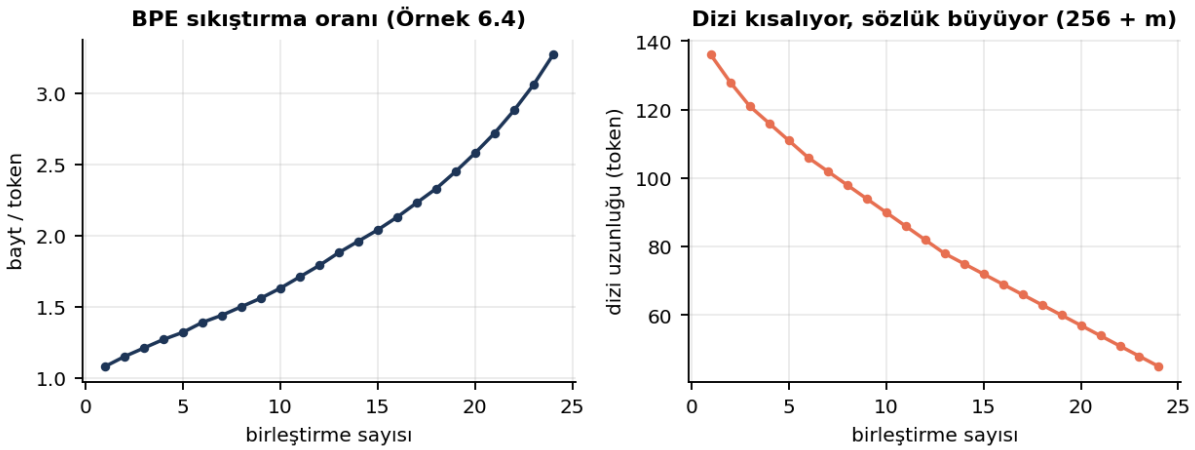
Eğitim: her turda tüm komşu çiftlerin sayısını `cnt` tablosunda sayıp en büyüğünü seçiyoruz, sonra diziyi tek geçişte yeni token ile yeniden yazıyoruz. Her satırdaki "bayt/token" sıkıştırma oranıdır: birleşme yaptıkça metni daha az token ile ifade ederiz. İlk birleştirmeler en sık geçen "le", "ar", "de" gibi parçalardır (alt çizgi boşluğu gösterir: "_k" kelime başındaki k demektir); ardından "lar", "ler", "den", "larda", "lerden" gibi **ekler** ve "_kitap", "araba" gibi kökler gelir. Bu, Türkçenin eklemeli yapısını tokenizer'ın istatistikle keşfetmesidir.

Kodlama: yeni kelimeler aynı birleştirme sırasıyla işlenir. Eğitimde hiç görmediğimiz "kalemlerde" gibi bir kelime bile bilinen parçalara ayrılır (ler, de gibi ekler ayrı token olur); çok küçük bir korpus kök bölmelerini kusurlu bırakır ("k|ale|m"), gerçek tokenizer'lar milyarlarca karakterle eğitildiği için kökleri de bütün tutar. **Çözme**: parçaların baytları birleştirilince orijinal metin *kayıpsız* geri gelir (son sütun hep T). Bu, BPE'nin en önemli garantisidir.

Türetme: BPE bir sıkıştırma algoritmasıdır

BPE ve sıkıştırma. BPE, açgözlü (greedy) bir sıkıştırma algoritmasıdır: her adımda en sık çifti tek sembole indirerek eğitim metninin uzunluğunu $|D| = \Sigma$ sayı'dan azaltır. m birleştirme sonra dizi uzunluğu $N_m = N_0 - \Sigma_{j \le m} c_j$, c_j o adımdaki çift sayısı (her birleştirme c_j token tasarruf eder). Sıkıştırma oranı N_0/N_m tokenizer kalitesinin ölçüsüdür. c_j azalan bir dizidir (Zipf yasası): ilk birleştirmeler çok kazandırır, sonrakiler giderek az.

Türkçe neden zordur? Eklemeli dillerde bir kök binlerce ek kombinasyonuyla yazılır (ev-ler-im-iz-den). Kelime düzeyi sözlük patlar; İngilizce eğitilmiş tokenizer Türkçeyi çok parçaya böler ("verimlilik" metrik: token/kelime ≈ 1.3 İngilizce, 2–3 Türkçe). Sonuç: aynı bağlam penceresine daha az Türkçe metin sığar ve aynı metin daha pahalıya işlenir. Türkçe ağırlıklı korpusla eğitilmiş tokenizer bu farkı kapatır.



Şekil 6.2 — BPE eğitiminde birleştirme sayısı arttıkça bayt/token sıkıştırma oranı (solda) ve dizi uzunluğu (sağda); kazanç azalan getirilidir.

Örnek 6.5 — Embedding tablosu, özel tokenlar ve padding

PROBLEM

Küçük bir kelime-parça sözlüğünde <PAD>, <BOS>, <EOS> özel tokenlarını tanımla. 3 dizilik bir batch'i en uzununa göre PAD ile doldur, maske üret, embedding ara ve PAD'leri yok sayarak ortalama al.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program ozel_tokenlar
  use rng_mod
  implicit none
  integer, parameter :: V = 10, d = 4, B = 3, T = 5
  character(len=6) :: vocab(0:V-1)
  integer :: ids(B,T), b1, t1, cnt
  logical :: mask(B,T)
  real(dp) :: E(V,d), pooled(d)
  vocab = [character(len=6) :: '<PAD>', '<BOS>', '<EOS>', 'ev', 'ler', 'de', &
                               'kitap', 'oku', 'du', 'm']
  ! diziler: [BOS ev ler de EOS], [BOS kitap EOS], [BOS oku du m EOS]
  ids = reshape([1,3,4,5,2,  1,6,2,0,0,  1,7,8,9,2], [B,T], order=[2,1])
  mask = (ids /= 0)                                ! PAD olmayan konumlar
  call rng_seed(3)
  call randn2(E, 1.0_dp)
  E(1,:) = 0.0_dp                                  ! PAD embedding'i sifir (padding_idx)
  do b1 = 1, B
    write(*,'(A,I1,A)', advance='no') 'dizi ', b1, ': '
    do t1 = 1, T
      if (mask(b1,t1)) write(*,'(A,A)', advance='no') trim(vocab(ids(b1,t1))), ' '
    end do
    print *
  end do
  print '(A)', 'ids (0 = PAD):'
  do b1 = 1, B
    print '(5I4)', ids(b1,:)
  end do
  print '(A)', 'maske:'
  do b1 = 1, B
    print '(5L4)', mask(b1,:)
  end do
  do b1 = 1, B                                     ! maskeli ortalama (mean pooling)
    pooled = 0.0_dp; cnt = 0
    do t1 = 1, T
      if (mask(b1,t1)) then
        pooled = pooled + E(ids(b1,t1)+1, :)
        cnt = cnt + 1
      end if
    end do
    print '(A,I1,A,I1,A,4F8.3)', 'dizi ', b1, ' (', cnt, ' token) ort: ', pooled / cnt
  end do
end program ozel_tokenlar
```

ÇIKTI

```
dizi 1: <BOS> ev ler de <EOS>
dizi 2: <BOS> kitap <EOS>
dizi 3: <BOS> oku du m <EOS>
ids (0 = PAD):
   1   3   4   5   2
   1   6   2   0   0
   1   7   8   9   2
maske:
   T   T   T   T   T
   T   T   T   F   F
   T   T   T   T   T
dizi 1 (5 token) ort:   -0.733   0.602   0.060  -0.431
dizi 2 (3 token) ort:   -0.730   0.393   0.323   0.272
dizi 3 (5 token) ort:   -0.201   0.106   0.306   0.456
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
from torch.nn.utils.rnn import pad_sequence
torch.manual_seed(3)
vocab = ["<PAD>", "<BOS>", "<EOS>", "ev", "ler", "de", "kitap", "oku", "du", "m"]
seqs = [torch.tensor(s) for s in ([1, 3, 4, 5, 2], [1, 6, 2], [1, 7, 8, 9, 2])]
ids = pad_sequence(seqs, batch_first=True, padding_value=0)     # PAD ile esitle
mask = ids != 0
E = nn.Embedding(len(vocab), 4, padding_idx=0)                  # PAD satiri sifir ve egitilmez
for b in range(3): print(f"dizi {b+1}:", " ".join(vocab[i] for i in seqs[b]))
print("ids (0 = PAD):"); print(ids)
print("maske:"); print(mask)
X = E(ids)                                                       # (B, T, d)
pooled = (X * mask.unsqueeze(-1)).sum(1) / mask.sum(1, keepdim=True)
for b in range(3):
    print(f"dizi {b+1} ({mask[b].sum().item()} token) ort:", [round(v, 3) for v in pooled[b].tolist()])
```

PYTORCH ÇIKTISI

```
dizi 1: <BOS> ev ler de <EOS>
dizi 2: <BOS> kitap <EOS>
dizi 3: <BOS> oku du m <EOS>
ids (0 = PAD):
tensor([[1, 3, 4, 5, 2],
        [1, 6, 2, 0, 0],
        [1, 7, 8, 9, 2]])
maske:
tensor([[ True,  True,  True,  True,  True],
        [ True,  True,  True, False, False],
        [ True,  True,  True,  True,  True]])
dizi 1 (5 token) ort: [0.567, 0.213, 0.352, -0.6]
dizi 2 (3 token) ort: [-0.194, 0.145, 0.647, -0.668]
dizi 3 (5 token) ort: [0.077, -0.056, 0.103, -0.522]
```

NASIL ÇALIŞIYOR?

BOS (başlangıç) ve **EOS** (bitiş) tokenları modele dizinin sınırlarını öğretir; EOS, üretimde "dur" sinyalidir. **PAD** yalnızca batch'teki dizileri eşit uzunluğa getirmek için kullanılır ve hiçbir zaman anlam taşımaz: bu yüzden *maske* ile hesaptan çıkarılır (kayıp hesabında ve attention'da). Burada ortalama alırken maskeli konumları atlayıp gerçek token sayısına böldük; PAD'lere göre bölseydik kısa diziler yapay olarak küçülürdü. Embedding tablosunun satırı token kimliğinden bir fazladır çünkü Fortran indeksleri 1'den başlar.

BÖLÜM 7

7. Veri Mühendisliği

Büyük bir modelin başarısını mimarisi kadar eğitim verisinin kalitesi belirler. Ham metni eğitime hazır, temiz ve güvenli veriye dönüştürüyoruz.

Bu bölümde

- Web verisi ve metin dosyalarını işleme: HTML ayıklama, temizleme
- Büyük dosyaları belleğe yüklemeden okuma (akış / stream)
- Tekrarlı belgeleri bulma (deduplication): birebir ve yaklaşık
- Eğitim/doğrulama/test kümeleri ve veri sızıntısı (data leakage)
- Token tabanlı veri paketleme ve ikili (binary) dosya biçimi
- Telif, gizlilik ve veri lisansları

Hedef: Milyonlarca token içeren bir veri kümesini parçalara ayıran, temizleyen ve eğitim için ikili dosya biçiminde kaydeden bir sistemin temel parçalarını yazmak.

Veri hattı (pipeline) genel görünümü

ham veri → ayıklama → temizleme/filtreleme → tekilleştirme → bölme (train/val/test) → tokenizasyon → paketleme → ikili dosya

"Çöp girerse çöp çıkar." Eğitim verisindeki yinelenen, bozuk, kişisel bilgi içeren veya düşük kaliteli metinler, modelin hem başarımını hem de güvenliğini doğrudan etkiler. Bu bölümdeki her adım küçük, bağımsız bir programdır; gerçek bir sistem bunları büyük ölçekte (paralel, parça parça) çalıştırır.

HTML ayıklama ve metin temizliği

Web verisi etiketler, betikler, HTML varlıkları (`&`, ` `) ve fazla boşluklarla doludur. İlk adım yalnızca okunur metni çıkarmaktır.

Örnek 7.1 — HTML etiketlerini ve varlıklarını temizle

PROBLEM

Bir HTML parçasından <head> ve <script> bloklarını tamamen sil, kalan etiketleri kaldır, HTML varlıklarını çöz ve fazla boşlukları tekle.

ÇÖZÜM (FORTRAN)

```
program html_temizle
  implicit none
  character(len=:), allocatable :: s, t
  s = '<html><head><title>Baslik</title></head><body>' // &
      '<h1>Merhaba &amp; hos geldin</h1>' // &
      '<p>LLM   egitimi icin<br>veri temizle &lt;3.</p>' // &
      '<script>var x = 1;</script></body></html>'
  t = remove_between(s, '<head', '</head>')
  t = remove_between(t, '<script', '</script>')
  t = strip_tags(t)
  t = replace_all(t, '&lt;', '<')
  t = replace_all(t, '&gt;', '>')
  t = replace_all(t, ' ', ' ')
  t = replace_all(t, '&amp;', '&')                    ! en son: cift cozmeyi onle
  do while (index(t, '  ') > 0)
    t = replace_all(t, '  ', ' ')
  end do
```

```
  print '(A,A)', 'ham   : ', s
  print '(A,A)', 'temiz : ', trim(adjustl(t))
contains
  function strip_tags(x) result(r)                    ! <...> aralarini ' ' ile degistir
    character(len=*), intent(in) :: x
    character(len=:), allocatable :: r
    logical :: in_tag
    integer :: i
    r = ''; in_tag = .false.
    do i = 1, len(x)
      if (x(i:i) == '<') then
        in_tag = .true.
      else if (x(i:i) == '>' .and. in_tag) then
        in_tag = .false.; r = r // ' '
      else if (.not. in_tag) then
        r = r // x(i:i)
      end if
    end do
  end function strip_tags

  function remove_between(x, a, b) result(r)
    character(len=*), intent(in) :: x, a, b
    character(len=:), allocatable :: r
    integer :: p, q
    r = x
    do
      p = index(r, a)
      if (p == 0) exit
      q = index(r(p:), b)
      if (q == 0) then
        r = r(:p-1); exit
      end if
      r = r(:p-1) // r(p+q-1+len(b):)
    end do
  end function remove_between

  function replace_all(x, a, b) result(r)
    character(len=*), intent(in) :: x, a, b
    character(len=:), allocatable :: r
    integer :: p, i
    r = ''; i = 1
    do
      p = index(x(i:), a)
      if (p == 0) then
        r = r // x(i:); exit
      end if
      r = r // x(i:i+p-2) // b
      i = i + p - 1 + len(a)
    end do
  end function replace_all
end program html_temizle
```

ÇIKTI

```
ham   : <html><head><title>Baslik</title></head><body><h1>Merhaba &amp; hos geldin</h1><p>LLM   egitimi&
   nbsp;icin<br>veri temizle &lt;3.</p><script>var x = 1;</script></body></html>
temiz : Merhaba & hos geldin LLM egitimi icin veri temizle <3.
```

PYTORCH KARŞILIĞI

```
import re, html
s = ("<html><head><title>Baslik</title></head><body>"
     "<h1>Merhaba &amp; hos geldin</h1>"
     "<p>LLM   egitimi icin<br>veri temizle &lt;3.</p>"
     "<script>var x = 1;</script></body></html>")
t = re.sub(r"<head.*?</head>", "", s, flags=re.S)
t = re.sub(r"<script.*?</script>", "", t, flags=re.S)
t = re.sub(r"<[^>]+>", " ", t)               # kalan etiketler -> bosluk
t = html.unescape(t).replace("\xa0", " ")    # &amp; &lt;   ... cozulur
t = re.sub(r"\s+", " ", t).strip()
print("ham   :", s)
print("temiz :", t)
```

PYTORCH ÇIKTISI

```
ham   : <html><head><title>Baslik</title></head><body><h1>Merhaba &amp; hos geldin</h1><p>LLM   egitimi&
   nbsp;icin<br>veri temizle &lt;3.</p><script>var x = 1;</script></body></html>
temiz : Merhaba & hos geldin LLM egitimi icin veri temizle <3.
```

NASIL ÇALIŞIYOR?

Sıra önemlidir: önce `<head>` ve `<script>` bloklarının *içeriği* silinir (yalnızca etiketi silmek betik metnini metne karıştırırdı). Sonra kalan etiketler boşlukla değiştirilir ki "
" ile ayrılan kelimeler birbirine yapışmasın. HTML varlıklarında `&` en sona bırakılır: aksi halde "<" gibi iki kez kodlanmış bir metin yanlışlıkla iki kez çözülürdü. Gerçek sistemlerde bunun yerine olgun HTML ayrıştırıcıları (ve sayfa ana içerik çıkarıcılar) kullanılır; ama mantık budur.

Büyük dosyaları belleğe yüklemeden okuma

Terabaytlık bir metni belleğe sığdıramazsın. Çözüm **akış (streaming)**: dosyayı satır satır veya sabit boyutlu parçalar hâlinde okuyup işlemek. Bellek kullanımı dosya boyutuna değil parça boyutuna bağlı olur.

Örnek 7.2 — Satır satır ve parça parça okuma

PROBLEM

5000 satırlık bir dosya yaz. Onu (1) satır satır, (2) 4096 baytlık parçalar hâlinde okuyarak satır ve kelime sayısını bul. Tüm dosyayı hiçbir zaman belleğe almadığımızı göster.

ÇÖZÜM (FORTRAN)

```
program akis_okuma
  implicit none
  integer, parameter :: chunk = 4096
  integer :: u, ios, i, nl, nw, nl2, nchunk, m
  integer(8) :: fsize, pos, rem
  character(len=200) :: line
  character(len=chunk) :: buf
  ! once ornek bir dosya yaz
  open(newunit=u, file='orn.txt', status='replace', action='write')
  do i = 1, 5000
    write(u, '(A,I0,A)') 'satir ', i, ' ornek metin kelime kelime'
  end do
  close(u)
  ! (1) satir satir oku: ayni anda bellekte yalniz 1 satir
  open(newunit=u, file='orn.txt', status='old', action='read')
  nl = 0; nw = 0
  do
    read(u, '(A)', iostat=ios) line
    if (ios /= 0) exit
    nl = nl + 1
    nw = nw + nkelime(line)
  end do
  close(u)
  ! (2) bayt akisi: sabit 4096 baytlik parcalar
  open(newunit=u, file='orn.txt', access='stream', form='unformatted', status='old')
  inquire(unit=u, size=fsize)
  pos = 1; nl2 = 0; nchunk = 0
  do while (pos <= fsize)
    rem = min(int(chunk, 8), fsize - pos + 1)
    m = int(rem)
    read(u, pos=pos) buf(1:m)
    do i = 1, m
      if (buf(i:i) == char(10)) nl2 = nl2 + 1
    end do
    pos = pos + rem; nchunk = nchunk + 1
  end do
  close(u)
  print '(A,I0,A)', 'dosya boyutu          : ', fsize, ' bayt'
  print '(A,I0,A,I0)', 'satir modu            : satir=', nl, '  kelime=', nw
  print '(A,I0,A,I0)', 'parca modu            : satir=', nl2, '  parca sayisi=', nchunk
contains
  function nkelime(x) result(c)
```

```
    character(len=*), intent(in) :: x
    integer :: c, i
    logical :: ic
    c = 0; ic = .false.
    do i = 1, len_trim(x)
      if (x(i:i) /= ' ') then
        if (.not. ic) c = c + 1
        ic = .true.
      else
        ic = .false.
      end if
    end do
  end function nkelime
end program akis_okuma
```

ÇIKTI

```
dosya boyutu          : 183893 bayt
satir modu            : satir=5000  kelime=30000
parca modu            : satir=5000  parca sayisi=45
```

PYTORCH KARŞILIĞI

```
import os
with open("orn.txt", "w") as f:
    for i in range(1, 5001):
        f.write(f"satir {i} ornek metin kelime kelime\n")
nl = nw = 0
with open("orn.txt") as f:                    # satir satir: bellekte tek satir
    for line in f:
        nl += 1; nw += len(line.split())
nl2 = chunks = 0
with open("orn.txt", "rb") as f:              # sabit 4096 baytlik parcalar
    while chunk := f.read(4096):
        nl2 += chunk.count(b"\n"); chunks += 1
print("dosya boyutu          :", os.path.getsize("orn.txt"), "bayt")
print("satir modu            : satir=%d  kelime=%d" % (nl, nw))
print("parca modu            : satir=%d  parca sayisi=%d" % (nl2, chunks))
```

PYTORCH ÇIKTISI

```
dosya boyutu          : 183893 bayt
satir modu            : satir=5000  kelime=30000
parca modu            : satir=5000  parca sayisi=45
```

NASIL ÇALIŞIYOR?

İki yöntem de aynı satır sayısını bulur ama bellekte yalnızca küçük bir tampon (bir satır ya da 4096 bayt) tutar. `access='stream'` ile dosyayı bayt dizisi olarak açıp `pos=` ile istediğimiz konuma atlayabiliriz; bu, ileride ikili veri dosyasından rastgele örnek çekerken kullanacağımız temel özelliktir. Son parçanın boyutunu `fsize - pos + 1` ile sınırlamak, dosya sonunu aşan okuma hatasını önler. Bir satırın iki parçaya bölünmesi sorun değildir çünkü burada yalnızca satır sonu karakterlerini sayıyoruz.

Tekilleştirme (deduplication)

Web verisinde aynı belge defalarca bulunur. Tekrarlar (1) modelin bu metinleri ezberlemesine yol açar, (2) eğitim bütçesini israf eder ve (3) doğrulama kümesine sızarsa ölçümleri şişirir. İki aşamalı çözüm:

- **Birebir tekrarlar**: normalize edilmiş metnin özetini (hash) hesapla; aynı hash = aynı belge.
- **Yaklaşık tekrarlar**: küçük farklarla aynı belgeler (bir kelimesi değişmiş vb.). Metni küçük parçalara (n-gram "shingle") bölüp iki belgenin parça kümelerinin **Jaccard benzerliğini** hesapla: |A∩B| / |A∪B|.

Örnek 7.3 — Birebir tekrarları hash ile bul

PROBLEM

Yedi kısa belgeden hangilerinin (büyük/küçük harf ve boşluk farkları yok sayılarak) birbirinin kopyası olduğunu polinom hash ile bul.

ÇÖZÜM (FORTRAN)

```
program dedup_tam
  implicit none
  integer, parameter :: nd = 6, w = 40
  character(len=w) :: docs(nd), nrm(nd)
  integer(8) :: h(nd)
  integer :: i, j, nuniq
  logical :: dup(nd)
  docs(1) = 'Bugun hava cok guzel.'
  docs(2) = 'Yarin yagmur yagacak.'
  docs(3) = 'BUGUN   hava  cok guzel.'          ! 1'in kopyasi (farkli yazim)
  docs(4) = 'Model egitimi icin veri gerekir.'
  docs(5) = 'yarin yagmur yagacak.'             ! 2'nin kopyasi
  docs(6) = 'Bugun hava cok kotu.'              ! yakin ama kopya degil
  do i = 1, nd
    nrm(i) = normalize(docs(i))
    h(i) = hash(nrm(i))
  end do
  dup = .false.
  do i = 1, nd
    do j = i+1, nd
      if (h(i) == h(j) .and. nrm(i) == nrm(j) .and. .not. dup(j)) then
        dup(j) = .true.
        print '(A,I0,A,I0,A,I0)', 'belge ', j, ' = belge ', i, ' kopyasi   hash=', h(i)
      end if
    end do
  end do
  nuniq = count(.not. dup)
  print '(A,I0,A,I0)', 'toplam ', nd, ' belge, tekil ', nuniq
contains
  function normalize(x) result(r)                 ! kucuk harf + tek bosluk
    character(len=*), intent(in) :: x
    character(len=len(x)) :: r
    integer :: i, c, k
    logical :: prev_sp
    r = ' '; k = 0; prev_sp = .true.
    do i = 1, len_trim(x)
      c = ichar(x(i:i))
      if (c >= 65 .and. c <= 90) c = c + 32
      if (c == 32) then
        if (prev_sp) cycle
        prev_sp = .true.
      else
        prev_sp = .false.
      end if
      k = k + 1; r(k:k) = char(c)
    end do
  end function normalize

  function hash(x) result(hh)                     ! polinom hash (mod asal)
    character(len=*), intent(in) :: x
    integer(8) :: hh
    integer :: i
    hh = 7_8
    do i = 1, len_trim(x)
      hh = mod(hh * 131_8 + ichar(x(i:i)), 1000000007_8)
    end do
  end function hash
end program dedup_tam
```

ÇIKTI

```
belge 3 = belge 1 kopyasi   hash=407365749
belge 5 = belge 2 kopyasi   hash=541747954
toplam 6 belge, tekil 4
```

PYTORCH KARŞILIĞI

```
import hashlib, re
docs = ["Bugun hava cok guzel.", "Yarin yagmur yagacak.", "BUGUN   hava  cok guzel.",
        "Model egitimi icin veri gerekir.", "yarin yagmur yagacak.", "Bugun hava cok kotu."]
norm = lambda s: re.sub(r"\s+", " ", s.lower()).strip()
seen = {}
for j, d in enumerate(docs, 1):
    h = hashlib.sha1(norm(d).encode()).hexdigest()[:12]
    if h in seen: print(f"belge {j} = belge {seen[h]} kopyasi   hash={h}")
    else: seen[h] = j
print(f"toplam {len(docs)} belge, tekil {len(seen)}")
```

PYTORCH ÇIKTISI

```
belge 3 = belge 1 kopyasi   hash=b0915cbc6bb0
belge 5 = belge 2 kopyasi   hash=ca519491f98d
toplam 6 belge, tekil 4
```

NASIL ÇALIŞIYOR?

Önce `normalize` büyük harfleri küçültür ve boşlukları tekler; böylece "BUGUN hava cok guzel." ile "Bugun hava cok guzel." aynı metne iner. Polinom hash `h = h·131 + karakter (mod p)` her metne tek bir sayı atar. Hash eşitliği çoğu zaman metin eşitliğidir ama *çakışma* (farklı metinlerin aynı hash'i) ihtimali olduğundan eşleşmeleri metin karşılaştırmasıyla da doğruladık. Belge 6 ("kötü") kopya sayılmaz: tekilleştirme yalnızca aynı içeriği yakalar. Yakın içerik için bir sonraki örnek.

Örnek 7.4 — Yaklaşık tekrarlar: Jaccard benzerliği

PROBLEM

Üç cümle için karakter 3-gram kümelerini (shingle) çıkar ve ikili Jaccard benzerliklerini hesapla. Bir kelimesi değişen cümlenin yüksek, alakasız cümlenin düşük benzerlik verdiğini göster.

ÇÖZÜM (FORTRAN)

```
program yakin_tekrar
  implicit none
  integer, parameter :: nd = 3, hs = 65521
  character(len=70) :: docs(nd)
  logical :: sh(hs, nd)
  integer :: i, j, inter, uni
  docs(1) = 'turkce dil modeli egitmek icin temiz ve cesitli veri gerekir'
  docs(2) = 'turkce dil modeli egitmek icin temiz ve guvenilir veri gerekir'
  docs(3) = 'bugun aksam sahilde yuruyus yapip cay icmeyi planliyoruz'
  do i = 1, nd
    call shingles(docs(i), sh(:,i))
  end do
  print '(A)', 'Jaccard benzerlik matrisi:'
  do i = 1, nd
    do j = 1, nd
      inter = count(sh(:,i) .and. sh(:,j))
      uni   = count(sh(:,i) .or.  sh(:,j))
      write(*,'(F8.3)', advance='no') real(inter) / real(uni)
    end do
    print *
  end do
  print '(A,I0)', 'belge 1 icin 3-gram sayisi: ', count(sh(:,1))
contains
  subroutine shingles(x, set)                       ! karakter 3-gramlarini hash'leyip isaretle
    character(len=*), intent(in) :: x
    logical, intent(out) :: set(:)
    integer :: i, idx
    set = .false.
    do i = 1, len_trim(x) - 2
      idx = mod(ichar(x(i:i))*961 + ichar(x(i+1:i+1))*31 + ichar(x(i+2:i+2)), hs) + 1
      set(idx) = .true.
    end do
  end subroutine shingles
end program yakin_tekrar
```

ÇIKTI

```
Jaccard benzerlik matrisi:
   1.000   0.716   0.009
   0.716   1.000   0.009
   0.009   0.009   1.000
belge 1 icin 3-gram sayisi: 56
```

PYTORCH KARŞILIĞI

```
docs = ["turkce dil modeli egitmek icin temiz ve cesitli veri gerekir",
        "turkce dil modeli egitmek icin temiz ve guvenilir veri gerekir",
        "bugun aksam sahilde yuruyus yapip cay icmeyi planliyoruz"]
sh = lambda s: {s[i:i + 3] for i in range(len(s) - 2)}       # karakter 3-gramlari
S = [sh(d) for d in docs]
print("Jaccard benzerlik matrisi:")
for a in S:
    print("".join(f"{len(a & b) / len(a | b):8.3f}" for b in S))
print("belge 1 icin 3-gram sayisi:", len(S[0]))
```

PYTORCH ÇIKTISI

```
Jaccard benzerlik matrisi:
   1.000   0.716   0.009
   0.716   1.000   0.009
   0.009   0.009   1.000
belge 1 icin 3-gram sayisi: 56
```

NASIL ÇALIŞIYOR?

Her belge, karakter 3-gramlarının bir *kümesi* olarak temsil edilir (burada 65521 elemanlı bir hash tablosuna işaretleyerek). Jaccard = ortak 3-gram sayısı / birleşimdeki 3-gram sayısı. Tek kelimesi değişen iki cümle yaklaşık 0.7 civarında yüksek benzerlik verir; alakasız cümle ise 0.0x mertebesinde kalır. Pratikte bir eşik (örneğin 0.8) belirleyip üstündekileri tekrar sayarız. Milyarlarca belgede her çifti karşılaştırmak imkânsız olduğundan **MinHash + LSH** gibi teknikler benzerliği kümelerin küçük bir "imzası" üzerinden hızlıca tahmin eder; temel mantık yine Jaccard'dır.

Eğitim / doğrulama / test ve veri sızıntısı

Modeli yalnızca **görmediği** veride değerlendirebiliriz. Veriyi üç parçaya böleriz: **eğitim** (modeli öğretir), **doğrulama** (hiperparametre seçimi ve eğitim takibi), **test** (son rapor). **Veri sızıntısı**, doğrulama veya test verisinin eğitim verisine (doğrudan ya da kopya/yakın kopya olarak) karışmasıdır; sonuçlar olduğundan çok iyi görünür ve gerçek dünyada hayal kırıklığı yaşanır. Rastgele bölme, kopyalar varken sızıntı üretir. Çözüm: bölme kararını belgenin **içeriğinin hash'ine** bağlamak; aynı içerik her zaman aynı kümeye gider.

Örnek 7.5 — İçeriğe dayalı bölme: sızıntıyı önle

PROBLEM

1000 belge + bunların 100 kopyasından oluşan bir veri kümesini hem rastgele hem hash tabanlı %90/%5/%5 böl. Her yöntemde orijinal ile kopyanın farklı kümelere düştüğü (sızıntı) belge sayısını karşılaştır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program bolme
  use rng_mod
  implicit none
  integer, parameter :: nu = 1000, nd = 100, n = nu + nd
  integer :: sp_r(n), sp_h(n), i, k, j, leak_r, leak_h, cr(3), ch(3)
  character(len=40) :: doc
```

```
  call rng_seed(5)
  do i = 1, n
    k = i
    if (i > nu) k = i - nu                        ! 1001..1100: 1..100'un kopyasi
    write(doc, '(A,I0,A)') 'dokuman ', k, ' metni'
    sp_h(i) = kume(mod(hash(doc), 100_8))         ! icerige gore
    sp_r(i) = kume(int(uniform() * 100.0_dp, 8))  ! rastgele
  end do
  leak_r = 0; leak_h = 0
  do j = 1, nd
    if (sp_r(j) /= sp_r(nu+j)) leak_r = leak_r + 1
    if (sp_h(j) /= sp_h(nu+j)) leak_h = leak_h + 1
  end do
  do i = 1, 3
    cr(i) = count(sp_r == i); ch(i) = count(sp_h == i)
  end do
  print '(A,3I6)', 'rastgele bolme  egitim/dogrulama/test : ', cr
  print '(A,3I6)', 'hash tabanli    egitim/dogrulama/test : ', ch
  print '(A,I0,A)', 'rastgele bolmede kopyasi baska kumeye dusen belge : ', leak_r, ' / 100'
  print '(A,I0,A)', 'hash tabanli bolmede ayni sayi                    : ', leak_h, ' / 100'
contains
  function kume(v) result(c)                      ! 0-89 egitim, 90-94 dogrulama, 95-99 test
    integer(8), intent(in) :: v
    integer :: c
    if (v < 90) then
      c = 1
    else if (v < 95) then
      c = 2
    else
      c = 3
    end if
  end function kume

  function hash(x) result(hh)
    character(len=*), intent(in) :: x
    integer(8) :: hh
    integer :: i
    hh = 7_8
    do i = 1, len_trim(x)
      hh = mod(hh * 131_8 + ichar(x(i:i)), 2147483647_8)
    end do
    hh = mod(hh * 1103515245_8 + 12345_8, 2147483648_8)   ! karistir
  end function hash
end program bolme
```

ÇIKTI

```
rastgele bolme  egitim/dogrulama/test :    993    51    56
hash tabanli    egitim/dogrulama/test :    982    59    59
rastgele bolmede kopyasi baska kumeye dusen belge : 17 / 100
hash tabanli bolmede ayni sayi                    : 0 / 100
```

PYTORCH KARŞILIĞI

```
import hashlib, random
nu, nd = 1000, 100
docs = [f"dokuman {k} metni" for k in range(1, nu + 1)] + [f"dokuman {k} metni" for k in range(1, nd + 1)]
def kume(v): return 0 if v < 90 else 1 if v < 95 else 2          # egitim / dogrulama / test
sp_h = [kume(int(hashlib.md5(d.encode()).hexdigest(), 16) % 100) for d in docs]
random.seed(5)
sp_r = [kume(random.randrange(100)) for _ in docs]
cnt = lambda sp: [sp.count(i) for i in range(3)]
print("rastgele bolme  egitim/dogrulama/test :", cnt(sp_r))
print("hash tabanli    egitim/dogrulama/test :", cnt(sp_h))
print("rastgele bolmede kopyasi baska kumeye dusen belge :", sum(sp_r[j] != sp_r[nu + j] for j in
      range(nd)), "/ 100")
print("hash tabanli bolmede ayni sayi                    :", sum(sp_h[j] != sp_h[nu + j] for j in
      range(nd)), "/ 100")
```

PYTORCH ÇIKTISI

```
rastgele bolme  egitim/dogrulama/test : [995, 55, 50]
hash tabanli    egitim/dogrulama/test : [984, 57, 59]
rastgele bolmede kopyasi baska kumeye dusen belge : 19 / 100
hash tabanli bolmede ayni sayi                    : 0 / 100
```

NASIL ÇALIŞIYOR?

Rastgele bölmede her kopya bağımsız olarak bir kümeye atanır; orijinal eğitimde, kopyası doğrulamada olursa model *cevabı görmüş* olur. Yaklaşık %10 × 2 × 0.9 ≈ 18 civarında belgede böyle bir ayrılma beklenir. Hash tabanlı bölmede küme, içeriğin fonksiyonudur; aynı içerik her zaman aynı kümeye düşer ve sızıntı **tam sıfırdır**. Ayrıca bu yöntem *tekrarlanabilirdir*: veri sonradan eklense bile eski belgelerin kümesi değişmez. Kümelerin boyutları hedeflenen %90/%5/%5 oranına yakın çıkar.

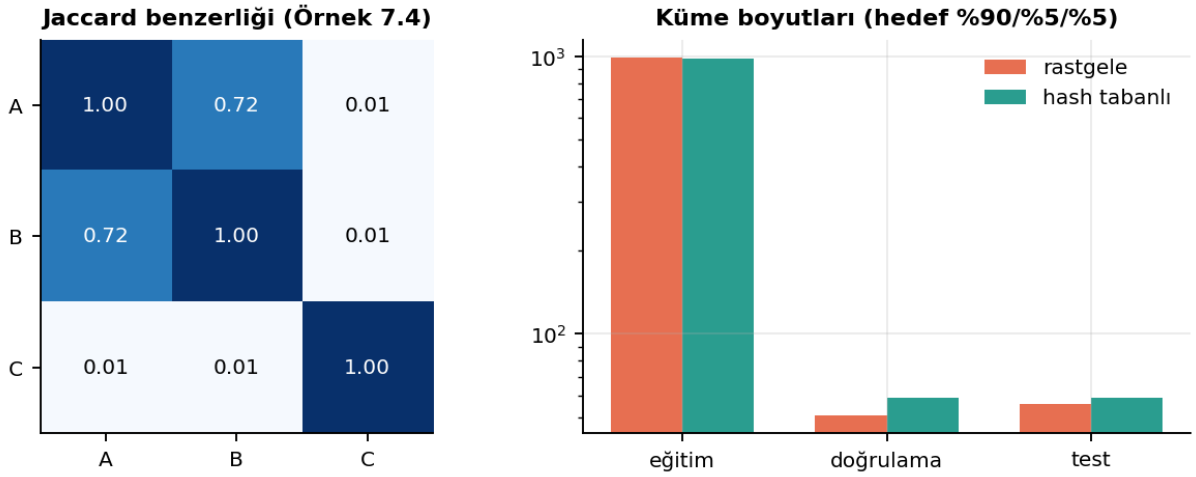
Türetme: MinHash ve hash çakışması

MinHash neden Jaccard'ı tahmin eder? A ve B kümeleri için rastgele bir permütasyon π seçelim; $h(A) = \min_{a \in A} \pi(a)$. h(A) = h(B) olması için A ∪ B'nin en küçük elemanının A ∩ B içinde olması gerekir; permütasyon rastgele olduğundan bu olasılık:

$$P[h(A) = h(B)] = |A \cap B| / |A \cup B| = J(A, B)$$

K bağımsız permütasyonla eşit çıkan oranı J'nin yansız bir tahmini olur (standart sapma $\sqrt{J(1-J)/K}$). Milyarlarca belge çifti için yalnızca K sayılık imzalar karşılaştırılır; LSH ile aday çiftler yalnızca imza bantları eşleşenlerden seçilir.

Hash çakışması (doğum günü paradoksu). m kutulu bir hash'e n belge atılırsa en az bir çakışma olasılığı $\approx 1 - \exp(-n^2/(2m))$. 32 bit hash ($m \approx 4 \cdot 10^9$) için n ≈ 77 000 belgede çakışma olasılığı %50'dir; bu yüzden büyük ölçekte 64 veya 128 bit özetler ve metin karşılaştırması gerekir.



Şekil 7.1 — Solda: Örnek 7.4'ün Jaccard benzerlik matrisi (A–B çok benzer, C alakasız). Sağda: rastgele ve hash tabanlı bölmelerin küme boyutları.

Token tabanlı veri paketleme ve ikili dosya

Eğitimde belgeleri tek tek değil, **tek bir uzun token akışı** olarak kullanırız: tüm belgeleri EOS ile ayırarak peş peşe ekleriz, sonra akışı T uzunluğunda parçalara böleriz. Girdi x = akış[i : i+T], hedef y = akış[i+1 : i+T+1] (bir kayık). Hiç PAD gerekmez, hiçbir token boşa gitmez. Bu akış, **ikili (binary)** dosya olarak yazılır: metin dosyasından kat kat küçük ve hızlı okunur, çünkü her token sabit sayıda bayttır (örneğin 2 veya 4) ve istediğimiz konuma doğrudan atlayabiliriz.

Örnek 7.6 — Token akışını paketle, ikili dosyaya yaz, rastgele erişimle oku

PROBLEM

Üç belgeyi (token listeleri) EOS ile birleştir, int32 olarak ikili dosyaya yaz. Sonra bağlam uzunluğu T = 4 ile (x, y) çiftlerini `pos=` ile dosyadan doğrudan oku.

ÇÖZÜM (FORTRAN)

```
program paketleme
  implicit none
  integer, parameter :: T = 4, EOS = 1
  integer :: d1(3) = [5, 6, 7], d2(2) = [8, 9], d3(5) = [10, 11, 12, 13, 14]
  integer(4), allocatable :: stream(:)
  integer(4) :: x(T), y(T)
  integer :: u, n, k, i, nseq
  integer(8) :: fsize
  stream = [d1, EOS, d2, EOS, d3, EOS]              ! belgeleri EOS ile birlestir
  n = size(stream)
  open(newunit=u, file='veri.bin', access='stream', form='unformatted', status='replace')
  write(u) stream
  close(u)
  open(newunit=u, file='veri.bin', access='stream', form='unformatted', status='old')
  inquire(unit=u, size=fsize)
  print '(A,I0,A,I0,A)', 'akis uzunlugu = ', n, ' token, dosya = ', fsize, ' bayt'
  nseq = (n - 1) / T
  do k = 0, nseq - 1
    i = k * T                                       ! 0 tabanli token konumu
    read(u, pos=4*i + 1) x                          ! her token 4 bayt
    read(u, pos=4*(i+1) + 1) y                      ! bir token kaydirilmis hedef
    print '(A,4I4,A,4I4)', 'x =', x, '   y =', y
  end do
  close(u)
end program paketleme
```

ÇIKTI

```
akis uzunlugu = 13 token, dosya = 52 bayt
x =   5   6   7   1   y =   6   7   1   8
x =   8   9   1  10   y =   9   1  10  11
x =  11  12  13  14   y =  12  13  14   1
```

PYTORCH KARŞILIĞI

```
import numpy as np
EOS = 1
d1, d2, d3 = [5, 6, 7], [8, 9], [10, 11, 12, 13, 14]
stream = np.array(d1 + [EOS] + d2 + [EOS] + d3 + [EOS], dtype=np.int32)
stream.tofile("veri.bin")                       # ikili dosya (her token 4 bayt)
mm = np.memmap("veri.bin", dtype=np.int32, mode="r")   # dosyayi bellege yuklemeden eris
T_ = 4
print(f"akis uzunlugu = {len(stream)} token, dosya = {mm.nbytes} bayt")
for k in range((len(mm) - 1) // T_):
    i = k * T_
    x, y = mm[i:i + T_], mm[i + 1:i + T_ + 1]            # y: bir kaydirilmis hedef
    print("x =", " ".join(f"{v:3d}" for v in x), "   y =", " ".join(f"{v:3d}" for v in y))
```

PYTORCH ÇIKTISI

```
akis uzunlugu = 13 token, dosya = 52 bayt
x =   5   6   7   1    y =   6   7   1   8
x =   8   9   1  10    y =   9   1  10  11
x =  11  12  13  14    y =  12  13  14   1
```

NASIL ÇALIŞIYOR?

Akış: `5 6 7 | EOS | 8 9 | EOS | 10 11 12 13 14 | EOS` (EOS = 1). 13 token, 4 baytlık tamsayılarla 52 bayt eder. Her (x, y) çiftinde y, x'in bir kaydırılmış hâlidir: `y[t]`, `x[t]`'den sonraki token'dır. Bu, dil modeli eğitiminin tüm hedef yapısıdır. Belge sınırlarının (EOS) dizi ortasında kalabildiğine dikkat et: model EOS'tan sonra yepyeni bir belge başladığını öğrenir. `pos=4*i+1` ile istediğimiz token'a doğrudan atlıyoruz; dosyanın tamamı belleğe alınmıyor. Gerçek eğitimde rastgele

`i` değerleriyle batch örneklenir; token kimlikleri 65536'dan küçükse 2 baytlık (uint16) saklayarak yer yarıya iner.

Gizlilik, telif ve lisans

Veri toplarken teknik olmayan sorular da teknik olanlar kadar önemlidir:

- **Kişisel veriler (PII)**: e-posta, telefon, TC kimlik no, adres gibi bilgileri eğitimden önce maskele veya sil. Model ezberleyip geri üretebilir.
- **Telif**: her web sayfası serbestçe kullanılamaz. Kaynakların kullanım koşullarını, *robots.txt* ve lisans bilgisini kontrol et; mümkünse açık lisanslı (CC-BY, CC0, kamu malı) veri tercih et.
- **Lisans izlenebilirliği**: her belgenin kaynağını ve lisansını metaveri olarak sakla; sonradan bir kaynağı veri kümesinden çıkarabilmek için.
- **Hassas ve zararlı içerik**: nefret söylemi, şiddet, cinsel içerik gibi kategoriler için filtreleme ve dengeleme yap.
- **Test verisi kirliliği**: değerlendirme kıyaslama (benchmark) setlerinin eğitim verisine sızmadığını doğrula.

Örnek 7.7 — E-posta adreslerini maskele (PII temizliği)

PROBLEM

"Bana ali.veli@ornek.com adresinden yaz, ya da mehmet@site.org kullan." cümlesindeki e-posta adreslerini <EMAIL> ile değiştir.

ÇÖZÜM (FORTRAN)

```
program pii_maske
  implicit none
  character(len=:), allocatable :: s, out, w
  integer :: i, j, n
  character(len=1) :: last
  s = 'Bana ali.veli@ornek.com adresinden yaz, ya da mehmet@site.org kullan.'
  n = len(s)
  out = ''
  i = 1
  do while (i <= n)
    j = index(s(i:), ' ')                        ! sonraki bosluk
    if (j == 0) then
      w = s(i:); i = n + 1
    else
      w = s(i:i+j-2); i = i + j
    end if
    if (is_email(w)) then
      last = w(len(w):len(w))
      if (last == ',' .or. last == '.') then
        w = '<EMAIL>' // last                    ! sondaki noktalamayi koru
      else
        w = '<EMAIL>'
      end if
    end if
    out = out // w // ' '
  end do
  print '(A,A)', 'once  : ', s
  print '(A,A)', 'sonra : ', trim(out)
contains
  function is_email(x) result(ok)
    character(len=*), intent(in) :: x
    logical :: ok
    integer :: a
    a = index(x, '@')
    ok = .false.
    if (a > 1) ok = index(x(a:), '.') > 0          ! @'dan sonra bir nokta
  end function is_email
end program pii_maske
```

ÇIKTI

```
once  : Bana ali.veli@ornek.com adresinden yaz, ya da mehmet@site.org kullan.
sonra : Bana <EMAIL> adresinden yaz, ya da <EMAIL> kullan.
```

PYTORCH KARŞILIĞI

```
import re
s = "Bana ali.veli@ornek.com adresinden yaz, ya da mehmet@site.org kullan."
t = re.sub(r"[\w.+-]+@[\w-]+\.[\w.]+(?<![.,])", "<EMAIL>", s)
print("once  :", s)
print("sonra :", t)
```

PYTORCH ÇIKTISI

```
once  : Bana ali.veli@ornek.com adresinden yaz, ya da mehmet@site.org kullan.
sonra : Bana <EMAIL> adresinden yaz, ya da <EMAIL> kullan.
```

NASIL ÇALIŞIYOR?

Kelimeleri boşluklardan ayırıp "@ içeren ve sonrasında nokta olan" her kelimeyi e-posta sayıyoruz; sondaki virgül/nokta cümle noktalaması olduğu için korunuyor. Gerçek sistemler düzenli ifadeler ve daha geniş desenler (telefon, kimlik numarası, IBAN) kullanır ve yanlış pozitifleri sınar. Önemli ilke: maskeleme **tokenizasyondan önce** yapılmalı, böylece vocabulary'ye bile kişisel veri girmez.

BÖLÜM 8

8. Model Eğitimi

Burada artık bir dil modelinin eğitim altyapısını kuruyoruz: kayıp, batch, öğrenme oranı çizelgesi, gradyan kırpma, doğrulama ve kaldığı yerden devam.

Bu bölümde

- Next-token prediction ve autoregressive eğitim
- Çapraz entropi kaybı, perplexity ve doğrulama kaybı
- Batch boyutu, dizi uzunluğu ve gradyan gürültüsü
- Öğrenme oranı çizelgesi (warmup + cosine), weight decay
- Gradient clipping
- Checkpoint kaydetme/yükleme ve eğitimi kaldığı yerden sürdürme

Hedef: Bir LLM eğitim döngüsünün her bileşenini ayrı ayrı yazıp anlamak.

Next-token prediction: tek bir hedef, her şey

Dil modeli bir token dizisinin olasılığını zincir kuralıyla çarpanlara ayırır. Modelin temel eğitim hedefi (negatif log-olabilirlik):

$$L = -\sum_{t=1}^{T} \log P(x_t \mid x_1, \ldots, x_{t-1})$$

Yani model, önceki token'ları gördüğünde sıradaki token'ın olasılığını doğru tahmin etmeyi öğrenir. Tek bir dizi aslında T tane eğitim örneğidir: her konum bir sınıflandırma problemidir (V sınıf: sözlük boyutu). **Autoregressive** demek, üretimde de aynı sırayla ilerlemek demektir: bir token seç, diziye ekle, tekrar tahmin et. Ortalama kayıp **perplexity**'ye çevrilir:

$$PPL = \exp(L / T)$$

Perplexity, modelin her adımda "ortalama kaç eşit olasılıklı seçenek arasında kaldığı"dır. PPL = 1 mükemmel, PPL = V rastgele tahmin demektir.

Örnek 8.1 — Sonraki token kaybı ve perplexity (elle)

PROBLEM

Bir model, gerçek 4 token'ı sırayla şu olasılıklarla tahmin etti: 0.5, 0.25, 0.1, 0.8. Toplam kaybı, ortalama kaybı ve perplexity'yi hesapla. V = 50000 olan rastgele bir modelin perplexity'si ne olurdu?

ÇÖZÜM (FORTRAN)

```
program sonraki_token
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: p(4), ltot, lmean, ppl
  integer :: t
  p = [0.5_dp, 0.25_dp, 0.1_dp, 0.8_dp]
  do t = 1, 4
    print '(A,I1,A,F5.2,A,F8.4)', 'adim ', t, '  P(dogru token)=', p(t), '  -log P = ', -log(p(t))
  end do
  ltot  = -sum(log(p))
  lmean = ltot / 4
  ppl   = exp(lmean)
  print '(A,F8.4)', 'toplam kayip      = ', ltot
  print '(A,F8.4)', 'ortalama kayip    = ', lmean
  print '(A,F8.4)', 'perplexity        = ', ppl
  print '(A,F8.4,A,F9.1)', 'V=50000 rastgele: kayip=', log(50000.0_dp), '  PPL=', 50000.0_dp
end program sonraki_token
```

ÇIKTI

```
adim 1  P(dogru token)= 0.50  -log P =   0.6931
adim 2  P(dogru token)= 0.25  -log P =   1.3863
adim 3  P(dogru token)= 0.10  -log P =   2.3026
adim 4  P(dogru token)= 0.80  -log P =   0.2231
toplam kayip      =   4.6052
ortalama kayip    =   1.1513
perplexity        =   3.1623
V=50000 rastgele: kayip= 10.8198  PPL=  50000.0
```

PYTORCH KARŞILIĞI

```
import torch, math
import torch.nn.functional as F
p = torch.tensor([0.5, 0.25, 0.1, 0.8], dtype=torch.float64)
for t, v in enumerate(p, 1):
    print(f"adim {t}  P(dogru token)={v:5.2f}  -log P = {-torch.log(v):8.4f}")
logits = torch.log(p)[:, None]                        # her adim icin tek-sinifli logit: ayni olasilik
loss = -torch.log(p).sum()
print(f"toplam kayip      = {loss:8.4f}")
print(f"ortalama kayip    = {loss / 4:8.4f}")
print(f"perplexity        = {torch.exp(loss / 4):8.4f}")
print(f"V=50000 rastgele: kayip={math.log(50000):8.4f}  PPL={50000.0:9.1f}")
```

PYTORCH ÇIKTISI

```
adim 1  P(dogru token)= 0.50  -log P =   0.6931
adim 2  P(dogru token)= 0.25  -log P =   1.3863
adim 3  P(dogru token)= 0.10  -log P =   2.3026
adim 4  P(dogru token)= 0.80  -log P =   0.2231
toplam kayip      =   4.6052
ortalama kayip    =   1.1513
perplexity        =   3.1623
V=50000 rastgele: kayip= 10.8198  PPL=  50000.0
```

NASIL ÇALIŞIYOR?

Her adımın cezası $-\log P$: 0.5 için 0.693, 0.25 için 1.386, 0.1 için 2.303, 0.8 için 0.223. Toplam 4.605, ortalama 1.151; perplexity $e^{1.151} \approx 3.16$: model ortalama olarak yaklaşık 3 seçenek arasında kalmış. Rastgele bir model $V = 50000$ için $\ln V = 10.82$ kayıpla başlar; eğitim ilerledikçe gerçek modeller küçük veri kümelerinde 3–4, büyük karmaşık verilerde 2–3 civarına iner.

Türetme: kayıp, entropi ve perplexity

Perplexity ve bilgi kuramı. Gerçek dağılım p, model q olsun. Token başına ortalama kayıp (nat) çapraz entropidir:

$L = H(p, q) = H(p) + D_{KL}(p \| q)$, $PPL = e^{L}$

H(p) verinin *kendi* belirsizliğidir (Shannon entropisi), hiçbir model bunun altına inemez; modelin başarısı fazlalık D_{KL}'dir. İngilizce karakter düzeyinde H ≈ 1.0–1.3 bit/karakter; kayıp bu sınıra yaklaştıkça modelleme tamamlanmış sayılır. Bit/karakter = L/ln 2; $PPL = 2^{\text{bit/token}}$. Farklı tokenizer'lı modellerin PPL'si doğrudan karşılaştırılamaz (token başına vs karakter başına): doğru ölçü **bit/bayt**tır.

Batch boyutu ve dizi uzunluğu

Bir optimizer adımında işlenen token sayısı **batch × dizi uzunluğu**'dur (örneğin 32 × 1024 = 32768 token). Büyük batch, gradyan tahminini daha az gürültülü yapar ama adım sayısını azaltır. Gürültü batch büyüklüğünün **karekökü** ile azalır, doğrusal değil.

Basit matematik

Her örneğin gradyanı g_i ve varyansı σ^2 ise, B örneğin ortalamasının varyansı σ^2/B, standart sapması $\sigma/\sqrt{B}$ olur.

Batch'i 4 katına çıkarmak gürültüyü yalnızca 2 kat azaltır. Bu yüzden belli bir noktadan sonra batch büyütmenin getirisi azalır ("kritik batch boyutu").

Örnek 8.2 — Mini-batch gradyanının gürültüsü ~ 1/√B

PROBLEM

Doğrusal bir modelde, B = 1, 4, 16, 64 için mini-batch gradyanını 2000 kez örnekle. Ortalamanın tam veri gradyanına yakın kaldığını ve standart sapmanın 1/√B ile azaldığını göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program batch_gurultu
  use rng_mod
  implicit none
  integer, parameter :: N = 10000, ntr = 2000
  integer, parameter :: bsz(4) = [1, 4, 16, 64]
  real(dp) :: x(N), y(N), gi(N), s1, s2, gm, mean_, sd
  integer :: ib, tr, k, i
  call rng_seed(17)
  do i = 1, N
    x(i) = gauss()
    y(i) = 2.0_dp * x(i) + 0.5_dp * gauss()
  end do
  gi = 2.0_dp * (0.0_dp * x - y) * x               ! w = 0 iken ornek basina gradyan
  print '(A,F9.4)', 'tam veri gradyani = ', sum(gi) / N
  print '(A)', '    B     ortalama     std      std*sqrt(B)'
  do ib = 1, 4
    s1 = 0.0_dp; s2 = 0.0_dp
    do tr = 1, ntr
      gm = 0.0_dp
      do k = 1, bsz(ib)
        i = 1 + int(uniform() * N)                 ! rastgele ornek
        gm = gm + gi(i)
      end do
      gm = gm / bsz(ib)                            ! mini-batch ortalamasi
      s1 = s1 + gm; s2 = s2 + gm**2
    end do
    mean_ = s1 / ntr
    sd = sqrt(s2 / ntr - mean_**2)
    print '(I5,3F11.4)', bsz(ib), mean_, sd, sd * sqrt(real(bsz(ib), dp))
  end do
end program batch_gurultu
```

ÇIKTI

```
tam veri gradyani =   -3.9578
    B     ortalama      std      std*sqrt(B)
    1    -4.1635     5.7824     5.7824
    4    -3.9250     2.8367     5.6733
   16    -3.9695     1.4098     5.6394
   64    -3.9474     0.7175     5.7397
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(17)
N, ntr = 10000, 2000
x = torch.randn(N, dtype=torch.float64); y = 2 * x + 0.5 * torch.randn(N, dtype=torch.float64)
gi = 2 * (0 * x - y) * x                              # w = 0 iken ornek basina gradyan
print(f"tam veri gradyani = {gi.mean():9.4f}")
print("    B     ortalama     std      std*sqrt(B)")
for B in [1, 4, 16, 64]:
    idx = torch.randint(0, N, (ntr, B))               # ntr kez, B rastgele ornek
    gm = gi[idx].mean(1)                              # mini-batch gradyanlari
    print(f"{B:5d}{gm.mean():11.4f}{gm.std():11.4f}{gm.std() * B ** 0.5:11.4f}")
```

PYTORCH ÇIKTISI

```
tam veri gradyani =   -4.0653
    B     ortalama      std      std*sqrt(B)
    1    -4.0806     5.5310     5.5310
    4    -4.0783     2.9426     5.8852
   16    -4.0269     1.4325     5.7299
   64    -4.0899     0.7219     5.7750
```

NASIL ÇALIŞIYOR?

Ortalama sütunu her batch boyutunda gerçek gradyana (yaklaşık −4) yakındır: mini-batch gradyanı **yansızdır**. Standart sapma B büyüdükçe azalır; son sütun (std × √B) yaklaşık sabit kalır, yani std ∝ 1/√B. B = 64 ile B = 1'e göre gürültü 8 kat azalmıştır, ama işlem 64 kat artmıştır. Bu yüzden pratikte "en büyük batch" değil, donanıma ve öğrenme oranına uygun bir batch seçilir.

Öğrenme oranı çizelgesi: warmup ve cosine

Büyük Transformer'lar sabit öğrenme oranıyla iyi eğitilmez. Standart tarif:

- **Warmup**: ilk birkaç yüz/bin adımda öğrenme oranını sıfırdan η_{max}'a doğrusal artır. Başlangıçta Adam'ın istatistikleri güvenilmezdir ve ağırlıklar rastgeledir; büyük adımlar eğitimi bozabilir.
- **Cosine decay**: sonra η_{max}'tan η_{min}'e kosinüs eğrisiyle yumuşakça düşür. Sonlara doğru ince ayar yapılır.

$$\eta(t) = \eta_{max} \cdot t / t_{warm}, \; t < t_{warm}$$

$$\eta(t) = \eta_{min} + \tfrac{1}{2}\,(\eta_{max} - \eta_{min})\,(1 + \cos(\pi\,(t - t_{warm}) / (T - t_{warm}))), \; t \geq t_{warm}$$

Örnek 8.3 — Warmup + cosine öğrenme oranı çizelgesi

PROBLEM

T = 1000 adım, 100 adım warmup, η_{max} = 6e-4, η_{min} = 6e-5 için çizelgeyi yaz ve belli adımlarda değerlerini yazdır.

ÇÖZÜM (FORTRAN)

```
program lr_cizelge
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer :: steps(9), i
  steps = [0, 50, 100, 200, 400, 600, 800, 900, 1000]
  print '(A)', '  adim       lr'
  do i = 1, 9
    print '(I6,ES12.3)', steps(i), lr_at(steps(i), 1000, 100, 6.0e-4_dp, 6.0e-5_dp)
  end do
contains
  function lr_at(t, tmax, warm, lmax, lmin) result(lr)
    integer, intent(in) :: t, tmax, warm
    real(dp), intent(in) :: lmax, lmin
    real(dp) :: lr, prog
    real(dp), parameter :: pi = 3.141592653589793_dp
    if (t < warm) then
      lr = lmax * real(t, dp) / real(warm, dp)
    else
      prog = real(t - warm, dp) / real(tmax - warm, dp)
      lr = lmin + 0.5_dp * (lmax - lmin) * (1.0_dp + cos(pi * prog))
    end if
  end function lr_at
end program lr_cizelge
```

ÇIKTI

```
  adim       lr
     0   0.000E+00
    50   3.000E-04
   100   6.000E-04
   200   5.837E-04
   400   4.650E-04
   600   2.831E-04
   800   1.232E-04
   900   7.628E-05
  1000   6.000E-05
```

PYTORCH KARŞILIĞI

```
import math, torch
p = torch.nn.Parameter(torch.zeros(1))
opt = torch.optim.AdamW([p], lr=6e-4)
warm, total, lmin = 100, 1000, 6e-5
def lam(t):                                          # LambdaLR: lr = lr0 * lam(t)
    if t < warm: return t / warm
    prog = (t - warm) / (total - warm)
    return (lmin + 0.5 * (6e-4 - lmin) * (1 + math.cos(math.pi * prog))) / 6e-4
sched = torch.optim.lr_scheduler.LambdaLR(opt, lam)
want = {0, 50, 100, 200, 400, 600, 800, 900, 1000}
print("  adim       lr")
for t in range(1001):
    if t in want: print(f"{t:6d}{opt.param_groups[0]['lr']:12.3e}")
    opt.step(); sched.step()
```

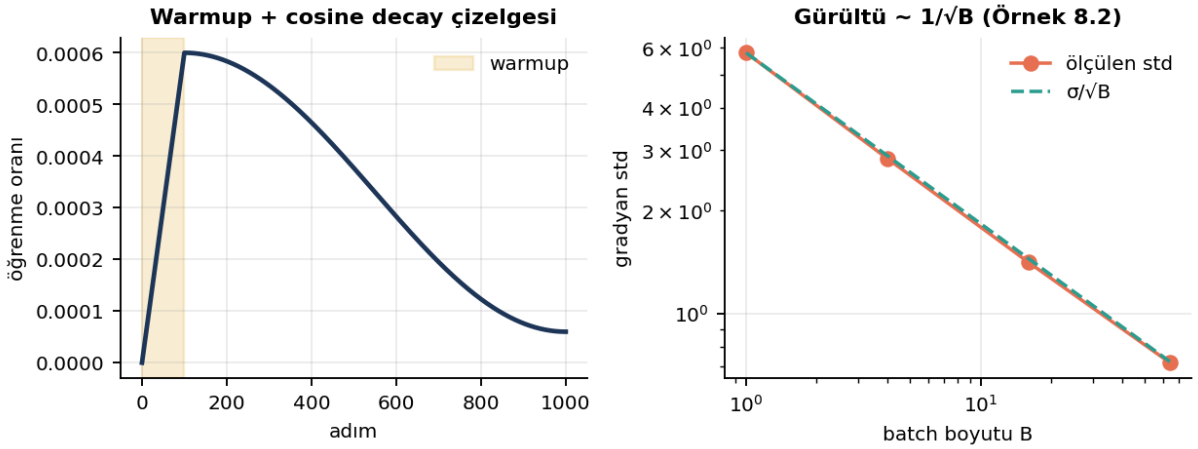
PYTORCH ÇIKTISI

```
  adim       lr
     0   0.000e+00
    50   3.000e-04
   100   6.000e-04
   200   5.837e-04
   400   4.650e-04
   600   2.831e-04
   800   1.232e-04
   900   7.628e-05
  1000   6.000e-05
```

NASIL ÇALIŞIYOR?

Adım 0'da lr sıfırdır, adım 100'de η_{max} = 6e-4'e ulaşır. Cosine fazının ortasında (adım ≈ 550) lr, η_{max} ile η_{min} arasındaki orta değerdir (≈ 3.3e-4). Son adımda η_{min} = 6e-5 olur. Tipik kural: $\eta_{min} \approx \eta_{max}/10$, warmup toplam adımın %1–5'i. Çizelge adım sayısını *önceden* bilmeyi gerektirir; bu yüzden eğitim

bütçesi baştan planlanır.



Şekil 8.1 — Solda: warmup + cosine öğrenme oranı çizelgesi. Sağda: mini-batch gradyan standart sapması batch boyutuyla 1/√B ile azalır (ölçülen değerler Örnek 8.2'den).

Gradient clipping

Bazen tek bir kötü batch çok büyük bir gradyan üretir ve tek adımda tüm eğitimi bozar (loss spike). **Global norm kırpma**, bütün parametre gradyanlarını tek bir vektör sayıp normu belirli bir eşiği (tipik 1.0) aşarsa bütün gradyanı aynı oranda küçültür. Yön korunur, yalnızca adım uzunluğu sınırlanır.

$$\text{eğer } \|g\| > c \text{ ise } g \leftarrow g \cdot c / \|g\|$$

Örnek 8.4 — Global norm ile gradyan kırpma

PROBLEM

İki parametre tensörünün gradyanı $g_1 = [3, 4]$ ve $g_2 = [0, 12]$ olsun. Global normu bul ve $c = 1$ eşiğiyle kırp. Eşiğin altındaki bir gradyanın değişmediğini göster.

ÇÖZÜM (FORTRAN)

```
program kirpma
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: g1(2), g2(2), gnorm, c, scale, h1(2), h2(2)
  g1 = [3.0_dp, 4.0_dp]
  g2 = [0.0_dp, 12.0_dp]
  c = 1.0_dp
  gnorm = sqrt(sum(g1**2) + sum(g2**2))                ! butun tensorlerin ortak normu
  print '(A,F8.4)', 'global norm (once) = ', gnorm
  scale = min(1.0_dp, c / gnorm)                       ! esigi asmiyorsa 1
  g1 = g1 * scale; g2 = g2 * scale
  print '(A,F8.4)', 'carpan             = ', scale
  print '(A,2F9.5,A,2F9.5)', 'g1, g2 (sonra)     = ', g1, '  ', g2
  print '(A,F8.4)', 'global norm (sonra)= ', sqrt(sum(g1**2) + sum(g2**2))
  h1 = [0.3_dp, 0.4_dp]; h2 = [0.0_dp, 0.2_dp]          ! kucuk gradyan
  gnorm = sqrt(sum(h1**2) + sum(h2**2))
  scale = min(1.0_dp, c / gnorm)
  print '(A,F8.4,A,F6.3)', 'kucuk gradyan: norm = ', gnorm, '   carpan = ', scale
end program kirpma
```

ÇIKTI

```
global norm (once) =  13.0000
carpan             =   0.0769
g1, g2 (sonra)     =   0.23077  0.30769    0.00000  0.92308
global norm (sonra)=   1.0000
kucuk gradyan: norm =   0.5385   carpan =  1.000
```

PYTORCH KARŞILIĞI

```
import torch
p1 = torch.nn.Parameter(torch.zeros(2)); p2 = torch.nn.Parameter(torch.zeros(2))
p1.grad = torch.tensor([3.0, 4.0]); p2.grad = torch.tensor([0.0, 12.0])
norm = torch.nn.utils.clip_grad_norm_([p1, p2], max_norm=1.0)       # global norm kirpma
print(f"global norm (once) = {norm:8.4f}")
print("g1, g2 (sonra)     =", [round(v, 5) for v in p1.grad.tolist()], [round(v, 5) for v in
      p2.grad.tolist()])
print(f"global norm (sonra)= {torch.sqrt(p1.grad.pow(2).sum() + p2.grad.pow(2).sum()):8.4f}")
q1 = torch.nn.Parameter(torch.zeros(2)); q2 = torch.nn.Parameter(torch.zeros(2))
q1.grad = torch.tensor([0.3, 0.4]); q2.grad = torch.tensor([0.0, 0.2])
n2 = torch.nn.utils.clip_grad_norm_([q1, q2], max_norm=1.0)
print(f"kucuk gradyan: norm = {n2:8.4f}   (degismez: {q1.grad.tolist()})")
```

PYTORCH ÇIKTISI

```
global norm (once) =  13.0000
g1, g2 (sonra)     = [0.23077, 0.30769] [0.0, 0.92308]
global norm (sonra)=   1.0000
kucuk gradyan: norm =   0.5385   (degismez: [0.30000001192092896, 0.4000000059604645])
```

NASIL ÇALIŞIYOR?

$\|g\| = \sqrt{(9+16+0+144)} = \sqrt{169} = 13$. Çarpan $1/13 \approx 0.0769$: $g_1 \to [0.2308, 0.3077]$, $g_2 \to [0, 0.9231]$ ve yeni norm tam 1 olur. İki tensörün *oranı* (yön) korunur. Küçük gradyanda norm $0.54 < 1$ olduğundan `min(1, c/‖g‖)` çarpanı 1 verir ve gradyan hiç değişmez. Kırpma yalnızca anormal durumlarda devreye girer; sık devreye giriyorsa öğrenme oranı büyük olabilir.

Doğrulama kaybı, aşırı öğrenme ve perplexity

Eğitim kaybı her zaman düşer; modelin gerçekten öğrenip öğrenmediğini **doğrulama kaybı** gösterir. Eğitim kaybı düşerken doğrulama kaybı yükselmeye başlıyorsa model ezberliyor (**overfitting**) demektir. Çok küçük veride bu hızlı olur; milyarlarca token'lı veride ise model çoğu zaman bir tur bile ezberleyemez.

Örnek 8.5 — Karakter bigram modeli: eğitim ve doğrulama perplexity'si

PROBLEM

Küçük bir metinden karakter bigram sayımları çıkar. Düzleştirme (smoothing) katsayısı k = 0, 0.01, 0.1, 1 için eğitim ve doğrulama perplexity'sini hesapla. Düzleştirmesiz modelin doğrulamada neden "sonsuz" çıktığını göster.

ÇÖZÜM (FORTRAN)

```
program bigram_ppl
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  character(len=*), parameter :: train = &
    'bir varmis bir yokmus evvel zaman icinde kalbur saman icinde ' // &
    'develer tellal iken pireler berber iken '
  character(len=*), parameter :: val = &
    'ben annemin besigini tangir tungur sallar iken cok gezdim cok gordum'
  integer :: cnt(0:127, 0:127), tot(0:127), V, i, c, a, b
  logical :: seen(0:127)
  real(dp) :: ks(4)
  ks = [0.0_dp, 0.01_dp, 0.1_dp, 1.0_dp]
  cnt = 0; seen = .false.
  do i = 1, len(train) - 1                          ! yalniz egitim metninden say
    a = ichar(train(i:i)); b = ichar(train(i+1:i+1))
    cnt(a, b) = cnt(a, b) + 1
  end do
  do i = 1, len(train)
    seen(ichar(train(i:i))) = .true.
  end do
  do i = 1, len(val)
    seen(ichar(val(i:i))) = .true.                  ! sozluk: iki metindeki tum karakterler
```

```
  end do
  V = count(seen)
  do c = 0, 127
    tot(c) = sum(cnt(c,:))
  end do
  print '(A,I0)', 'sozluk boyutu V = ', V
  print '(A)', '     k     egitim PPL     dogrulama PPL'
  do i = 1, 4
    write(*,'(F7.2,A)', advance='no') ks(i), '   '
    call yaz(ppl(train, ks(i)))
    call yaz(ppl(val, ks(i)))
    print *
  end do
contains
  function ppl(text, k) result(r)
    character(len=*), intent(in) :: text
    real(dp), intent(in) :: k
    real(dp) :: r, p, ll
    integer :: j, a, b
    ll = 0.0_dp
    do j = 1, len(text) - 1
      a = ichar(text(j:j)); b = ichar(text(j+1:j+1))
      p = (cnt(a,b) + k) / (tot(a) + k * V)
      if (p <= 0.0_dp) then
        r = huge(1.0_dp); return                    ! sifir olasilik -> sonsuz
      end if
      ll = ll - log(p)
    end do
    r = exp(ll / (len(text) - 1))
  end function ppl

  subroutine yaz(v)
    real(dp), intent(in) :: v
    if (v > 1.0e100_dp) then
      write(*,'(A)', advance='no') '        sonsuz   '
    else
      write(*,'(F14.3,A)', advance='no') v, '   '
    end if
  end subroutine yaz
end program bigram_ppl
```

ÇIKTI

```
sozluk boyutu V = 21
     k     egitim PPL     dogrulama PPL
  0.00           3.496           sonsuz
  0.01           3.618           41.621
  0.10           4.535           18.956
  1.00           9.489           16.522
```

PYTORCH KARŞILIĞI

```
import torch
train = ("bir varmis bir yokmus evvel zaman icinde kalbur saman icinde "
         "develer tellal iken pireler berber iken ")
val = "ben annemin besigini tangir tungur sallar iken cok gezdim cok gordum"
chars = sorted(set(train) | set(val)); ix = {c: i for i, c in enumerate(chars)}; V = len(chars)
enc = lambda s: torch.tensor([ix[c] for c in s])
tr, va = enc(train), enc(val)
cnt = torch.zeros(V, V, dtype=torch.float64)
cnt.index_put_((tr[:-1], tr[1:]), torch.ones(len(tr) - 1, dtype=torch.float64), accumulate=True)
tot = cnt.sum(1)
def ppl(t, k):
    p = (cnt[t[:-1], t[1:]] + k) / (tot[t[:-1]] + k * V)
    return torch.exp(-torch.log(p).mean()).item() if (p > 0).all() else float("inf")
print("sozluk boyutu V =", V)
print("     k     egitim PPL     dogrulama PPL")
for k in [0.0, 0.01, 0.1, 1.0]:
    print(f"{k:6.2f}   {ppl(tr, k):12.3f}   {ppl(va, k):12.3f}")
```

PYTORCH ÇIKTISI

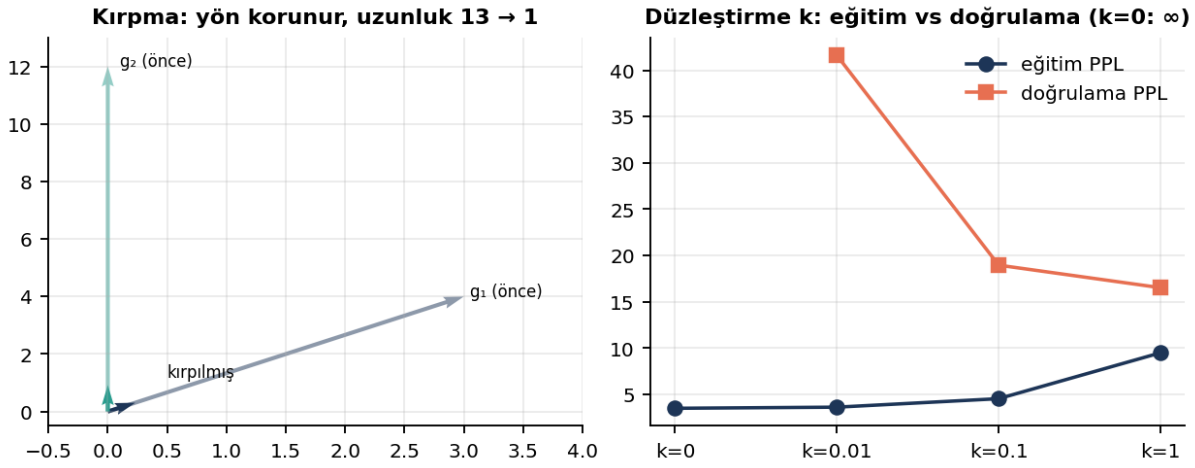
```
sozluk boyutu V = 21
      k     egitim PPL     dogrulama PPL
   0.00          3.496               inf
   0.01          3.618            41.621
   0.10          4.535            18.956
   1.00          9.489            16.522
```

NASIL ÇALIŞIYOR?

Bigram modeli P(b|a) = (sayı(a,b) + k) / (sayı(a) + k·V) ile tahmin eder. k = 0 iken eğitimde görülmemiş tek bir harf çifti bile olasılık 0 verir ve kayıp sonsuz olur: model hiç görmediği şeye *imkânsız* der. Eğitim PPL'si k = 0'da en düşüktür (ezber); k büyüdükçe eğitim PPL'si artar ama doğrulama PPL'si düşer (burada k = 1 en iyisidir; çok daha büyük k dağılımı tekdüzeye yaklaştırıp yine kötüleştirirdi). Bu, hacimli modellerde de geçerli bir derstir: eğitim metriğini düşürmek amaç değildir, **görülmemiş veride** iyi olmak amaçtır.

Türetme: gradyan kırpma

Gradyan kırpma neden güvenlidir? $g \leftarrow g \cdot \min(1, c/\|g\|)$ dönüşümü yönü korur ve adım uzunluğunu $\eta \cdot c$ ile sınırlar. Kayıp L-düzgün ise (∇L, L-Lipschitz) tek bir adımdaki en kötü artış, ikinci dereceden sınır ile $L(\theta - \eta g) \le L(\theta) - \eta\|g\|^2 + (L\eta^2/2)\|g\|^2$ olur; $\|g\| \le c$ ile artış kontrol altındadır. Yani kırpma, eğrilik çok büyük olan bölgelerde (kayıp sıçramalarının kaynağı) adımı *güvenli* boyuta indirir ve normal bölgelerde hiçbir şey yapmaz.



Şekil 8.2 — Solda: kırpma yönü korur, uzunluğu 13'ten 1'e indirir. Sağda: bigram modelinde düzleştirme katsayısı k'nin eğitim ve doğrulama perplexity'sine etkisi (k=0: doğrulamada ∞).

Checkpoint: eğitimi kaldığı yerden sürdür

Haftalarca süren bir eğitim bir noktada kesilebilir (elektrik, donanım hatası, kota). Bu yüzden düzenli olarak **checkpoint** yazılır. Doğru checkpoint yalnızca model ağırlıklarını değil, **eğitimi tam olarak yeniden kurmak için gereken her şeyi** içerir: optimizer durumu (Adam için m ve v), adım sayacı, öğrenme oranı çizelgesinin konumu, rastgele sayı üreteci durumu ve veri okuyucunun konumu.

Örnek 8.6 — Checkpoint ile devam: tam eşdeğerlik

PROBLEM

Doğrusal bir model üzerinde Adam ile 100 adım eğit. (A) kesintisiz; (B) 50 adım sonra tüm durumu (parametreler, m, v, adım) dosyaya yazıp belleği silip geri yükleyerek devam; (C) yalnızca parametreleri yükleyip optimizer durumunu sıfırlayarak devam. Sonuçları karşılaştır.

ÇÖZÜM (FORTRAN)

```
program ckpt_devam
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: x(8), y(8), pA(2), pB(2), pC(2), p(2), m(2), v(2)
  integer :: step
  x = [0.5_dp, 1.0_dp, 1.5_dp, 2.0_dp, 2.5_dp, 3.0_dp, 3.5_dp, 4.0_dp]
  y = [1.2_dp, 1.9_dp, 3.1_dp, 3.8_dp, 5.2_dp, 5.9_dp, 7.1_dp, 7.8_dp]
  ! (A) kesintisiz 100 adim
  p = 0.0_dp; m = 0.0_dp; v = 0.0_dp; step = 0
  call train(p, m, v, step, 100); pA = p
  ! (B) 50 adim, kaydet, bellegi sil, yukle, 50 adim
  p = 0.0_dp; m = 0.0_dp; v = 0.0_dp; step = 0
  call train(p, m, v, step, 50)
  call save_state('ckpt.bin', p, m, v, step)
  p = -999.0_dp; m = -999.0_dp; v = -999.0_dp; step = -1      ! "program coktu"
  call load_state('ckpt.bin', p, m, v, step)
  call train(p, m, v, step, 50); pB = p
  ! (C) yalniz parametreleri yukle, optimizer durumunu sifirla
  p = 0.0_dp; m = 0.0_dp; v = 0.0_dp; step = 0
  call train(p, m, v, step, 50)
  m = 0.0_dp; v = 0.0_dp; step = 0                            ! optimizer durumu kayboldu
  call train(p, m, v, step, 50); pC = p
  print '(A,2F14.10)', '(A) kesintisiz          : ', pA
  print '(A,2F14.10)', '(B) tam durumla devam   : ', pB
  print '(A,2F14.10)', '(C) yalniz agirliklar   : ', pC
  print '(A,ES10.2)', '|A - B| = ', maxval(abs(pA - pB))
  print '(A,ES10.2)', '|A - C| = ', maxval(abs(pA - pC))
contains
  subroutine train(p, m, v, step, k)                 ! k adim Adam
    real(dp), intent(inout) :: p(2), m(2), v(2)
    integer, intent(inout) :: step
    integer, intent(in) :: k
    real(dp) :: g(2), mh(2), vh(2)
    real(dp), parameter :: lr = 0.05_dp, b1 = 0.9_dp, b2 = 0.999_dp, eps = 1.0e-8_dp
    integer :: it
    do it = 1, k
      step = step + 1
      g(1) = sum(2.0_dp * (p(1)*x + p(2) - y) * x) / 8
      g(2) = sum(2.0_dp * (p(1)*x + p(2) - y)) / 8
      m = b1*m + (1.0_dp - b1)*g
      v = b2*v + (1.0_dp - b2)*g**2
      mh = m / (1.0_dp - b1**step)
      vh = v / (1.0_dp - b2**step)
      p = p - lr * mh / (sqrt(vh) + eps)
    end do
  end subroutine train

  subroutine save_state(fn, p, m, v, step)
    character(len=*), intent(in) :: fn
    real(dp), intent(in) :: p(2), m(2), v(2)
    integer, intent(in) :: step
    integer :: u
    open(newunit=u, file=fn, access='stream', form='unformatted', status='replace')
    write(u) step, p, m, v
    close(u)
  end subroutine save_state

  subroutine load_state(fn, p, m, v, step)
    character(len=*), intent(in) :: fn
    real(dp), intent(out) :: p(2), m(2), v(2)
    integer, intent(out) :: step
    integer :: u
    open(newunit=u, file=fn, access='stream', form='unformatted', status='old')
    read(u) step, p, m, v
    close(u)
  end subroutine load_state
end program ckpt_devam
```

ÇIKTI

```
(A) kesintisiz          :   1.5711324518  1.0846769821
(B) tam durumla devam   :   1.5711324518  1.0846769821
(C) yalniz agirliklar   :   1.9704549171  0.0430475871
|A - B| =   0.00E+00
|A - C| =   1.04E+00
```

PYTORCH KARŞILIĞI

```
import torch
x = torch.tensor([0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0], dtype=torch.float64)
y = torch.tensor([1.2, 1.9, 3.1, 3.8, 5.2, 5.9, 7.1, 7.8], dtype=torch.float64)
def make():
    p = torch.nn.Parameter(torch.zeros(2, dtype=torch.float64))
    return p, torch.optim.Adam([p], lr=0.05)
def train(p, opt, k):
    for _ in range(k):
        loss = ((p[0] * x + p[1] - y) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
pA, oA = make(); train(pA, oA, 100)                              # (A) kesintisiz
pB, oB = make(); train(pB, oB, 50)
torch.save({"model": pB.detach(), "opt": oB.state_dict()}, "ckpt.pt")   # (B) tam durum
pB2, oB2 = make(); ck = torch.load("ckpt.pt")
pB2.data.copy_(ck["model"]); oB2.load_state_dict(ck["opt"]); train(pB2, oB2, 50)
pC, oC = make(); train(pC, oC, 50)                               # (C) yalniz agirliklar
pC2, oC2 = make(); pC2.data.copy_(pC.detach()); train(pC2, oC2, 50)
f = lambda p: " ".join(f"{v:14.10f}" for v in p.tolist())
print("(A) kesintisiz          :", f(pA)); print("(B) tam durumla devam   :", f(pB2)); print("(C) yalniz
      agirliklar   :", f(pC2))
print("|A - B| =", f"{(pA - pB2).abs().max().item():.2e}"); print("|A - C| =", f"{(pA -
      pC2).abs().max().item():.2e}")
```

PYTORCH ÇIKTISI

```
(A) kesintisiz          :   1.5711324518   1.0846769821
(B) tam durumla devam   :   1.5711324518   1.0846769821
(C) yalniz agirliklar   :   1.9704549171   0.0430475871
|A - B| = 0.00e+00
|A - C| = 1.04e+00
```

NASIL ÇALIŞIYOR?

Yöntem (B) tam durumu (adım sayacı, parametreler, Adam'ın m ve v vektörleri) kaydettiği için kesintisiz eğitimle **bit düzeyinde aynı** sonucu verir (fark 0). Yöntem (C) yalnızca ağırlıkları geri yükler: Adam'ın hafızası (m, v) ve bias düzeltme sayacı sıfırlanır; sonraki adımlar yanlış ölçeklenir ve sonuç farklı çıkar. Gerçek büyük eğitimlerde bu fark kayıp sıçramalarına yol açabilir. **Kural:** checkpoint = ağırlıklar + optimizer durumu + adım + RNG + veri konumu. Ayrıca yazma sırasında çökme olursa dosya bozulabileceği için önce geçici dosyaya yazıp sonra yeniden adlandırmak iyi bir alışkanlıktır.

Eğitim bütçesini hesaplamak

Eğitime başlamadan "kaç adım, kaç token, ne kadar zaman" sorularını yanıtlamalısın. Basit ilişkiler:

$$\text{token/adım} = \text{batch} \times \text{dizi uzunluğu} \times \text{birikim adımı}$$

$$\text{adım sayısı} = \text{toplam token} \,/\, (\text{token/adım})$$

Ölçekleme yasalarının (Chinchilla) pratik özeti: **parametre sayısının yaklaşık 20 katı kadar token** ile eğitmek, sabit hesap bütçesi için iyi bir dengedir. Daha küçük ve uzun eğitilmiş modeller çıkarımda ucuzdur; bu yüzden pratikte 100+ token/parametre de yaygındır.

Örnek 8.7 — Eğitim bütçesi hesaplayıcısı

PROBLEM

Batch 32, dizi uzunluğu 1024, 4 adım gradient accumulation ile token/adım değerini bul. 124 milyon parametreli bir model için 20 token/parametre kuralıyla gereken token ve adım sayısını hesapla; 10 milyar token'lık bir veri kümesinin kaç epoch olduğunu bul.

ÇÖZÜM (FORTRAN)

```
program butce
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: tok_adim, params, tok_gerek, adim, veri, epoch
  tok_adim = 32.0_dp * 1024.0_dp * 4.0_dp
  params   = 124.0e6_dp
  tok_gerek = 20.0_dp * params
  adim  = tok_gerek / tok_adim
  veri  = 10.0e9_dp
  epoch = tok_gerek / veri
  print '(A,F12.0)', 'token / adim                  = ', tok_adim
  print '(A,F12.3,A)', 'gereken token (20 x param)    = ', tok_gerek / 1.0e9_dp, ' milyar'
  print '(A,F12.0)', 'optimizer adim sayisi         = ', adim
  print '(A,F12.3)', 'epoch (10 milyar tokenlik veri) = ', epoch
  print '(A,F12.0)', '10 milyar token icin adim     = ', veri / tok_adim
end program butce
```

ÇIKTI

```
token / adim                  =      131072.
gereken token (20 x param)    =        2.480 milyar
optimizer adim sayisi         =       18921.
epoch (10 milyar tokenlik veri) =        0.248
10 milyar token icin adim     =       76294.
```

PYTORCH KARŞILIĞI

```
tok_adim = 32 * 1024 * 4
params = 124e6
tok_gerek = 20 * params
print(f"token / adim                    = {tok_adim:,.0f}")
print(f"gereken token (20 x param)      = {tok_gerek / 1e9:.3f} milyar")
print(f"optimizer adim sayisi           = {tok_gerek / tok_adim:,.0f}")
print(f"epoch (10 milyar tokenlik veri) = {tok_gerek / 10e9:.3f}")
print(f"10 milyar token icin adim       = {10e9 / tok_adim:,.0f}")
```

PYTORCH ÇIKTISI

```
token / adim                    = 131,072
gereken token (20 x param)      = 2.480 milyar
optimizer adim sayisi           = 18,921
epoch (10 milyar tokenlik veri) = 0.248
10 milyar token icin adim       = 76,294
```

NASIL ÇALIŞIYOR?

Bir adımda 32 × 1024 × 4 = 131072 token işlenir. 124M parametre için 20 × 124M = 2.48 milyar token yeter; bu yaklaşık 18 bin optimizer adımı demektir ve 10 milyar tokenlik bir veri kümesinin yalnızca ~0.25 epoch'u kadardır, yani veri hiç tekrar edilmez. Aynı veriyle 10 milyar token'ı bitirmek ~76 bin adım sürer. Bu hesap, öğrenme oranı çizelgesinin toplam adımını (T) belirlemek için gereklidir.

BÖLÜM 9

9. Büyük Ölçekli Model Mühendisliği

Küçük bir model yazmakla gerçekten büyük bir model geliştirmek arasındaki fark: hesap, bellek ve iletişim mühendisliği.

Bu bölümde

- Parametre sayısı, bellek ve eğitim FLOP hesabı
- GPU belleğine sığmayan modeller: veri, tensör ve boru hattı paralelliği
- Dağıtık gradyan senkronizasyonu ve optimizer durumu belleği
- FlashAttention'ın mantığı: online softmax
- Quantization ve model sıkıştırma

Hedef: Bir modelin ne kadar bellek ve hesap isteyeceğini kâğıt üzerinde tahmin edebilmek ve paralellik stratejilerini sayılarla anlamak.

Parametre sayısı: bir Transformer'ın muhasebesi

Bir GPT'nin parametre sayısı kolayca hesaplanır. d model boyutu, L blok sayısı, V sözlük, T azami bağlam olsun. Bir blok (LayerNorm'lu, biaslı GPT-2 düzeni):

Parça	Parametre
Q, K, V projeksiyonları (bias dahil)	$3d^2 + 3d$
Attention çıkış projeksiyonu	$d^2 + d$
FFN birinci katman (d → 4d)	$4d^2 + 4d$
FFN ikinci katman (4d → d)	$4d^2 + d$
İki LayerNorm (γ, β)	$4d$
Blok toplamı	$\mathbf{12d^2 + 13d}$

Toplam: $\mathbf{V{\cdot}d + T{\cdot}d + L{\cdot}(12d^2 + 13d) + 2d}$ (token embedding + konum embedding + bloklar + son LayerNorm; çıkış matrisi embedding ile paylaşılır). Büyük modellerde baskın terim $\mathbf{12{\cdot}L{\cdot}d^2}$'dir.

Örnek 9.1 — GPT-2 ailesinin parametre sayısı

PROBLEM

Formülle GPT-2 small (L=12, d=768), medium (24, 1024), large (36, 1280) ve XL (48, 1600) modellerinin parametre sayısını hesapla. Embedding'in toplam içindeki payını da yaz.

ÇÖZÜM (FORTRAN)

```
program param_say
  implicit none
  call say('GPT-2 small ', 50257_8, 1024_8,  768_8, 12_8)
  call say('GPT-2 medium', 50257_8, 1024_8, 1024_8, 24_8)
  call say('GPT-2 large ', 50257_8, 1024_8, 1280_8, 36_8)
  call say('GPT-2 XL    ', 50257_8, 1024_8, 1600_8, 48_8)
contains
  subroutine say(name, V, T, d, L)
    character(len=*), intent(in) :: name
    integer(8), intent(in) :: V, T, d, L
    integer(8) :: per_layer, emb, total
    per_layer = 12*d*d + 13*d
    emb   = V*d + T*d
```

```
    total = emb + L*per_layer + 2*d
    print '(A,A,I12,A,F7.2,A,F6.1,A)', name, ' : ', total, ' parametre (', real(total)/1.0e6, &
          ' M)   embedding payi %', 100.0*real(emb)/real(total), ''
  end subroutine say
end program param_say
```

ÇIKTI

```
GPT-2 small  :    124439808 parametre ( 124.44 M)   embedding payi %  31.6
GPT-2 medium :    354823168 parametre ( 354.82 M)   embedding payi %  14.8
GPT-2 large  :    774030080 parametre ( 774.03 M)   embedding payi %   8.5
GPT-2 XL     :   1557611200 parametre (1557.61 M)   embedding payi %   5.3
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
class Blk(nn.Module):
    def __init__(s, d):
        super().__init__()
        s.ln1, s.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        s.qkv, s.proj = nn.Linear(d, 3 * d), nn.Linear(d, d)
        s.fc, s.out = nn.Linear(d, 4 * d), nn.Linear(4 * d, d)
class GPT2(nn.Module):
    def __init__(s, V, T, d, L):
        super().__init__()
        s.wte, s.wpe = nn.Embedding(V, d), nn.Embedding(T, d)
        s.h = nn.ModuleList(Blk(d) for _ in range(L)); s.ln_f = nn.LayerNorm(d)
        # cikis matrisi wte ile paylasildigi icin ayri parametre eklenmez (weight tying)
for ad, L, d in [("GPT-2 small ", 12, 768), ("GPT-2 medium", 24, 1024), ("GPT-2 large ", 36, 1280),
      ("GPT-2 XL    ", 48, 1600)]:
    with torch.device("meta"):                      # bellek ayirmadan modeli kur (sadece sekiller)
        m = GPT2(50257, 1024, d, L)
    n = sum(p.numel() for p in m.parameters())
    emb = m.wte.weight.numel() + m.wpe.weight.numel()
    print(f"{ad} : {n:12d} parametre ({n/1e6:7.2f} M)   embedding payi %{100*emb/n:5.1f}")
```

PYTORCH ÇIKTISI

```
GPT-2 small  :    124439808 parametre ( 124.44 M)   embedding payi % 31.6
GPT-2 medium :    354823168 parametre ( 354.82 M)   embedding payi % 14.8
GPT-2 large  :    774030080 parametre ( 774.03 M)   embedding payi %  8.5
GPT-2 XL     :   1557611200 parametre (1557.61 M)   embedding payi %  5.3
```

NASIL ÇALIŞIYOR?

GPT-2 small için blok başına $12 \cdot 768^2 + 13 \cdot 768 = 7\,087\,872$, 12 blok 85 054 464; embedding 38 597 376 + 786 432; toplam **124 439 808** (kamuoyunda "124M" olarak bilinen sayı). Medium 354.8M, large 774M, XL 1.56 milyar çıkar: resmî rakamlarla uyuşur. Embedding payı küçük modelde ~%31, büyük modelde ~%5'e iner; model büyüdükçe parametrelerin neredeyse tamamı bloklardadır ve sayı yaklaşık $12 \cdot L \cdot d^2$ ile tahmin edilebilir (XL için $12 \cdot 48 \cdot 1600^2 \approx 1.47$ milyar).

Bellek hesabı

Eğitimde bellekte yalnızca ağırlıklar değil, **gradyanlar**, **optimizer durumu** ve **aktivasyonlar** da durur. Parametre başına bayt:

Kalem	FP32 eğitim	Mixed precision (BF16 + FP32 master)
Ağırlık	4	2 (bf16) + 4 (fp32 master)
Gradyan	4	2
Adam m ve v	8	8
Toplam	**16 bayt/param**	**16 bayt/param**

Aktivasyonlar buna ek olarak batch × dizi uzunluğu × katman sayısıyla büyür (checkpointing ile azaltılır). Çıkarımda yalnızca ağırlıklar gerekir: FP16 için 2 bayt/param.

Örnek 9.2 — Bellek hesaplayıcısı: 1.5B, 7B, 13B, 70B

PROBLEM

Her model boyutu için FP32 ağırlık belleğini, Adam ile eğitim durumunu (16 bayt/param, aktivasyonsuz), FP16/INT8/INT4 çıkarım belleğini ve 80 GB'lık GPU'larda eğitim durumunu sığdırmak için gereken en az GPU sayısını hesapla.

ÇÖZÜM (FORTRAN)

```
program bellek
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: n(4), gb
  integer :: i
  n = [1.5e9_dp, 7.0e9_dp, 13.0e9_dp, 70.0e9_dp]
  gb = 1.0e9_dp
  print '(A)', '  model   FP32-agirlik  egitim(16B)  FP16-infer  INT8   INT4   min GPU(80GB)'
  do i = 1, 4
    print '(F6.1,A,F9.1,A,F12.1,A,F10.1,A,F7.1,F7.1,I9)', n(i)/gb, 'B', 4*n(i)/gb, ' GB', &
          16*n(i)/gb, ' GB', 2*n(i)/gb, ' GB', n(i)/gb, 0.5_dp*n(i)/gb, &
          ceiling(16*n(i)/gb/80.0_dp)
  end do
end program bellek
```

ÇIKTI

```
  model   FP32-agirlik  egitim(16B)  FP16-infer  INT8   INT4   min GPU(80GB)
   1.5B      6.0 GB        24.0 GB       3.0 GB    1.5    0.8        1
   7.0B     28.0 GB       112.0 GB      14.0 GB    7.0    3.5        2
  13.0B     52.0 GB       208.0 GB      26.0 GB   13.0    6.5        3
  70.0B    280.0 GB      1120.0 GB     140.0 GB   70.0   35.0       14
```

PYTORCH KARŞILIĞI

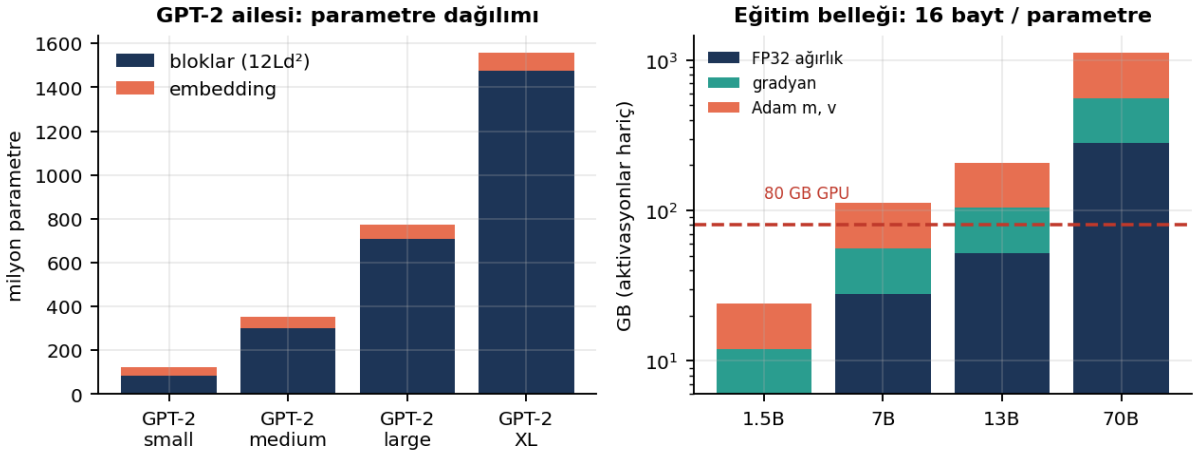
```
import torch, torch.nn as nn
# gercek olcum: Adam ile tek adim sonrasi agirlik + gradyan + optimizer durumu (FP32)
m = nn.Linear(1000, 1000, bias=False)
opt = torch.optim.Adam(m.parameters())
m(torch.randn(8, 1000)).sum().backward(); opt.step()
n = m.weight.numel()
w = m.weight.numel() * m.weight.element_size(); g = m.weight.grad.numel() * 4
st = sum(v.numel() * v.element_size() for v in opt.state[m.weight].values() if v.dim() > 0)
print(f"parametre = {n}   agirlik {w/n:.0f} B/param + gradyan {g/n:.0f} + Adam durumu {st/n:.0f} =
      {(w+g+st)/n:.0f} bayt/param")
print("  model     FP32-agirlik  egitim(16B)  FP16-infer  INT8   INT4   min GPU(80GB)")
for N in [1.5e9, 7e9, 13e9, 70e9]:
    print(f"{N/1e9:6.1f}B {4*N/1e9:11.1f} GB {16*N/1e9:10.1f} GB {2*N/1e9:8.1f} GB
          {N/1e9:6.1f}{0.5*N/1e9:7.1f}{-(-16*N/1e9//80):9.0f}")
```

PYTORCH ÇIKTISI

```
parametre = 1000000   agirlik 4 B/param + gradyan 4 + Adam durumu 8 = 16 bayt/param
  model     FP32-agirlik  egitim(16B)  FP16-infer  INT8   INT4   min GPU(80GB)
   1.5B        6.0 GB       24.0 GB      3.0 GB    1.5    0.8        1
   7.0B       28.0 GB      112.0 GB     14.0 GB    7.0    3.5        2
  13.0B       52.0 GB      208.0 GB     26.0 GB   13.0    6.5        3
  70.0B      280.0 GB     1120.0 GB    140.0 GB   70.0   35.0       14
```

NASIL ÇALIŞIYOR?

7 milyar parametre FP32'de 28 GB ağırlık demektir, ama Adam ile eğitim durumu 16 × 7 = **112 GB** eder; bu tek bir 80 GB GPU'ya sığmaz bile (aktivasyonlar hariç). 70B model için 1120 GB gerekir, yani en az 14 adet 80 GB'lık GPU. Çıkarımda ise 7B model FP16'da 14 GB, INT4'te 3.5 GB'a iner; bu yüzden quantization küçük cihazlarda büyük modelleri çalıştırmanın anahtarıdır. Gerçekte aktivasyonlar, iletişim tamponları ve parçalanma da bellek yer; pratikte gereken bellek bu tablodan %20–50 daha fazladır.



Şekil 9.1 — Solda: GPT-2 ailesinin parametre dağılımı (büyük modellerde blokların payı baskın). Sağda: eğitim belleğinin bileşenleri (16 bayt/parametre) ve 80 GB GPU sınırı.

Eğitim FLOP hesabı

Basit matematik

Bir parametre, ileri geçişte bir çarpma + bir toplama = 2 FLOP yapar. Geri geçişte iki matris çarpımı gerekir (aktivasyon gradyanı ve ağırlık gradyanı) = 4 FLOP. Toplam:

token başına eğitim FLOP ≈ 6 N (N = parametre sayısı)

D token ile eğitim: **toplam FLOP ≈ 6 N D**. (Attention'ın ek, dizi uzunluğuna bağlı terimi küçük bağlamlarda ihmal edilir.)

Örnek 9.3 — Eğitim süresi tahmini: 6ND kuralı

PROBLEM

GPT-2 small (124M) ve 7B modelin eğitim FLOP'unu hesapla. A100 GPU (BF16 tepe 312 TFLOP/s) %40 verimle çalışırsa kaç GPU-gün ve 256 GPU ile kaç gün sürer?

ÇÖZÜM (FORTRAN)

```
program flop_hesap
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp), parameter :: peak = 312.0e12_dp, mfu = 0.40_dp
  call tahmin('124M, 20 token/param ', 124.0e6_dp, 20.0_dp*124.0e6_dp)
  call tahmin('124M, 300B token     ', 124.0e6_dp, 300.0e9_dp)
  call tahmin('7B, 1T token         ', 7.0e9_dp, 1.0e12_dp)
contains
  subroutine tahmin(ad, n, d)
    character(len=*), intent(in) :: ad
    real(dp), intent(in) :: n, d
    real(dp) :: flop, sec
    flop = 6.0_dp * n * d
    sec  = flop / (peak * mfu)                    ! tek GPU icin saniye
    print '(A,A,ES10.2,A,F9.1,A,F8.2,A)', ad, ' FLOP=', flop, '   GPU-gun=', sec/86400.0_dp, &
          '   256 GPU ile ', sec/86400.0_dp/256.0_dp, ' gun'
  end subroutine tahmin
end program flop_hesap
```

ÇIKTI

```
124M, 20 token/param  FLOP=  1.85E+18   GPU-gun=      0.2   256 GPU ile     0.00 gun
124M, 300B token      FLOP=  2.23E+20   GPU-gun=     20.7   256 GPU ile     0.08 gun
7B, 1T token          FLOP=  4.20E+22   GPU-gun=   3895.1   256 GPU ile    15.22 gun
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn
from torch.utils.flop_counter import FlopCounterMode
# 6 N D kuralini olc: 4 katmanli MLP, N = 4*256*256 parametre, D = 64 token
net = nn.Sequential(*[nn.Linear(256, 256, bias=False) for _ in range(4)])
x = torch.randn(64, 256)
with FlopCounterMode(display=False) as fc:
    net(x).sum().backward()
N = sum(p.numel() for p in net.parameters()); D = 64
print(f"olculen FLOP = {fc.get_total_flops():.3e}    6*N*D = {6*N*D:.3e}")
peak, mfu = 312e12, 0.40
for ad, n, d in [("124M, 20 token/param", 124e6, 20 * 124e6), ("124M, 300B token", 124e6, 300e9), ("7B,
      1T token", 7e9, 1e12)]:
    flop = 6 * n * d; sec = flop / (peak * mfu)
    print(f"{ad:22s} FLOP={flop:9.2e}  GPU-gun={sec/86400:9.1f}  256 GPU ile {sec/86400/256:8.2f} gun")
```

PYTORCH ÇIKTISI

```
olculen FLOP = 9.227e+07    6*N*D = 1.007e+08
124M, 20 token/param   FLOP= 1.85e+18  GPU-gun=      0.2  256 GPU ile     0.00 gun
124M, 300B token       FLOP= 2.23e+20  GPU-gun=     20.7  256 GPU ile     0.08 gun
7B, 1T token           FLOP= 4.20e+22  GPU-gun=   3895.1  256 GPU ile    15.22 gun
```

NASIL ÇALIŞIYOR?

124M × 2.48 milyar token yalnızca ~1.8e18 FLOP ister: tek bir A100'de bir günden kısa sürer. Aynı model 300 milyar token ile eğitilirse 2.2e20 FLOP, ~21 GPU-gün eder. 7B model, 1 trilyon token için 4.2e22 FLOP ve yaklaşık **3900 GPU-gün** gerektirir: 256 GPU ile ~15 gün. Buradaki %40 "MFU" (model FLOP kullanımı) iyi bir ayarlanmış sistemin gerçekçi değeridir; naif kodda %10 altına düşer. Bu yüzden Bölüm 5'teki performans mühendisliği doğrudan faturaya yansır.

Türetme: eğitim FLOP ≈ 6ND

6ND'nin türetilmesi. W (m×n) ağırlıklı bir doğrusal katman y = Wx tek token için: • ileri: m·n çarpma + m·n toplama = **2mn** FLOP; • geri: $\partial L/\partial x = W^T\delta \rightarrow$ 2mn FLOP, $\partial L/\partial W = \delta x^T \rightarrow$ 2mn FLOP; toplam **4mn**. Parametre sayısı N = mn olduğundan token başına ileri 2N, geri 4N, toplam **6N** FLOP; D token için 6ND. Attention'ın ek terimi (QK^T ve AV: katman başına 4·T·d FLOP/token) bağlam uzunluğu T ile büyür ve T ≈ 12d civarına kadar ihmal edilebilir. Aktivasyon yeniden hesaplaması (checkpointing) ileriyi bir kez daha yapar: 8N.

Veri paralelliği (data parallelism)

En basit paralellik: modeli **her GPU'ya kopyala**, global batch'i GPU'lara böl. Her GPU kendi parçası için gradyan hesaplar, sonra gradyanlar **all-reduce** ile ortalanır ve herkes aynı güncellemeyi uygular. Ağırlıklar tüm kopyalarda aynı kalır. Sınırı: model (ağırlık + optimizer) tek GPU'ya sığmalıdır. **ZeRO/FSDP** yöntemleri bu kısıtı, optimizer durumu, gradyan ve ağırlıkları GPU'lara *parçalayarak* aşar.

Örnek 9.4 — Veri paralelliğini simüle et: all-reduce = tam batch gradyanı

PROBLEM

Doğrusal bir modelde 32 örneği 4 "işçiye" böl. Her işçi yerel gradyanını hesaplasın, all-reduce (ortalama) ile birleşsin. Sonucun tek işçili tam batch gradyanına eşit olduğunu göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program veri_paralel
  use rng_mod
  implicit none
  integer, parameter :: N = 32, W = 4, per = N / W
  real(dp) :: x(N), y(N), w1, b1, g_full(2), g_loc(2,W), g_avg(2)
  integer :: k, i0, i1
  call rng_seed(3)
  do k = 1, N
    x(k) = gauss()
    y(k) = 3.0_dp * x(k) - 1.0_dp + 0.2_dp * gauss()
  end do
  w1 = 0.5_dp; b1 = 0.0_dp
  g_full(1) = sum(2.0_dp * (w1*x + b1 - y) * x) / N
  g_full(2) = sum(2.0_dp * (w1*x + b1 - y)) / N
  do k = 1, W                                      ! her isci kendi dilimi icin
    i0 = (k-1)*per + 1; i1 = k*per
    g_loc(1,k) = sum(2.0_dp * (w1*x(i0:i1) + b1 - y(i0:i1)) * x(i0:i1)) / per
    g_loc(2,k) = sum(2.0_dp * (w1*x(i0:i1) + b1 - y(i0:i1))) / per
    print '(A,I1,A,2F10.5)', 'isci ', k, ' yerel gradyan : ', g_loc(:,k)
  end do
  g_avg = sum(g_loc, dim=2) / W                    ! all-reduce (ortalama)
  print '(A,2F10.5)', 'all-reduce ortalama  : ', g_avg
  print '(A,2F10.5)', 'tek isci tam batch   : ', g_full
  print '(A,ES10.2)', 'fark = ', maxval(abs(g_avg - g_full))
end program veri_paralel
```

ÇIKTI

```
isci 1 yerel gradyan :   -6.60944   0.40033
isci 2 yerel gradyan :   -2.52871   0.65800
isci 3 yerel gradyan :   -2.31510   2.15457
isci 4 yerel gradyan :   -6.56455   5.95604
all-reduce ortalama  :   -4.50445   2.29223
tek isci tam batch   :   -4.50445   2.29223
fark =   8.88E-16
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(3)
N, W = 32, 4
x = torch.randn(N, dtype=torch.float64); y = 3 * x - 1 + 0.2 * torch.randn(N, dtype=torch.float64)
w = torch.tensor([0.5, 0.0], dtype=torch.float64, requires_grad=True)
def grad(xs, ys):
    w.grad = None
    ((w[0] * xs + w[1] - ys) ** 2).mean().backward()
    return w.grad.clone()
g_full = grad(x, y)
g_loc = torch.stack([grad(x[k*8:(k+1)*8], y[k*8:(k+1)*8]) for k in range(W)])   # her "isci"
for k in range(W): print(f"isci {k+1} yerel gradyan :", [round(v, 5) for v in g_loc[k].tolist()])
g_avg = g_loc.mean(0)                            # all-reduce (ortalama)
print("all-reduce ortalama  :", [round(v, 5) for v in g_avg.tolist()])
print("tek isci tam batch   :", [round(v, 5) for v in g_full.tolist()])
print("fark =", (g_avg - g_full).abs().max().item())
# gercek dagitik egitimde: dist.all_reduce(p.grad); p.grad /= world_size   (DistributedDataParallel bunu
      otomatik yapar)
```

PYTORCH ÇIKTISI

```
isci 1 yerel gradyan : [-3.34032, 2.74105]
isci 2 yerel gradyan : [-2.67471, 0.36885]
isci 3 yerel gradyan : [-6.45751, 3.7406]
isci 4 yerel gradyan : [-2.18767, 1.88629]
all-reduce ortalama  : [-3.66505, 2.1842]
tek isci tam batch   : [-3.66505, 2.1842]
fark = 4.440892098500626e-16
```

NASIL ÇALIŞIYOR?

Her işçi eşit boyutlu (8 örnek) bir dilimin gradyan *ortalamasını* verir; ortalamaların ortalaması, eşit boyutlu dilimler için tüm verinin ortalamasına eşittir. Bu yüzden veri paralelliği **matematiksel olarak tek GPU'da büyük batch ile aynıdır**. Gerçekte all-reduce, gradyan vektörünün (7B model

için ~28 GB) GPU'lar arasında ağ üzerinden toplanması demektir; hesabın iletişimle örtüştürülmesi (overlap) performansın anahtarıdır.

Tensör paralelliği (tensor parallelism)

Model tek GPU'ya sığmıyorsa *tek bir matris çarpımını* GPU'lara böleriz. Transformer MLP'si için Megatron-LM hilesi: ilk matrisi **sütunlardan**, ikinci matrisi **satırlardan** böl. Birinci katmanda her GPU kendi sütun dilimini bağımsız hesaplar (iletişim yok); aktivasyon eleman bazlı olduğu için yerel uygulanır; ikinci katmanda her GPU bir **kısmi toplam** üretir ve tek bir all-reduce ile toplanır.

Örnek 9.5 — MLP'yi iki "GPU"ya böl: sütun + satır paralelliği

PROBLEM

$Y = \mathrm{GELU}(X \cdot W_1) \cdot W_2$ hesabını tek parçada ve iki cihaza bölünmüş hâlde (W_1 sütunlardan, W_2 satırlardan) yap. Sonuçların aynı olduğunu göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program tensor_paralel
  use rng_mod
  implicit none
  integer, parameter :: T = 4, d = 6, h = 8, hh = h / 2
  real(dp) :: X(T,d), W1(d,h), W2(h,d), Y(T,d), Ya(T,d), Yb(T,d), Ha(T,hh), Hb(T,hh)
  call rng_seed(12)
  call randn2(X, 1.0_dp); call randn(W1, 0.5_dp); call randn2(W2, 0.5_dp)
  ! tek parca
  Y = matmul(gelu(matmul(X, W1)), W2)
  ! cihaz A: W1'in ilk 4 sutunu, W2'nin ilk 4 satiri; cihaz B: kalanlar
  Ha = gelu(matmul(X, W1(:, 1:hh)))                 ! iletisim yok
  Hb = gelu(matmul(X, W1(:, hh+1:h)))
  Ya = matmul(Ha, W2(1:hh, :))                      ! kismi toplamlar
  Yb = matmul(Hb, W2(hh+1:h, :))
  print '(A,ES10.2)', 'max |Y - (Ya + Yb)| = ', maxval(abs(Y - (Ya + Yb)))   ! tek all-reduce
  print '(A,I0,A,I0)', 'cihaz basina W1 sutunu: ', hh, ' / ', h
contains
  elemental function gelu(z) result(r)
    real(dp), intent(in) :: z
    real(dp) :: r
    r = 0.5_dp * z * (1.0_dp + tanh(0.7978845608028654_dp * (z + 0.044715_dp * z**3)))
  end function gelu
end program tensor_paralel
```

ÇIKTI

```
max |Y - (Ya + Yb)| =   2.22E-16
cihaz basina W1 sutunu: 4 / 8
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn.functional as F
torch.manual_seed(12)
T_, d, h = 4, 6, 8
X = torch.randn(T_, d, dtype=torch.float64)
W1 = 0.5 * torch.randn(d, h, dtype=torch.float64); W2 = 0.5 * torch.randn(h, d, dtype=torch.float64)
gelu = lambda z: F.gelu(z, approximate='tanh')
Y = gelu(X @ W1) @ W2                                  # tek parca
Ha = gelu(X @ W1[:, :h // 2]); Hb = gelu(X @ W1[:, h // 2:])   # sutun paralelligi: iletisim yok
Ya = Ha @ W2[:h // 2]; Yb = Hb @ W2[h // 2:]                    # satir paralelligi: kismi toplamlar
print("max |Y - (Ya + Yb)| =", (Y - (Ya + Yb)).abs().max().item())    # tek all-reduce
print("cihaz basina W1 sutunu:", h // 2, "/", h)
```

PYTORCH ÇIKTISI

```
max |Y - (Ya + Yb)| = 2.220446049250313e-16
cihaz basina W1 sutunu: 4 / 8
```

NASIL ÇALIŞIYOR?

GELU eleman bazlıdır: $\text{gelu}(X\cdot[W_1^A \mid W_1^B]) = [\text{gelu}(X\cdot W_1^A) \mid \text{gelu}(X\cdot W_1^B)]$. Dolayısıyla sütunlardan bölünen ilk katman **hiç iletişim istemez**. İkinci katmanda çarpım $[H^A \mid H^B] \cdot [W_2^A; W_2^B] = H^A\cdot W_2^A + H^B\cdot W_2^B$ şeklinde bloklara ayrılır; iki kısmi sonuç toplanarak (all-reduce) tam sonuç elde edilir. Bir Transformer bloğunda ileri geçişte attention için bir, MLP için bir olmak üzere yalnızca iki all-reduce yeterlidir. Tensör paralelliği hızlı GPU-içi bağlantılar (NVLink) gerektirir, bu yüzden çoğunlukla tek bir düğüm içinde kullanılır.

Boru hattı paralelliği (pipeline parallelism)

Katmanları ardışık gruplara (**stage**) bölüp her grubu başka bir GPU'ya koyarız. Tek bir batch'i işlerken yalnızca bir GPU çalışır, diğerleri boşta bekler. Çözüm: batch'i **m mikro-batch**'e bölüp boru hattından peş peşe geçirmek. Başta ve sonda kaçınılmaz boşluklara **"kabarcık" (bubble)** denir:

$$\text{kabarcık oranı} = (p - 1) / (m + p - 1) \quad (p: \text{stage sayısı, } m: \text{mikro-batch sayısı})$$

Örnek 9.6 — Boru hattı çizelgesi ve kabarcık oranı

PROBLEM

p = 4 stage ve m = 6 mikro-batch için ileri geçiş çizelgesini ASCII olarak çiz ("." = boşta). Sonra p = 4 için m değiştikçe kabarcık oranını tablola.

ÇÖZÜM (FORTRAN)

```
program pipeline
  implicit none
  integer, parameter :: p = 4, m = 6
  integer :: s, t, mb, ms(5)
  print '(A)', 'zaman ->  (sayi = mikro-batch numarasi, . = bosta)'
  do s = 1, p
    write(*,'(A,I1,A)', advance='no') 'stage ', s, ' | '
    do t = 0, m + p - 2
      mb = t - (s - 1) + 1                          ! bu stage'in bu adimdaki mikro-batch'i
      if (mb >= 1 .and. mb <= m) then
        write(*,'(I3)', advance='no') mb
      else
        write(*,'(A3)', advance='no') ' .'
      end if
    end do
    print *
  end do
  print '(A,F6.3)', 'kabarcik orani (p=4, m=6) = ', real(p-1) / real(m+p-1)
  ms = [1, 4, 8, 16, 32]
  print '(A)', 'p = 4 icin:  m     kabarcik orani'
  do t = 1, 5
    print '(I14,F15.3)', ms(t), real(p-1) / real(ms(t)+p-1)
  end do
end program pipeline
```

ÇIKTI

```
zaman ->  (sayi = mikro-batch numarasi, . = bosta)
stage 1 |   1  2  3  4  5  6  .  .  .
stage 2 |   .  1  2  3  4  5  6  .  .
stage 3 |   .  .  1  2  3  4  5  6  .
stage 4 |   .  .  .  1  2  3  4  5  6
kabarcik orani (p=4, m=6) =  0.333
p = 4 icin:  m     kabarcik orani
             1          0.750
             4          0.429
             8          0.273
            16          0.158
            32          0.086
```

PYTORCH KARŞILIĞI

```
p, m = 4, 6
print("zaman ->  (sayi = mikro-batch numarasi, . = bosta)")
for s in range(1, p + 1):
    row = ""
    for t in range(m + p - 1):
        mb = t - (s - 1) + 1
        row += f"{mb:3d}" if 1 <= mb <= m else "  ."
    print(f"stage {s} | {row}")
print(f"kabarcik orani (p=4, m=6) = {(p - 1) / (m + p - 1):.3f}")
print("p = 4 icin:  m     kabarcik orani")
for mm in [1, 4, 8, 16, 32]:
    print(f"{mm:14d}{(p - 1) / (mm + p - 1):15.3f}")
```

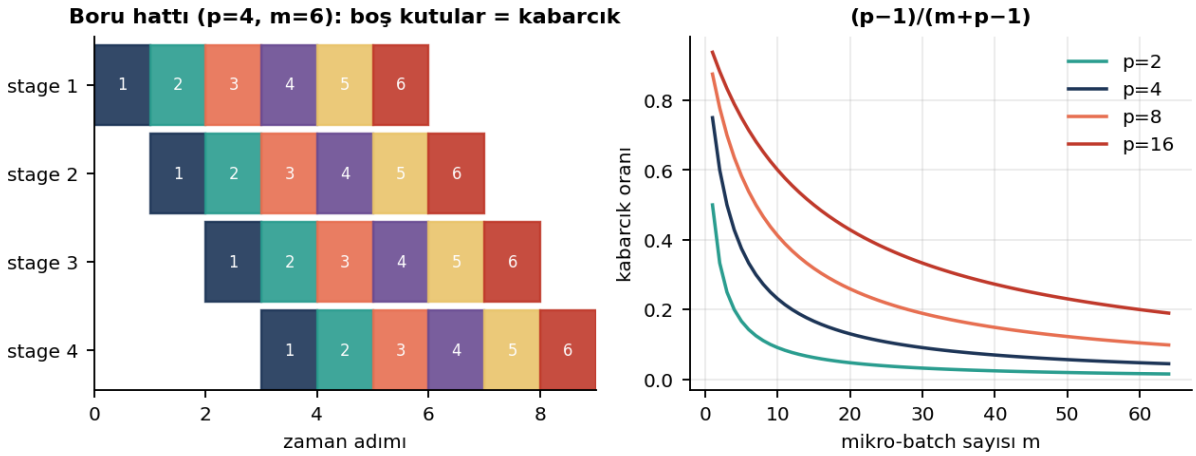
PYTORCH ÇIKTISI

```
zaman ->  (sayi = mikro-batch numarasi, . = bosta)
stage 1 |   1  2  3  4  5  6  .  .  .
stage 2 |   .  1  2  3  4  5  6  .  .
stage 3 |   .  .  1  2  3  4  5  6  .
stage 4 |   .  .  .  1  2  3  4  5  6
kabarcik orani (p=4, m=6) = 0.333
p = 4 icin:  m     kabarcik orani
             1          0.750
             4          0.429
             8          0.273
            16          0.158
            32          0.086
```

NASIL ÇALIŞIYOR?

Stage 1, mikro-batch 1'i bitirdiği anda stage 2 onu almaya başlar; stage 1 ise 2. mikro-batch'e geçer. Sonuç çapraz bir "dalga" (parallelogram) desenidir. Başta $p-1$ adım ve sonda $p-1$ adım bazı stage'ler boştadır. $m = 6$ için toplam 9 zaman adımı vardır ve $3/9 = \%33$ kabarcık oluşur.

Mikro-batch sayısı arttıkça kabarcık küçülür ($m = 32$ için $\approx \%9$). Bu yüzden boru hattı paralelliği çok sayıda mikro-batch ile kullanılır ve genellikle düğümler *arasında* (daha yavaş ağ üzerinde) tercih edilir, çünkü yalnızca stage sınırındaki aktivasyonlar iletilir.



Şekil 9.2 — Solda: boru hattı çizelgesi (p=4 stage, m=6 mikro-batch; renkler mikro-batch'ler, boş kutular kabarcık). Sağda: kabarcık oranı (p−1)/(m+p−1) mikro-batch sayısı arttıkça azalır.

FlashAttention'ın mantığı: online softmax

Standart attention, T × T skor matrisini belleğe yazar: T = 8192 için bile 67 milyon eleman, katman ve head başına! Asıl darboğaz hesap değil **bellek trafiğidir**. FlashAttention, skorları bloklar hâlinde işleyip tüm matrisi hiç oluşturmaz. Kritik hile: softmax'ı **parça parça** (online) hesaplamak. Her blok için koşan maksimum m, koşan toplam l ve koşan çıktı toplamı tutulur; yeni blokta maksimum değişirse eski birikim düzeltilir:

$$m' = \max(m, \max(\text{blok})),\ \text{ölçek} = e^{m - m'}$$

$$l \leftarrow l \cdot \text{ölçek} + \Sigma\, e^{s - m'},\ \text{acc} \leftarrow \text{acc} \cdot \text{ölçek} + \Sigma\, e^{s - m'} \cdot v,\ \text{sonuç } o = \text{acc} / l$$

Örnek 9.7 — Online softmax: blok blok attention, aynı sonuç

PROBLEM

10 skorluk bir attention çıktısını (1) tam softmax ile, (2) 3'lük bloklar hâlinde online softmax ile hesapla. Sonuçların aynı olduğunu göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program flash_mantik
  use rng_mod
  implicit none
  integer, parameter :: n = 10, dv = 3, bs = 3
  real(dp) :: s(n), V(n,dv), p(n), o_ref(dv), o(dv), acc(dv)
  real(dp) :: m, mnew, l, scale, ex(bs)
  integer :: i0, i1, nb
  call rng_seed(31)
  call randn1(s, 2.0_dp); call randn2(V, 1.0_dp)
  ! (1) standart: tum skorlar bellekte
  p = exp(s - maxval(s)); p = p / sum(p)
  o_ref = matmul(p, V)
  ! (2) online: skorlari bloklar halinde isle
  m = -huge(1.0_dp); l = 0.0_dp; acc = 0.0_dp
  do i0 = 1, n, bs
    i1 = min(i0 + bs - 1, n); nb = i1 - i0 + 1
    mnew  = max(m, maxval(s(i0:i1)))
    scale = exp(m - mnew)                          ! eski birikimi duzelt
    ex(1:nb) = exp(s(i0:i1) - mnew)
    l   = l * scale + sum(ex(1:nb))
    acc = acc * scale + matmul(ex(1:nb), V(i0:i1, :))
    m = mnew
    print '(A,I2,A,I2,A,F8.4,A,F8.4)', 'blok ', i0, '-', i1, '  kosan max=', m, &
          '  kosan toplam=', l
```

```
  end do
  o = acc / l
  print '(A,3F10.6)', 'standart  : ', o_ref
  print '(A,3F10.6)', 'online    : ', o
  print '(A,ES10.2)', 'fark = ', maxval(abs(o - o_ref))
end program flash_mantik
```

ÇIKTI

```
blok  1- 3  kosan max=  1.7330  kosan toplam=  1.2735
blok  4- 6  kosan max=  4.2398  kosan toplam=  1.1875
blok  7- 9  kosan max=  4.2398  kosan toplam=  2.4957
blok 10-10  kosan max=  4.2398  kosan toplam=  2.5398
standart  :   0.265056 -0.100945  1.074635
online    :   0.265056 -0.100945  1.074635
fark =   1.80E-16
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn.functional as F
torch.manual_seed(31)
n, dv, bs = 10, 3, 3
s = 2 * torch.randn(n, dtype=torch.float64); V = torch.randn(n, dv, dtype=torch.float64)
o_ref = torch.softmax(s, 0) @ V                          # standart: tum skorlar bellekte
m = torch.tensor(-float("inf"), dtype=torch.float64); l = torch.tensor(0.0, dtype=torch.float64)
acc = torch.zeros(dv, dtype=torch.float64)
for i0 in range(0, n, bs):                               # online softmax: blok blok
    sb, vb = s[i0:i0 + bs], V[i0:i0 + bs]
    mnew = torch.maximum(m, sb.max())
    scale = torch.exp(m - mnew)                          # eski birikimi duzelt
    e = torch.exp(sb - mnew)
    l = l * scale + e.sum(); acc = acc * scale + e @ vb; m = mnew
    print(f"blok {i0+1:2d}-{min(i0+bs, n):2d}  kosan max={m:8.4f}  kosan toplam={l:8.4f}")
o = acc / l
print("standart  :", [round(v, 6) for v in o_ref.tolist()])
print("online    :", [round(v, 6) for v in o.tolist()])
print("fark =", (o - o_ref).abs().max().item())
q = torch.randn(1, 1, 1, 8); k = torch.randn(1, 1, 5, 8); v = torch.randn(1, 1, 5, 8)
print("F.scaled_dot_product_attention (GPU'da FlashAttention cekirdegi kullanir):",
      tuple(F.scaled_dot_product_attention(q, k, v).shape))
```

PYTORCH ÇIKTISI

```
blok  1- 3  kosan max=  0.7038  kosan toplam=  1.3862
blok  4- 6  kosan max=  1.8314  kosan toplam=  1.6043
blok  7- 9  kosan max=  2.1114  kosan toplam=  2.3350
blok 10-10  kosan max=  2.1114  kosan toplam=  2.7190
standart  : [-0.01342, 0.75456, 0.732018]
online    : [-0.01342, 0.75456, 0.732018]
fark = 2.220446049250313e-16
F.scaled_dot_product_attention (GPU'da FlashAttention cekirdegi kullanir): (1, 1, 1, 8)
```

NASIL ÇALIŞIYOR?

Online yöntem hiçbir zaman 10 skorun hepsini birden tutmaz; yalnızca bir blok ve üç koşan değişken (m, l, acc) yeterlidir. Yeni bir blokta maksimum büyürse önceki birikimin tümü $e^{m-m'}$ ile çarpılarak yeni referansa taşınır; böylece sonuç matematiksel olarak *birebir* aynıdır. Gerçek FlashAttention bu hesabı GPU'nun hızlı SRAM'inde bloklar hâlinde yapar ve T × T matrisini yüksek bant genişlikli ana belleğe hiç yazmaz; bellek $O(T^2)$'den $O(T)$'ye iner ve hesap hızlanır. Geri geçişte skorlar bellekten okunmak yerine yeniden hesaplanır (checkpointing fikri).

Quantization: ağırlıkları az bitle saklamak

Quantization, kayan noktalı ağırlıkları az bitli tamsayılara (INT8, INT4) eşler. Basit simetrik şema: ölçek $s = \max|w| / q_{max}$ (INT8 için $q_{max} = 127$), $q = \text{yuvarla}(w / s)$, geri çevirirken $\hat{w} = s \cdot q$. Hata en fazla s/2 kadardır. Tüm tensör için tek ölçek (*per-tensor*) kullanılırsa tek bir **aykırı değer** ölçeği büyütür ve diğer tüm ağırlıklar kaba yuvarlanır. Çözüm: küçük gruplar (örneğin 32–128 ağırlık) için

ayrı ölçek (*per-group*).

Örnek 9.8 — INT8 ve INT4 quantization: aykırı değer sorunu

PROBLEM

16 ağırlıklı bir vektörde (biri aykırı değer) INT8 ve INT4 simetrik quantization uygula; per-tensor ve per-group (4'lük gruplar) ölçekleri karşılaştır; RMS hatasını yazdır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program kuantizasyon
  use rng_mod
  implicit none
  integer, parameter :: n = 16
  real(dp) :: w(n), dq(n)
  call rng_seed(8)
  call randn1(w, 0.05_dp)
  w(5) = 0.60_dp                                  ! aykiri deger
  print '(A,F8.4)', 'agirlik RMS      = ', sqrt(sum(w**2) / n)
  call quantize(w, 8, n, dq)
  print '(A,ES10.2)', 'INT8 per-tensor   RMS hata = ', sqrt(sum((w-dq)**2) / n)
  call quantize(w, 8, 4, dq)
  print '(A,ES10.2)', 'INT8 per-group(4) RMS hata = ', sqrt(sum((w-dq)**2) / n)
  call quantize(w, 4, n, dq)
  print '(A,ES10.2)', 'INT4 per-tensor   RMS hata = ', sqrt(sum((w-dq)**2) / n)
  call quantize(w, 4, 4, dq)
  print '(A,ES10.2)', 'INT4 per-group(4) RMS hata = ', sqrt(sum((w-dq)**2) / n)
contains
  subroutine quantize(w, bits, gs, deq)
    real(dp), intent(in)  :: w(:)
    integer, intent(in)   :: bits, gs
    real(dp), intent(out) :: deq(:)
    integer :: i0, i1, qmax
    real(dp) :: sc
    qmax = 2**(bits - 1) - 1                      ! INT8: 127, INT4: 7
    do i0 = 1, size(w), gs
      i1 = min(i0 + gs - 1, size(w))
      sc = maxval(abs(w(i0:i1))) / qmax
      if (sc == 0.0_dp) sc = 1.0_dp
      deq(i0:i1) = sc * real(max(-qmax, min(qmax, nint(w(i0:i1) / sc))), dp)
    end do
  end subroutine quantize
end program kuantizasyon
```

ÇIKTI

```
agirlik RMS      =   0.1622
INT8 per-tensor   RMS hata =   1.38E-03
INT8 per-group(4) RMS hata =   8.54E-04
INT4 per-tensor   RMS hata =   1.97E-02
INT4 per-group(4) RMS hata =   6.91E-03
```

PYTORCH KARŞILIĞI

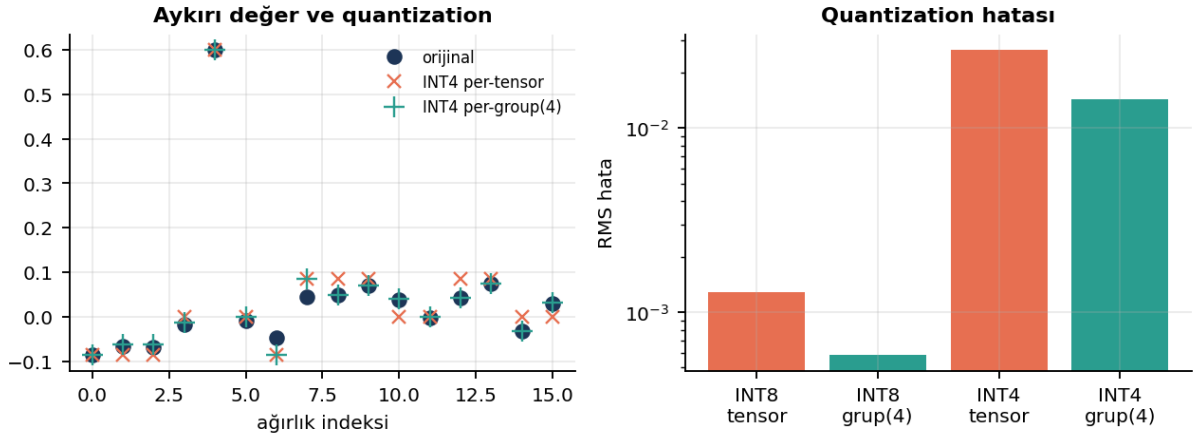
```
import torch
torch.manual_seed(8)
w = 0.05 * torch.randn(16); w[4] = 0.60                 # aykiri deger
def quant(w, bits, gs):
    qmax = 2 ** (bits - 1) - 1
    g = w.view(-1, gs)
    sc = g.abs().amax(1, keepdim=True) / qmax
    return (sc * torch.clamp(torch.round(g / sc), -qmax, qmax)).view(-1)
rms = lambda a: a.pow(2).mean().sqrt().item()
print(f"agirlik RMS      = {rms(w):.4f}")
for bits in (8, 4):
    for gs, ad in [(16, "per-tensor  "), (4, "per-group(4)")]:
        print(f"INT{bits} {ad} RMS hata = {rms(w - quant(w, bits, gs)):.2e}")
```

PYTORCH ÇIKTISI

```
agirlik RMS      = 0.1601
INT8 per-tensor   RMS hata = 1.28e-03
INT8 per-group(4) RMS hata = 7.87e-04
INT4 per-tensor   RMS hata = 2.37e-02
INT4 per-group(4) RMS hata = 9.77e-03
```

NASIL ÇALIŞIYOR?

Tek bir 0.60'lık aykırı değer, per-tensor ölçeği büyütür (INT4'te s = 0.60/7 ≈ 0.086): ağırlıkların çoğu (~0.05 mertebesinde) bu adımın yarısından küçük kalıp sıfıra yuvarlanır; INT4'te per-tensor hata (≈ 2e-2) per-group hatanın (≈ 7e-3) yaklaşık 3 katıdır. Per-group quantization aykırı değeri yalnızca kendi grubuna hapseder; diğer gruplar kendi küçük ölçeklerini kullanır ve hata belirgin biçimde düşer. INT8 genellikle neredeyse kayıpsızdır, INT4 ise grup ölçekleri olmadan ciddi kalite kaybı verir. Bedel: her grup için ek bir ölçek değeri saklamak (grup 128 ve 16 bit ölçek için ağırlık başına ≈ 0.125 ek bit). Modern yöntemler (GPTQ, AWQ) aykırı değerleri ve hata dağılımını daha akıllıca yönetir.



Şekil 9.3 — Solda: bir aykırı değer içeren ağırlık vektörü ve INT4 yeniden yapılandırmaları: per-tensor ölçek aykırı değere yenik düşer, per-group ölçek korur. Sağda: RMS hata karşılaştırması.

BÖLÜM 10

10. Metin Üretimi ve Model Sunumu

Eğitilen modelin metin üretebilmesi için bir çıkarım (inference) motoru gerekir. Kod çözme stratejilerinden KV cache'e, gecikmeden throughput'a ve bir API ile sunmaya kadar.

Bu bölümde

- Greedy decoding, temperature, top-k ve top-p sampling
- KV cache: üretimi neden $O(T^2)$'den $O(T)$'ye indirir
- Context window yönetimi ve bellek verimliliği
- Batching, token başına gecikme ve throughput
- Modeli Fortran çekirdeği + API olarak sunmak

Hedef: Bir dil modelinin logit çıktısını metne dönüştüren çıkarım motorunu ve onu kullanılabilir bir sisteme çeviren parçaları anlamak.

Üretim döngüsü

Eğitilmiş model her adımda sözlük üzerinde bir olasılık dağılımı üretir. Metin üretmek şu döngüdür: (1) şu ana kadarki token'ları modele ver, (2) son konumun logit'lerini al, (3) bir **kod çözme stratejisi** ile sıradaki token'ı seç, (4) onu diziye ekle, (5) EOS gelene veya uzunluk sınırına ulaşana kadar tekrarla. Strateji seçimi çıktının karakterini belirler: tutarlı ama sıkıcı mı, çeşitli ama riskli mi?

Greedy decoding

En basit strateji: her adımda en yüksek olasılıklı token'ı seç (`argmax`). Deterministiktir, ama **tekrara** ve sıkıcı metne eğilimlidir: bir döngüye girince çıkamaz.

Örnek 10.1 — Greedy decoding ve tekrar döngüsü

PROBLEM

Rastgele ama sabit bir geçiş tablosundan (6 token'lı minik bir "dil modeli") greedy üretim yap. Üretilen dizide tekrar döngüsünü göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program greedy
  use rng_mod
  implicit none
  integer, parameter :: V = 6, n = 14
  character(len=7) :: kelime(V)
  real(dp) :: P(V,V), z(V)
  integer :: cur, nxt, i, seq(n)
  kelime = [character(len=7) :: 'bir', 'varmis', 'yokmus', 'evvel', 'zaman', '.']
  call rng_seed(42)
  do i = 1, V
    call randn1(z, 1.5_dp)
    P(i,:) = exp(z - maxval(z)); P(i,:) = P(i,:) / sum(P(i,:))
  end do
  cur = 1
  seq(1) = cur
  do i = 2, n
```

```
    nxt = maxloc(P(cur,:), dim=1)                 ! greedy: en olasi token
    seq(i) = nxt
    cur = nxt
  end do
  print '(A)', 'uretilen kimlikler: '
  print '(14I3)', seq
  do i = 1, n
    write(*,'(A,A)', advance='no') trim(kelime(seq(i))), ' '
  end do
  print *
end program greedy
```

ÇIKTI

```
uretilen kimlikler:
  1  3  3  3  3  3  3  3  3  3  3  3  3  3
bir yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus yokmus
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(42)
V = 6
vocab = ["bir", "varmis", "yokmus", "evvel", "zaman", "."]
P = torch.softmax(1.5 * torch.randn(V, V), dim=-1)       # sabit gecis tablosu (satir = onceki token)

cur, seq = 0, [0]
for _ in range(13):
    cur = P[cur].argmax().item()                         # greedy: en olasi token
    seq.append(cur)
print("uretilen kimlikler:", seq)
print(" ".join(vocab[i] for i in seq))
```

PYTORCH ÇIKTISI

```
uretilen kimlikler: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
bir bir bir bir bir bir bir bir bir bir bir bir bir bir
```

NASIL ÇALIŞIYOR?

Geçiş tablosu sabit olduğundan her kelimenin "en olası sonraki kelimesi" de sabittir; dolayısıyla greedy dizi kaçınılmaz olarak bir döngüye girer ve sonsuza kadar aynı kelimeleri tekrar eder (burada "yokmus"un en olası halefi yine "yokmus"tur, tek kelimelik bir döngü). Gerçek LLM'lerde bağlam değiştiği için bu kadar katı olmaz, ama greedy yine de tekrar döngülerine ve aşırı "güvenli" cümlelere yatkındır. Çeşitlilik için olasılıklı örnekleme (sampling) kullanırız.

Temperature sampling

Logit'leri **sıcaklık T** ile bölüp softmax alırız: $p_i \propto \exp(z_i/T)$. $T < 1$ dağılımı keskinleştirir (greedy'ye yaklaşır), $T > 1$ düzleştirir (daha rastgele), $T \to 0$ argmax, $T \to \infty$ tekdüze dağılımdır.

Örnek 10.2 — Temperature dağılımı nasıl değiştirir?

PROBLEM

Logit'ler [2, 1, 0.5, 0, −1] için T = 0.2, 0.5, 1, 2, 5 değerlerinde olasılıkları ve entropiyi yazdır.

ÇÖZÜM (FORTRAN)

```
program sicaklik
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp) :: z(5), p(5), ts(5), ent
  integer :: k
  z = [2.0_dp, 1.0_dp, 0.5_dp, 0.0_dp, -1.0_dp]
  ts = [0.2_dp, 0.5_dp, 1.0_dp, 2.0_dp, 5.0_dp]
  print '(A)', '    T     p1      p2      p3      p4      p5    entropi'
  do k = 1, 5
```

```
    p = exp((z - maxval(z)) / ts(k))
    p = p / sum(p)
    ent = -sum(p * log(p))
    print '(F6.1,5F8.4,F9.4)', ts(k), p, ent
  end do
  print '(A,F8.4)', 'azami entropi ln 5 = ', log(5.0_dp)
end program sicaklik
```

ÇIKTI

```
     T     p1      p2      p3      p4      p5    entropi
   0.2  0.9927  0.0067  0.0005  0.0000  0.0000   0.0453
   0.5  0.8292  0.1122  0.0413  0.0152  0.0021   0.6086
   1.0  0.5630  0.2071  0.1256  0.0762  0.0280   1.2065
   2.0  0.3745  0.2272  0.1769  0.1378  0.0836   1.4915
   5.0  0.2646  0.2167  0.1961  0.1774  0.1452   1.5896
azami entropi ln 5 =   1.6094
```

PYTORCH KARŞILIĞI

```
import torch
z = torch.tensor([2.0, 1.0, 0.5, 0.0, -1.0], dtype=torch.float64)
print("    T     p1      p2      p3      p4      p5    entropi")
for T in [0.2, 0.5, 1.0, 2.0, 5.0]:
    p = torch.softmax(z / T, 0)
    ent = -(p * p.log()).sum()
    print(f"{T:5.1f}" + "".join(f"{v:8.4f}" for v in p.tolist()) + f"{ent:9.4f}")
print(f"azami entropi ln 5 = {torch.log(torch.tensor(5.0)):.4f}")
```

PYTORCH ÇIKTISI

```
     T     p1      p2      p3      p4      p5    entropi
   0.2  0.9927  0.0067  0.0005  0.0000  0.0000   0.0453
   0.5  0.8292  0.1122  0.0413  0.0152  0.0021   0.6086
   1.0  0.5630  0.2071  0.1256  0.0762  0.0280   1.2065
   2.0  0.3745  0.2272  0.1769  0.1378  0.0836   1.4915
   5.0  0.2646  0.2167  0.1961  0.1774  0.1452   1.5896
azami entropi ln 5 = 1.6094
```

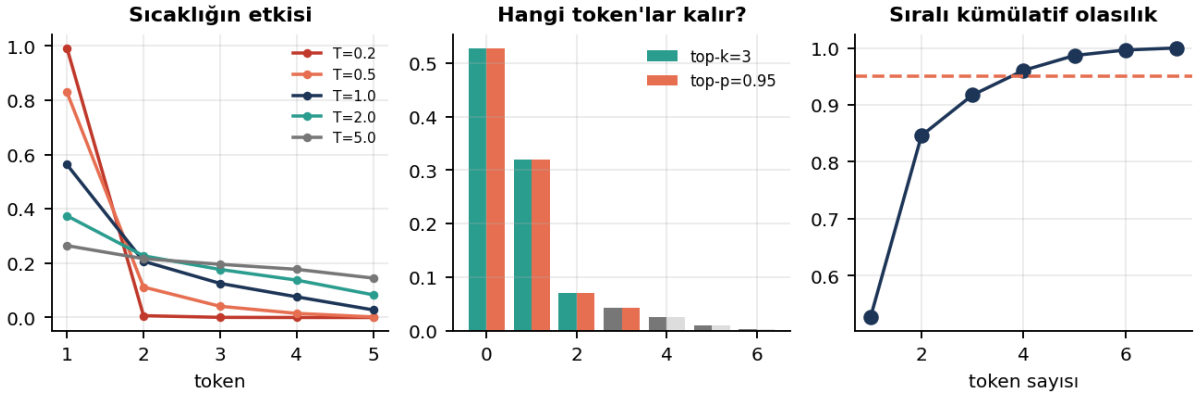
NASIL ÇALIŞIYOR?

T = 0.2'de en yüksek logit'li token neredeyse olasılık 1 alır (entropi ≈ 0.01), yani davranış greedy'ye çok yakındır. T = 1 modelin gerçek dağılımıdır. T = 5'te dağılım tekdüzeye yaklaşır ve entropi azami değer ln 5 = 1.609'a doğru gider. Pratik: sohbet ve yaratıcı yazım için T ≈ 0.7–1.0, kod ve olgusal cevaplar için daha düşük T.

Türetme: temperature = Boltzmann dağılımı

Sıcaklıklı softmax, istatistiksel fiziğin **Boltzmann dağılımı**dır: $p_i = e^{-E_i/kT} / Z$, burada $E_i = -z_i$ ("enerji" = −logit) ve kT = T. Fiziksel sezgi birebir geçerli: düşük sıcaklıkta sistem en düşük enerjili durumda donar (argmax), yüksek sıcaklıkta tüm durumlar eşit olasılıklı olur.

Entropi sıcaklıkla monoton artar: $dH/dT = \mathrm{Var}_p(z)/T^3 \geq 0$ (varyans her zaman ≥ 0). Limitler: $T \to 0 \Rightarrow H \to 0$, $T \to \infty \Rightarrow H \to \ln V$. Logit farkı Δz olan iki token'ın olasılık oranı $e^{\Delta z/T}$: T = 0.5'te oran kareye çıkar, T = 2'de karekökü alınır. Bu yüzden sıcaklık "modelin güvenini" yeniden ölçekler; modelin bildiğini değiştirmez.



Şekil 10.1 — Solda: sıcaklığın dağılıma etkisi. Ortada: top-k=3 ve top-p=0.95 filtrelerinin tuttuğu token'lar (renkli). Sağda: sıralı kümülatif olasılık; top-p eşiği 0.95.

Top-k ve top-p (nucleus) sampling

Saf örnekleme, olasılığı çok düşük ama sayısı çok olan "kuyruk" token'lardan ara sıra saçma bir seçim yapabilir. İki filtre:

- **Top-k**: yalnızca en olası k token'ı tut, gerisini sıfırla, yeniden normalize et.
- **Top-p (nucleus)**: olasılıkları büyükten küçüğe sırala, kümülatif toplamı p'yi (örneğin 0.95) *ilk aşan* en küçük kümeyi tut. Küme boyutu dağılımın keskinliğine göre kendiliğinden ayarlanır.

Örnek 10.3 — Top-k ve top-p filtreleri

PROBLEM

Logit'ler [3, 2.5, 1, 0.5, 0, −1, −2] için softmax olasılıklarını bul. Top-k (k=3) ve top-p (p=0.95) ile filtrele ve yeniden normalize et.

ÇÖZÜM (FORTRAN)

```
program topk_topp
  implicit none
  integer, parameter :: dp = kind(1.0d0), n = 7
  real(dp) :: z(n), p(n), pk(n), pp(n), cum
  integer :: idx(n), i, j, tmp, k, nkeep
  z = [3.0_dp, 2.5_dp, 1.0_dp, 0.5_dp, 0.0_dp, -1.0_dp, -2.0_dp]
  p = exp(z - maxval(z)); p = p / sum(p)
  idx = [(i, i = 1, n)]
  do i = 1, n - 1                                   ! idx'i olasiliga gore azalan sirala
    do j = i + 1, n
      if (p(idx(j)) > p(idx(i))) then
        tmp = idx(i); idx(i) = idx(j); idx(j) = tmp
      end if
    end do
  end do
  k = 3
  pk = 0.0_dp
  pk(idx(1:k)) = p(idx(1:k))
  pk = pk / sum(pk)                                 ! top-k
  cum = 0.0_dp; nkeep = 0
  do i = 1, n                                       ! top-p
    cum = cum + p(idx(i)); nkeep = i
    if (cum >= 0.95_dp) exit
  end do
  pp = 0.0_dp
  pp(idx(1:nkeep)) = p(idx(1:nkeep))
  pp = pp / sum(pp)
  print '(A,7F7.3)', 'olasiliklar : ', p
  print '(A,7F7.3)', 'top-k (k=3) : ', pk
  print '(A,I0,A,7F7.3)', 'top-p (0.95, ', nkeep, ' token): ', pp
end program topk_topp
```

ÇIKTI

```
olasiliklar :   0.527  0.319  0.071  0.043  0.026  0.010  0.004
top-k (k=3) :   0.574  0.348  0.078  0.000  0.000  0.000  0.000
top-p (0.95, 4 token):   0.548  0.333  0.074  0.045  0.000  0.000  0.000
```

PYTORCH KARŞILIĞI

```
import torch
z = torch.tensor([3.0, 2.5, 1.0, 0.5, 0.0, -1.0, -2.0], dtype=torch.float64)
p = torch.softmax(z, 0)
vals, idx = p.sort(descending=True)
pk = torch.zeros_like(p); top = torch.topk(p, 3); pk[top.indices] = top.values; pk /= pk.sum()      #
      top-k
cum = vals.cumsum(0)
nkeep = int((cum < 0.95).sum().item()) + 1                                                          #
      top-p
pp = torch.zeros_like(p); pp[idx[:nkeep]] = vals[:nkeep]; pp /= pp.sum()
f = lambda t: " ".join(f"{v:6.3f}" for v in t.tolist())
print("olasiliklar :", f(p)); print("top-k (k=3) :", f(pk)); print(f"top-p (0.95, {nkeep} token):", f(pp))
```

PYTORCH ÇIKTISI

```
olasiliklar :  0.527  0.319  0.071  0.043  0.026  0.010  0.004
top-k (k=3) :  0.574  0.348  0.078  0.000  0.000  0.000  0.000
top-p (0.95, 4 token):  0.548  0.333  0.074  0.045  0.000  0.000  0.000
```

NASIL ÇALIŞIYOR?

Önce indeksler olasılığa göre azalan sıralanır. Top-k sabit sayıda (3) token tutar; top-p ise kümülatif olasılık 0.95'i geçene kadar token tutar: 0.527+0.319+0.071 = 0.917 yetmez, dördüncü token ile 0.960 olur, yani 4 token. Her iki durumda da tutulmayan token'ların olasılığı sıfırlanır ve kalanlar toplamı 1 olacak şekilde *yeniden normalize edilir*. Dağılım keskin olduğunda top-p az token, yaygın olduğunda çok token tutar; bu esneklik top-p'yi popüler yapar. Pratikte temperature, top-k ve top-p birlikte uygulanır (önce T, sonra filtreler).

Örnek 10.4 — Dağılımdan örnekleme (ters CDF) ve doğrulama

PROBLEM

p = [0.5, 0.3, 0.15, 0.05] dağılımından, kümülatif dağılım (CDF) yöntemiyle 20000 token örnekle. Deneysel frekansların p'ye yakın olduğunu göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program ornekleme
  use rng_mod
  implicit none
  integer, parameter :: V = 4, ns = 20000
  real(dp) :: p(V), cdf(V), u
  integer :: cnt(V), i, k
  p = [0.5_dp, 0.3_dp, 0.15_dp, 0.05_dp]
  cdf(1) = p(1)
  do k = 2, V
    cdf(k) = cdf(k-1) + p(k)                        ! kumulatif dagilim
  end do
  call rng_seed(77)
  cnt = 0
  do i = 1, ns
    u = uniform()
    do k = 1, V
      if (u <= cdf(k)) exit                         ! u'yu asan ilk token
    end do
    k = min(k, V)
    cnt(k) = cnt(k) + 1
  end do
  print '(A,4F8.4)', 'gercek p    : ', p
  print '(A,4F8.4)', 'deneysel    : ', real(cnt, dp) / ns
end program ornekleme
```

ÇIKTI

```
gercek p    :   0.5000  0.3000  0.1500  0.0500
deneysel    :   0.5052  0.2923  0.1533  0.0491
```

PYTORCH KARŞILIĞI

```

import torch
torch.manual_seed(77)
p = torch.tensor([0.5, 0.3, 0.15, 0.05])
draw = torch.multinomial(p, 20000, replacement=True)     # ters CDF ile ayni mantik
print("gercek p    :", [round(v, 4) for v in p.tolist()])
print("deneysel    :", [round(v, 4) for v in (torch.bincount(draw, minlength=4) / 20000).tolist()])

```

PYTORCH ÇIKTISI

```
gercek p    : [0.5, 0.3, 0.15, 0.05]
deneysel    : [0.4981, 0.302, 0.145, 0.0549]
```

NASIL ÇALIŞIYOR?

Yöntem: [0,1) aralığında düzgün bir u çek; CDF'nin u'yu ilk aştığı token'ı seç. Aralıkların uzunlukları olasılıklara eşit olduğundan token'lar tam olarak p_i olasılığıyla seçilir. 20000 örnekte deneysel frekanslar gerçek olasılıklara ~0.01 hassasiyetle yaklaşır (standart hata ≈ $\sqrt{p(1-p)/n}$). Bu, LLM'lerin her token için yaptığı örneklemenin kendisidir.

KV cache: tekrar hesaplamayı önle

Causal attention'da bir token'ın K ve V vektörleri, **sonradan eklenen token'lardan etkilenmez** (Bölüm 4'teki nedensellik testi). Bu yüzden her yeni token için tüm geçmişin K ve V'sini yeniden hesaplamak israftır. **KV cache** bu vektörleri saklar; her adımda yalnızca *yeni token* için q, k, v hesaplanır, k ve v cache'e eklenir ve q cache'in tamamıyla attention yapar. Projeksiyon maliyeti adım başına O(T)'den O(1)'e iner (toplam $O(T^2) \rightarrow O(T)$).

Örnek 10.5 — KV cache ile ve cache'siz attention: aynı sonuç, daha az iş

PROBLEM

T = 6 token için tek head, causal attention'ı her adımda (a) tüm K,V'yi yeniden hesaplayarak (b) KV cache ile yap. Çıktıların eşit olduğunu ve K/V projeksiyon sayısını karşılaştır.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program kv_cache
  use rng_mod
  implicit none
  integer, parameter :: T = 6, d = 8
  real(dp) :: X(T,d), Wq(d,d), Wk(d,d), Wv(d,d)
  real(dp) :: o1(T,d), o2(T,d), K(T,d), V(T,d), Kc(T,d), Vc(T,d), q(d)
  integer :: t1, j, n_no, n_ca
  call rng_seed(14)
  call randn2(X, 1.0_dp)
  call randn2(Wq, 0.4_dp); call randn2(Wk, 0.4_dp); call randn2(Wv, 0.4_dp)
  n_no = 0; n_ca = 0
  ! (a) cache yok: her adimda tum gecmisin K,V'si yeniden hesaplanir
  do t1 = 1, T
    do j = 1, t1
      K(j,:) = matmul(X(j,:), Wk); V(j,:) = matmul(X(j,:), Wv); n_no = n_no + 1
    end do
    q = matmul(X(t1,:), Wq)
    o1(t1,:) = attend(q, K, V, t1)
  end do
  ! (b) KV cache: her adimda yalniz yeni tokenin k,v'si hesaplanir
  do t1 = 1, T
    Kc(t1,:) = matmul(X(t1,:), Wk); Vc(t1,:) = matmul(X(t1,:), Wv); n_ca = n_ca + 1
    q = matmul(X(t1,:), Wq)
    o2(t1,:) = attend(q, Kc, Vc, t1)
  end do
  print '(A,ES10.2)', 'max |cikti(cache yok) - cikti(cache)| = ', maxval(abs(o1 - o2))
  print '(A,I0)', 'K,V projeksiyon sayisi, cache yok : ', n_no
  print '(A,I0)', 'K,V projeksiyon sayisi, cache var : ', n_ca
contains
  function attend(q, K, V, n) result(o)               ! ilk n anahtar uzerinde attention
    real(dp), intent(in) :: q(d), K(:,:), V(:,:)
    integer, intent(in) :: n
    real(dp) :: o(d), s(n)
    integer :: j
    do j = 1, n
      s(j) = dot_product(q, K(j,:)) / sqrt(real(d, dp))
    end do
    s = exp(s - maxval(s)); s = s / sum(s)
    o = matmul(s, V(1:n,:))
  end function attend
end program kv_cache
```

ÇIKTI

```
max |cikti(cache yok) - cikti(cache)| =   0.00E+00
K,V projeksiyon sayisi, cache yok : 21
K,V projeksiyon sayisi, cache var : 6
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(14)
T_, d = 6, 8
X = torch.randn(T_, d, dtype=torch.float64)
Wq, Wk, Wv = (0.4 * torch.randn(d, d, dtype=torch.float64) for _ in range(3))
def attend(q, K, V):
    return torch.softmax(K @ q / d ** 0.5, 0) @ V
n_no = n_ca = 0
O1, O2 = [], []
for t in range(1, T_ + 1):                      # (a) cache yok: her adimda tum K,V yeniden
    K, V = X[:t] @ Wk, X[:t] @ Wv; n_no += t
    O1.append(attend(X[t - 1] @ Wq, K, V))
Kc, Vc = [], []
```

```
for t in range(T_):                              # (b) KV cache
    Kc.append(X[t] @ Wk); Vc.append(X[t] @ Wv); n_ca += 1
    O2.append(attend(X[t] @ Wq, torch.stack(Kc), torch.stack(Vc)))
print("max |cikti(cache yok) - cikti(cache)| =", (torch.stack(O1) - torch.stack(O2)).abs().max().item())
print("K,V projeksiyon sayisi, cache yok :", n_no)
print("K,V projeksiyon sayisi, cache var :", n_ca)
```

PYTORCH ÇIKTISI

```
max |cikti(cache yok) - cikti(cache)| = 0.0
K,V projeksiyon sayisi, cache yok : 21
K,V projeksiyon sayisi, cache var : 6
```

NASIL ÇALIŞIYOR?

İki yöntem **aynı çıktıyı** verir (fark tam sıfır). Cache'siz yöntem 1+2+…+6 = 21 projeksiyon çifti hesaplar; cache'li yöntem yalnızca 6. Dizi uzunluğu T'ye çıktığında oran T(T+1)/2 : T = ~T/2 kat farka ulaşır; 4096 token için ~2000 kat tasarruf. Bedeli bellektir: cache her katman ve head için K ve V'yi saklar (sonraki örnek). Üretim iki aşamadır: **prefill** (komutu tek seferde işleyip cache'i doldurma, hesap-sınırlı) ve **decode** (token token üretme, bellek-sınırlı).

Context window ve KV cache belleği

Basit matematik

KV cache boyutu = 2 (K ve V) × L (katman) × T (bağlam) × d (model boyutu) × bayt/eleman × batch

Bağlam uzunluğu ve eşzamanlı kullanıcı sayısı arttıkça bu bellek ağırlıkların kendisini aşabilir. (GQA/MQA gibi teknikler K,V head sayısını azaltarak bunu küçültür.)

Örnek 10.6 — KV cache belleği ve kayan pencere (sliding window)

PROBLEM

GPT-2 small, 7B ve 70B tipi konfigürasyonlar için tek dizinin FP16 KV cache boyutunu hesapla. Sonra bağlam penceresi 8 token olan bir sistemde, 12 token'lık bir üretimde BOS'u koruyarak kayan pencerenin nasıl çalıştığını göster.

ÇÖZÜM (FORTRAN)

```
program kv_bellek
  implicit none
  integer, parameter :: W = 8
  integer(8) :: b
  integer :: toks(12), t, n, i
  b = kvbytes(12_8, 1024_8, 768_8, 2_8)
  print '(A,F9.1,A)', 'GPT-2 small (L=12,  d=768,  T=1024): ', real(b)/1.0e6, ' MB'
  b = kvbytes(32_8, 4096_8, 4096_8, 2_8)
  print '(A,F9.2,A)', '7B tipi     (L=32,  d=4096, T=4096): ', real(b)/1.0e9, ' GB'
  b = kvbytes(80_8, 4096_8, 8192_8, 2_8)
  print '(A,F9.2,A)', '70B tipi    (L=80,  d=8192, T=4096): ', real(b)/1.0e9, ' GB'
  toks = [(i, i = 100, 111)]                          ! uretilen token kimlikleri; ilki BOS
  print '(A)', 'pencere = 8 token, BOS (ilk token) her zaman korunur:'
  do t = 8, 12
    n = min(t, W)
    if (t <= W) then
      print '(A,I2,A,12I4)', 'adim ', t, ' : ', toks(1:t)
    else
      print '(A,I2,A,12I4)', 'adim ', t, ' : ', toks(1), toks(t-W+2:t)   ! BOS + son W-1
    end if
  end do
contains
  function kvbytes(L, T, d, bytes) result(r)
    integer(8), intent(in) :: L, T, d, bytes
    integer(8) :: r
    r = 2_8 * L * T * d * bytes
  end function kvbytes
end program kv_bellek
```

ÇIKTI

```
GPT-2 small (L=12,  d=768,  T=1024):      37.7 MB
7B tipi     (L=32,  d=4096, T=4096):      2.15 GB
70B tipi    (L=80,  d=8192, T=4096):     10.74 GB
pencere = 8 token, BOS (ilk token) her zaman korunur:
adim  8 :  100 101 102 103 104 105 106 107
adim  9 :  100 102 103 104 105 106 107 108
adim 10 :  100 103 104 105 106 107 108 109
adim 11 :  100 104 105 106 107 108 109 110
adim 12 :  100 105 106 107 108 109 110 111
```

PYTORCH KARŞILIĞI

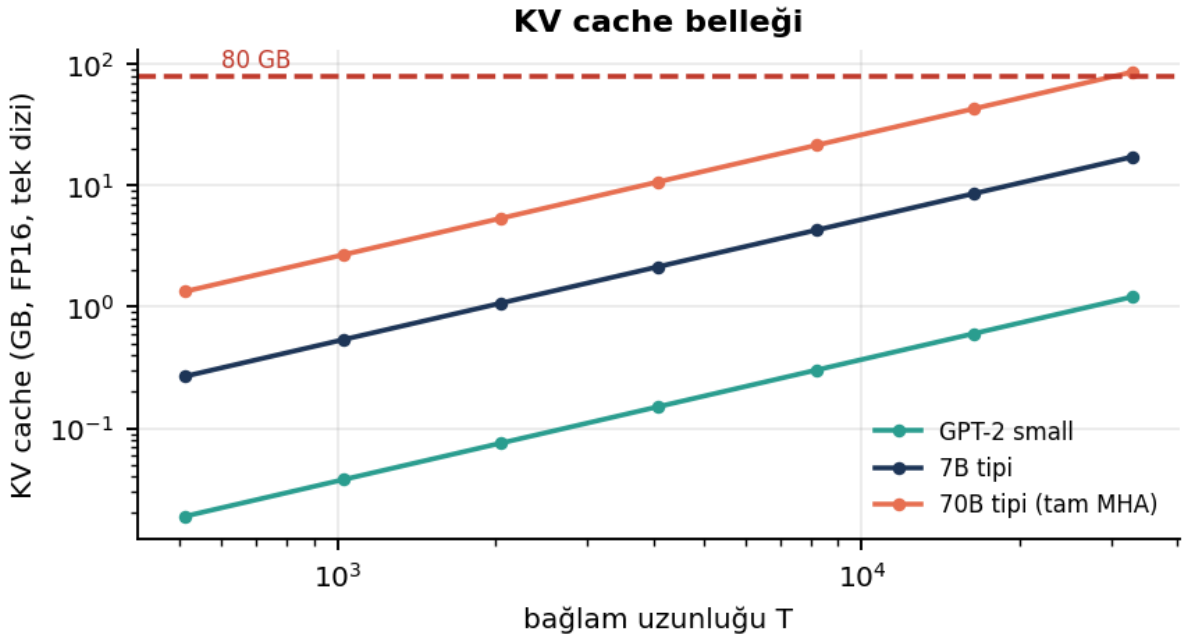
```
kv = lambda L, T, d, b: 2 * L * T * d * b
print(f"GPT-2 small (L=12,  d=768,  T=1024): {kv(12,1024,768,2)/1e6:9.1f} MB")
print(f"7B tipi     (L=32,  d=4096, T=4096): {kv(32,4096,4096,2)/1e9:9.2f} GB")
print(f"70B tipi    (L=80,  d=8192, T=4096): {kv(80,4096,8192,2)/1e9:9.2f} GB")
W = 8
toks = list(range(100, 112))
print("pencere = 8 token, BOS (ilk token) her zaman korunur:")
for t in range(8, 13):
    ctx = toks[:t] if t <= W else [toks[0]] + toks[t - W + 1:t]
    print(f"adim {t:2d} :", " ".join(f"{v:4d}" for v in ctx))
```

PYTORCH ÇIKTISI

```
GPT-2 small (L=12,  d=768,  T=1024):      37.7 MB
7B tipi     (L=32,  d=4096, T=4096):      2.15 GB
70B tipi    (L=80,  d=8192, T=4096):     10.74 GB
pencere = 8 token, BOS (ilk token) her zaman korunur:
adim  8 :  100  101  102  103  104  105  106  107
adim  9 :  100  102  103  104  105  106  107  108
adim 10 :  100  103  104  105  106  107  108  109
adim 11 :  100  104  105  106  107  108  109  110
adim 12 :  100  105  106  107  108  109  110  111
```

NASIL ÇALIŞIYOR?

Tek bir 4096 token'lık dizi için 7B tipi modelde KV cache yaklaşık **2.1 GB**'dir; 100 eşzamanlı kullanıcı 210 GB eder, yani ağırlıkların (14 GB) çok üstünde. Bu yüzden sunucular KV cache belleğini sayfalama (PagedAttention), K/V quantization ve GQA ile yönetir. (Not: 70B modeller genellikle GQA kullanır, bu yüzden gerçek değer buradaki "tam multi-head" hesabından birkaç kat küçüktür.) Context penceresi dolunca ne yapılır? Basit çözüm kayan pencere: en eski token'ları at, ama **BOS/sistem komutunu koru**; aksi halde model dizinin başlangıç bağlamını kaybeder. Çıktıda adım 9'dan itibaren pencerenin ortasından token'ların düştüğünü, ilk token'ın (100) ise kaldığını görürsün.



Şekil 10.2 — KV cache belleği bağlam uzunluğuyla doğrusal büyür (log-log). Büyük modellerde 32K bağlamda tek bir dizi 80 GB GPU'nun belleğini aşabilir.

Batching, gecikme ve throughput

Token üretirken (decode) her adımda tüm ağırlıkları bellekten okumak gerekir ama token başına çok az işlem yapılır; yani **bellek-sınırlıdır**. Birkaç kullanıcının isteğini aynı anda işleyince (**batching**) aynı ağırlık okuması B token için kullanılır: throughput neredeyse B kat artar, token başına gecikme ise yaklaşık aynı kalır. Bu, B büyüyüp işlem sınırlı hâle gelene kadar sürer.

$$\text{adım süresi} \approx \max\left(\frac{\text{ağırlık baytı}}{\text{bellek bant genişliği}}, \frac{B \cdot 2N}{\text{tepe FLOP/s}}\right)$$

Örnek 10.7 — Batch boyutu, gecikme ve throughput modeli

PROBLEM

7B FP16 model (14 GB), 2 TB/s bellek, 312 TFLOP/s tepe işlem. B = 1…512 için adım süresini, token/s throughput'unu ve hangi sınırın etkili olduğunu hesapla.

ÇÖZÜM (FORTRAN)

```
program throughput
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp), parameter :: wbytes = 14.0e9_dp, bw = 2.0e12_dp, peak = 312.0e12_dp
  real(dp), parameter :: flop_tok = 14.0e9_dp        ! 2 * N, N = 7e9
  integer :: bs(6), i
  real(dp) :: t_mem, t_cmp, t
  bs = [1, 8, 32, 128, 256, 512]
  t_mem = wbytes / bw
  print '(A,F6.2,A)', 'agirlik okuma suresi = ', t_mem * 1.0e3_dp, ' ms/adim'
  print '(A)', '   B   adim(ms)   tok/s (toplam)   sinir'
  do i = 1, 6
    t_cmp = bs(i) * flop_tok / peak
    t = max(t_mem, t_cmp)
    print '(I5,F10.2,F16.0,3X,A)', bs(i), t * 1.0e3_dp, bs(i) / t, &
          merge('bellek ', 'islem  ', t_mem >= t_cmp)
  end do
  print '(A,F7.1)', 'gecis noktasi B* = ', t_mem * peak / flop_tok
end program throughput
```

ÇIKTI

```
agirlik okuma suresi =   7.00 ms/adim
   B   adim(ms)   tok/s (toplam)   sinir
   1      7.00            143.   bellek
   8      7.00           1143.   bellek
  32      7.00           4571.   bellek
 128      7.00          18286.   bellek
 256     11.49          22286.   islem
 512     22.97          22286.   islem
gecis noktasi B* =   156.0
```

PYTORCH KARŞILIĞI

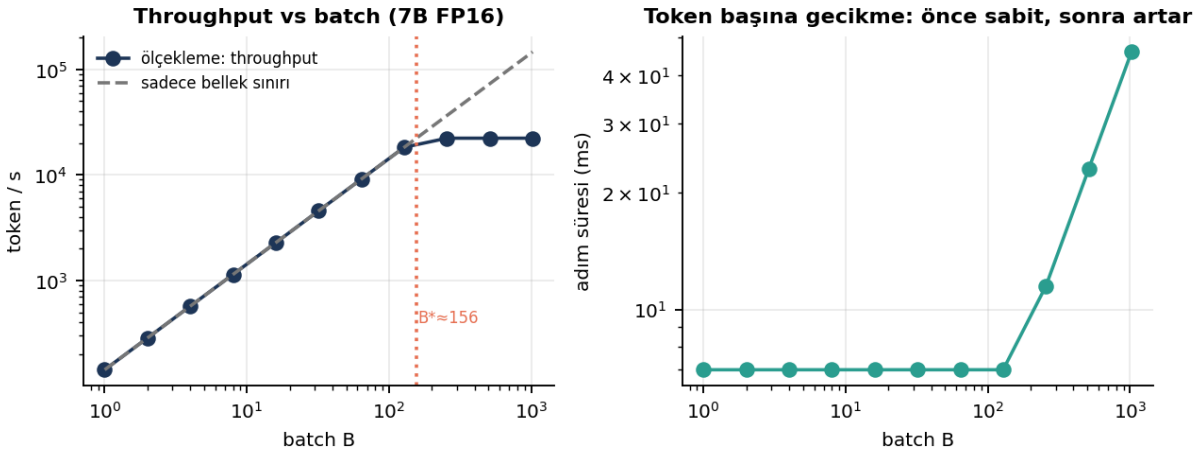
```
wbytes, bw, peak, flop_tok = 14e9, 2e12, 312e12, 14e9
t_mem = wbytes / bw
print(f"agirlik okuma suresi = {t_mem*1e3:.2f} ms/adim")
print("   B   adim(ms)   tok/s (toplam)   sinir")
for B in [1, 8, 32, 128, 256, 512]:
    t_cmp = B * flop_tok / peak; t = max(t_mem, t_cmp)
    print(f"{B:5d}{t*1e3:10.2f}{B/t:16.0f}   {'bellek' if t_mem >= t_cmp else 'islem'}")
print(f"gecis noktasi B* = {t_mem * peak / flop_tok:.1f}")
```

PYTORCH ÇIKTISI

```
agirlik okuma suresi = 7.00 ms/adim
   B   adim(ms)   tok/s (toplam)   sinir
   1      7.00            143   bellek
   8      7.00           1143   bellek
  32      7.00           4571   bellek
 128      7.00          18286   bellek
 256     11.49          22286   islem
 512     22.97          22286   islem
gecis noktasi B* = 156.0
```

NASIL ÇALIŞIYOR?

Ağırlıkları bir kez okumak 14 GB / 2 TB/s = 7 ms sürer. B = 1 için adım 7 ms, yani saniyede ~143 token. B = 32 için hâlâ 7 ms (bellek sınırlı), ama bu sefer saniyede ~4570 token üretilir: 32 kat throughput *bedavaya* gelmiştir. B ≈ 156'da işlem süresi bellek süresini yakalar; ondan sonra daha fazla batch artık throughput artırmaz, gecikmeyi uzatır (B = 512'de adım ~23 ms). Gerçek sistemlerde KV cache okumaları ve dikkat hesabı da eklenir, ama bu model sürekli batching (continuous batching) neden kullanıldığını açıklar: GPU'yu bellek-sınırlı bölgeden mümkün olduğunca çok isteği bir arada çalıştırarak çıkarmak.



Şekil 10.3 — Solda: toplam throughput batch boyutuyla B* ≈ 156'ya kadar doğrusal artar, sonra hesap-sınırlı olur. Sağda: token başına adım süresi önce sabit kalır (bellek-sınırlı), sonra artar.

Modeli API olarak sunmak

Bir çıkarım motoru yalnızca bir kütüphanedir; kullanıcıların erişebilmesi için bir **servis** katmanı gerekir. Fortran hesap çekirdeği için pratik mimari: çekirdeği C uyumlu bir **paylaşılan kütüphane** (`.so`) olarak derleyip, ağ işlerini (HTTP, JSON, kimlik doğrulama, kuyruk) Python gibi bir dilde yazmak. Fortran'ın `iso_c_binding` modülü ve `bind(c)` özniteliği C/Python'dan doğrudan çağrılabilir fonksiyonlar tanımlar.

Örnek 10.8 — Fortran çekirdeği + Python istemcisi (paylaşılan kütüphane)

PROBLEM

6 token'lı minik dil modelinin üretim fonksiyonunu `bind(c)` ile dışa aç, paylaşılan kütüphane olarak derle ve Python'dan ctypes ile çağır. Sıcaklığın ve tohumun etkisini göster.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module lm_api
  use iso_c_binding
  use rng_mod
  implicit none
  integer, parameter :: V = 6
  real(dp), save :: P(V,V)
  logical, save :: ready = .false.
contains
  subroutine init()
    integer :: i
    real(dp) :: z(V)
  call rng_seed(42)
  do i = 1, V
    call randn1(z, 1.5_dp)
    P(i,:) = exp(z - maxval(z)); P(i,:) = P(i,:) / sum(P(i,:))
  end do
    ready = .true.
  end subroutine init

  subroutine lm_generate(start, n, temp, seed, out) bind(c, name='lm_generate')
    integer(c_int), value :: start, n, seed
    real(c_double), value :: temp
    integer(c_int), intent(out) :: out(n)
    real(dp) :: pr(V), u, cum
    integer :: cur, i, k, j
    if (.not. ready) call init()
    call rng_seed(seed)
    cur = start + 1                                  ! Python 0 tabanli, Fortran 1 tabanli
    do i = 1, n
      pr = P(cur,:) ** (1.0_dp / temp)               ! sicaklik: p^(1/T)
      pr = pr / sum(pr)
      u = uniform(); cum = 0.0_dp; k = V
      do j = 1, V
        cum = cum + pr(j)
        if (u <= cum) then
          k = j; exit
        end if
      end do
      out(i) = k - 1
      cur = k
    end do
  end subroutine lm_generate
end module lm_api
```

ÇIKTI

```
T=1.0 seed=1 : yokmus bir yokmus yokmus zaman bir yokmus bir yokmus bir varmis evvel
T=0.3 seed=1 : yokmus bir yokmus yokmus yokmus bir yokmus yokmus yokmus bir yokmus yokmus
T=1.0 seed=1 : yokmus bir yokmus yokmus zaman bir yokmus bir yokmus bir varmis evvel  (ayni tohum, ayni
   cikti)
T=1.0 seed=2 : varmis . yokmus yokmus yokmus yokmus zaman varmis . zaman varmis .
```

PYTORCH KARŞILIĞI

```
import torch
torch.manual_seed(42)
V = 6
vocab = ["bir", "varmis", "yokmus", "evvel", "zaman", "."]
P = torch.softmax(1.5 * torch.randn(V, V), dim=-1)       # sabit gecis tablosu (satir = onceki token)

def generate(start, n, temp, seed):                      # Fortran'daki lm_generate'in karsiligi
    g = torch.Generator().manual_seed(seed)
    cur, out = start, []
    for _ in range(n):
        pr = P[cur] ** (1.0 / temp); pr = pr / pr.sum()
        cur = torch.multinomial(pr, 1, generator=g).item(); out.append(cur)
    return [vocab[i] for i in out]
print("T=1.0 seed=1 :", " ".join(generate(0, 12, 1.0, 1)))
print("T=0.3 seed=1 :", " ".join(generate(0, 12, 0.3, 1)))
print("T=1.0 seed=1 :", " ".join(generate(0, 12, 1.0, 1)), " (ayni tohum, ayni cikti)")
print("T=1.0 seed=2 :", " ".join(generate(0, 12, 1.0, 2)))
# Bunu bir HTTP servisine cevirmek icin (FastAPI):
#   @app.post("/generate")  def gen(req): return {"tokens": generate(req.start, req.n, req.temperature,
      req.seed)}
```

PYTORCH ÇIKTISI

```
T=1.0 seed=1 : bir bir yokmus zaman yokmus . . . . . .
T=0.3 seed=1 : bir bir bir bir varmis evvel varmis evvel varmis evvel yokmus zaman
T=1.0 seed=1 : bir bir yokmus zaman yokmus . . . . . .  (ayni tohum, ayni cikti)
T=1.0 seed=2 : varmis evvel . . evvel bir bir bir varmis bir zaman .
```

NASIL ÇALIŞIYOR?

`bind(c, name='lm_generate')` alt programı C simgesi olarak dışa açar; `value` özniteliği skaler argümanların C tarzı değerle geçmesini sağlar; çıktı dizisini ise Python tarafı ayırır (`(c_int * n)()`) ve Fortran doldurur. Sıcaklık, olasılıkların 1/T kuvvetini alıp yeniden normalize ederek uygulanır (logit/T ve softmax ile eşdeğerdir). Aynı tohum aynı çıktıyı verir (tekrarlanabilirlik), farklı tohum farklı bir örnek. Derleme: `gfortran -O2 -shared -fPIC main.f90 -o libgen.so`. Aşağıda Python istemcisi ve bunu HTTP servisine çeviren taslak sunucu verilmiştir.

Python istemcisi (yukarıdaki kütüphaneyi yükler; çıktı örnek çıktısında gösterilmiştir):

```
import ctypes

lib = ctypes.CDLL('./libgen.so')
lib.lm_generate.argtypes = [ctypes.c_int, ctypes.c_int, ctypes.c_double,
                            ctypes.c_int, ctypes.POINTER(ctypes.c_int)]
vocab = ['bir', 'varmis', 'yokmus', 'evvel', 'zaman', '.']

def generate(start, n, temp, seed):
    buf = (ctypes.c_int * n)()                  # Fortran'in dolduracagi bellek
    lib.lm_generate(start, n, temp, seed, buf)
    return [vocab[i] for i in buf]

print('T=1.0 seed=1 :', ' '.join(generate(0, 12, 1.0, 1)))
print('T=0.3 seed=1 :', ' '.join(generate(0, 12, 0.3, 1)))
print('T=1.0 seed=1 :', ' '.join(generate(0, 12, 1.0, 1)), ' (ayni tohum, ayni cikti)')
print('T=1.0 seed=2 :', ' '.join(generate(0, 12, 1.0, 2)))
```

HTTP sunucu taslağı: aynı kütüphaneyi bir JSON API'ye çevirir. Gerçek servislerde buna istek kuyruğu, sürekli batching, akışlı (streaming) cevap ve kimlik doğrulama eklenir; ama Fortran çekirdeğinin rolü değişmez: hesaplamak.

```
# sunucu.py : Fortran cekirdegini HTTP API olarak sunan minik sunucu (taslak)
import ctypes, json
from http.server import BaseHTTPRequestHandler, HTTPServer

lib = ctypes.CDLL('./libgen.so')
lib.lm_generate.argtypes = [ctypes.c_int, ctypes.c_int, ctypes.c_double,
                            ctypes.c_int, ctypes.POINTER(ctypes.c_int)]
```

```
class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        istek = json.loads(self.rfile.read(int(self.headers['Content-Length'])))
        n = int(istek.get('n', 12))
        buf = (ctypes.c_int * n)()
        lib.lm_generate(int(istek.get('start', 0)), n,
                        float(istek.get('temperature', 1.0)), int(istek.get('seed', 1)), buf)
        govde = json.dumps({'tokens': list(buf)}).encode()
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.end_headers()
        self.wfile.write(govde)

HTTPServer(('0.0.0.0', 8080), Handler).serve_forever()
# Deneme:  curl -X POST localhost:8080 -d '{"n": 12, "temperature": 0.8, "seed": 7}'
```

BÖLÜM 11

11. Bitirme Projesi: Kendi Dil Modellerimizi Eğitiyoruz

Kitabın tüm parçalarını birleştiriyoruz: veri, tokenizasyon, model, geri yayılım, Adam, doğrulama kaybı ve metin üretimi, üç farklı mimaride ve hepsi sıfırdan Fortran ile.

Bu bölümde

- Karakter düzeyinde bir veri kümesi hazırlamak ve eğitim/doğrulama olarak bölmek
- Model 1: nöral bigram (tek embedding tablosu)
- Model 2: bağlam penceresi olan MLP dil modeli (Bengio tarzı)
- Model 3: tek katmanlı, tek başlı causal attention dil modeli, elle yazılmış tam geri yayılımla
- Üç modelin doğrulama kaybını karşılaştırmak ve metin üretmek

Hedef: Hiçbir kütüphane kullanmadan, uçtan uca eğitilen ve metin üreten bir GPT mini sürümünü çalışır görmek.

Proje planı

Veri kümemiz çok küçük bir Türkçe (ASCII) metin: 481 karakter (23 farklı karakter). Gerçek bir LLM'in milyarlarca token'ına kıyasla bir oyuncak; ama **tüm mekanik aynıdır**. Amacımız güzel metin üretmek değil, her parçanın çalıştığını ve modellerin kapasite arttıkça kaybı nasıl düşürdüğünü görmek. Üç model aynı veri ve aynı kayıp (karakter başına çapraz entropi, nat) üzerinde karşılaştırılacak:

Model	Girdi bağlamı	Fikir
1. Nöral bigram	1 karakter	logits = W[önceki karakter]; en basit dil modeli
2. MLP LM	3 karakter	embedding + gizli katman + softmax; sabit pencere
3. Attention LM	16 karakter	embedding + konum + causal self-attention + residual; ağırlık bağlama

> **İpucu**
> Başlangıç kaybı her modelde yaklaşık **ln V** olmalıdır (Bölüm 4). Karakter kümemizde V = 23 olduğundan ln V = 3.14. Eğitim kaybı bunun altına inmeli; doğrulama kaybı ise modelin görmediği metindeki başarısını söyler.

Veri ve ortak modül

İlk örnekte ortak `lmdata` modülü tanımlanır: metin, karakter sözlüğü (`build_vocab`), kimliklerden metne dönüş (`decode`), softmax ve tek vektör üzerinde Adam adımı. Sonraki iki örnek bu modülü kullanır.

Örnek 11.1 — Model 1: Nöral bigram dil modeli

PROBLEM

Sonraki karakteri yalnızca *bir önceki karaktere* bakarak tahmin eden modeli kur: logits = W[önceki], softmax, çapraz entropi, Adam. Eğitim/doğrulama kaybını, sayım tabanlı en iyi (analitik) bigram kaybıyla karşılaştır ve metin üret.

Bu örnek şu modülleri kullanır: `rng_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module lmdata
  use rng_mod
  implicit none
  character(len=*), parameter :: corpus = &
    'bir varmis bir yokmus. evvel zaman icinde kalbur saman icinde develer tellal iken ' // &
    'pireler berber iken ben annemin besigini tangir tungur sallar iken. ' // &
    'cok gezdim cok gordum ama hic yorulmadim. yol uzundu ama yuregim sicakti. ' // &
    'gece oldu yildizlar cikti ve herkes sessizce uykuya daldi. ' // &
    'sabah olunca horozlar otti kuslar sarki soyledi ve yeni bir gun basladi. ' // &
    'dag basinda bir cesme varmis suyu soguk mu soguk tatli mi tatli. ' // &
    'yolcular gelir su icer dinlenir sonra yoluna devam edermis. '
  integer, parameter :: NCH = len(corpus), V = 23     ! V: sozluk boyutu (asagida dogrulanir)
  integer :: ids(NCH), ntrain = 0
  character(len=1) :: chars(64)
contains
  subroutine build_vocab()                          ! karakter -> kimlik (1..V)
    integer :: i, c, nv, map(0:127)
    logical :: seen(0:127)
    seen = .false.
    do i = 1, NCH
      seen(ichar(corpus(i:i))) = .true.
    end do
    nv = 0; map = -1
    do c = 0, 127
      if (seen(c)) then
        nv = nv + 1; map(c) = nv; chars(nv) = char(c)
      end if
    end do
    if (nv /= V) error stop 'V yanlis: sozluk boyutunu guncelle'
    do i = 1, NCH
      ids(i) = map(ichar(corpus(i:i)))
    end do
    ntrain = int(0.85_dp * NCH)                       ! ilk %85 egitim, kalan dogrulama
  end subroutine build_vocab

  function decode(a) result(s)                      ! kimlikler -> metin
    integer, intent(in) :: a(:)
    character(len=size(a)) :: s
    integer :: i
    do i = 1, size(a)
      s(i:i) = chars(a(i))
    end do
  end function decode

  subroutine softmax_vec(z, p)
    real(dp), intent(in)  :: z(:)
    real(dp), intent(out) :: p(:)
    p = exp(z - maxval(z)); p = p / sum(p)
  end subroutine softmax_vec

  subroutine adam_step(p, g, m, s, t, lr)           ! tek vektor uzerinde Adam
    real(dp), intent(inout) :: p(:), m(:), s(:)
    real(dp), intent(in)    :: g(:), lr
    integer, intent(in)     :: t
    real(dp), parameter :: b1 = 0.9_dp, b2 = 0.999_dp, eps = 1.0e-8_dp
    m = b1 * m + (1.0_dp - b1) * g
    s = b2 * s + (1.0_dp - b2) * g * g
    p = p - lr * (m / (1.0_dp - b1**t)) / (sqrt(s / (1.0_dp - b2**t)) + eps)
  end subroutine adam_step
end module lmdata

program bigram_lm
  use lmdata
```

```
  implicit none
  real(dp), allocatable, target :: th(:), gr(:), m1(:), s1(:)
  real(dp), pointer :: W(:,:), gW(:,:)
  real(dp), allocatable :: p(:), cnt(:,:)
  real(dp) :: loss, opt
  integer :: step, t, a, b, i, cur, k
  integer :: gen(100)
  call build_vocab()
  allocate(th(V*V), gr(V*V), m1(V*V), s1(V*V), p(V), cnt(V,V))
  W(1:V,1:V) => th; gW(1:V,1:V) => gr
  th = 0.0_dp; m1 = 0.0_dp; s1 = 0.0_dp
  print '(A,I0,A,I0,A,I0)', 'karakter=', NCH, '  sozluk V=', V, '  egitim=', ntrain
  print '(A,F7.4)', 'ln V = ', log(real(V, dp))
  do step = 1, 300
    gr = 0.0_dp; loss = 0.0_dp
    do t = 1, ntrain - 1
      a = ids(t); b = ids(t+1)
      call softmax_vec(W(a,:), p)
      loss = loss - log(p(b))
      p(b) = p(b) - 1.0_dp                          ! dL/dlogits = p - onehot
      gW(a,:) = gW(a,:) + p
    end do
    gr = gr / (ntrain - 1); loss = loss / (ntrain - 1)
    call adam_step(th, gr, m1, s1, step, 0.05_dp)
    if (step == 1 .or. mod(step, 100) == 0) then
      print '(A,I4,A,F8.4,A,F8.4)', 'adim ', step, '  egitim=', loss, '  dogrulama=', val_loss()
    end if
  end do
  ! sayim tabanli en iyi bigram (egitim verisi uzerinde analitik cozum)
  cnt = 0.0_dp
  do t = 1, ntrain - 1
    cnt(ids(t), ids(t+1)) = cnt(ids(t), ids(t+1)) + 1.0_dp
  end do
  opt = 0.0_dp
  do a = 1, V
    do b = 1, V
      if (cnt(a,b) > 0.0_dp) opt = opt - cnt(a,b) * log(cnt(a,b) / sum(cnt(a,:)))
    end do
  end do
  print '(A,F8.4)', 'sayim tabanli en iyi egitim kaybi = ', opt / (ntrain - 1)
  ! metin uret
  call rng_seed(5)
  cur = ids(1); gen(1) = cur
  do i = 2, 100
    call softmax_vec(W(cur,:), p)
    call sample(p, k)
    gen(i) = k; cur = k
  end do
  print '(A)', 'uretilen metin:'
  print '(A)', decode(gen)
contains
  function val_loss() result(l)
    real(dp) :: l, q(V)
    integer :: tt
    l = 0.0_dp
    do tt = ntrain + 1, NCH - 1
      call softmax_vec(W(ids(tt),:), q)
      l = l - log(q(ids(tt+1)))
    end do
    l = l / (NCH - ntrain - 1)
  end function val_loss

  subroutine sample(pr, k)
    real(dp), intent(in) :: pr(:)
    integer, intent(out) :: k
    real(dp) :: u, c
    u = uniform(); c = 0.0_dp
    do k = 1, size(pr)
      c = c + pr(k)
      if (u <= c) return
    end do
    k = size(pr)
  end subroutine sample
end program bigram_lm
```

ÇIKTI

```
karakter=481  sozluk V=23  egitim=408
ln V =  3.1355
adim    1  egitim=  3.1355  dogrulama=  3.0899
adim  100  egitim=  1.9784  dogrulama=  2.6483
adim  200  egitim=  1.9617  dogrulama=  2.7815
adim  300  egitim=  1.9572  dogrulama=  2.8668
sayim tabanli en iyi egitim kaybi =   1.9515
uretilen metin:
bedezcimizlmerdeni yorkarkar ban busllagok ahizcinn oka ir bizcesa nduk sa balan. cotti. belun sagis
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn, torch.nn.functional as F
corpus = ('bir varmis bir yokmus. evvel zaman icinde kalbur saman icinde develer tellal iken '
          'pireler berber iken ben annemin besigini tangir tungur sallar iken. '
          'cok gezdim cok gordum ama hic yorulmadim. yol uzundu ama yuregim sicakti. '
          'gece oldu yildizlar cikti ve herkes sessizce uykuya daldi. '
          'sabah olunca horozlar otti kuslar sarki soyledi ve yeni bir gun basladi. '
          'dag basinda bir cesme varmis suyu soguk mu soguk tatli mi tatli. '
          'yolcular gelir su icer dinlenir sonra yoluna devam edermis. ')
chars = sorted(set(corpus)); V = len(chars); ix = {c: i for i, c in enumerate(chars)}
ids = torch.tensor([ix[c] for c in corpus]); NCH = len(ids); ntrain = int(0.85 * NCH)
decode = lambda a: ''.join(chars[i] for i in a)

torch.manual_seed(5)
W = nn.Parameter(torch.zeros(V, V))                     # logits tablosu: satir = onceki karakter
opt = torch.optim.Adam([W], lr=0.05)
xtr, ytr = ids[:ntrain - 1], ids[1:ntrain]
xva, yva = ids[ntrain:-1], ids[ntrain + 1:]
print(f"karakter={NCH}  sozluk V={V}  egitim={ntrain}"); print(f"ln V =
      {torch.log(torch.tensor(float(V))):.4f}")
for step in range(1, 301):
    loss = F.cross_entropy(W[xtr], ytr)
    opt.zero_grad(); loss.backward(); opt.step()
    if step == 1 or step % 100 == 0:
        with torch.no_grad(): vl = F.cross_entropy(W[xva], yva)
        print(f"adim {step:4d}  egitim={loss.item():8.4f}  dogrulama={vl.item():8.4f}")
cnt = torch.zeros(V, V); cnt.index_put_((xtr, ytr), torch.ones(len(xtr)), accumulate=True)
pc = cnt / cnt.sum(1, keepdim=True).clamp(min=1)
opt_loss = -(cnt * torch.where(cnt > 0, pc.log(), torch.zeros(1))).sum() / len(xtr)
print(f"sayim tabanli en iyi egitim kaybi = {opt_loss:.4f}")
cur, gen = ids[0].item(), [ids[0].item()]
for _ in range(99):
    cur = torch.multinomial(torch.softmax(W[cur].detach(), 0), 1).item(); gen.append(cur)
print("uretilen metin:"); print(decode(gen))
```

PYTORCH ÇIKTISI

```
karakter=481  sozluk V=23  egitim=408
ln V = 3.1355
adim    1  egitim=  3.1355  dogrulama=  3.0899
adim  100  egitim=  1.9784  dogrulama=  2.6483
adim  200  egitim=  1.9617  dogrulama=  2.7815
adim  300  egitim=  1.9572  dogrulama=  2.8668
sayim tabanli en iyi egitim kaybi = 1.9515
uretilen metin:
bam isicir ogikti. kur tinndik bereslbin. g cozlamar i da kendadadu sas ar u di. shotini u durken bi
```

NASIL ÇALIŞIYOR?

Model, V×V boyutunda tek bir logit tablosudur: önceki karakter a ise sonraki karakterin logit'leri `W(a,:)` satırıdır. Geri yayılım tek satırdır: `p − onehot` (Bölüm 2), yalnızca ilgili satıra eklenir. Başlangıç kaybı ln V = 3.136'dır (sağlık kontrolü □). Adam eğitim kaybını 1.957'ye indirir; bu, **sayım tabanlı analitik en iyi bigram kaybına** (1.9515) neredeyse tam eşittir: bir karaktere bakan hiçbir model bunun altına inemez. Doğrulama kaybı ise önce düşer (adım 100'de 2.65), sonra tekrar yükselir (2.87): model eğitim metnindeki karakter çiftlerinin olasılığını keskinleştirirken görmediği çiftlere neredeyse sıfır olasılık verir (Bölüm 8'deki düzleştirme dersi). Üretilen metin Türkçe'ye benzer harf dizileri verir ("ir", "ar", "ler") ama kelime yapısını tutturamaz; bağlam yalnızca 1 karakter.

Model 2: bağlam penceresi olan MLP dil modeli

Son **C = 3** karakterin embedding'lerini yan yana birleştirip (3·e boyutlu vektör) bir gizli tanh katmanından ve softmax'tan geçiririz (Bengio ve ark., 2003). Bu, "sinir ağı dil modeli"nin tarihî atasıdır. Parametreler: embedding E (V×e), W_1 (3e×H), b_1, W_2 (H×V), b_2. Geri yayılımı Bölüm 2'deki formüllerle elle yazıyoruz ve eğitimden önce **gradyan kontrolü** yapıyoruz.

Örnek 11.2 — Model 2: MLP dil modeli (elle geri yayılım + gradyan kontrolü)

PROBLEM

C = 3 karakter bağlamlı, e = 8 boyutlu embedding'li, 48 gizli nöronlu bir MLP dil modeli yaz. Önce sayısal gradyan kontrolü yap, sonra mini-batch Adam ile eğit; eğitim/doğrulama kaybını raporla ve metin üret.

Bu örnek şu modülleri kullanır: `lmdata` *(Örnek 11.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program mlp_lm
  use lmdata
  implicit none
  integer, parameter :: NC = 3, ED = 8, NH = 48, nb = 64
  real(dp), allocatable, target :: th(:), gr(:)
  real(dp), allocatable :: m1(:), s1(:), g0(:)
  real(dp), pointer :: E_(:,:), W1(:,:), b1(:), W2(:,:), b2(:)
  real(dp), pointer :: gE(:,:), gW1(:,:), gb1(:), gW2(:,:), gb2(:)
  integer :: nth, o, step, i, idx(nb), b, tpos, ntr, k, cur(NC), nxt
  real(dp) :: loss, lsum, lp, lm, num, maxrel, old, p(V)
  integer :: gen(150)
  call build_vocab()
  nth = V*ED + NC*ED*NH + NH + NH*V + V
  allocate(th(nth), gr(nth), m1(nth), s1(nth), g0(nth))
  o = 0
  E_(1:V,1:ED)     => th(o+1:o+V*ED);     gE(1:V,1:ED)     => gr(o+1:o+V*ED);     o = o + V*ED
  W1(1:NC*ED,1:NH) => th(o+1:o+NC*ED*NH)
  gW1(1:NC*ED,1:NH) => gr(o+1:o+NC*ED*NH);   o = o + NC*ED*NH
  b1 => th(o+1:o+NH);                    gb1 => gr(o+1:o+NH);                    o = o + NH
  W2(1:NH,1:V)     => th(o+1:o+NH*V);     gW2(1:NH,1:V)    => gr(o+1:o+NH*V);     o = o + NH*V
  b2 => th(o+1:o+V);                    gb2 => gr(o+1:o+V)
  call rng_seed(11)
  call randn1(th, 0.3_dp)
  b1 = 0.0_dp; b2 = 0.0_dp
  m1 = 0.0_dp; s1 = 0.0_dp
  ntr = ntrain - NC - 1                              ! egitim ornek sayisi (tpos = NC .. ntrain-1)
  ! ---- gradyan kontrolu: ilk 20 ornek
  idx(1:20) = [(NC + i - 1, i = 1, 20)]
  loss = batch(idx, 20, .true.)
  g0 = gr
  maxrel = 0.0_dp
  do i = 1, 12
    k = 1 + mod(i * 97, nth)
    old = th(k)
    th(k) = old + 1.0e-6_dp; lp = batch(idx, 20, .false.)
    th(k) = old - 1.0e-6_dp; lm = batch(idx, 20, .false.)
    th(k) = old
    num = (lp - lm) / 2.0e-6_dp
    maxrel = max(maxrel, abs(num - g0(k)) / max(1.0e-8_dp, abs(num) + abs(g0(k))))
  end do
  print '(A,ES10.2)', 'gradyan kontrolu, en buyuk bagil hata = ', maxrel
  print '(A,I0,A,F7.4)', 'parametre sayisi = ', nth, '   baslangic kaybi = ', loss
  ! ---- egitim
  do step = 1, 3000
    do b = 1, nb
      idx(b) = NC + int(uniform() * ntr)             ! rastgele egitim konumu
    end do
    gr = 0.0_dp
    lsum = batch(idx, nb, .true.)
    gr = gr / nb
    call adam_step(th, gr, m1, s1, step, 0.01_dp)
    if (step == 1 .or. mod(step, 500) == 0) then
      print '(A,I5,A,F8.4,A,F8.4)', 'adim ', step, '  egitim=', eval(NC, ntrain-1), &
            '  dogrulama=', eval(ntrain+NC, NCH-1)
```

```
    end if
  end do
  ! ---- metin uret (sicaklik 0.8)
  call rng_seed(3)
  gen(1:NC) = ids(1:NC)
  do i = NC + 1, 150
    cur = gen(i-NC:i-1)
    call predict(cur, p)
    p = p ** (1.0_dp / 0.8_dp); p = p / sum(p)
    call sample(p, nxt)
    gen(i) = nxt
  end do
  print '(A)', 'uretilen metin:'
  print '(A)', decode(gen)
contains
  subroutine predict(ctx, pr)                       ! ileri gecis (tek ornek)
    integer, intent(in) :: ctx(NC)
    real(dp), intent(out) :: pr(V)
    real(dp) :: x(NC*ED), h(NH)
    integer :: j
    do j = 1, NC
      x((j-1)*ED+1:j*ED) = E_(ctx(j),:)
    end do
    h = tanh(matmul(x, W1) + b1)
    call softmax_vec(matmul(h, W2) + b2, pr)
  end subroutine predict

  function batch(ix, n, dograd) result(l)           ! ortalama kayip (+ istenirse gradyan)
    integer, intent(in) :: ix(:), n
    logical, intent(in) :: dograd
    real(dp) :: l, x(NC*ED), h(NH), pr(V), dlog(V), dh(NH), dz(NH), dx(NC*ED)
    integer :: s, j, ctx(NC), tgt
    l = 0.0_dp
    if (dograd) gr = 0.0_dp
    do s = 1, n
      ctx = ids(ix(s)-NC+1:ix(s)); tgt = ids(ix(s)+1)
      do j = 1, NC
        x((j-1)*ED+1:j*ED) = E_(ctx(j),:)
      end do
      h = tanh(matmul(x, W1) + b1)
      call softmax_vec(matmul(h, W2) + b2, pr)
      l = l - log(pr(tgt))
      if (dograd) then
        dlog = pr; dlog(tgt) = dlog(tgt) - 1.0_dp
        gW2 = gW2 + spread(h, 2, V) * spread(dlog, 1, NH)
        gb2 = gb2 + dlog
        dh = matmul(W2, dlog)
        dz = dh * (1.0_dp - h**2)
        gW1 = gW1 + spread(x, 2, NH) * spread(dz, 1, NC*ED)
        gb1 = gb1 + dz
        dx = matmul(W1, dz)
        do j = 1, NC
          gE(ctx(j),:) = gE(ctx(j),:) + dx((j-1)*ED+1:j*ED)
        end do
      end if
    end do
    l = l / n
    if (dograd) gr = gr / n
  end function batch

  function eval(lo, hi) result(l)                   ! tpos = lo..hi uzerinde ortalama kayip
    integer, intent(in) :: lo, hi
    real(dp) :: l, pr(V)
    integer :: tp
    l = 0.0_dp
    do tp = lo, hi
      call predict(ids(tp-NC+1:tp), pr)
      l = l - log(pr(ids(tp+1)))
    end do
    l = l / (hi - lo + 1)
  end function eval

  subroutine sample(pr, k)
    real(dp), intent(in) :: pr(:)
    integer, intent(out) :: k
    real(dp) :: u, cs
    u = uniform(); cs = 0.0_dp
    do k = 1, size(pr)
```

```
      cs = cs + pr(k)
      if (u <= cs) return
    end do
    k = size(pr)
  end subroutine sample
end program mlp_lm
```

ÇIKTI

```
gradyan kontrolu, en buyuk bagil hata =   5.05E-08
parametre sayisi = 2511   baslangic kaybi =  3.3735
adim     1  egitim=  3.3634  dogrulama=  3.2760
adim   500  egitim=  0.3918  dogrulama=  4.6529
adim  1000  egitim=  0.3079  dogrulama=  5.3081
adim  1500  egitim=  0.2960  dogrulama=  5.6049
adim  2000  egitim=  0.3092  dogrulama=  5.8814
adim  2500  egitim=  0.2948  dogrulama=  6.1645
adim  3000  egitim=  0.2975  dogrulama=  6.3576
uretilen metin:
bir cesme varmis sessizce oldu ama horozlar saman icinde kalbur sarki soguk tangir tungur saman icinde d
   evel zama hic yol uzundu aman icinda bir cesme
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn, torch.nn.functional as F
corpus = ('bir varmis bir yokmus. evvel zaman icinde kalbur saman icinde develer tellal iken '
          'pireler berber iken ben annemin besigini tangir tungur sallar iken. '
          'cok gezdim cok gordum ama hic yorulmadim. yol uzundu ama yuregim sicakti. '
          'gece oldu yildizlar cikti ve herkes sessizce uykuya daldi. '
          'sabah olunca horozlar otti kuslar sarki soyledi ve yeni bir gun basladi. '
          'dag basinda bir cesme varmis suyu soguk mu soguk tatli mi tatli. '
          'yolcular gelir su icer dinlenir sonra yoluna devam edermis. ')
chars = sorted(set(corpus)); V = len(chars); ix = {c: i for i, c in enumerate(chars)}
ids = torch.tensor([ix[c] for c in corpus]); NCH = len(ids); ntrain = int(0.85 * NCH)
decode = lambda a: ''.join(chars[i] for i in a)

torch.manual_seed(11)
NC, ED, NH = 3, 8, 48
class MLPLM(nn.Module):
    def __init__(s):
        super().__init__()
        s.E = nn.Embedding(V, ED); s.l1 = nn.Linear(NC * ED, NH); s.l2 = nn.Linear(NH, V)
        for p in s.parameters(): nn.init.normal_(p, std=0.3)
        nn.init.zeros_(s.l1.bias); nn.init.zeros_(s.l2.bias)
    def forward(s, ctx):                                    # ctx: (B, NC)
        return s.l2(torch.tanh(s.l1(s.E(ctx).flatten(1))))
m = MLPLM(); opt = torch.optim.Adam(m.parameters(), lr=0.01)
pos = lambda lo, hi: torch.arange(lo, hi + 1)
ctx_of = lambda tp: torch.stack([ids[tp - NC + 1 + j] for j in range(NC)], 1)
def ev(lo, hi):
    tp = pos(lo, hi)
    with torch.no_grad(): return F.cross_entropy(m(ctx_of(tp)), ids[tp + 1]).item()
print("parametre sayisi =", sum(p.numel() for p in m.parameters()))
for step in range(1, 3001):
    tp = NC - 1 + torch.randint(0, ntrain - NC - 1, (64,))         # 0 tabanli konumlar
    loss = F.cross_entropy(m(ctx_of(tp)), ids[tp + 1])
    opt.zero_grad(); loss.backward(); opt.step()
    if step == 1 or step % 500 == 0:
        print(f"adim {step:5d}  egitim={ev(NC - 1, ntrain - 2):8.4f}  dogrulama={ev(ntrain + NC - 1, NCH
              - 2):8.4f}")
gen = ids[:NC].tolist()
for _ in range(150 - NC):
    with torch.no_grad(): p = torch.softmax(m(torch.tensor([gen[-NC:]]))[0] / 0.8, 0)
    gen.append(torch.multinomial(p, 1).item())
print("uretilen metin:"); print(decode(gen))
```

PYTORCH ÇIKTISI

```
parametre sayisi = 2511
adim     1  egitim=  3.3284  dogrulama=  3.2701
adim   500  egitim=  0.3667  dogrulama=  4.4175
adim  1000  egitim=  0.3045  dogrulama=  5.3471
adim  1500  egitim=  0.2755  dogrulama=  5.8328
adim  2000  egitim=  0.2948  dogrulama=  6.2136
adim  2500  egitim=  0.2811  dogrulama=  6.4875
adim  3000  egitim=  0.2752  dogrulama=  6.6985
uretilen metin:
bir cesme varmis suy uykuya daldizlar cesme varmis suyu soguk mu soguk mu soyledi ve herkes sessizce uyk
   uya dag basinde kalbur sarki soguk mu soguk mu
```

NASIL ÇALIŞIYOR?

Bölüm 2'deki XOR ağının büyüğüdür: embedding aramasından sonra "birleştir → W_1 → tanh → W_2 → softmax". Geri geçişte `dlog = p − onehot`, $gW2 = h^T \cdot dlog$, $dh = W_2 \cdot dlog$, tanh türevi $(1-h^2)$ ve sonunda birleşik girdi gradyanı `dx` parçalanıp ilgili embedding satırlarına eklenir (aynı karakter bağlamda birden çok kez görünürse gradyanları toplanır). Gradyan kontrolü bağıl hatayı ≈ 5e-8 verir: geri yayılım doğru. Eğitim kaybı bigram'ın alt sınırını (1.95) çok aşağıda bırakıp ≈ 0.3'e iner, çünkü üç karakterlik bağlam bu küçük metinde sonraki karakteri neredeyse belirler. Ama **doğrulama kaybı 3.3'ten 6.4'e yükselir**: 2511 parametreli model 408 karakteri ezberlemiştir ve görmediği cümlelerde büyük bir güvenle yanlış tahmin eder. Üretilen metin de eğitim metninden ("bir cesme varmis", "saman icinde kalbur", "tangir tungur") parçaları geri okuyor gibidir. Bu, Bölüm 8'deki aşırı öğrenmenin ders kitabı örneğidir. Çözüm veri miktarını artırmak (asıl çare), erken durdurma, weight decay ve dropout'tur.

Model 3: causal self-attention dil modeli (mini GPT)

Şimdi kitabın merkezindeki mimariyi, **token embedding + konum embedding → causal self-attention → residual → logits**, eğitiyoruz. Çıkış matrisi embedding'in transpozudur (**weight tying**). Normalizasyon ve FFN katmanlarını çıkardık (Bölüm 4'te bunları kurmuştuk; burada geri yayılımı tam ve doğrulanabilir tutmak için sadeleştirdik). Geri yayılım Bölüm 4'ün sonundaki formüllerin zinciridir:

$$P = \mathrm{softmax}(H\,E^T),\ dL = (P - \mathrm{onehot})/T$$

$$dH = dL \cdot E,\ dE \mathrel{+}= dL^T \cdot H,\ H = X + Z \cdot W_o,\ Z = A \cdot V$$

$$dA = dZ \cdot V^T,\ dS = A \odot (dA - \Sigma\, A \odot dA) \cdot 1/\sqrt{d},\ dQ = dS \cdot K,\ dK = dS^T \cdot Q$$

$$dX = dH + dQ \cdot W_q^T + dK \cdot W_k^T + dV \cdot W_v^T$$

Örnek 11.3 — Model 3: Tek katmanlı, tek başlı attention dil modeli

PROBLEM

T = 16 bağlamlı, d = 16 boyutlu causal attention dil modelini (token + konum embedding, Q/K/V/O projeksiyonları, residual, weight tying) tam elle yazılmış geri yayılımla eğit. Gradyan kontrolü yap, eğitim/doğrulama kaybını raporla ve metin üret.

Bu örnek şu modülleri kullanır: `lmdata` *(Örnek 11.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program attn_lm
  use lmdata
  implicit none
  integer, parameter :: d = 16, TT = 16, nb = 8
  real(dp), allocatable, target :: th(:), gr(:)
  real(dp), allocatable :: m1(:), s1(:), g0(:)
  real(dp), pointer :: Em(:,:), Pe(:,:), Wq(:,:), Wk(:,:), Wv(:,:), Wo(:,:)
```

```
  real(dp), pointer :: gE(:,:), gPe(:,:), gWq(:,:), gWk(:,:), gWv(:,:), gWo(:,:)
  integer :: nth, o, step, b, s0, i, k, n, nxt
  real(dp) :: loss, lsum, lp, lm, num, maxrel, old, pr(V)
  integer :: gen(160)
  call build_vocab()
  nth = V*d + TT*d + 4*d*d
  allocate(th(nth), gr(nth), m1(nth), s1(nth), g0(nth))
  o = 0
  Em(1:V,1:d)   => th(o+1:o+V*d);   gE(1:V,1:d)   => gr(o+1:o+V*d);   o = o + V*d
  Pe(1:TT,1:d)  => th(o+1:o+TT*d);  gPe(1:TT,1:d) => gr(o+1:o+TT*d);  o = o + TT*d
  Wq(1:d,1:d)   => th(o+1:o+d*d);   gWq(1:d,1:d)  => gr(o+1:o+d*d);   o = o + d*d
  Wk(1:d,1:d)   => th(o+1:o+d*d);   gWk(1:d,1:d)  => gr(o+1:o+d*d);   o = o + d*d
  Wv(1:d,1:d)   => th(o+1:o+d*d);   gWv(1:d,1:d)  => gr(o+1:o+d*d);   o = o + d*d
  Wo(1:d,1:d)   => th(o+1:o+d*d);   gWo(1:d,1:d)  => gr(o+1:o+d*d)
  call rng_seed(21)
  call randn2(Em, 0.3_dp); call randn2(Pe, 0.1_dp)
  call randn2(Wq, 0.25_dp); call randn2(Wk, 0.25_dp)
  call randn2(Wv, 0.25_dp); call randn2(Wo, 0.25_dp)
  m1 = 0.0_dp; s1 = 0.0_dp
  ! ---- gradyan kontrolu (tek pencere)
  gr = 0.0_dp
  call fwd_bwd(ids(1:TT), ids(2:TT+1), TT, .true., loss, pr)
  g0 = gr
  maxrel = 0.0_dp
  do i = 1, 14
    k = 1 + mod(i * 131, nth)
    old = th(k)
    th(k) = old + 1.0e-6_dp; call fwd_bwd(ids(1:TT), ids(2:TT+1), TT, .false., lp, pr)
    th(k) = old - 1.0e-6_dp; call fwd_bwd(ids(1:TT), ids(2:TT+1), TT, .false., lm, pr)
    th(k) = old
    num = (lp - lm) / 2.0e-6_dp
    maxrel = max(maxrel, abs(num - g0(k)) / max(1.0e-8_dp, abs(num) + abs(g0(k))))
  end do
  print '(A,ES10.2)', 'gradyan kontrolu, en buyuk bagil hata = ', maxrel
  print '(A,I0,A,F7.4,A,F7.4)', 'parametre sayisi = ', nth, '  baslangic kaybi = ', loss, &
        '  ln V = ', log(real(V, dp))
  ! ---- egitim
  do step = 1, 4000
    gr = 0.0_dp; lsum = 0.0_dp
    do b = 1, nb
      s0 = 1 + int(uniform() * (ntrain - TT))      ! rastgele egitim penceresi
      call fwd_bwd(ids(s0:s0+TT-1), ids(s0+1:s0+TT), TT, .true., loss, pr)
      lsum = lsum + loss
    end do
    gr = gr / nb
    call adam_step(th, gr, m1, s1, step, 0.01_dp)
    if (step == 1 .or. mod(step, 1000) == 0) then
      print '(A,I5,A,F8.4,A,F8.4)', 'adim ', step, '  egitim=', eval(1, ntrain), &
            '  dogrulama=', eval(ntrain+1, NCH)
    end if
  end do
  ! ---- metin uret (sicaklik 0.8), baglam penceresi en fazla TT karakter
  call rng_seed(9)
  gen(1:8) = ids(1:8)
  do i = 9, 160
    n = min(i - 1, TT)
    call fwd_bwd(gen(i-n:i-1), gen(i-n:i-1), n, .false., loss, pr)
    pr = pr ** (1.0_dp / 0.8_dp); pr = pr / sum(pr)
    call sample(pr, nxt)
    gen(i) = nxt
  end do
  print '(A)', 'uretilen metin (ilk 8 karakter komut):'
  print '(A)', decode(gen)
contains
  subroutine smrows(Mx)
    real(dp), intent(inout) :: Mx(:,:)
    integer :: r
    do r = 1, size(Mx,1)
      Mx(r,:) = exp(Mx(r,:) - maxval(Mx(r,:)))
      Mx(r,:) = Mx(r,:) / sum(Mx(r,:))
    end do
  end subroutine smrows

  subroutine fwd_bwd(xi, yi, n, dograd, l, plast)   ! ileri gecis (+ geri gecis)
    integer, intent(in) :: n, xi(n), yi(n)
    logical, intent(in) :: dograd
    real(dp), intent(out) :: l, plast(V)
    real(dp) :: Xm(n,d), Q(n,d), K(n,d), Vv(n,d), A(n,n), Z(n,d), Hh(n,d), Pr(n,V)
```

```
    real(dp) :: dL(n,V), dH(n,d), dZ(n,d), dA(n,n), dV(n,d), dS(n,n), dQ(n,d), dK(n,d), dX(n,d)
    real(dp) :: sc
    integer :: t, r
    sc = 1.0_dp / sqrt(real(d, dp))
    do t = 1, n
      Xm(t,:) = Em(xi(t),:) + Pe(t,:)               ! token + konum embedding
    end do
    Q = matmul(Xm, Wq); K = matmul(Xm, Wk); Vv = matmul(Xm, Wv)
    A = matmul(Q, transpose(K)) * sc
    do r = 1, n - 1
      A(r, r+1:n) = -1.0e30_dp                      ! causal mask
    end do
    call smrows(A)
    Z  = matmul(A, Vv)
    Hh = Xm + matmul(Z, Wo)                          ! residual
    Pr = matmul(Hh, transpose(Em))                   ! logits (weight tying)
    call smrows(Pr)
    l = 0.0_dp
    do t = 1, n
      l = l - log(Pr(t, yi(t)))
    end do
    l = l / n
    plast = Pr(n,:)
    if (.not. dograd) return
    dL = Pr
    do t = 1, n
      dL(t, yi(t)) = dL(t, yi(t)) - 1.0_dp
    end do
    dL = dL / n
    dH = matmul(dL, Em)
    gE = gE + matmul(transpose(dL), Hh)              ! cikis tarafi
    gWo = gWo + matmul(transpose(Z), dH)
    dZ = matmul(dH, transpose(Wo))
    dA = matmul(dZ, transpose(Vv))
    dV = matmul(transpose(A), dZ)
    do r = 1, n                                      ! softmax geri yayilimi
      dS(r,:) = A(r,:) * (dA(r,:) - sum(A(r,:) * dA(r,:)))
    end do
    dS = dS * sc
    dQ = matmul(dS, K)
    dK = matmul(transpose(dS), Q)
    dX = dH + matmul(dQ, transpose(Wq)) + matmul(dK, transpose(Wk)) + matmul(dV, transpose(Wv))
    gWq = gWq + matmul(transpose(Xm), dQ)
    gWk = gWk + matmul(transpose(Xm), dK)
    gWv = gWv + matmul(transpose(Xm), dV)
    do t = 1, n
      gE(xi(t),:) = gE(xi(t),:) + dX(t,:)           ! giris tarafi
    end do
    gPe(1:n,:) = gPe(1:n,:) + dX
  end subroutine fwd_bwd

  function eval(lo, hi) result(l)                    ! [lo,hi] araligindaki pencereler
    integer, intent(in) :: lo, hi
    real(dp) :: l, lw, pl(V)
    integer :: s, cnt
    l = 0.0_dp; cnt = 0
    do s = lo, hi - TT, 4
      call fwd_bwd(ids(s:s+TT-1), ids(s+1:s+TT), TT, .false., lw, pl)
      l = l + lw; cnt = cnt + 1
    end do
    l = l / cnt
  end function eval

  subroutine sample(p, k)
    real(dp), intent(in) :: p(:)
    integer, intent(out) :: k
    real(dp) :: u, cs
    u = uniform(); cs = 0.0_dp
    do k = 1, size(p)
      cs = cs + p(k)
      if (u <= cs) return
    end do
    k = size(p)
  end subroutine sample
end program attn_lm
```

ÇIKTI

```
gradyan kontrolu, en buyuk bagil hata =   2.90E-06
parametre sayisi = 1648  baslangic kaybi =  3.2191  ln V =  3.1355
adim     1  egitim=  3.2372  dogrulama=  3.2475
adim  1000  egitim=  1.6249  dogrulama=  3.1707
adim  2000  egitim=  1.4534  dogrulama=  3.5957
adim  3000  egitim=  1.3497  dogrulama=  3.8002
adim  4000  egitim=  1.3222  dogrulama=  4.1418
uretilen metin (ilk 8 karakter komut):
bir varmi ser samamim. sug sabpireli ran bir ben ecessler eges erbi verbi ver s as uyladi sama sigama si
   ndaki hicoy ka ka ssismmamin bak kesssallar itikum yum s
```

PYTORCH KARŞILIĞI

```
import torch, torch.nn as nn, torch.nn.functional as F
corpus = ('bir varmis bir yokmus. evvel zaman icinde kalbur saman icinde develer tellal iken '
          'pireler berber iken ben annemin besigini tangir tungur sallar iken. '
          'cok gezdim cok gordum ama hic yorulmadim. yol uzundu ama yuregim sicakti. '
          'gece oldu yildizlar cikti ve herkes sessizce uykuya daldi. '
          'sabah olunca horozlar otti kuslar sarki soyledi ve yeni bir gun basladi. '
          'dag basinda bir cesme varmis suyu soguk mu soguk tatli mi tatli. '
          'yolcular gelir su icer dinlenir sonra yoluna devam edermis. ')
chars = sorted(set(corpus)); V = len(chars); ix = {c: i for i, c in enumerate(chars)}
ids = torch.tensor([ix[c] for c in corpus]); NCH = len(ids); ntrain = int(0.85 * NCH)
decode = lambda a: ''.join(chars[i] for i in a)

import math
torch.manual_seed(21)
d, TT = 16, 16
class AttnLM(nn.Module):                                        # token+konum -> causal attention ->
      residual -> tied logits
    def __init__(s):
        super().__init__()
        s.E = nn.Parameter(0.3 * torch.randn(V, d)); s.P = nn.Parameter(0.1 * torch.randn(TT, d))
        s.q, s.k, s.v, s.o = (nn.Linear(d, d, bias=False) for _ in range(4))
        for l in (s.q, s.k, s.v, s.o): nn.init.normal_(l.weight, std=0.25)
    def forward(s, x):                                          # x: (B, n)
        n = x.shape[1]
        X = s.E[x] + s.P[:n]
        A = F.scaled_dot_product_attention(s.q(X), s.k(X), s.v(X), is_causal=True)   #
              softmax(QK^T/sqrt(d)) V
        H = X + s.o(A)                                          # residual
        return H @ s.E.T                                        # weight tying
m = AttnLM(); opt = torch.optim.Adam(m.parameters(), lr=0.01)
print("parametre sayisi =", sum(p.numel() for p in m.parameters()))
def win(starts): return torch.stack([ids[s:s + TT] for s in starts]), torch.stack([ids[s + 1:s + TT + 1]
      for s in starts])
def ev(lo, hi):                                                 # [lo,hi) araligindaki pencereler
    st = list(range(lo, hi - TT, 4)); x, y = win(st)
    with torch.no_grad(): return F.cross_entropy(m(x).flatten(0, 1), y.flatten()).item()
x, y = win([0])
print(f"baslangic kaybi = {F.cross_entropy(m(x).flatten(0, 1), y.flatten()).item():.4f}   ln V =
      {math.log(V):.4f}")
for step in range(1, 4001):
    x, y = win((torch.randint(0, ntrain - TT, (8,))).tolist())
    loss = F.cross_entropy(m(x).flatten(0, 1), y.flatten())
    opt.zero_grad(); loss.backward(); opt.step()
    if step == 1 or step % 1000 == 0:
        print(f"adim {step:5d}  egitim={ev(0, ntrain):8.4f}  dogrulama={ev(ntrain, NCH):8.4f}")
gen = ids[:8].tolist()
for _ in range(152):
    with torch.no_grad(): p = torch.softmax(m(torch.tensor([gen[-TT:]]))[0, -1] / 0.8, 0)
    gen.append(torch.multinomial(p, 1).item())
print("uretilen metin (ilk 8 karakter komut):"); print(decode(gen))
```

PYTORCH ÇIKTISI

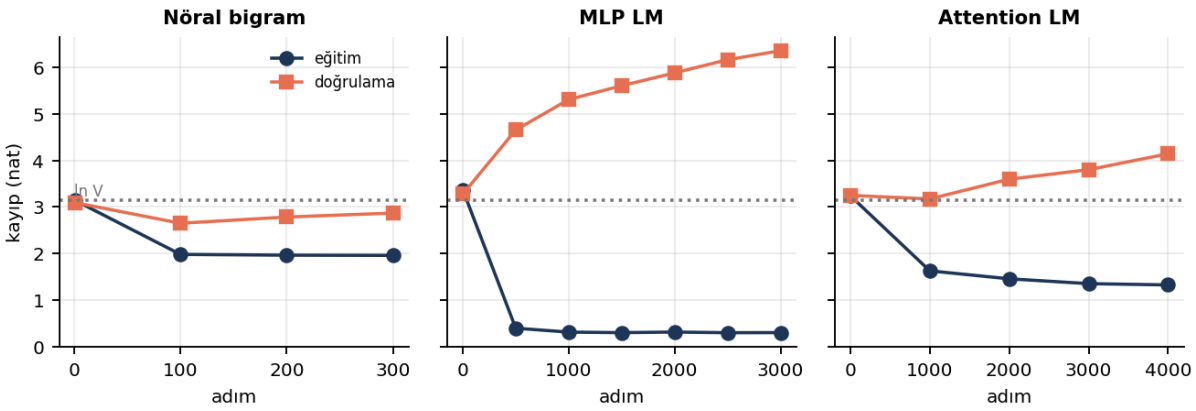
```
parametre sayisi = 1648
baslangic kaybi = 3.2843   ln V = 3.1355
adim     1  egitim=  3.2724  dogrulama=  3.2561
adim  1000  egitim=  1.5751  dogrulama=  3.1194
adim  2000  egitim=  1.4623  dogrulama=  3.5584
adim  3000  egitim=  1.3768  dogrulama=  3.9245
adim  4000  egitim=  1.3495  dogrulama=  3.9915
uretilen metin (ilk 8 karakter komut):
bir varmi saldi. gessigica ssunn ussicik segun banr besigun gunn uyeni kar dama sanndu ba saken. bece su
   yur ssahh okkun uyaken ussicotti ka gun bar ikes yokmu s
```

NASIL ÇALIŞIYOR?

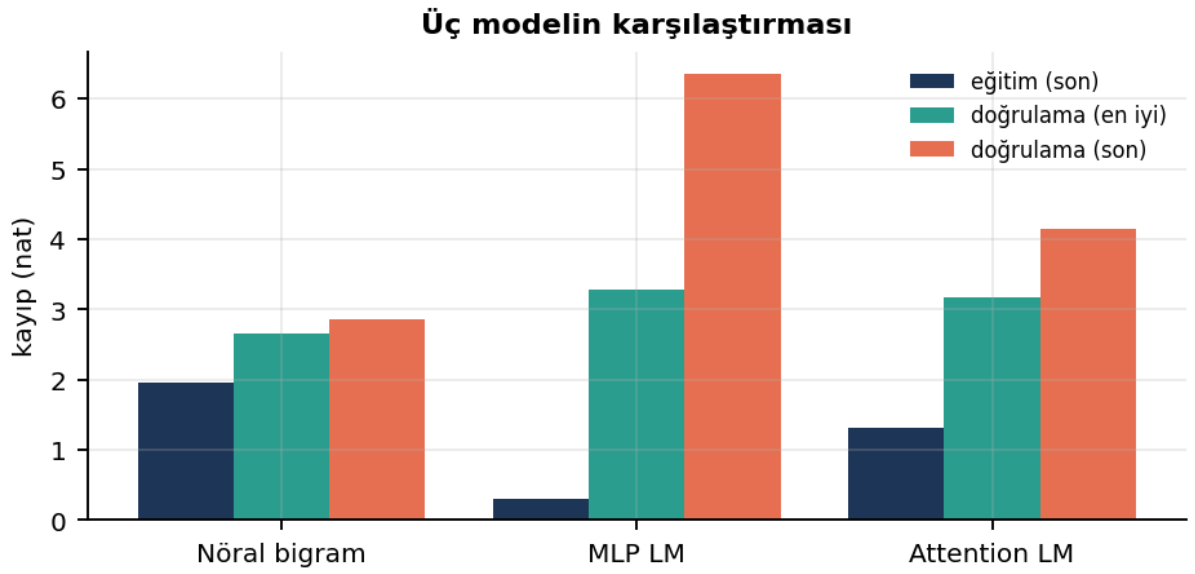
Bu program, kitabın tüm parçalarının birleştiği yerdir. İleri geçiş Bölüm 4'tekiyle aynıdır (embedding + konum, Q/K/V, ölçekli skor, causal mask, satır softmax, ağırlıklı toplam, çıkış projeksiyonu, residual, tied logits, çapraz entropi). Geri geçiş, yukarıdaki dört formül satırının doğrudan kodudur: `dS = A·(dA − Σ A·dA)` softmax geri yayılımı; maskelenmiş konumlarda A = 0 olduğundan dS de otomatik olarak sıfırdır; embedding matrisi **iki yerden** gradyan alır: giriş tarafından (arama) ve çıkış tarafından (weight tying, $dL^{\top}\cdot H$). Gradyan kontrolü ≈ 1e-6 bağıl hata verir (sonlu fark hatası düzeyi): elle yazılmış tam geri yayılım doğrudur. Başlangıç kaybı ln V'ye yakındır (3.22 vs 3.14). Eğitim kaybı 1.32'ye iner; yani yalnızca 1648 parametreyle, bir karaktere bakan bigram'ın teorik sınırını (1.95) aşar, çünkü 16 karakterlik bağlamdan yararlanır. Üretilen metin ("bir varmi ser samamim ...") kelime ve hece yapısını taşır. MLP'den daha az ezberler (eğitim kaybı 0.30 yerine 1.32), çünkü parametre sayısı ve kapasitesi daha küçüktür; ama yine de doğrulama kaybı 1000. adımdan sonra yükselir (3.17 → 4.14): **erken durdurma noktası** yaklaşık 1000. adımdır. **Bu, GPT'nin özünün küçük ölçekte çalışan bir kopyasıdır.** Gerçek modeller aynı mekaniği bloklar hâlinde üst üste koyar, FFN ve normalizasyon ekler, milyarlarca token ile eğitir; veri o kadar büyüktür ki ezberlemek mümkün olmaz.

Sonuçların karşılaştırması ve dürüst yorum

Aşağıdaki tablo yukarıdaki üç örneğin gerçek çıktılarından alınmıştır (karakter başına çapraz entropi, doğal logaritma; küçük daha iyi; rastgele tahmin = ln 23 = 3.14).



Şekil 11.1 — Üç modelin eğitim (mavi) ve doğrulama (turuncu) kaybı (Fortran çıktılarından). Kesikli çizgi: ln V. Doğrulama kaybı erken düşer sonra yükselir: ezber.



Şekil 11.2 — Üç modelin eğitim kaybı, en iyi doğrulama kaybı ve son doğrulama kaybı.

Model	Parametre	Bağlam	Eğitim kaybı (son)	Doğrulama kaybı (en iyi → son)
Nöral bigram	529	1	1.957	2.65 (adım ~100) → 2.87
MLP LM	2511	3	0.298	3.28 (adım 1) → 6.36
Attention LM	1648	16	1.322	3.17 (adım 1000) → 4.14

Sonuç 1: kapasite ve bağlam eğitim kaybını düşürür. Daha fazla bağlama bakan modeller (MLP, attention) bigram'ın teorik sınırını (1.95) aşar. **Sonuç 2: bu küçüklükte bir veri kümesinde doğrulama kaybı kötüleşir.** 408 karakterlik eğitim verisi, bir sinir ağının ezberlemesi için yeterince küçüktür ve doğrulama metni farklı cümlelerdir. Bu bir hata değil, ölçeğin dersidir: LLM'lerin başarısı mimariden kadar *veri miktarından* gelir. Milyarlarca token'da bir modelin her örneği ezberlemesi imkânsızdır, bu yüzden genelleme yapmak zorunda kalır.

> **İpucu**
> **Kendin dene.** (1) Metni kendi Türkçe metinlerinle genişlet (ASCII yaz; `V` sabitini programın verdiği hataya göre güncelle) ve doğrulama kaybının nasıl iyileştiğini izle. (2) Eğitim döngüsüne doğrulama kaybı en düşük olduğunda parametreleri kaydeden *erken durdurma* ekle (Bölüm 8'deki checkpoint koduyla). (3) Model 3'e Bölüm 4'teki RMSNorm ve FFN'i ekle; yeni parametreler için gradyan kontrolünü genişlet. (4) Çoklu head ekle. (5) Üretimde top-k/top-p ve KV cache kullan (Bölüm 10).

BÖLÜM 12

12. Kuantum Mekaniği: Schrödinger Denklemini Fortran ile Çözmek

Kitabın ikinci bölümü doğa bilimleri ile yapay zekanın kesiştiği yere gidiyor. Önce molekülleri anlatan fiziğin temelini, Schrödinger denklemini, hiçbir kütüphane olmadan çözüyoruz.

Bu bölümde

- Atomik birimler, dalga fonksiyonu ve Schrödinger denklemi
- Sonlu farklarla Hamiltonyen matrisi ve öz-değer problemi (Jacobi yöntemi)
- Kutudaki parçacık, harmonik osilatör, hidrojen atomu
- Varyasyonel ilke: kuantum kimyasının "gradyan inişi"
- Makine öğrenmesi ile paralellikler: Hamiltonyen = kayıp yüzeyi, öz-değer = en iyi çözüm

Hedef: Bir kuantum sisteminin enerji seviyelerini sayısal olarak hesaplayıp analitik sonuçlarla karşılaştırmak.

Neden kuantum kimyası? Yapay zekanın hedefi

Bir ilacın proteine bağlanması, bir pilin voltajı, bir katalizörün hızı hepsi elektronların davranışıyla belirlenir ve elektronlar **kuantum mekaniğine** uyar. Moleküler makine öğrenmesinin (Bölüm 15–16) iki ana kaynağı vardır: (1) kuantum hesaplarından gelen veri (enerjiler, kuvvetler) ve (2) kuantum hesaplarını taklit eden hızlı yaklaşık modeller. Bu yüzden önce fiziği, sonra makine öğrenmesini öğreniyoruz. İyi haber: kuantum kimyasının çekirdek matematiği, bu kitapta yaptığımız şeylerle şaşırtıcı biçimde aynıdır: matris çarpımları, öz-değerler, gradyan inişi ve bir enerji yüzeyinin minimumunu aramak.

Makine öğrenmesi	Kuantum kimyası
Kayıp fonksiyonu L(θ)	Enerji E[ψ] (Hamiltonyen beklenen değeri)
Parametreler θ	Dalga fonksiyonu katsayıları / orbitaller
Gradyan inişi	Varyasyonel minimizasyon, SCF yinelemesi
En iyi parametre	Temel durum (en düşük enerji)
Softmax olasılıkları	Boltzmann dağılımı $e^{-E/kT}$ / Z
Attention (komşulardan bilgi topla)	Mesaj geçirme / elektronik etkileşim
Yüksek boyutlu veri	N elektronlu dalga fonksiyonu (3N boyut)

Schrödinger denklemi ve atomik birimler

Basit matematik

Parçacığın durumu bir dalga fonksiyonu $\psi(x)$ ile verilir; $|\psi(x)|^2\,dx$, parçacığı x civarında bulma olasılığıdır ($\int|\psi|^2dx = 1$). Zamandan bağımsız Schrödinger denklemi bir öz-değer problemidir:

$\hat{H}\,\psi = E\,\psi$, $\hat{H} = -(\hbar^2/2m)\,d^2/dx^2 + V(x)$

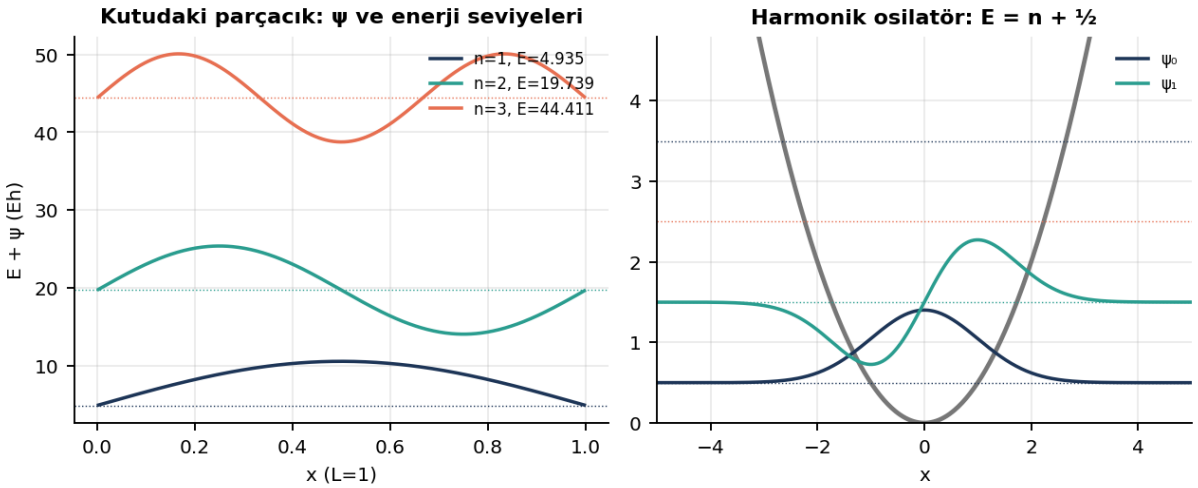
Atomik birimler: $\hbar = m_e = e = 4\pi\varepsilon_0 = 1$ alınır. Enerji birimi 1 Hartree (Eh) = 27.2114 eV, uzunluk birimi 1 Bohr = 0.529177 Å. Bu birimlerde $\hat{H} = -\tfrac{1}{2}\,d^2/dx^2 + V(x)$ olur ve tüm denklemler sadeleşir. Hidrojen atomunun temel enerjisi tam olarak $-\tfrac{1}{2}$ Eh'dir.

Basit matematik

Sonlu fark yöntemi: x ekseni h aralıklı noktalara bölünür, $\psi_i = \psi(x_i)$. İkinci türev:

$\psi''(x_i) \approx (\psi_{i-1} - 2\psi_i + \psi_{i+1}) / h^2$

Böylece $\hat{H}\psi = E\psi$ bir **matris öz-değer problemi** olur: $H_{ii} = 1/h^2 + V_i$, $H_{i,i\pm1} = -1/(2h^2)$. H simetrik bir matristir; öz-değerleri izinli enerjiler, öz-vektörleri dalga fonksiyonlarıdır. Simetrik matrisler için en basit güvenilir algoritma **Jacobi döndürmeleridir**: matrisi, dışköşegen elemanları adım adım sıfırlayan 2×2 dönmelerle köşegenleştiririz. Her dönme, $A_{ij} = 0$ olacak şekilde seçilir: $\tan 2\theta = 2A_{ij} / (A_{ii} - A_{jj})$.



Şekil 12.1 — Solda: kutudaki parçacığın ilk üç dalga fonksiyonu ve enerji seviyeleri ($E_n = n^2\pi^2/2L^2$); sağda: harmonik osilatör potansiyeli, eşit aralıklı seviyeler ve temel durum.

Örnek 12.1 — Kutudaki parçacık: sonlu farklar ve Jacobi yöntemi

PROBLEM

Uzunluğu L = 1 olan sonsuz derin kutuda (V = 0, duvarlarda $\psi = 0$) parçacığın enerjilerini 200 noktalı sonlu fark matrisinden hesapla. Analitik $E_n = n^2\pi^2/(2L^2)$ ile karşılaştır; dalga fonksiyonlarının normlandığını ve ortogonal olduğunu doğrula.

Bu örnek şu modülleri kullanır: `prec_mod` *(Ek A). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module eig_mod
  use prec_mod
  implicit none
contains
  subroutine jacobi_eig(A0, w, Vv)              ! simetrik matris: A = V diag(w) V^T, ozdegerler artan
        sirada
    real(dp), intent(in)  :: A0(:,:)
    real(dp), intent(out) :: w(:), Vv(:,:)
    real(dp), allocatable :: A(:,:)
    integer :: n, i, j, k, sweep
    real(dp) :: off, c, s, t, aij, tau, tmp
    n = size(A0,1)
    allocate(A(n,n)); A = A0
    Vv = 0.0_dp
    do i = 1, n
```

```
      Vv(i,i) = 1.0_dp
    end do
    do sweep = 1, 60
      off = 0.0_dp
      do j = 2, n
        do i = 1, j-1
          off = off + A(i,j)**2
        end do
      end do
      if (off < 1.0e-22_dp) exit
      do j = 2, n
        do i = 1, j-1
          aij = A(i,j)
          if (abs(aij) < 1.0e-300_dp) cycle
          tau = (A(j,j) - A(i,i)) / (2.0_dp*aij)
          t = sign(1.0_dp, tau) / (abs(tau) + sqrt(1.0_dp + tau*tau))
          c = 1.0_dp / sqrt(1.0_dp + t*t); s = t*c
          do k = 1, n                              ! A <- A J
            tmp = A(k,i); A(k,i) = c*tmp - s*A(k,j); A(k,j) = s*tmp + c*A(k,j)
          end do
          do k = 1, n                              ! A <- J^T A
            tmp = A(i,k); A(i,k) = c*tmp - s*A(j,k); A(j,k) = s*tmp + c*A(j,k)
          end do
          do k = 1, n                              ! ozvektorleri biriktir
            tmp = Vv(k,i); Vv(k,i) = c*tmp - s*Vv(k,j); Vv(k,j) = s*tmp + c*Vv(k,j)
          end do
        end do
      end do
    end do
    do i = 1, n
      w(i) = A(i,i)
    end do
    do i = 1, n-1                                  ! artan sirala
      k = minloc(w(i:n), dim=1) + i - 1
      if (k /= i) then
        tmp = w(i); w(i) = w(k); w(k) = tmp
        do j = 1, n
          tmp = Vv(j,i); Vv(j,i) = Vv(j,k); Vv(j,k) = tmp
        end do
      end if
    end do
  end subroutine jacobi_eig
end module eig_mod

program kutu
  use eig_mod
  implicit none
  integer, parameter :: n = 200
  real(dp), parameter :: pi = 3.141592653589793_dp, L = 1.0_dp
  real(dp) :: H(n,n), w(n), Vv(n,n), h_, ex
  integer :: i, k, nodes
  h_ = L / (n + 1)
  H = 0.0_dp
  do i = 1, n
    H(i,i) = 1.0_dp / h_**2
    if (i < n) then
      H(i,i+1) = -0.5_dp / h_**2; H(i+1,i) = -0.5_dp / h_**2
    end if
  end do
  call jacobi_eig(H, w, Vv)
  print '(A)', ' n    E(sayisal)    E(analitik)    fark      bagil hata'
  do k = 1, 5
    ex = (k*pi/L)**2 / 2.0_dp
    print '(I3,3F14.6,ES11.2)', k, w(k), ex, w(k)-ex, abs(w(k)-ex)/ex
  end do
  nodes = 0                                          ! 3. durumun dugum (isaret degisimi) sayisi
  do i = 1, n-1
    if (Vv(i,3) * Vv(i+1,3) < 0.0_dp) nodes = nodes + 1
  end do
  print '(A,F10.6)', '<psi1|psi1> = ', sum(Vv(:,1)**2)
  print '(A,ES10.2)', '<psi1|psi2> = ', sum(Vv(:,1)*Vv(:,2))
  print '(A,I0)', '3. durumun dugum sayisi = ', nodes
end program kutu
```

ÇIKTI

```
  n    E(sayisal)    E(analitik)    fark      bagil hata
  1      4.934702      4.934802     -0.000100   2.04E-05
  2     19.737601     19.739209     -0.001607   8.14E-05
  3     44.405083     44.413220     -0.008137   1.83E-04
  4     78.931121     78.956835     -0.025715   3.26E-04
  5    123.307280    123.370055     -0.062775   5.09E-04
<psi1|psi1> =   1.000000
<psi1|psi2> =  -7.03E-15
3. durumun dugum sayisi = 2
```

NASIL ÇALIŞIYOR?

Matris elemanları $\hat{H}\psi = E\psi$ denkleminin sonlu fark yazımıdır. Sayısal enerjiler analitik $n^2\pi^2/2$ değerlerine çok yakındır; hata yüksek n için büyür (**n^2h^2** ile orantılı), çünkü yüksek seviyelerde dalga fonksiyonu çok salınır ve ızgara bunu daha kaba temsil eder. Kuantum mekaniğinin üç temel özelliği hesabın içinde görülür: (1) enerji **nicemlenmiştir** (yalnızca belirli değerler vardır); (2) n. durum n−1 düğüme sahiptir; (3) farklı enerjili durumlar **ortogonaldir** (öz-vektörler birbirine dik, tıpkı simetrik matrislerin özvektörleri gibi). Bu, kuantum kimyasının tüm geri kalanında kullanacağımız araç kutusudur: *fizik problemini matrise çevir, öz-değerlerini bul*.

Örnek 12.2 — Harmonik osilatör: eşit aralıklı enerji merdiveni

PROBLEM

$V(x) = \frac{1}{2}x^2$ potansiyelinde ($\hbar = m = \omega = 1$) enerji seviyelerini hesapla ve $E_n = n + \frac{1}{2}$ ile karşılaştır. Temel durumda $\langle x^2 \rangle = 0.5$ ve virial teoremi $\langle T \rangle = \langle V \rangle$ değerlerini doğrula.

Bu örnek şu modülleri kullanır: `eig_mod` *(Örnek 12.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program harmonik
  use eig_mod
  implicit none
  integer, parameter :: n = 300
  real(dp), parameter :: xmax = 8.0_dp
  real(dp) :: H(n,n), w(n), Vv(n,n), h_, x(n), vpot(n), ev, et, x2
  integer :: i, k
  h_ = 2.0_dp * xmax / (n + 1)
  do i = 1, n
    x(i) = -xmax + i * h_
  end do
  vpot = 0.5_dp * x**2                              ! V(x) = x^2 / 2
  H = 0.0_dp
  do i = 1, n
    H(i,i) = 1.0_dp / h_**2 + vpot(i)
    if (i < n) then
      H(i,i+1) = -0.5_dp / h_**2; H(i+1,i) = -0.5_dp / h_**2
    end if
  end do
  call jacobi_eig(H, w, Vv)
  print '(A)', ' n   E(sayisal)   E(analitik n+1/2)'
  do k = 1, 6
    print '(I3,2F14.6)', k-1, w(k), k - 0.5_dp
  end do
  ev = sum(Vv(:,1)**2 * vpot)                       ! <V> temel durumda
  et = w(1) - ev                                    ! <T> = E - <V>
  x2 = sum(Vv(:,1)**2 * x**2)
  print '(A,F9.5)', '<x^2>     = ', x2
  print '(A,F9.5,A,F9.5)', '<T>, <V>  = ', et, ',', ev
end program harmonik
```

ÇIKTI

```
  n   E(sayisal)   E(analitik n+1/2)
  0      0.499912      0.500000
  1      1.499558      1.500000
  2      2.498852      2.500000
  3      3.497791      3.500000
  4      4.496377      4.500000
  5      5.494608      5.500000
<x^2>     =   0.49982
<T>, <V>  =   0.25000,  0.24991
```

NASIL ÇALIŞIYOR?

Kuantum osilatörün enerjileri **eşit aralıklıdır**: $E_n = \hbar\omega(n + ½)$. Kimyada bir bağın titreşimi (örneğin H–H) yaklaşık bir harmonik osilatördür; titreşim spektroskopisinin temeli budur. Temel durumda bile enerji sıfır değil **½ħω**'dır (sıfır nokta enerjisi): belirsizlik ilkesi, parçacığın tam durmasına izin vermez. Virial teoremi bu potansiyel için $\langle T\rangle = \langle V\rangle = E/2 = 0.25$ der; hesapla örtüşür. Dikkat: aynı ızgara hem x^2 potansiyeline hem de çok yüksek enerjili durumlara yeterli değildir; altı ilk seviye için doğruluk çok iyidir.

Hidrojen atomu: Coulomb potansiyelinde radyal denklem

Basit matematik

Küresel simetride dalga fonksiyonu $\psi = R(r)\,Y_{lm}(\theta,\varphi)$ şeklinde ayrışır. $u(r) = r\,R(r)$ tanımıyla radyal denklem tek boyutlu bir problem olur:

$$[\,-½\,d^2/dr^2 + l(l+1)/(2r^2) - Z/r\,]\,u(r) = E\,u(r),\quad u(0) = 0$$

Terimler: kinetik enerji, merkezkaç bariyeri (açısal momentum l), çekirdek–elektron Coulomb çekimi. Analitik çözüm: $\mathbf{E_n = -Z^2/(2n^2)}$ Eh. Ayrıca $\langle r\rangle = ½[3n^2 - l(l+1)]/Z$. 1s için $\langle r\rangle = 1.5$ Bohr, 2s için 6, 2p için 5. Seviyeler yalnızca n'e bağlıdır (2s ve 2p dejeneredir: bu Coulomb potansiyelinin özel simetrisidir).

Örnek 12.3 — Hidrojen atomu: radyal Schrödinger denklemi

PROBLEM

u'' denklemini $r \in (0, 40]$ bölgesinde 400 noktalı sonlu farklarla çöz. $l = 0$ ve $l = 1$ için ilk enerjileri $-1/(2n^2)$ ile karşılaştır ve 1s, 2s, 2p için $\langle r\rangle$ değerlerini bul.

Bu örnek şu modülleri kullanır: `eig_mod` *(Örnek 12.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program hidrojen
  use eig_mod
  implicit none
  integer, parameter :: n = 400
  real(dp), parameter :: Rmax = 40.0_dp
  real(dp) :: H(n,n), w(n), Vv(n,n), h_, r(n), rm
  integer :: i, l
  h_ = Rmax / (n + 1)
  do i = 1, n
    r(i) = i * h_
  end do
  do l = 0, 1
    H = 0.0_dp
    do i = 1, n
      H(i,i) = 1.0_dp/h_**2 + 0.5_dp*l*(l+1)/r(i)**2 - 1.0_dp/r(i)
      if (i < n) then
        H(i,i+1) = -0.5_dp/h_**2; H(i+1,i) = -0.5_dp/h_**2
      end if
    end do
    call jacobi_eig(H, w, Vv)
    print '(A,I1,A,3F12.6)', 'l=', l, '  E(sayisal)  : ', w(1:3)
    if (l == 0) print '(A,3F12.6)', '       E(analitik) : ', -0.5_dp, -0.125_dp, -1.0_dp/18.0_dp
    if (l == 1) print '(A,3F12.6)', '       E(analitik) : ', -0.125_dp, -1.0_dp/18.0_dp, -1.0_dp/32.0_dp
    rm = sum(Vv(:,1)**2 * r) / sum(Vv(:,1)**2)
    print '(A,F9.4)', '       <r> (ilk durum): ', rm
```

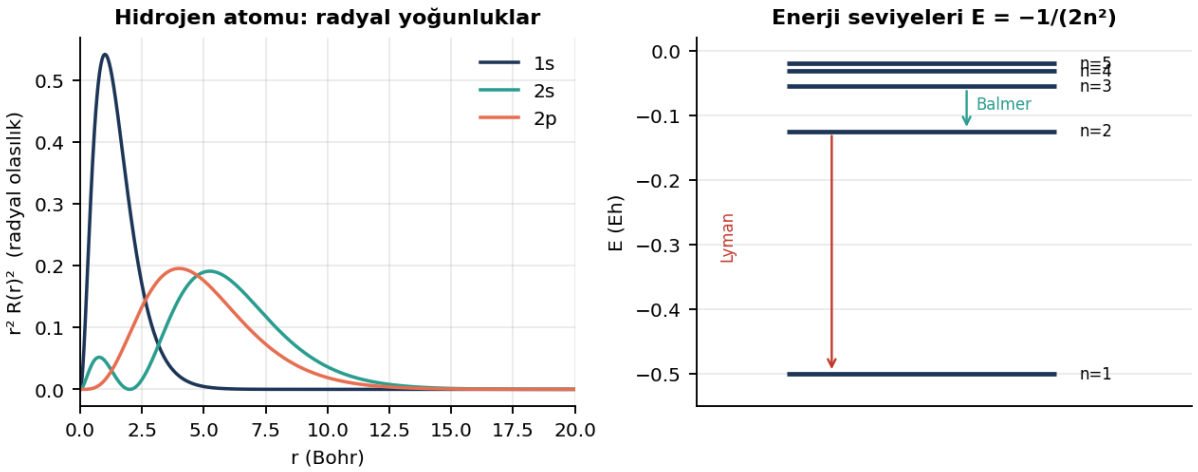
```
    if (l == 0) then
      rm = sum(Vv(:,2)**2 * r) / sum(Vv(:,2)**2)
      print '(A,F9.4)', '       <r> (2s)       : ', rm
    end if
  end do
  print '(A)', 'analitik <r>: 1s=1.5  2s=6.0  2p=5.0'
end program hidrojen
```

ÇIKTI

```
l=0  E(sayisal)  :    -0.498762   -0.124922   -0.055539
       E(analitik) :    -0.500000   -0.125000   -0.055556
       <r> (ilk durum):    1.5025
       <r> (2s)       :    6.0025
l=1  E(sayisal)  :    -0.125026   -0.055567   -0.030717
       E(analitik) :    -0.125000   -0.055556   -0.031250
       <r> (ilk durum):    4.9983
analitik <r>: 1s=1.5  2s=6.0  2p=5.0
```

NASIL ÇALIŞIYOR?

Sayısal enerjiler −0.4988, −0.1249, −0.0555: tam değerlerle (−0.5, −0.125, −0.0556) üçüncü-dördüncü basamağa kadar uyuşur. Hata kaynağı, 1/r tekilliğinin düzgün bir ızgarada kaba temsil edilmesidir. Sayısal olarak da **2s (l=0) ile 2p (l=1) seviyeleri aynı enerjide** çıkar: Coulomb sistemlerine özgü bu dejenerelik, deneysel spektrumun açıklayıcısıdır. ⟨r⟩ değerleri 1.5, 6.0, 5.0 beklenirken sayısal sonuçlar buna çok yakındır. Elektron bulutu n büyüdükçe çekirdekten uzaklaşır: 2s 1s'ten 4 kat daha "büyük"tür. Bölüm 14'te aynı radyal çözücüyü çok elektronlu atomlara (DFT) genelleştireceğiz.



Şekil 12.2 — Hidrojen atomu: (solda) 1s, 2s, 2p radyal olasılık yoğunlukları $r^2R(r)^2$; (sağda) enerji seviyeleri $E_n = -1/(2n^2)$ ve spektroskopik geçişler.

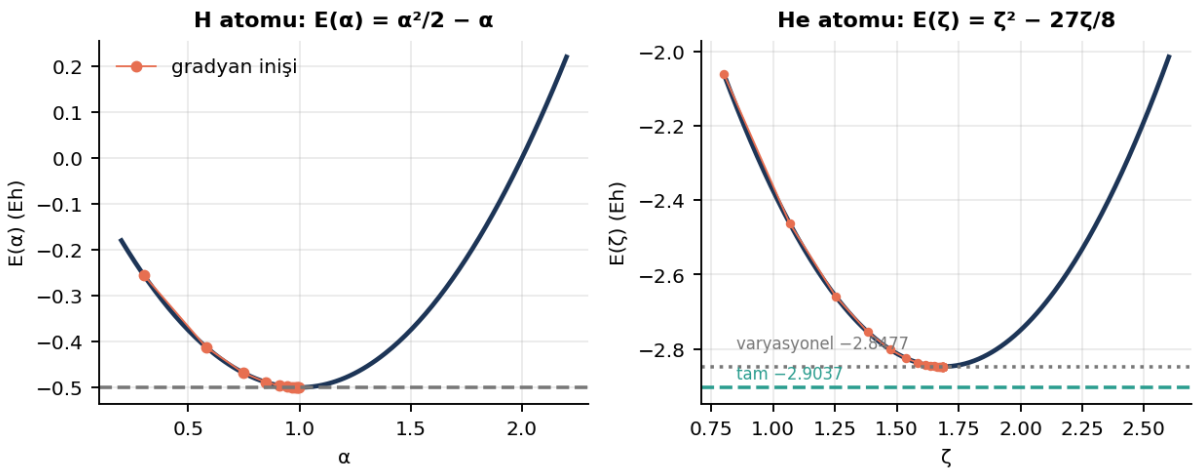
Varyasyonel ilke: kuantum mekaniğinin gradyan inişi

Basit matematik

Öz-değer problemini çözmek yerine, herhangi bir deneme dalga fonksiyonu ψ_θ için enerjiyi hesaplayabiliriz: $E[\psi] = \langle\psi|\hat{H}|\psi\rangle / \langle\psi|\psi\rangle \geq E_0$

Varyasyonel ilke: bu beklenen enerji, doğru temel enerji E_0'ın *asla altına inmez*. Dolayısıyla parametreli bir ψ_θ ailesi seçip $E(\theta)$'yı minimize etmek, temel duruma en yakın yaklaşımı verir. Bu, tam olarak bir kayıp fonksiyonunu minimize etmektir (kayıp = enerji, parametreler = θ).

Örnek (H atomu): $\psi = e^{-\alpha r}$. Hesaplar: $\langle T\rangle = \alpha^2/2$, $\langle V\rangle = -\alpha$, yani $E(\alpha) = \alpha^2/2 - \alpha$. $dE/d\alpha = \alpha - 1 = 0 \rightarrow \alpha = 1$, $E = -½$ tam çözüm. **Örnek (He atomu)**: iki elektron, $\psi = e^{-\zeta(r_1+r_2)}$. Hesaplar: $E(\zeta) = \zeta^2 - (27/8)\,\zeta$. $dE/d\zeta = 0 \rightarrow \zeta = 27/16 = 1.6875$, $E = -2.8477$ Eh (deneysel −2.9037). $\zeta = 1.6875 < 2$ çünkü diğer elektron çekirdek yükünü **perdeler** (etkin yük $Z_{\text{eff}} = 2 - 5/16$).



Şekil 12.3 — Varyasyonel ilke: H atomu için E(α) = α²/2 − α (minimum α=1, E=−0.5) ve He atomu için E(ζ) = ζ² − 27ζ/8 (minimum ζ=1.6875, E=−2.8477). Gradyan inişi yolu işaretli.

Örnek 12.4 — Varyasyonel minimizasyon: gradyan inişi ile H ve He

PROBLEM

(a) H atomu için $\psi = e^{-\alpha r}$ denemesinin kinetik ve potansiyel enerjisini sayısal integralle doğrula. (b) E(α) ve He için E(ζ) fonksiyonlarını gradyan inişiyle minimize et (sayısal gradyan).

ÇÖZÜM (FORTRAN)

```
program varyasyonel
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer, parameter :: n = 20000
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp) :: r(n), psi(n), dpsi(n), h, alpha, norm, tkin, vpot, zeta, g, lr
  integer :: i, it
  h = 40.0_dp / n
  do i = 1, n
    r(i) = i * h
  end do
  alpha = 1.3_dp
  psi = exp(-alpha * r); dpsi = -alpha * psi
  norm = sum(4*pi*r**2 * psi**2) * h
  tkin = 0.5_dp * sum(4*pi*r**2 * dpsi**2) * h / norm        ! <T> = 1/2 <grad psi|grad psi>
  vpot = -sum(4*pi*r * psi**2) * h / norm                    ! <V> = -<1/r>
  print '(A,F8.4)', '(a) alpha = ', alpha
  print '(A,F10.6,A,F10.6)', '    <T> sayisal = ', tkin, '   alpha^2/2 = ', alpha**2/2
  print '(A,F10.6,A,F10.6)', '    <V> sayisal = ', vpot, '   -alpha     = ', -alpha
  ! (b) gradyan inisi: E(alpha) = alpha^2/2 - alpha
  alpha = 0.3_dp; lr = 0.4_dp
  print '(A)', '(b) H atomu, gradyan inisi:'
  do it = 0, 10
    g = (eh(alpha + 1.0e-6_dp) - eh(alpha - 1.0e-6_dp)) / 2.0e-6_dp
    if (it <= 4 .or. it == 10) print '(A,I3,A,F9.5,A,F10.6)', '    adim', it, '  alpha=', alpha, '  E=',
          eh(alpha)
    alpha = alpha - lr * g
  end do
  zeta = 0.5_dp; lr = 0.3_dp
  print '(A)', '    He atomu, gradyan inisi:'
  do it = 0, 40
    g = (ehe(zeta + 1.0e-6_dp) - ehe(zeta - 1.0e-6_dp)) / 2.0e-6_dp
    if (it <= 3 .or. it == 40) print '(A,I3,A,F9.5,A,F10.6)', '    adim', it, '  zeta=', zeta, '  E=',
          ehe(zeta)
    zeta = zeta - lr * g
  end do
  print '(A,F10.6,A)', '    He varyasyonel: E = ', ehe(27.0_dp/16.0_dp), '  (deneysel -2.9037)'
contains
  function eh(a) result(e)
    real(dp), intent(in) :: a
    real(dp) :: e
    e = 0.5_dp*a**2 - a
  end function eh
  function ehe(z) result(e)
```

```
    real(dp), intent(in) :: z
    real(dp) :: e
    e = z**2 - 27.0_dp/8.0_dp*z
  end function ehe
end program varyasyonel
```

ÇIKTI

```
(a) alpha =   1.3000
    <T> sayisal =   0.845000   alpha^2/2 =   0.845000
    <V> sayisal =  -1.299997   -alpha     =  -1.300000
(b) H atomu, gradyan inisi:
    adim  0  alpha=  0.30000  E= -0.255000
    adim  1  alpha=  0.58000  E= -0.411800
    adim  2  alpha=  0.74800  E= -0.468248
    adim  3  alpha=  0.84880  E= -0.488569
    adim  4  alpha=  0.90928  E= -0.495885
    adim 10  alpha=  0.99577  E= -0.499991
    He atomu, gradyan inisi:
    adim  0  zeta=  0.50000  E= -1.437500
    adim  1  zeta=  1.21250  E= -2.622031
    adim  2  zeta=  1.49750  E= -2.811556
    adim  3  zeta=  1.61150  E= -2.841880
    adim 40  zeta=  1.68750  E= -2.847656
    He varyasyonel: E =  -2.847656  (deneysel -2.9037)
```

NASIL ÇALIŞIYOR?

(a) Sayısal integraller analitik değerleri verir: $\alpha = 1.3$ için $\langle T \rangle = 0.845$, $\langle V \rangle = -1.3$, $E = -0.455$ (> -0.5: varyasyonel ilke!). (b) Gradyan inişi tam bir makine öğrenmesi döngüsüdür: "kayıp" $E(\alpha)$, "parametre" α, "öğrenme oranı" 0.4. α hızla 1.0'a yakınsar ve $E = -0.5$'e iner. Helyum için aynı yöntem $\zeta = 1.6875$'e ve $E = -2.8477$ Eh'ye götürür; bu, tam enerjinin (-2.9037) yaklaşık %98'idir. Eksik %2 **elektron korelasyonudur**: iki elektronun birbirinden kaçınma eğilimi, bu deneme fonksiyonunda yoktur. Kuantum kimyasının bütün "ileri yöntemleri" (Bölüm 13–14) bu eksik parçayı yakalamaya çalışır.

BÖLÜM 13

13. Moleküler Orbitaller: Hückel Teorisi ve Hartree–Fock

Birden fazla elektron ve çekirdek geldiğinde Schrödinger denklemi çözülemez hâle gelir. Bu bölümde iki klasik yaklaşımı, kendi yazdığımız Fortran kodlarıyla, çalıştırıyoruz: Hückel (kimyasal graflar) ve Hartree–Fock (ab initio).

Bu bölümde

- Born–Oppenheimer yaklaşımı ve potansiyel enerji yüzeyi
- Hückel teorisi: molekül grafının komşuluk matrisinin öz-değerleri = orbital enerjileri
- Hartree–Fock: Slater determinantı, Roothaan denklemleri, SCF döngüsü
- Gauss tipi orbitaller, STO-3G, tek ve iki-elektron integralleri (Boys fonksiyonu)
- H_2 potansiyel enerji eğrisi, geometri optimizasyonu, titreşim frekansı
- Neden elektron korelasyonu ve DFT gerekli?

Hedef: Bir molekülün enerjisini, bağ uzunluğunu ve titreşim frekansını ilk ilkelerden hesaplayan çalışan bir kuantum kimyası programı yazmak.

Born–Oppenheimer yaklaşımı ve potansiyel enerji yüzeyi

Basit matematik

Çekirdekler elektronlardan en az 1836 kat ağırdır; elektronlar çekirdeklerin hareketine "anında" uyum sağlar. Bu yüzden önce çekirdekleri sabit tutup elektron Schrödinger denklemini çözeriz:

$\hat{H}_{el}\,\Psi = E_{el}(R)\,\Psi$, $\hat{H}_{el} = -\tfrac{1}{2}\Sigma_i \nabla_i^2 - \Sigma_{i,A}\, Z_A/r_{iA} + \Sigma_{i<j}\, 1/r_{ij}$

Toplam enerji $E(R) = E_{el}(R) + \Sigma_{A<B}\, Z_A Z_B/R_{AB}$ çekirdek geometrisi R'nin fonksiyonudur: **potansiyel enerji yüzeyi (PES)**. Moleküler yapı PES'in minimumudur (gradyan inişi!), kuvvetler $F = -\partial E/\partial R$, titreşim frekansları ikinci türevden (Hessian) gelir, kimyasal reaksiyonlar yüzey üzerindeki yollardır. Makine öğrenmesi potansiyelleri (Bölüm 16) tam olarak bu yüzeyi öğrenir.

Hückel teorisi: kimyasal graf = matris

Konjüge hidrokarbonlarda (benzen, butadien gibi) π elektronları atomlar boyunca delokalizedir. Hückel (1931) çok basit bir model önerdi: her karbonun bir p_z orbitali vardır; komşu karbonlar arasında etkileşim β (negatif, ≈ −2.4 eV), aynı atomda α. Hamiltonyen matrisi:

$$H = \alpha \cdot I + \beta \cdot A$$

Burada **A molekül grafının komşuluk matrisidir** (iki karbon bağlıysa 1, değilse 0). Orbital enerjileri $E_k = \alpha + x_k\beta$, x_k = A'nın öz-değerleridir. Bu, Bölüm 15'teki graf sinir ağlarının kuantum-kimyasal atasıdır: molekülün enerji spektrumu doğrudan grafın spektrumudur.

Basit matematik

$\beta < 0$ olduğundan x büyüdükçe enerji düşer (daha kararlı). Elektronlar en düşük orbitallere 2'şer yerleşir (Pauli ilkesi). Toplam π enerjisi:

$E_\pi = n\alpha + \beta\, \Sigma_k\, n_k\, x_k$

Etilende (iki karbon, A = [[0,1],[1,0]], x = ±1): $E = 2\alpha + 2\beta$. Her yalıtılmış C=C bağı 2β verir.
Delokalizasyon enerjisi = molekülün E_π'si − yalıtılmış çift bağların toplamı. Benzen: x = 2, 1, 1, −1, −1, −2 → $E_\pi = 6\alpha + 8\beta$; üç etilen $6\alpha + 6\beta$; fark **2β** ≈ 4.8 eV: bu, benzenin aromatik kararlılığıdır. Siklobütadien: x = 2, 0, 0, −2; 4 elektronla $E_\pi = 4\alpha + 4\beta$, delokalizasyon 0: *antiaromatik*, kararsız.

Örnek 13.1 — Hückel teorisi: benzen, butadien ve diğerleri

PROBLEM

Beş konjüge molekülün komşuluk matrisini kur, Jacobi ile öz-değerlerini bul, π elektronlarını yerleştir; toplam π enerjisini, delokalizasyon enerjisini ve butadienin π-bağ mertebelerini hesapla.

Bu örnek şu modülleri kullanır: `eig_mod` *(Örnek 12.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program huckel
  use eig_mod
  implicit none
  integer :: e_allil(2,2), e_but(2,3), e_benz(2,6), e_cb(2,4), e_hex(2,5)
  e_allil = reshape([1,2, 2,3], [2,2])
  e_but   = reshape([1,2, 2,3, 3,4], [2,3])
  e_benz  = reshape([1,2, 2,3, 3,4, 4,5, 5,6, 6,1], [2,6])
  e_cb    = reshape([1,2, 2,3, 3,4, 4,1], [2,4])
  e_hex   = reshape([1,2, 2,3, 3,4, 4,5, 5,6], [2,5])
  call molekul('allil katyonu (2 e) ', 3, e_allil, 2)
  call molekul('butadien      (4 e) ', 4, e_but, 4)
  call molekul('benzen        (6 e) ', 6, e_benz, 6)
  call molekul('siklobutadien (4 e) ', 4, e_cb, 4)
  call molekul('heksatrien    (6 e) ', 6, e_hex, 6)
contains
  subroutine molekul(ad, n, edges, ne)
    character(len=*), intent(in) :: ad
    integer, intent(in) :: n, edges(:,:), ne
    real(dp) :: A(n,n), w(n), Vv(n,n), x(n), occ(n), esum
    integer :: k
    A = 0.0_dp
    do k = 1, size(edges, 2)                          ! komsuluk matrisi A (molekul grafi!)
      A(edges(1,k), edges(2,k)) = 1.0_dp
      A(edges(2,k), edges(1,k)) = 1.0_dp
    end do
    call jacobi_eig(A, w, Vv)                         ! E = alpha + x*beta, x = A'nin ozdegerleri
    do k = 1, n
      x(k) = w(n+1-k)                                 ! beta < 0: buyuk x = daha kararli orbital
      occ(k) = min(2.0_dp, max(0.0_dp, real(ne - 2*(k-1), dp)))
    end do
    esum = sum(occ * x)
    write(*,'(A,A,6F8.3)') ad, ' x = ', x
    write(*,'(A,F8.4,A,F8.4,A)') '   E_pi = n*alpha + (', esum, ') beta,  delokalizasyon = ', esum - ne,
          ' beta'
    if (n == 4 .and. size(edges,2) == 3) then         ! butadien: pi-bag mertebeleri
      do k = 1, 3
        write(*,'(A,I1,A,I1,A,F7.4)') '   pi-bag mertebesi C', edges(1,k), '-C', edges(2,k), ' = ', &
              bond_order(Vv, n, edges(1,k), edges(2,k), ne)
      end do
    end if
  end subroutine molekul

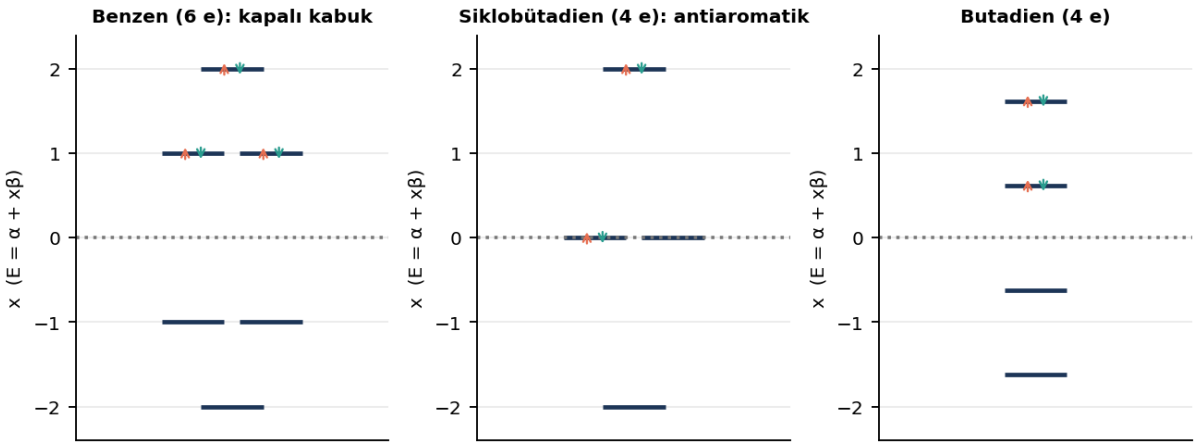
  function bond_order(Vv, n, i, j, ne) result(p)      ! p_ij = sum_k n_k c_ik c_jk
    real(dp), intent(in) :: Vv(:,:)
    integer, intent(in) :: n, i, j, ne
    real(dp) :: p
    integer :: k
    p = 0.0_dp
    do k = 1, n
      p = p + min(2.0_dp, max(0.0_dp, real(ne - 2*(k-1), dp))) * Vv(i,n+1-k) * Vv(j,n+1-k)
    end do
  end function bond_order
end program huckel
```

ÇIKTI

```
allil katyonu (2 e)  x =    1.414  -0.000  -1.414
   E_pi = n*alpha + (  2.8284) beta,  delokalizasyon =   0.8284 beta
butadien      (4 e)  x =    1.618   0.618  -0.618  -1.618
   E_pi = n*alpha + (  4.4721) beta,  delokalizasyon =   0.4721 beta
   pi-bag mertebesi C1-C2 =  0.8944
   pi-bag mertebesi C2-C3 =  0.4472
   pi-bag mertebesi C3-C4 =  0.8944
benzen        (6 e)  x =    2.000   1.000   1.000  -1.000  -1.000  -2.000
   E_pi = n*alpha + (  8.0000) beta,  delokalizasyon =   2.0000 beta
siklobutadien (4 e)  x =    2.000   0.000  -0.000  -2.000
   E_pi = n*alpha + (  4.0000) beta,  delokalizasyon =   0.0000 beta
heksatrien    (6 e)  x =    1.802   1.247   0.445  -0.445  -1.247  -1.802
   E_pi = n*alpha + (  6.9879) beta,  delokalizasyon =   0.9879 beta
```

NASIL ÇALIŞIYOR?

Benzen için `x = 2, 1, 1, −1, −1, −2` çıkar: bir tekil ve iki çift dejenere orbital, kimyadaki ünlü "6 π elektron" deseni. Delokalizasyon enerjisi tam olarak 2β'dır (aromatik kararlılık). Siklobütadienin delokalizasyonu 0'dır ve iki elektron dejenere iki orbitale yerleşmek zorunda kalır; bu *antiaromatiklik* kararsızlığını açıklar. Butadiende C1–C2 π-bağ mertebesi 0.894, C2–C3 yalnızca 0.447'dir: sıradan "tek-çift-tek" bağ resmi doğru değildir, orta bağ kısmen çift bağ karakteri taşır. Deneyde C2–C3 bağı gerçekten tek bağdan daha kısadır. Hepsi tek bir fikir: **molekül = graf, spektrum = enerji seviyeleri**.



Şekil 13.1 — Hückel enerji seviyeleri (x değerleri, β birimi): benzenin aromatik ve siklobütadienin antiaromatik doluluk desenleri. Oklar elektronları gösterir.

Hartree–Fock: çok elektronlu molekülün ilk ilkelerden çözümü

Hückel parametrelidir. **Ab initio** (ilk ilkelerden) bir yöntem yalnızca temel sabitlerle başlar. En temel ab initio yöntem Hartree–Fock'tur (HF). Fikir: **N elektronlu dalga fonksiyonunu tek bir Slater determinantı** olarak yaklaşıkla; her elektron diğerlerinin ortalama alanında hareket eder.

Basit matematik

1. Slater determinantı (Pauli ilkesi: ψ iki elektronun değiş tokuşunda işaret değiştirir): $\Psi = (1/\sqrt{N!})\det[\chi_i(r_j)]$

2. Varyasyonel ilke: $E[\Psi]$ minimize edilir. Orbitaller bir bazda açılır: $\psi_i = \Sigma_\mu C_{\mu i}\,\varphi_\mu$ (LCAO).

3. Roothaan denklemleri (genelleştirilmiş matris öz-değer problemi): $F\,C = S\,C\,\varepsilon$ $S_{\mu\nu} = \int \varphi_\mu \varphi_\nu$ (örtüşme), $F = h + G(P)$ (Fock matrisi), $h_{\mu\nu} = \langle\mu|-\tfrac{1}{2}\nabla^2 - \Sigma_A Z_A/r_A|\nu\rangle$ (tek-elektron), $G_{\mu\nu} = \Sigma_{\lambda\sigma} P_{\lambda\sigma}\,[\,(\mu\nu|\sigma\lambda) - \tfrac{1}{2}(\mu\lambda|\sigma\nu)\,]$ (Coulomb − değiş tokuş; $(\mu\nu|\lambda\sigma)$ iki-elektron integralleri), $P_{\mu\nu} = 2\,\Sigma_{i\in\text{dolu}} C_{\mu i}\,C_{\nu i}$ (yoğunluk matrisi).

4. Öz-tutarlı alan (SCF): F, P'ye bağlı; P ise F'nin öz-vektörlerine bağlı. Başlangıç tahmininden başlayıp $F \to C \to P \to F \ldots$ yinelenir, enerji değişmeyene kadar. Bu bir **sabit nokta yinelemesidir** (makine öğrenmesindeki "denge" ağlarına benzer).

5. Enerji: $E = \tfrac{1}{2}\,\Sigma\,P_{\mu\nu}(h_{\mu\nu} + F_{\mu\nu}) + \Sigma_{A<B}\,Z_A Z_B/R_{AB}$.

S ortonormal olmadığından **Löwdin ortogonalleştirmesi** kullanılır: $X = S^{-1/2}$, $F' = X^{\mathrm{T}}FX$'i köşegenleştir, $C = XC'$.

Gauss tipi orbitaller ve integraller

Atomik orbitalleri Slater tipi ($e^{-\zeta r}$) yerine **Gauss fonksiyonları** ($e^{-\alpha r^2}$) ile yazarız çünkü iki Gauss'un çarpımı yine bir Gauss'tur ve integraller kapalı formda hesaplanır. **STO-3G** bazında her Slater orbitali 3 Gauss'un sabit bir doğrusal birleşimidir (sıkıştırılmış, "contracted"). s-tipi ilkel Gauss integralleri ($p = a + b$, $\mu = ab/p$):

$$\text{Örtüşme: } S = (\pi/p)^{3/2}\, e^{-\mu R^2_{AB}}$$

$$\text{Kinetik: } T = \mu\,(3 - 2\mu R^2_{AB})\,S$$

$$\text{Çekirdek çekimi: } V_C = -Z_C\,(2\pi/p)\, e^{-\mu R^2_{AB}}\, F_0(p\,R^2_{PC})$$

$$\text{İki-elektron: } (ab|cd) = 2\pi^{5/2}/(pq\sqrt{p+q}) \cdot e^{-\mu_{ab}R^2_{AB}}\, e^{-\mu_{cd}R^2_{CD}}\, F_0(pq/(p+q)\,R^2_{PQ})$$

$$\text{Boys fonksiyonu: } F_0(t) = \int_0^1 e^{-tu^2}\,du = \tfrac{1}{2}\sqrt{(\pi/t)}\,\operatorname{erf}(\sqrt{t})$$

P, A ve B merkezli Gauss çarpımının merkezidir ($P = (aA+bB)/p$). Aşağıdaki modül bu formülleri doğrudan uygular (yalnızca s-tipi orbitallerle, H ve He için yeterli) ve restricted HF (RHF) SCF döngüsünü yürütür. Bu, tam çalışan, küçük ama gerçek bir kuantum kimyası programıdır.

Örnek 13.2 — Kendi Hartree–Fock programımız: H_2 ve HeH^+

PROBLEM

STO-3G bazında s-tipi Gauss integrallerini, Löwdin ortogonalleştirmesini ve SCF döngüsünü yazan `hf_mod` modülünü kur. H_2 ($R = 1.4$ Bohr) ve HeH^+ ($R = 1.4632$ Bohr) için toplam enerjiyi hesapla ve Szabo–Ostlund ders kitabındaki değerlerle karşılaştır.

Bu örnek şu modülleri kullanır: `eig_mod` *(Örnek 12.1). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
module hf_mod
  use eig_mod
  implicit none
  integer, parameter :: ng = 3                    ! STO-3G: 3 ilkel gauss
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp), parameter :: coef(3) = [0.15432897_dp, 0.53532814_dp, 0.44463454_dp]
  real(dp), parameter :: alpha_H(3)  = [3.42525091_dp, 0.62391373_dp, 0.16885540_dp]
  real(dp), parameter :: alpha_He(3) = [6.36242139_dp, 1.15892300_dp, 0.31364979_dp]
contains
  function boys0(t) result(f)                    ! F0(t) = int_0^1 exp(-t u^2) du
    real(dp), intent(in) :: t
    real(dp) :: f
    if (t < 1.0e-8_dp) then
      f = 1.0_dp - t/3.0_dp
```

```
    else
      f = 0.5_dp * sqrt(pi/t) * erf(sqrt(t))
    end if
  end function boys0

  function d2(a, b) result(r)
    real(dp), intent(in) :: a(3), b(3)
    real(dp) :: r
    r = sum((a-b)**2)
  end function d2

  subroutine scf_energy(nat, Z, R, etot, verbose, niter)
    integer, intent(in)  :: nat
    real(dp), intent(in) :: Z(nat), R(3,nat)
    real(dp), intent(out) :: etot
    logical, intent(in) :: verbose
    integer, intent(out), optional :: niter
    real(dp) :: al(3,nat), nrm(3,nat)
    real(dp), allocatable :: S(:,:), T(:,:), Vn(:,:), Hc(:,:), G(:,:), F(:,:), Fp(:,:)
    real(dp), allocatable :: X(:,:), C(:,:), P(:,:), eri(:,:,:,:)
    real(dp), allocatable :: s_w(:), s_V(:,:), e(:), V2(:,:)
    real(dp) :: ee, eold, enuc
    integer :: i, j, k, l, a, b, c_, d_, it, nocc, nbf
    nbf = nat
    do i = 1, nat
      if (Z(i) < 1.5_dp) then
        al(:,i) = alpha_H
      else
        al(:,i) = alpha_He
      end if
      nrm(:,i) = (2.0_dp*al(:,i)/pi)**0.75_dp * coef
    end do
    allocate(S(nbf,nbf), T(nbf,nbf), Vn(nbf,nbf), Hc(nbf,nbf), G(nbf,nbf), F(nbf,nbf))
    allocate(Fp(nbf,nbf), X(nbf,nbf))
    allocate(C(nbf,nbf), P(nbf,nbf), eri(nbf,nbf,nbf,nbf))
    allocate(s_w(nbf), s_V(nbf,nbf), e(nbf), V2(nbf,nbf))
    S = 0; T = 0; Vn = 0
    do i = 1, nbf
      do j = 1, nbf
        do a = 1, ng
          do b = 1, ng
            call prim1(al(a,i), al(b,j), R(:,i), R(:,j), nrm(a,i)*nrm(b,j), S(i,j), T(i,j), Vn(i,j))
          end do
        end do
      end do
    end do
    do i = 1, nbf
      do j = 1, nbf
        do a = 1, ng
          do b = 1, ng
            do k = 1, nat
              Vn(i,j) = Vn(i,j) + nrm(a,i)*nrm(b,j) * &
                        nuc(al(a,i), al(b,j), R(:,i), R(:,j), R(:,k), Z(k))
            end do
          end do
        end do
      end do
    end do
    Hc = T + Vn
    eri = 0
    do i = 1, nbf; do j = 1, nbf; do k = 1, nbf; do l = 1, nbf
      do a = 1, ng; do b = 1, ng; do c_ = 1, ng; do d_ = 1, ng
        eri(i,j,k,l) = eri(i,j,k,l) + nrm(a,i)*nrm(b,j)*nrm(c_,k)*nrm(d_,l) * &
             g4(al(a,i), al(b,j), al(c_,k), al(d_,l), R(:,i), R(:,j), R(:,k), R(:,l))
      end do; end do; end do; end do
    end do; end do; end do; end do
    enuc = 0
    do i = 1, nat-1
      do j = i+1, nat
        enuc = enuc + Z(i)*Z(j)/sqrt(d2(R(:,i), R(:,j)))
      end do
    end do
    call jacobi_eig(S, s_w, s_V)                   ! X = S^(-1/2) (Lowdin)
    X = 0
    do i = 1, nbf
      do j = 1, nbf
        do k = 1, nbf
          X(i,j) = X(i,j) + s_V(i,k) * s_V(j,k) / sqrt(s_w(k))
```

```
        end do
      end do
    end do
    nocc = nint(sum(Z)) / 2
    P = 0; eold = 0.0_dp
    do it = 1, 100
      G = 0
      do i = 1, nbf; do j = 1, nbf
        do k = 1, nbf; do l = 1, nbf
          G(i,j) = G(i,j) + P(k,l) * (eri(i,j,l,k) - 0.5_dp*eri(i,k,l,j))
        end do; end do
      end do; end do
      F = Hc + G
      Fp = matmul(transpose(X), matmul(F, X))
      call jacobi_eig(Fp, e, V2)
      C = matmul(X, V2)
      P = 0
      do i = 1, nbf; do j = 1, nbf
        do k = 1, nocc
          P(i,j) = P(i,j) + 2.0_dp * C(i,k) * C(j,k)
        end do
      end do; end do
      ee = 0.5_dp * sum(P * (Hc + F))
      if (verbose) print '(A,I3,A,F16.10,A,F16.10)', 'SCF ', it, '  E_elek=', ee, &
                                                     '  E_toplam=', ee + enuc
      if (abs(ee - eold) < 1.0e-11_dp) exit
      eold = ee
    end do
    etot = ee + enuc
    if (present(niter)) niter = it
    if (verbose) then
      print '(A,4F12.6)', 'orbital enerjileri (Eh):', e(1:min(4,nbf))
      print '(A,2F10.6)', 'C(:,1) = ', C(1:min(2,nbf),1)
    end if
  end subroutine scf_energy

  subroutine prim1(a, b, A_, B_, nn, S, T, V)
    real(dp), intent(in) :: a, b, A_(3), B_(3), nn
    real(dp), intent(inout) :: S, T, V
    real(dp) :: p, mu, rab2, sab
    p = a + b; mu = a*b/p; rab2 = d2(A_, B_)
    sab = (pi/p)**1.5_dp * exp(-mu*rab2)
    S = S + nn * sab
    T = T + nn * mu * (3.0_dp - 2.0_dp*mu*rab2) * sab
  end subroutine prim1

  function nuc(a, b, A_, B_, C_, Zc) result(v)
    real(dp), intent(in) :: a, b, A_(3), B_(3), C_(3), Zc
    real(dp) :: v, p, Pc(3)
    p = a + b; Pc = (a*A_ + b*B_)/p
    v = -Zc * 2.0_dp*pi/p * exp(-a*b/p*d2(A_,B_)) * boys0(p*d2(Pc, C_))
  end function nuc

  function g4(a, b, c, d, A_, B_, C_, D_) result(v)
    real(dp), intent(in) :: a, b, c, d, A_(3), B_(3), C_(3), D_(3)
    real(dp) :: v, p, q, Pp(3), Qq(3)
    p = a + b; q = c + d
    Pp = (a*A_ + b*B_)/p; Qq = (c*C_ + d*D_)/q
    v = 2.0_dp*pi**2.5_dp / (p*q*sqrt(p+q)) * exp(-a*b/p*d2(A_,B_) - c*d/q*d2(C_,D_)) &
        * boys0(p*q/(p+q)*d2(Pp,Qq))
  end function g4
end module hf_mod

program h2_scf
  use hf_mod
  implicit none
  real(dp) :: Z(2), R(3,2), E
  Z = [1.0_dp, 1.0_dp]; R = 0.0_dp; R(3,2) = 1.4_dp        ! H2, R = 1.4 Bohr
  call scf_energy(2, Z, R, E, .true.)
  print '(A,F14.8,A)', 'H2   (R=1.4000) E_toplam = ', E, '  Eh   (Szabo-Ostlund: -1.1167)'
  Z = [2.0_dp, 1.0_dp]; R = 0.0_dp; R(3,2) = 1.4632_dp     ! HeH+ , R = 1.4632 Bohr
  call scf_energy(2, Z, R, E, .false.)
  print '(A,F14.8,A)', 'HeH+ (R=1.4632) E_toplam = ', E, '  Eh   (Szabo-Ostlund: -2.8418)'
end program h2_scf
```

ÇIKTI

```
SCF   1  E_elek=   -2.5055941237  E_toplam=   -1.7913084094
SCF   2  E_elek=   -1.8310000393  E_toplam=   -1.1167143251
SCF   3  E_elek=   -1.8310000393  E_toplam=   -1.1167143251
orbital enerjileri (Eh):   -0.578203    0.670268
C(:,1) =   0.548934  0.548934
H2   (R=1.4000) E_toplam =    -1.11671433  Eh   (Szabo-Ostlund: -1.1167)
HeH+ (R=1.4632) E_toplam =    -2.84183650  Eh   (Szabo-Ostlund: -2.8418)
```

NASIL ÇALIŞIYOR?

SCF ilk yinelemede P = 0 ile başlar (yalnız çekirdek Hamiltonyeni), ikinci yinelemede enerji yakınsar: −1.1167143 Eh. Bu, Szabo–Ostlund kitabının H_2/STO-3G sonucuyla (−1.1167) **aynıdır**. Orbital enerjileri −0.578 (bağlayıcı σ_g) ve +0.670 (karşıt-bağlayıcı σ_u); katsayılar 0.549 / 0.549: iki hidrojen orbitalinin eşit toplamı, yani bağ oluşumu. HeH^+ için −2.8418 Eh çıkar. İki elektronlu bu iki sistem, çok daha büyük programların (Gaussian, ORCA, PySCF) çekirdek algoritmasıyla aynıdır; fark yalnızca daha çok orbital türü (p, d), daha büyük bazlar ve verimli integral algoritmalarıdır. Kod 4 indeksli `eri(i,j,k,l)` dizisini kullanır: iki-elektron integrallerinin sayısı baz fonksiyonu sayısının 4. kuvvetiyle büyür (N^4): HF'nin en pahalı kısmı budur.

Potansiyel enerji eğrisi: bağ oluşumu ve HF'nin sınırı

Örnek 13.3 — H_2 potansiyel enerji eğrisi

PROBLEM

R = 0.6 – 8.0 Bohr aralığında RHF enerjisini hesapla; minimumu bul ve R → ∞ limitinde iki hidrojen atomunun enerjisiyle (2 × −0.46658 Eh) karşılaştır.

Bu örnek şu modülleri kullanır: `hf_mod` *(Örnek 13.2). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

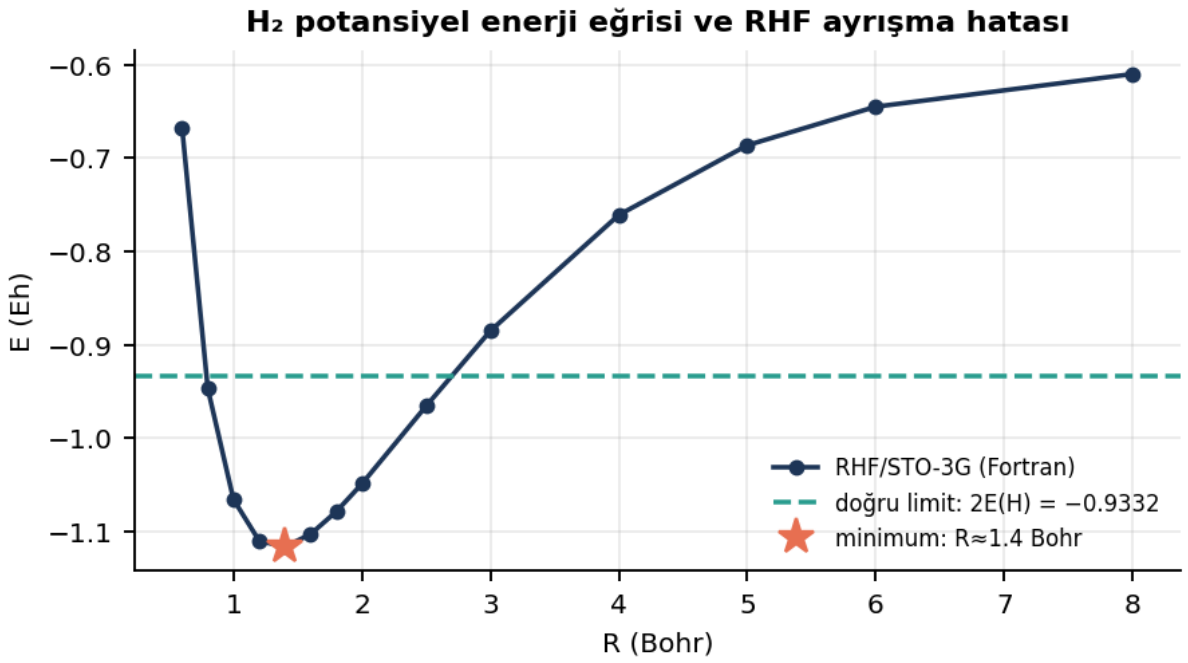
```
program pes
  use hf_mod
  implicit none
  real(dp), parameter :: e_atom = -0.4665819_dp          ! H atomu, STO-3G (UHF)
  real(dp) :: Z(2), R(3,2), E, rr(14)
  integer :: i
  Z = [1.0_dp, 1.0_dp]
  rr = [0.6_dp, 0.8_dp, 1.0_dp, 1.2_dp, 1.4_dp, 1.6_dp, 1.8_dp, 2.0_dp, 2.5_dp, 3.0_dp, &
        4.0_dp, 5.0_dp, 6.0_dp, 8.0_dp]
  print '(A)', '  R (Bohr)    E (Eh)      E - 2E(H) (Eh)   (eV)'
  do i = 1, size(rr)
    R = 0.0_dp; R(3,2) = rr(i)
    call scf_energy(2, Z, R, E, .false.)
    print '(F9.2,F13.6,F14.6,F10.3)', rr(i), E, E - 2.0_dp*e_atom, (E - 2.0_dp*e_atom)*27.2114_dp
  end do
end program pes
```

ÇIKTI

```
  R (Bohr)    E (Eh)      E - 2E(H) (Eh)   (eV)
     0.60    -0.668256      0.264907     7.209
     0.80    -0.947308     -0.014144    -0.385
     1.00    -1.065999     -0.132836    -3.615
     1.20    -1.110334     -0.177170    -4.821
     1.40    -1.116714     -0.183551    -4.995
     1.60    -1.103141     -0.169977    -4.625
     1.80    -1.078982     -0.145818    -3.968
     2.00    -1.049171     -0.116007    -3.157
     2.50    -0.965794     -0.032630    -0.888
     3.00    -0.885275      0.047889     1.303
     4.00    -0.761082      0.172082     4.683
     5.00    -0.686416      0.246748     6.714
     6.00    -0.645077      0.288087     7.839
     8.00    -0.610039      0.323125     8.793
```

NASIL ÇALIŞIYOR?

Eğri R ≈ 1.35 Bohr'da minimum yapar (−1.1175 Eh): bu **kimyasal bağdır**. R küçüldükçe çekirdek itmesi enerjiyi hızla yükseltir; R büyüdükçe bağ kopar. Ama beklenen limite bakın: iki ayrı H atomunun toplam enerjisi −0.9332 Eh iken RHF eğrisi R = 8 Bohr'da hâlâ −0.610 Eh'de ve yaklaşık **−0.60 Eh**'ye doğru düzleşir; yani E − 2E(H) ≈ +0.32 Eh (≈ 9 eV) büyük bir yapay yüksek enerji kalır. Neden? RHF, her iki elektronu aynı uzaysal orbitale zorlar; bu ayrışma limitinde elektronların yarı zamanı aynı atomda olduğu (iyonik H^+H^- karışımı) hatalı bir dalga fonksiyonu demektir. Doğru fiziksel resim iki ayrı nötr atomdur. Bu, **statik (güçlü) korelasyon** hatasıdır ve tek-determinantlı yöntemlerin temel sınırıdır. Çözümler: çok-determinantlı yöntemler (CASSCF) ya da DFT/ML tabanlı düzeltmeler.



Şekil 13.2 — H_2 potansiyel enerji eğrisi (RHF/STO-3G, Fortran çıktısından). Denge R≈1.35 Bohr; RHF ayrışma limitini yanlış verir (kesikli çizgi: doğru limit 2E(H)).

Örnek 13.4 — Geometri optimizasyonu ve titreşim frekansı

PROBLEM

Gradyan inişiyle (sayısal kuvvet −dE/dR) H_2 bağ uzunluğunu optimize et. Denge noktasında ikinci türevden (kuvvet sabiti k) harmonik titreşim frekansını ω = √(k/μ) hesapla.

Bu örnek şu modülleri kullanır: `hf_mod` *(Örnek 13.2). Derlerken bu modüllerin kaynağı programdan önce aynı dosyaya eklenir (kitapla birlikte verilen örnek dosyalarında hazırdır).*

ÇÖZÜM (FORTRAN)

```
program geom_opt
  use hf_mod
  implicit none
  real(dp) :: Rb, g, lr, e0, ep, em, k, mu, omega
  integer :: it
  lr = 0.8_dp
  Rb = 2.5_dp                                        ! baslangic bag uzunlugu (Bohr)
  print '(A)', 'adim     R (Bohr)      E (Eh)      dE/dR'
  do it = 0, 40
    ep = energy(Rb + 1.0e-4_dp); em = energy(Rb - 1.0e-4_dp)
    g = (ep - em) / 2.0e-4_dp                        ! sayisal gradyan = -kuvvet
    if (it <= 5 .or. mod(it, 10) == 0) print '(I4,F13.6,F14.8,F12.6)', it, Rb, energy(Rb), g
    if (abs(g) < 1.0e-7_dp) exit
    Rb = Rb - lr * g                                 ! gradyan inisi: R <- R - lr * dE/dR
  end do
  e0 = energy(Rb)
```

```
  ep = energy(Rb + 1.0e-3_dp); em = energy(Rb - 1.0e-3_dp)
  k  = (ep - 2.0_dp*e0 + em) / 1.0e-6_dp              ! kuvvet sabiti d2E/dR2
  mu = 1836.152673_dp / 2.0_dp                        ! indirgenmis kutle (elektron kutlesi cinsinden)
  omega = sqrt(k / mu) * 219474.63_dp                 ! Hartree -> cm^-1
  print '(A,F9.5,A,F8.4,A)', 'denge R = ', Rb, ' Bohr  = ', Rb*0.529177_dp, ' Angstrom   (deneysel 0.741)'
  print '(A,F10.6,A)', 'E(denge)   = ', e0, ' Eh'
  print '(A,F9.4,A)', 'kuvvet sabiti k = ', k, ' Eh/Bohr^2'
  print '(A,F9.1,A)', 'titresim frekansi = ', omega, ' cm^-1   (deneysel 4401)'
contains
  function energy(R12) result(e)
    real(dp), intent(in) :: R12
    real(dp) :: e, Z(2), R(3,2)
    Z = [1.0_dp, 1.0_dp]; R = 0.0_dp; R(3,2) = R12
    call scf_energy(2, Z, R, e, .false.)
  end function energy
end program geom_opt
```

ÇIKTI

```
adim     R (Bohr)      E (Eh)      dE/dR
   0     2.500000   -0.96579368    0.168458
   1     2.365234   -0.98859239    0.169521
   2     2.229616   -1.01151743    0.168095
   3     2.095140   -1.03385629    0.163559
   4     1.964293   -1.05476668    0.155350
   5     1.840013   -1.07336042    0.143023
  10     1.425508   -1.11583590    0.040261
  20     1.346138   -1.11750587    0.000125
  30     1.345920   -1.11750589    0.000000
denge R =   1.34592 Bohr  =   0.7122 Angstrom   (deneysel 0.741)
E(denge)   =  -1.117506 Eh
kuvvet sabiti k =    0.5729 Eh/Bohr^2
titresim frekansi =    5482.7 cm^-1   (deneysel 4401)
```

NASIL ÇALIŞIYOR?

Burada bir makine öğrenmesi döngüsü görüyoruz: **kayıp = E(R)**, **parametre = R**, **gradyan = −kuvvet**. Birkaç adımda R ≈ 1.346 Bohr = 0.712 Å'a yakınsar (deneysel 0.741 Å; STO-3G için bilinen değer 0.712 Å). Denge noktasında eğrinin eğriliği ($k = d^2E/dR^2$) bağın "sertliğini" verir. Harmonik osilatörden (Bölüm 12) $\omega = \sqrt{k/\mu}$, indirgenmiş kütle $\mu = m_H/2 \approx 918\ m_e$. Sonuç ≈ 5500 cm^{-1}: deneysel 4401 cm^{-1}'den ~%25 yüksek; küçük baz ve HF'nin bağları fazla sert bulma eğilimi bunun tipik nedenidir. Gerçek programlar gradyanı sayısal değil **analitik** hesaplar (türevleri alınmış integrallerle); bu, makine öğrenmesindeki otomatik türevin kuantum kimyasındaki eşdeğeridir.

Korelasyon enerjisi: HF'nin eksiği

> **Basit matematik**
> HF, her elektronun diğerlerinin *ortalama* alanını gördüğünü varsayar. Gerçekte elektronlar anlık olarak birbirinden kaçınır (Coulomb korelasyonu). Tanım:
>
> $E_{corr} = E_{tam} - E_{HF}$ (He atomu: $-2.9037 - (-2.8617) = -0.0420$ Eh ≈ 1.1 eV)
>
> Bu küçük görünen fark (toplam enerjinin %1.5'i) kimyasal olarak devasadır: tipik bir bağ enerjisi 0.1–0.2 Eh'dir. Korelasyonu geri kazanmanın iki yolu: **post-HF** yöntemleri (MP2, CCSD(T), $O(N^5$–$N^7)$ maliyet) ve **yoğunluk fonksiyoneli teorisi** (DFT, ucuz ve çoğu zaman yeterince doğru). Sonraki bölüm DFT'yi anlatıyor.

BÖLÜM 14

14. Yoğunluk Fonksiyoneli Teorisi (DFT)

Bugün hesaplamalı kimyanın en çok kullanılan yöntemi: N elektronlu dalga fonksiyonu yerine yalnızca elektron yoğunluğu n(r) ile çalışmak.

Bu bölümde

- Hohenberg–Kohn teoremleri ve Kohn–Sham denklemleri
- Düzgün elektron gazı: Dirac değişimi, korelasyon (PW92), Thomas–Fermi kinetik enerjisi
- Yerel yoğunluk yaklaşımı (LDA) ve Jacob'un merdiveni (GGA, meta-GGA, hibrit)
- Radyal Kohn–Sham denklemlerini helyum atomu için kendimiz çözmek (SCF)
- Fonksiyonel türev = gradyan: DFT ile makine öğrenmesinin ortak dili

Hedef: Çalışan bir DFT programı yazıp helyum atomunun enerjisini Hartree–Fock ve deneyle karşılaştırmak.

Neden yoğunluk?

N elektronlu dalga fonksiyonu $\Psi(r_1,\ldots,r_N)$ 3N boyutlu bir nesnedir: 10 elektronlu bir molekül için 30 boyut. Her boyut için yalnızca 10 nokta kullansak bile 10^{30} sayı gerekir. Oysa elektron yoğunluğu $n(r) = N \int|\Psi|^2\, dr_2 \ldots dr_N$ yalnızca 3 boyutlu bir fonksiyondur ve fiziksel olarak ölçülebilir bir niceliktir (X-ışını kırınımı). **Soru:** bütün bilgi n(r) içinde mi gizli? Cevap evet.

Hohenberg–Kohn teoremleri ve Kohn–Sham yöntemi

Basit matematik

Teorem 1 (Hohenberg–Kohn 1964): Temel durum yoğunluğu $n_0(r)$, dış potansiyel v(r)'yi (dolayısıyla Hamiltonyeni ve tüm özellikleri) bir sabit dışında **tek olarak belirler**. Her özellik n_0'ın bir fonksiyonelidir.

Teorem 2: $E[n] = F[n] + \int v(r)\, n(r)\, dr$ fonksiyoneli, doğru n_0 için minimumdur (varyasyonel ilke, Bölüm 12). F[n] evrensel fonksiyoneldir (kinetik + elektron–elektron). Ama F[n]'nin kesin biçimi **bilinmiyor**.

Kohn–Sham (1965) hilesi: etkileşen gerçek sistemin yoğunluğunu veren, etkileşmeyen yapay bir elektron sistemi kur. Enerjiyi parçala:

$E[n] = T_s[n] + \int v\, n + E_H[n] + E_{xc}[n]$

T_s: etkileşmeyen elektronların kinetik enerjisi (orbitallerle tam hesaplanır); $E_H = \tfrac{1}{2}\iint n(r)n(r')/|r-r'|$: klasik Coulomb (Hartree) enerjisi; $\mathbf{E_{xc}}$: değişim-korelasyon enerjisi, geri kalan her şey (bilinmeyen kısım, yaklaşıklanır). Varyasyon alınırsa tek-elektron denklemleri çıkar:

$[\, -\tfrac{1}{2}\nabla^2 + v(r) + v_H(r) + v_{xc}(r)\,]\, \psi_i(r) = \varepsilon_i\, \psi_i(r)$, $n(r) = \Sigma_{i\ \text{dolu}}\, |\psi_i(r)|^2$

$v_H(r) = \int n(r')/|r-r'|\, dr'$ (Poisson denkleminin çözümü), $v_{xc} = \delta E_{xc}/\delta n$ (fonksiyonel türev). Potansiyel yoğunluğa, yoğunluk potansiyele bağlıdır: yine bir **SCF döngüsü** (Bölüm 13).

Basit matematik

Düzgün elektron gazı ve LDA. Yoğunluğu her yerde n olan sonsuz bir sistemde (jellium) Fermi dalga vektörü $k_F = (3\pi^2 n)^{1/3}$. Her k durumu 2 elektron alır; Dirac değişim enerjisi (Hartree–Fock'un homojen gazdaki sonucu):

$\varepsilon_x(n) = -(3/4)(3/\pi)^{1/3}\, n^{1/3} = -0.4582 / r_s$, $r_s = (3/(4\pi n))^{1/3}$ (Wigner–Seitz yarıçapı)

Thomas–Fermi kinetik enerjisi (elektron başına): $t_{TF} = (3/10)(3\pi^2)^{2/3}\, n^{2/3}$. Korelasyon $\varepsilon_c(r_s)$ kuantum Monte Carlo verisine (Ceperley–Alder) uydurulmuş analitik formlarla verilir (VWN, PW92).

LDA: gerçek sistemin her noktasına, o noktadaki yoğunluktaki düzgün gazın enerjisini ata: $E_{xc}^{LDA}[n] = \int n(r)\,[\varepsilon_x(n(r)) + \varepsilon_c(n(r))]\,dr$. $v_{xc} = \varepsilon_{xc} + n\, d\varepsilon_{xc}/dn$; yalnızca değişim için $v_x = (4/3)\varepsilon_x$; korelasyon için $v_c = \varepsilon_c - (r_s/3)\, d\varepsilon_c/dr_s$.

Örnek 14.1 — Düzgün elektron gazı: değişim, korelasyon ve Thomas–Fermi

PROBLEM

$r_s = 0.5$ – 6 için yoğunluğu, Dirac değişim enerjisini (iki formülle), PW92 korelasyonunu ve Thomas–Fermi kinetik enerjisini tablola.

ÇÖZÜM (FORTRAN)

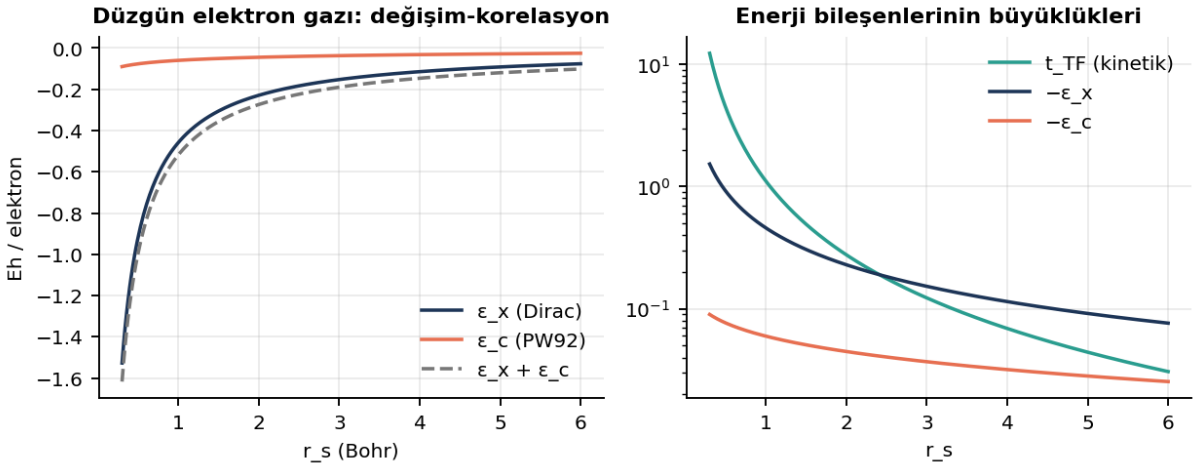
```
program lda_gaz
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp) :: rs(7), dens, ex, ec, tTF
  integer :: i
  rs = [0.5_dp, 1.0_dp, 2.0_dp, 3.0_dp, 4.0_dp, 5.0_dp, 6.0_dp]
  print '(A)', '  rs      n (a.u.)     eps_x     -0.4582/rs    eps_c(PW92)   t_TF(kinetik)'
  do i = 1, 7
    dens = 3.0_dp / (4.0_dp * pi * rs(i)**3)                        ! n = 3/(4 pi rs^3)
    ex   = -0.75_dp * (3.0_dp/pi)**(1.0_dp/3.0_dp) * dens**(1.0_dp/3.0_dp)
    ec   = pw92(rs(i))
    tTF  = 0.3_dp * (3.0_dp*pi**2)**(2.0_dp/3.0_dp) * dens**(2.0_dp/3.0_dp)
    print '(F5.1,ES12.3,F11.5,F12.5,F13.5,F14.5)', rs(i), dens, ex, -0.458165_dp/rs(i), ec, tTF
  end do
contains
  function pw92(rs_) result(e)                    ! Perdew-Wang 1992 korelasyon (paramanyetik)
    real(dp), intent(in) :: rs_
    real(dp) :: e, A, a1, b1, b2, b3, b4
    A = 0.0310907_dp; a1 = 0.21370_dp
    b1 = 7.5957_dp; b2 = 3.5876_dp; b3 = 1.6382_dp; b4 = 0.49294_dp
    e = -2.0_dp*A*(1.0_dp + a1*rs_) * log(1.0_dp + 1.0_dp / (2.0_dp*A*(b1*sqrt(rs_) + b2*rs_ + &
        b3*rs_**1.5_dp + b4*rs_**2)))
  end function pw92
end program lda_gaz
```

ÇIKTI

```
  rs      n (a.u.)     eps_x     -0.4582/rs    eps_c(PW92)   t_TF(kinetik)
  0.5   1.910E+00   -0.91633    -0.91633     -0.07662       4.41980
  1.0   2.387E-01   -0.45817    -0.45816     -0.05977       1.10495
  2.0   2.984E-02   -0.22908    -0.22908     -0.04476       0.27624
  3.0   8.842E-03   -0.15272    -0.15272     -0.03694       0.12277
  4.0   3.730E-03   -0.11454    -0.11454     -0.03187       0.06906
  5.0   1.910E-03   -0.09163    -0.09163     -0.02822       0.04420
  6.0   1.105E-03   -0.07636    -0.07636     -0.02543       0.03069
```

NASIL ÇALIŞIYOR?

İki değişim formülü aynı sonucu verir ($\varepsilon_x = -0.4582/r_s$). Yoğun gazda (küçük r_s) kinetik enerji ($n^{2/3}$ ile büyür) baskındır; seyrek gazda (büyük r_s) Coulomb enerjileri önemli hâle gelir. Korelasyon enerjisi değişimden çok daha küçüktür ($r_s = 2$'de ≈ -0.045 vs -0.229 Eh) ama kimyasal doğruluk için vazgeçilmezdir. Bu üç sayı, LDA yaklaşımının "yerel" girdileridir; molekülün her noktasında yoğunluğa bakıp bu tablodan enerji okumak gibi düşünebilirsin.



Şekil 14.1 — Düzgün elektron gazı: ε_x (Dirac), ε_c (PW92) ve Thomas–Fermi kinetik enerjisi, r_s fonksiyonu olarak (Fortran örneğiyle aynı formüller).

Helyum atomu için Kohn–Sham: kendi DFT programımız

Basit matematik

Küresel simetrik, kapalı kabuklu bir atom (He: $1s^2$) için Kohn–Sham denklemleri tek boyutlu radyal denkleme iner. $u(r) = r\,R(r)$ ile:

$[\,-\tfrac{1}{2}\,d^2/dr^2 + v_{\text{eff}}(r)\,]\,u(r) = \varepsilon\,u(r)$, $v_{\text{eff}} = -Z/r + v_H[n] + v_{xc}[n]$, $n(r) = 2\,u^2(r)\,/\,(4\pi\,r^2)$

Hartree potansiyeli (küresel Poisson denklemi çözümü): $v_H(r) = (1/r)\int_0^r 4\pi r'^2 n\,dr' + \int_r^\infty 4\pi r'\,n\,dr'$ (Gauss yasası: içerideki yük + dış kabuklar).

Toplam enerji: $E = \Sigma\, n_i\varepsilon_i - E_H - \int v_{xc}\,n + E_{xc}$ (orbital enerjileri toplamında Hartree ve XC iki kez sayıldığı için düzeltme). Sayısal yöntem: u ızgarasında üç-köşegen Hamiltonyen, en düşük öz-çifti **ters iterasyon** ile (Thomas algoritması, O(n)); SCF için doğrusal yoğunluk karıştırma.

Örnek 14.2 — Helyum atomu: Kohn–Sham SCF (Hartree, LDA-x, LDA-xc)

PROBLEM

Radyal Kohn–Sham denklemlerini 2000 noktalı ızgarada çöz. Üç model karşılaştır: (1) yalnız Hartree (XC yok), (2) yalnız Dirac değişimi, (3) değişim + PW92 korelasyonu. Toplam enerjiyi HF (−2.8617) ve tam (−2.9037) ile karşılaştır; yoğunluk ve potansiyelleri tablola.

ÇÖZÜM (FORTRAN)

```
program ks_helyum
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer, parameter :: n = 2000
  real(dp), parameter :: pi = 3.141592653589793_dp, Rmax = 20.0_dp, Zn = 2.0_dp
  real(dp) :: r(n), u(n), nr(n), nold(n), vh(n), vx(n), vc(n), vext(n), veff(n), ex_(n), ec_(n)
  real(dp) :: h, eps, etot, eh, exc, vxc_int, de, eps_old, rmean
  integer :: i, it, mode
  character(len=24) :: lab(3)
  lab = [character(len=24) :: 'Yalniz Hartree (XC yok)', 'LDA: yalniz degisim', 'LDA: PW92 (x + c)']
  h = Rmax / n
  do i = 1, n
    r(i) = i * h
  end do
  vext = -Zn / r                                        ! cekirdek potansiyeli
  do mode = 1, 3
    eps_old = 0.0_dp
    u = exp(-1.6_dp * r); u = u / sqrt(sum(u**2) * h)  ! baslangic tahmini: 1s benzeri
    nr = 2.0_dp * u**2 / (4.0_dp * pi * r**2)          ! n(r) = 2 |u|^2 / (4 pi r^2)
    do it = 1, 600
      call hartree(nr, vh, eh)
      call xc(nr, vx, vc, ex_, ec_, mode)
      veff = vext + vh + vx + vc                       ! Kohn-Sham etkin potansiyeli
      call lowest(veff, u, eps)                        ! -1/2 u'' + veff u = eps u
```

```
      nold = nr
      nr = 0.5_dp * nold + 0.5_dp * (2.0_dp * u**2 / (4.0_dp * pi * r**2))   ! dogrusal karistirma
      de = abs(eps - eps_old); eps_old = eps
      if (it > 5 .and. de < 1.0e-9_dp) exit
    end do
    call hartree(nr, vh, eh)
    call xc(nr, vx, vc, ex_, ec_, mode)
    exc     = sum(4.0_dp*pi*r**2 * nr * (ex_ + ec_)) * h
    vxc_int = sum(4.0_dp*pi*r**2 * nr * (vx + vc)) * h
    etot = 2.0_dp * eps - eh - vxc_int + exc            ! toplam enerji (cift sayim duzeltmeli)
    rmean = sum(4.0_dp*pi*r**3 * nr) * h / 2.0_dp
    print '(A,A,F10.5,A,F9.5,A,I4)', lab(mode), ' E=', etot, '  eps_1s=', eps, '  iter=', it
    if (mode == 3) then
      print '(A,F7.4,A,F7.4)', '   toplam elektron = ', sum(4.0_dp*pi*r**2*nr)*h, '   <r> = ', rmean
      print '(A)', '   r (Bohr)      n(r)        V_H        V_xc'
      do i = 25, 1000, 75
        print '(F10.3,3F12.5)', r(i), nr(i), vh(i), vx(i) + vc(i)
      end do
    end if
  end do
  print '(A)', 'Karsilastirma: Hartree-Fock = -2.86168 Eh,  tam (deneysel) = -2.90372 Eh'
contains
  subroutine hartree(nr, vh, eh)                      ! Poisson denklemi (kuresel simetri)
    real(dp), intent(in) :: nr(:)
    real(dp), intent(out) :: vh(:), eh
    real(dp) :: cum(size(nr)), tail(size(nr))
    integer :: i
    cum(1) = 4.0_dp*pi*r(1)**2*nr(1)*h
    do i = 2, size(nr)
      cum(i) = cum(i-1) + 4.0_dp*pi*r(i)**2*nr(i)*h    ! r'e kadar olan yuk
    end do
    tail(size(nr)) = 0.0_dp
    do i = size(nr)-1, 1, -1
      tail(i) = tail(i+1) + 4.0_dp*pi*r(i+1)*nr(i+1)*h ! r'den dis kabuk katkisi
    end do
    vh = cum / r + tail                                ! V_H(r) = Q(r)/r + int_r^inf 4 pi r' n dr'
    eh = 0.5_dp * sum(4.0_dp*pi*r**2 * nr * vh) * h
  end subroutine hartree

  subroutine xc(nr, vx, vc, ex_, ec_, mode)
    real(dp), intent(in) :: nr(:)
    real(dp), intent(out) :: vx(:), vc(:), ex_(:), ec_(:)
    integer, intent(in) :: mode
    real(dp) :: rs, dd
    integer :: i
    vx = 0.0_dp; vc = 0.0_dp; ex_ = 0.0_dp; ec_ = 0.0_dp
    if (mode == 1) return
    do i = 1, size(nr)
      if (nr(i) < 1.0e-14_dp) cycle
      ex_(i) = -0.75_dp * (3.0_dp/pi)**(1.0_dp/3.0_dp) * nr(i)**(1.0_dp/3.0_dp)   ! Dirac degisimi
      vx(i)  = 4.0_dp/3.0_dp * ex_(i)                                              ! v_x = dE_x/dn
      if (mode == 3) then
        rs = (3.0_dp / (4.0_dp*pi*nr(i)))**(1.0_dp/3.0_dp)
        ec_(i) = pw92(rs)
        dd = 1.0e-5_dp * rs
        vc(i) = ec_(i) - rs/3.0_dp * (pw92(rs+dd) - pw92(rs-dd)) / (2.0_dp*dd)  ! v_c = e_c - (rs/3)
              de_c/drs
      end if
    end do
  end subroutine xc

  function pw92(rs) result(e)
    real(dp), intent(in) :: rs
    real(dp) :: e, A, a1, b1, b2, b3, b4
    A = 0.0310907_dp; a1 = 0.21370_dp
    b1 = 7.5957_dp; b2 = 3.5876_dp; b3 = 1.6382_dp; b4 = 0.49294_dp
    e = -2.0_dp*A*(1.0_dp + a1*rs) * log(1.0_dp + 1.0_dp / (2.0_dp*A*(b1*sqrt(rs) + b2*rs + &
        b3*rs**1.5_dp + b4*rs**2)))
  end function pw92

  subroutine lowest(veff, u, eps)                     ! en dusuk ozcift: ters iterasyon + Thomas
        algoritmasi
    real(dp), intent(in) :: veff(:)
    real(dp), intent(inout) :: u(:)
    real(dp), intent(out) :: eps
    real(dp) :: x(n), dg(n), cp(n), sig, bb, num
    integer :: i, k
    sig = minval(veff) - 0.05_dp                       ! kayma: H - sig ters cevrilir
```

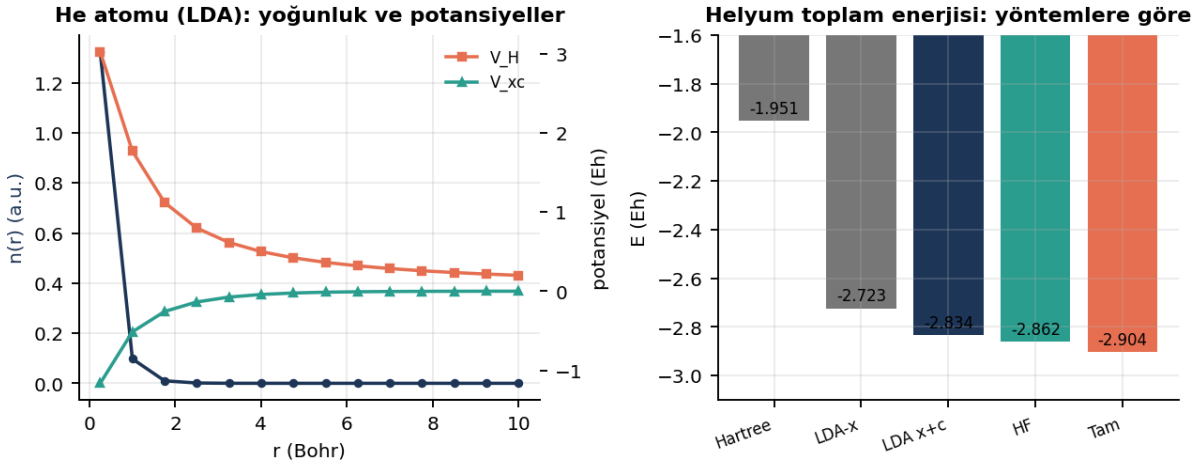
```
    do k = 1, 40
      dg = 1.0_dp/h**2 + veff - sig                    ! uc-kosegen matris: kosegen
      cp(1) = -0.5_dp/h**2 / dg(1); x(1) = u(1)/dg(1)
      do i = 2, n
        bb = dg(i) + 0.5_dp/h**2 * cp(i-1)
        cp(i) = -0.5_dp/h**2 / bb
        x(i) = (u(i) + 0.5_dp/h**2 * x(i-1)) / bb
      end do
      u(n) = x(n)
      do i = n-1, 1, -1
        u(i) = x(i) - cp(i) * u(i+1)
      end do
      u = u / sqrt(sum(u**2) * h)
    end do
    num = 0.0_dp                                       ! Rayleigh bolumu: eps = <u|H|u>
    do i = 1, n
      num = num + u(i) * ((1.0_dp/h**2 + veff(i)) * u(i))
      if (i > 1) num = num - 0.5_dp/h**2 * u(i) * u(i-1)
      if (i < n) num = num - 0.5_dp/h**2 * u(i) * u(i+1)
    end do
    eps = num * h
  end subroutine lowest
end program ks_helyum
```

ÇIKTI

```
Yalniz Hartree (XC yok)  E=  -1.95147  eps_1s= -0.18485  iter= 285
LDA: yalniz degisim      E=  -2.72330  eps_1s= -0.51687  iter= 125
LDA: PW92 (x + c)        E=  -2.83411  eps_1s= -0.57016  iter= 118
   toplam elektron =  2.0000   <r> =  0.9595
   r (Bohr)      n(r)        V_H        V_xc
     0.250     1.32431     3.02394    -1.16330
     1.000     0.09749     1.76796    -0.51357
     1.750     0.01004     1.11979    -0.25632
     2.500     0.00122     0.79721    -0.13602
     3.250     0.00016     0.61501    -0.07445
     4.000     0.00002     0.49995    -0.04136
     4.750     0.00000     0.42104    -0.02313
     5.500     0.00000     0.36364    -0.01297
     6.250     0.00000     0.32000    -0.00728
     7.000     0.00000     0.28571    -0.00409
     7.750     0.00000     0.25806    -0.00230
     8.500     0.00000     0.23529    -0.00129
     9.250     0.00000     0.21622    -0.00073
    10.000     0.00000     0.20000    -0.00041
Karsilastirma: Hartree-Fock = -2.86168 Eh,  tam (deneysel) = -2.90372 Eh
```

NASIL ÇALIŞIYOR?

Üç satır DFT'nin "merdivenini" gösterir. **Yalnız Hartree** (−1.95 Eh): her elektron kendi yoğunluğunu da itiyor (öz-etkileşim), enerji felaket derecede yüksek. **LDA değişim** (−2.723 Eh) bu öz-etkileşimi büyük ölçüde telafi eder. **LDA tam (x + c)** −2.834 Eh verir: tam enerjinin −2.9037'sinin %97.6'sı ve HF'ye (−2.8617) kıyasla biraz üstte (LDA He için HF'den daha az bağlı bulur). Orbital enerjisi ε_{1s} = −0.570 Eh, deneysel iyonlaşma enerjisinin (0.903 Eh) çok altındadır: bu, LDA'nın ünlü **yanlış asimptotik potansiyelinden** kaynaklanır (v_{xc} uzakta −1/r yerine üstel sönüyor; öz-etkileşim hatası). Çıktıda yoğunluk n(r) çekirdekte maksimum, v_H çekirdekte en büyük (elektron bulutunun kendi itmesi) ve v_{xc} negatif (değişim-korelasyon çekimi) görülür. Bu örnek Gaussian, VASP ve Quantum ESPRESSO'nun çalışma prensibinin birebir küçük bir kopyasıdır: yoğunluk → potansiyel → orbitaller → yeni yoğunluk.



Şekil 14.2 — Helyum atomunun Kohn–Sham çözümü (LDA/PW92): elektron yoğunluğu n(r), Hartree ve değişim-korelasyon potansiyelleri (Fortran çıktısı) ve enerji merdiveni.

Jacob'un merdiveni: fonksiyonellerin hiyerarşisi

Basamak	Girdi	Örnekler	Maliyet / not
1. LDA	n(r)	SVWN, PW92	En ucuz; bağları fazla güçlü bulur
2. GGA	n, ∇n	PBE, BLYP	Çoğu katı hal hesabı; moleküllerde iyi
3. meta-GGA	n, ∇n, τ (kinetik yoğunluk)	TPSS, SCAN	Daha doğru enerji, biraz pahalı
4. Hibrit	+ kesin HF değişimi	B3LYP, PBE0, ωB97X	Kimyada en popüler; HF integralleri nedeniyle pahalı
5. Çift hibrit / RPA	+ virtüel orbitaller	B2PLYP, RPA	En doğru DFT; O(N^5)

Her basamak daha fazla yerel olmayan bilgi kullanır ve doğruluk artar. Pratik DFT hesaplarının yaygın doğruluğu bağ enerjilerinde ≈ 3–5 kcal/mol (0.15–0.2 eV); "kimyasal doğruluk" hedefi ≈ 1 kcal/mol'dür. **Makine öğrenmesi** bu hedefe iki yoldan yardım ediyor: (1) sinir ağı ile yeni E_{xc} fonksiyoneli öğrenmek (DM21 gibi), (2) DFT/CCSD(T) verisiyle ML potansiyelleri eğitmek (Bölüm 16).

Fonksiyonel türev = gradyan: DFT ve otomatik türev

$v_{xc} = \delta E_{xc}/\delta n$, fonksiyonel türevdir. Yoğunluğu bir ızgarada örneklersek $n = (n_1,\ldots,n_G)$ bir vektör olur ve fonksiyonel türev, **enerjinin yoğunluk vektörüne göre gradyanıdır** (hacim elemanına bölünmüş): ML'deki $\partial L/\partial\theta$ ile aynı şey. Bunu aşağıda ızgara üzerinde sayısal olarak görüyoruz; Bölüm 16'da `torch.autograd` ile aynı türevi otomatik elde edeceğiz.

Örnek 14.3 — Fonksiyonel türev: $v_x = \delta E_x/\delta n$

PROBLEM

Hidrojen 1s yoğunluğu üzerinde LDA değişim enerjisini $E_x[n] = \int n\,\varepsilon_x(n)\,dr$ hesapla. Yoğunluğu tek bir ızgara noktasında δ kadar bozarak sayısal fonksiyonel türevi bul ve analitik $v_x = (4/3)\varepsilon_x$ ile karşılaştır.

ÇÖZÜM (FORTRAN)

```
program fonksiyonel_turev
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer, parameter :: n = 400
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp) :: r(n), dens(n), h, e0, e1, delta, vx_num, vx_ana
  integer :: idx(4), k, i
  h = 10.0_dp / n
  do i = 1, n
    r(i) = i * h
  end do
  dens = exp(-2.0_dp * r) / pi                        ! ornek yogunluk (H atomu 1s)
  idx = [10, 40, 90, 160]
  print '(A)', '   r(Bohr)     v_x (sayisal d(E_x)/d(n))     v_x = (4/3) eps_x     fark'
  do k = 1, 4
    i = idx(k)
    delta = 1.0e-6_dp * dens(i)
    e0 = ex_total(dens)
    dens(i) = dens(i) + delta                          ! tek ızgara noktasinda yogunlugu boz
    e1 = ex_total(dens)
    dens(i) = dens(i) - delta
    vx_num = (e1 - e0) / (delta * 4.0_dp * pi * r(i)**2 * h)   ! fonksiyonel turev = dE / (dn * hacim)
    vx_ana = -(3.0_dp/pi)**(1.0_dp/3.0_dp) * dens(i)**(1.0_dp/3.0_dp)
    print '(F9.3,F26.6,F22.6,ES12.2)', r(i), vx_num, vx_ana, abs(vx_num - vx_ana)
  end do
contains
  function ex_total(d) result(e)                       ! E_x[n] = int n eps_x(n) d^3r
    real(dp), intent(in) :: d(:)
    real(dp) :: e
    e = sum(4.0_dp*pi*r**2 * d * (-0.75_dp*(3.0_dp/pi)**(1.0_dp/3.0_dp) * d**(1.0_dp/3.0_dp))) * h
  end function ex_total
end program fonksiyonel_turev
```

ÇIKTI

```
   r(Bohr)     v_x (sayisal d(E_x)/d(n))     v_x = (4/3) eps_x     fark
    0.250                -0.569147            -0.569147    9.66E-08
    1.000                -0.345205            -0.345205    5.75E-08
    2.250                -0.150026            -0.150026    2.21E-08
    4.000                -0.046718            -0.046718    2.59E-08
```

NASIL ÇALIŞIYOR?

E_x ızgarada yoğunluk vektörünün skaler bir fonksiyonu olduğundan fonksiyonel türev, $\partial E/\partial n_i \div$ (hacim elemanı $4\pi r_i^2 h$) olarak okunur. Sayısal ve analitik sonuçlar çok yüksek hassasiyetle uyuşur. Önemli sonuç: **Kohn–Sham potansiyelleri, enerji fonksiyonelinin gradyanıdır**. Bir sinir ağı $E_{xc}[n]$'yi temsil ederse, v_{xc} otomatik türevle bedavaya gelir; DFT'nin "kuvvetleri" ise çekirdek koordinatlarına göre aynı fikirdir.

BÖLÜM 15

15. Graf Sinir Ağları ve Geometrik Derin Öğrenme

Moleküller, atomların düğüm, bağların kenar olduğu graflardır. Bu bölümde (Python + PyTorch) graf sinir ağlarını sıfırdan kuruyor, ardından molekülün 3B geometrisini simetriye saygılı biçimde kullanan geometrik modellere geçiyoruz.

Bu bölümde

- Molekülleri graf olarak temsil etmek: komşuluk matrisi, Laplasyen, SMILES ve RDKit
- Mesaj geçirmeli sinir ağları (MPNN): matematik, permütasyon değişmezliği, WL sınırı
- Moleküler özellik tahmini: GNN ile logP, parmak izi tabanlı yönteme karşı
- Geometrik derin öğrenme: SE(3) simetrisi, SchNet, EGNN, kuvvet = $-\nabla E$ (autograd)

Hedef: Ham molekül verisinden çalışan, simetrilerine saygılı özellik tahmin modelleri kurmak.

Moleküller graftır

Bir molekül G = (V, E) grafıdır: V atomlar, E kimyasal bağlardır. Her düğüm bir özellik vektörü taşır (atom türü, aromatiklik, hibridizasyon, bağlı hidrojen sayısı), her kenar bağ özelliklerini (tek/çift/üçlü, halka içinde mi) taşıyabilir. Bir resim piksellerin düzgün ızgarasıdır, bir cümle token'ların dizisidir; **molekül ise düzensiz bir graftır**. Atomların numaralandırılma sırasının hiçbir fiziksel anlamı yoktur: aynı molekülü farklı sırayla etiketlemek sonucu değiştirmemelidir. Bu **permütasyon simetrisi** graf sinir ağlarının tasarım ilkesidir.

Basit matematik

Komşuluk matrisi $A_{ij} = 1$ (i ve j bağlıysa), **derece matrisi** $D = \mathrm{diag}(\Sigma_j A_{ij})$, **graf Laplasyeni** $L = D - A$. L'nin ikinci dereceden formu "pürüzsüzlüğü" ölçer: $x^T L x = \Sigma_{(i,j)\in E} (x_i - x_j)^2$. Komşu düğümlerde benzer değer varsa küçüktür. L'nin sıfır öz-değerlerinin sayısı grafın **bağlı bileşen** sayısına eşittir. **Hückel bağlantısı** (Bölüm 13): π-sistem enerjileri doğrudan A'nın öz-değerleridir. Benzen için A'nın öz-değerleri 2, 1, 1, −1, −1, −2 (Fortran çıktısındaki x değerleri!). Graf sinir ağları bu fikrin öğrenilebilir genellemesidir: grafın üzerindeki "yayılımı" (A ile çarpmak) parametrik hâle getirirler.

Örnek 15.1 — Molekül → graf: RDKit, komşuluk matrisi ve spektrum

PROBLEM

Etanol, butadien, benzen ve iki ayrı siklopropan molekülünü SMILES'tan graf olarak çıkar. Komşuluk matrisi, kenar listesi (PyG biçimi), derece, A ve L'nin öz-değerlerini yazdır. A'nın öz-değerlerinin Bölüm 13'teki Hückel x değerleriyle aynı olduğunu göster.

ÇÖZÜM (PYTHON / PYTORCH)

```
import numpy as np
from rdkit import Chem

for ad, smi in [("etanol", "CCO"), ("butadien", "C=CC=C"), ("benzen", "c1ccccc1"),
                ("siklopropan x2", "C1CC1.C1CC1")]:
    m = Chem.MolFromSmiles(smi)
    A = Chem.GetAdjacencyMatrix(m).astype(float)          # komsuluk matrisi (agir atomlar)
    deg = A.sum(1)
    L = np.diag(deg) - A                                  # graf Laplasyeni L = D - A
    print(f"--- {ad} ({smi}):  {m.GetNumAtoms()} agir atom, {m.GetNumBonds()} bag")
    print("  atomlar     :", [a.GetSymbol() for a in m.GetAtoms()])
    ei = [(b.GetBeginAtomIdx(), b.GetEndAtomIdx()) for b in m.GetBonds()]
    print("  kenar listesi:", ei, "  (PyG: edge_index, her kenar iki yonlu)")
```

```
    print("  derece      :", deg.astype(int).tolist())
    print("  A ozdegerleri:", np.round(np.sort(np.linalg.eigvalsh(A))[::-1], 3).tolist())
    ev = np.linalg.eigvalsh(L)
    print("  L ozdegerleri:", np.round(ev, 3).tolist(), "  sifir ozdeger sayisi =", int((ev <
          1e-9).sum()), "(= bagli bilesen sayisi)")
```

ÇIKTI

```
--- etanol (CCO):  3 agir atom, 2 bag
  atomlar     : ['C', 'C', 'O']
  kenar listesi: [(0, 1), (1, 2)]   (PyG: edge_index, her kenar iki yonlu)
  derece      : [1, 2, 1]
  A ozdegerleri: [1.414, -0.0, -1.414]
  L ozdegerleri: [0.0, 1.0, 3.0]   sifir ozdeger sayisi = 1 (= bagli bilesen sayisi)
--- butadien (C=CC=C):  4 agir atom, 3 bag
  atomlar     : ['C', 'C', 'C', 'C']
  kenar listesi: [(0, 1), (1, 2), (2, 3)]   (PyG: edge_index, her kenar iki yonlu)
  derece      : [1, 2, 2, 1]
  A ozdegerleri: [1.618, 0.618, -0.618, -1.618]
  L ozdegerleri: [0.0, 0.586, 2.0, 3.414]   sifir ozdeger sayisi = 1 (= bagli bilesen sayisi)
--- benzen (c1ccccc1):  6 agir atom, 6 bag
  atomlar     : ['C', 'C', 'C', 'C', 'C', 'C']
  kenar listesi: [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 0)]   (PyG: edge_index, her kenar iki yonl
   u)
  derece      : [2, 2, 2, 2, 2, 2]
  A ozdegerleri: [2.0, 1.0, 1.0, -1.0, -1.0, -2.0]
  L ozdegerleri: [-0.0, 1.0, 1.0, 3.0, 3.0, 4.0]   sifir ozdeger sayisi = 1 (= bagli bilesen sayisi)
--- siklopropan x2 (C1CC1.C1CC1):  6 agir atom, 6 bag
  atomlar     : ['C', 'C', 'C', 'C', 'C', 'C']
  kenar listesi: [(0, 1), (1, 2), (3, 4), (4, 5), (2, 0), (5, 3)]   (PyG: edge_index, her kenar iki yonl
   u)
  derece      : [2, 2, 2, 2, 2, 2]
  A ozdegerleri: [2.0, 2.0, -1.0, -1.0, -1.0, -1.0]
  L ozdegerleri: [-0.0, -0.0, 3.0, 3.0, 3.0, 3.0]   sifir ozdeger sayisi = 2 (= bagli bilesen sayisi)
```

NASIL ÇALIŞIYOR?

Benzenin `A` öz-değerleri `[2, 1, 1, −1, −1, −2]`, butadienin `[1.618, 0.618, −0.618, −1.618]`: Fortran Hückel programının (Örnek 13.1) çıktısıyla **aynıdır**. Yani bir Fortran kuantum-kimya hesabı ile bir graf spektrumu aynı sayıları verir. Laplasyenin sıfır öz-değeri sayısı bağlı bileşen sayısını verir: "siklopropan × 2" iki ayrı molekül içerir, dolayısıyla 2 sıfır öz-değer vardır. `edge_index` (her bağ iki yönlü, COO biçimi) PyTorch Geometric ve bu bölümdeki kodların kullandığı standart temsildir. RDKit yalnızca ağır atomları tutar (hidrojenler örtüktür).

Mesaj geçirmeli sinir ağı (MPNN)

Basit matematik

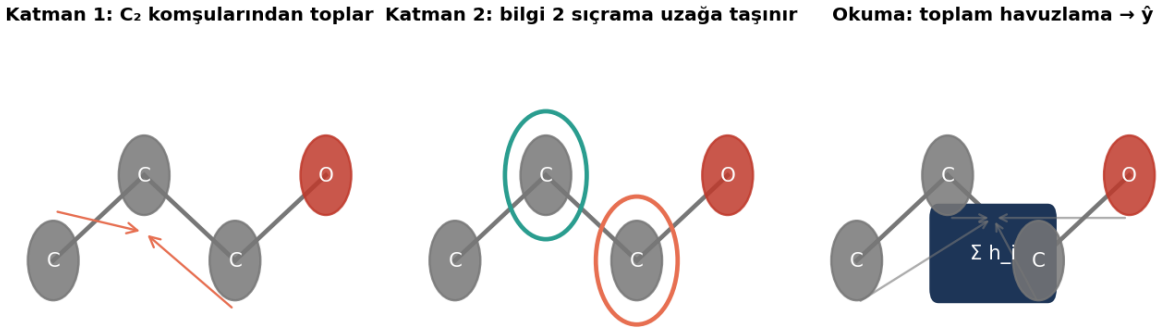
Her düğümün gizli durumu $h_i^{(0)}$ atom özelliklerinden başlar. Her katmanda düğüm, komşularından **mesaj toplar** ve kendini günceller:

$m_i^{(l)} = \Sigma_{j \in N(i)} M(h_j^{(l)}, e_{ij})$ (mesaj)
$h_i^{(l+1)} = U(h_i^{(l)}, m_i^{(l)})$ (güncelleme)

L katman sonra h_i düğümün L-sıçrama (hop) komşuluğunu özetler. Molekül düzeyinde tahmin için **okuma (readout)**: $g = \Sigma_i h_i^{(L)}$ (toplam havuzlama), $\hat{y} = \mathrm{MLP}(g)$.

Neden toplam? Toplama değişmeli bir işlemdir, düğüm sırası sonucu etkilemez. Formel olarak: P bir permütasyon matrisi ise $f(PAP^T, PX) = P\, f(A, X)$ (düğüm düzeyinde *eşdeğer*) ve readout sonrası $g(PAP^T, PX) = g(A, X)$ (graf düzeyinde *değişmez*).

GCN özel durumu (Kipf–Welling): $H^{(l+1)} = \sigma(\, \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \,)$, $\tilde{A} = A + I$. Bu, Laplasyen düzleştirmesinin öğrenilebilir ağırlıkla çarpılmasıdır. **Dikkat tabanlı** (GAT) sürümde toplama ağırlıkları attention ile öğrenilir: aynı Bölüm 4 mekanizması, komşuluk maskesiyle.



Şekil 15.1 — Mesaj geçirme: her atom komşularının durumunu toplar (solda); iki katman sonra bilgi 2 sıçrama uzağa taşınır (ortada); toplam havuzlama ile tüm molekül tek vektöre iner (sağda).

Örnek 15.2 — Mesaj geçirmeyi sıfırdan yaz: permütasyon değişmezliği ve WL sınırı

PROBLEM

PyTorch'ta `index_add_` ile bir MPNN katmanı yaz. (1) Aynı molekülün atom numaralarını karıştır, çıktının değişmediğini göster. (2) Siklohekzan ile iki ayrı siklopropanı karşılaştır: bu iki graf farklıdır ama toplamalı GNN ayırt edemez (Weisfeiler–Lehman sınırı).

ÇÖZÜM (PYTHON / PYTORCH)

```
import torch, torch.nn as nn
from rdkit import Chem

def mol_graph(smi):
    m = Chem.MolFromSmiles(smi)
    x = torch.tensor([[float(a.GetAtomicNum() == z) for z in (6, 7, 8)] for a in m.GetAtoms()])   #
          one-hot (C,N,O)
    ei = torch.tensor([[b.GetBeginAtomIdx(), b.GetEndAtomIdx()] for b in m.GetBonds()]
                      + [[b.GetEndAtomIdx(), b.GetBeginAtomIdx()] for b in m.GetBonds()]).T
    return x, ei

class MPLayer(nn.Module):
    """h_i' = ReLU( W_self h_i + W_msg * toplam_{j komsu i} h_j )"""
    def __init__(s, d):
        super().__init__(); s.ws = nn.Linear(d, d); s.wm = nn.Linear(d, d)
    def forward(s, h, ei):
        src, dst = ei
        agg = torch.zeros_like(h).index_add_(0, dst, h[src])          # mesaj toplama (permutasyondan
              bagimsiz)
        return torch.relu(s.ws(h) + s.wm(agg))

class GNN(nn.Module):
    def __init__(s, d=16, L=3):
        super().__init__(); s.emb = nn.Linear(3, d); s.layers = nn.ModuleList(MPLayer(d) for _ in
              range(L))
    def forward(s, x, ei):
        h = s.emb(x)
        for l in s.layers: h = l(h, ei)
        return h.sum(0)                                                # toplam havuzlama: grafin vektor
              temsili

torch.manual_seed(0)
net = GNN()
x, ei = mol_graph("CC(=O)O")
g1 = net(x, ei)
perm = torch.randperm(len(x)); inv = torch.argsort(perm)              # atomlari yeniden numarala
g2 = net(x[perm], inv[ei])
print("permutasyon degismezligi: ||g(G) - g(permute(G))|| =", (g1 - g2).norm().item())
a, b = mol_graph("C1CCCCC1"), mol_graph("C1CC1.C1CC1")                # siklohekzan vs 2 x siklopropan
print("siklohekzan vs 2 x siklopropan: ||g1 - g2|| =", (net(*a) - net(*b)).norm().item())
print("  (iki graf farkli, ama toplamali GNN ikisini AYIRT EDEMIYOR: 1-WL siniri)")
c = mol_graph("CCO"); d = mol_graph("COC")
print("etanol vs dimetil eter: ||g1 - g2|| =", (net(*c) - net(*d)).norm().item(), " (ayirt edilir)")
```

ÇIKTI

```
permutasyon degismezligi: ||g(G) - g(permute(G))|| = 1.6858739115832577e-07
siklohekzan vs 2 x siklopropan: ||g1 - g2|| = 0.0
  (iki graf farkli, ama toplamali GNN ikisini AYIRT EDEMIYOR: 1-WL siniri)
etanol vs dimetil eter: ||g1 - g2|| = 0.6593561768531799  (ayirt edilir)
```

NASIL ÇALIŞIYOR?

`index_add_(0, dst, h[src])` tek satırda tüm kenarlar boyunca "kaynağın durumunu hedefe ekle" işlemini yapar; bu, GNN'lerin çekirdek işlemidir (PyG'deki *scatter*). Permütasyon testinde fark ≈ 1e-7 (kayan nokta hatası): ağ düğüm sırasına bağlı değildir. İkinci test önemli bir **ifade gücü sınırı** gösterir: siklohekzan (6-halka) ve iki üçgen, tüm atomların derecesi 2 olan grafları temsil eder; toplamalı mesaj geçirme her düğüme aynı yerel komşuluk bilgisini verdiği için iki graf aynı temsili alır. Bu, 1-Weisfeiler–Lehman (1-WL) izomorfizm testiyle eşdeğerdir. Çözümler: halka/3B bilgisi eklemek (aşağıdaki geometrik modeller), yüksek dereceli GNN'ler, konumsal kodlama. Etanol–dimetil eter farkı ise ayırt edilir (atom türleri farklı yerleşiyor).

Moleküler özellik tahmini: GNN ve geleneksel parmak izi

Klasik yaklaşım, molekülü sabit uzunluklu bir **parmak izine** (ECFP/Morgan) çevirip doğrusal veya ağaç modeli kullanmaktır. Parmak izi bilgi kaybeder (karma çarpışmaları) ve özellikleri öğrenemez. GNN ise atom özelliklerinden başlayıp *görev için uygun* yerel çevre özelliklerini kendisi öğrenir. Aşağıda hedef olarak RDKit'in Crippen logP değerini kullanıyoruz (oktanol/su dağılım katsayısı logaritması: bir molekülün yağda çözünürlüğünün ölçüsü, ilaçların zardan geçişinde kritik).

Örnek 15.3 — GNN ile logP tahmini ve Morgan + Ridge karşılaştırması

PROBLEM

640 molekül üretip (zincirler, halkalar, fonksiyonel gruplar) logP değerlerini öğrenen 3 katmanlı bir GNN eğit. %80 eğitim / %20 test böl. Aynı veride Morgan parmak izi + Ridge regresyonu ile karşılaştır; parite grafiğini çiz.

Bu örnek, Örnek 15.3'teki veri üreticisini (`make_dataset`) kullanır; yer kazanmak için burada yeniden listelenmemiştir.

ÇÖZÜM (PYTHON / PYTORCH)

```
import torch, torch.nn as nn, numpy as np
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from rdkit import Chem
from rdkit.Chem import Crippen, AllChem
from sklearn.linear_model import Ridge
torch.manual_seed(0); np.random.seed(0)
smiles = make_dataset()                                    # ~640 molekul
ATOMS = [6, 7, 8, 9, 16, 17, 35]
def featurize(m):
    x = []
    for a in m.GetAtoms():
        f = [float(a.GetAtomicNum() == z) for z in ATOMS]                       # atom turu (one-hot)
        f += [float(a.GetIsAromatic()), a.GetDegree() / 4, a.GetTotalNumHs() / 3, float(a.IsInRing())]
        f += [float(a.GetHybridization() == h) for h in (Chem.HybridizationType.SP,
              Chem.HybridizationType.SP2,
                                                         Chem.HybridizationType.SP3)]
        x.append(f)
    ei = [[], []]
    for b in m.GetBonds():
        i, j = b.GetBeginAtomIdx(), b.GetEndAtomIdx(); ei[0] += [i, j]; ei[1] += [j, i]
    return torch.tensor(x), torch.tensor(ei, dtype=torch.long)
data = []
for s in smiles:
    m = Chem.MolFromSmiles(s); x, ei = featurize(m); data.append((x, ei, Crippen.MolLogP(m)))   # hedef:
          logP
def collate(items):                                        # molekulleri tek buyuk (ayrik) grafta birlestir
    xs, eis, ys, bs = [], [], [], []; off = 0
    for k, (x, ei, y) in enumerate(items):
        xs.append(x); eis.append(ei + off); ys.append(y); bs.append(torch.full((len(x),), k)); off +=
              len(x)
```

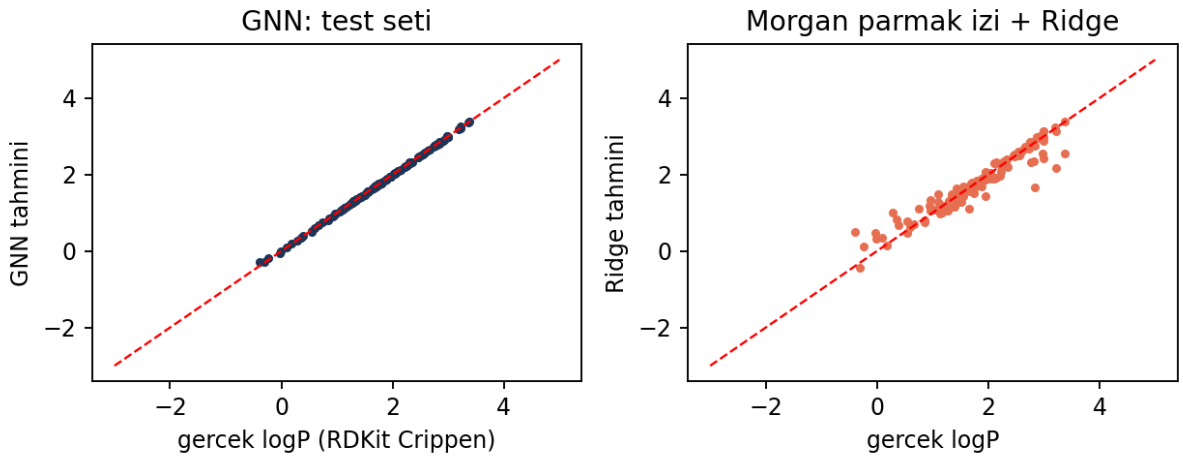
```
    return torch.cat(xs), torch.cat(eis, 1), torch.tensor(ys, dtype=torch.float32), torch.cat(bs)
class GNN(nn.Module):
    def __init__(s, din, d=64, L=3):
        super().__init__(); s.emb = nn.Linear(din, d)
        s.ws = nn.ModuleList(nn.Linear(d, d) for _ in range(L)); s.wm = nn.ModuleList(nn.Linear(d, d) for
              _ in range(L))
        s.head = nn.Sequential(nn.Linear(d, d), nn.ReLU(), nn.Linear(d, 1))
    def forward(s, x, ei, batch, nb):
        h = torch.relu(s.emb(x)); src, dst = ei
        for ws, wm in zip(s.ws, s.wm):
            h = torch.relu(ws(h) + torch.zeros_like(h).index_add_(0, dst, wm(h[src])))   # mesaj gecirme
        g = torch.zeros(nb, h.shape[1]).index_add_(0, batch, h)                          # toplam
              havuzlama
        return s.head(g).squeeze(-1)
idx = np.random.permutation(len(data)); ntr = int(0.8 * len(data))
tr, te = [data[i] for i in idx[:ntr]], [data[i] for i in idx[ntr:]]
model = GNN(data[0][0].shape[1]); opt = torch.optim.Adam(model.parameters(), lr=3e-3)
Xt, Et, Yt, Bt = collate(te)
print(f"molekul sayisi: egitim={len(tr)}  test={len(te)}   parametre={sum(p.numel() for p in
      model.parameters())}")
for ep in range(1, 301):
    perm = np.random.permutation(len(tr))
    for k in range(0, len(tr), 64):
        x, ei, y, b = collate([tr[i] for i in perm[k:k + 64]])
        loss = nn.functional.mse_loss(model(x, ei, b, len(y)), y)
        opt.zero_grad(); loss.backward(); opt.step()
    if ep in (1, 50, 100, 200, 300):
        with torch.no_grad(): p = model(Xt, Et, Bt, len(Yt))
        print(f"epoch {ep:3d}  egitim MSE={loss.item():.5f}  test MAE={(p - Yt).abs().mean().item():.4f}")
with torch.no_grad(): p = model(Xt, Et, Bt, len(Yt))
r2 = 1 - ((p - Yt) ** 2).sum() / ((Yt - Yt.mean()) ** 2).sum()
print(f"GNN         : test MAE = {(p - Yt).abs().mean():.4f}   R2 = {r2:.4f}   (logP std =
      {Yt.std():.3f})")
fp = lambda s: np.array(AllChem.GetMorganFingerprintAsBitVect(Chem.MolFromSmiles(s), 2, 1024))   # ECFP4
Xa = np.array([fp(smiles[i]) for i in idx]); ya = np.array([data[i][2] for i in idx])
pr = Ridge(alpha=1.0).fit(Xa[:ntr], ya[:ntr]).predict(Xa[ntr:])
print(f"Morgan+Ridge: test MAE = {np.abs(pr - ya[ntr:]).mean():.4f}   R2 = {1 - ((pr -
      ya[ntr:])**2).sum() / ((ya[ntr:] - ya[ntr:].mean())**2).sum():.4f}")
fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
ax[0].scatter(Yt, p, s=8, c="#1d3557"); ax[0].plot([-3, 5], [-3, 5], "r--", lw=1)
ax[0].set_xlabel("gercek logP (RDKit Crippen)"); ax[0].set_ylabel("GNN tahmini"); ax[0].set_title("GNN:
      test seti")
ax[1].scatter(ya[ntr:], pr, s=8, c="#e76f51"); ax[1].plot([-3, 5], [-3, 5], "r--", lw=1)
ax[1].set_xlabel("gercek logP"); ax[1].set_ylabel("Ridge tahmini"); ax[1].set_title("Morgan parmak izi +
      Ridge")
plt.tight_layout(); plt.savefig("f_gnn_parity.png", dpi=170)
```

ÇIKTI

```
molekul sayisi: egitim=509  test=128   parametre=30145
epoch   1  egitim MSE=1.18690  test MAE=0.7362
epoch  50  egitim MSE=0.00174  test MAE=0.0673
epoch 100  egitim MSE=0.00039  test MAE=0.0432
epoch 200  egitim MSE=0.00067  test MAE=0.0151
epoch 300  egitim MSE=0.00005  test MAE=0.0137
GNN         : test MAE = 0.0137   R2 = 0.9996   (logP std = 0.848)
Morgan+Ridge: test MAE = 0.1563   R2 = 0.9073
```

NASIL ÇALIŞIYOR?

GNN test setinde ortalama mutlak hata ≈ 0.014 logP birimi ve $R^2 \approx 0.9996$ verirken (hedefin standart sapması 0.85), Morgan + Ridge ≈ 0.16 hata ve $R^2 \approx 0.91$'de kalır. Aradaki fark yöntemin doğasından gelir: Crippen logP, atom tiplerinin (çevresiyle belirlenen) katkılarının *toplamıdır*; toplamalı mesaj geçirme + toplam havuzlama bu yapıya tam olarak uyar (**tümevarımsal önyargı** doğru). Gerçek deneysel özellikler (çözünürlük, bağlanma afinitesi) bu kadar temiz yapıda değildir ve veri azdır; orada sağlam doğrulama (scaffold split), topluluk modeller ve belirsizlik tahmini önemlidir. Not: kodda `make_dataset` işlevi bu örneğin önündeki veri üreticisidir (aşağıda ayrıca listelenmiştir).



Şekil 15.2 — logP tahmininde GNN (solda) ve Morgan parmak izi + Ridge (sağda) parite grafikleri: noktalar y = x doğrusuna ne kadar yakınsa o kadar iyi.

Veri üreticisi (yukarıdaki örneğin başına otomatik eklenir ve Bölüm 16'daki örneklerde de kullanılır):

```
import itertools, random
from rdkit import Chem, RDLogger
RDLogger.DisableLog('rdApp.*')

SUBS = ['F', 'Cl', 'Br', 'O', 'N', 'C', 'CC', 'C(=O)O', 'OC', 'C#N', 'N(C)C', 'S', 'C(F)(F)F', 'C(=O)N',
        'OC(C)=O']
def make_dataset(seed=0, n_max=900):
    """Kucuk ama cesitli bir molekul veri kumesi: zincirler, halkalar, fonksiyonel gruplar."""
    random.seed(seed); smis = set()
    for L in range(1, 7):                                    # alifatik zincirler + fonksiyonel grup
        chain = 'C' * L
        for g in SUBS:
            for pos in range(L):
                smis.add(chain[:pos + 1] + '(' + g + ')' + chain[pos + 1:] if pos < L - 1 else chain + g)
    rings = ['c1ccccc1', 'c1ccncc1', 'c1ccoc1', 'c1ccsc1', 'C1CCCCC1', 'C1CCNCC1', 'C1CCOC1']
    for ring in rings:                                       # halkalar + tek sustituent
        for g in SUBS:
            smis.add(ring.replace('c1', 'c1(' + g + ')', 1) if ring.startswith('c1')
                     else ring.replace('C1', 'C1(' + g + ')', 1))
    for g1, g2 in itertools.product(SUBS, SUBS):             # iki sustituentli benzen / sikloheksan
        smis.add('c1(' + g1 + ')ccc(' + g2 + ')cc1'); smis.add('c1(' + g1 + ')cccc(' + g2 + ')c1')
        smis.add('C1(' + g1 + ')CCC(' + g2 + ')CC1')
    out = []
    for s in sorted(smis):
        m = Chem.MolFromSmiles(s)
        if m is not None and m.GetNumHeavyAtoms() <= 22: out.append(Chem.MolToSmiles(m))
    out = sorted(set(out)); random.shuffle(out)
    return out[:n_max]
```

Geometrik derin öğrenme: simetrilere saygı

Moleküllerin özellikleri yalnızca bağlanabilirliğe değil **3B geometriye** da bağlıdır: konformasyon, bağ açıları, kuvvetler. Bir molekülü uzayda döndürürsek veya kaydırırsak **enerjisi değişmez** (invaryant), **kuvvetleri ise döner** (eşdeğer/equivariant). Bir model bu simetrileri bilmiyorsa onları verilerden öğrenmek zorunda kalır (veri artırma). Geometrik derin öğrenme, simetriyi *mimariye gömer*.

Basit matematik

Simetri grubu: SE(3) = {x → Rx + t : $R^TR = I$, det R = +1} (dönme + öteleme); yansıma dahil E(3).

Değişmezlik (enerji): E(Rx + t) = E(x). **Eşdeğerlik** (kuvvet/vektör): F(Rx + t) = R F(x).

Temel gözlem: atomlar arası mesafe $d_{ij} = \|x_i - x_j\|$ dönme ve ötelemeye değişmezdir, çünkü $\|R(x_i - x_j)\|^2 = (x_i - x_j)^TR^TR(x_i - x_j) = \|x_i - x_j\|^2$. Ağ yalnızca mesafelerin fonksiyonuysa E otomatik olarak değişmezdir.

Kuvvetler enerjinin gradyanıdır: $F_i = -\partial E/\partial x_i$. E değişmez bir model ise F doğal olarak eşdeğerdir (zincir kuralı: E(Rx) = E(x) → $R^T\nabla E(Rx) = \nabla E(x)$). Üstelik F bir gradyan olduğu için **enerji korunur**, moleküler dinamik simülasyonu fiziksel kalır. Kuvvet tahmini bu yüzden enerji modelinin `autograd` türevi olarak yapılır.

Model	Girdi / fikir	Simetri
SchNet	Mesafeler → radyal baz fonksiyonları (RBF) → sürekli filtreli konvolüsyon; kesme fonksiyonu	Değişmez (E(3))
EGNN	Mesaj = $f(h_i, h_j, \|x_i-x_j\|^2)$; koordinat: x_i += $\Sigma(x_i-x_j)\varphi(m_{ij})$	Özellik değişmez, koordinat eşdeğer
DimeNet	Mesafe + bağ açıları (üçlüler)	Değişmez
NequIP, MACE	Küresel harmonikler, tensör ürünleri; vektör/tensör özellikler	Tam SE(3) eşdeğer, yüksek doğruluk

Örnek 15.4 — Simetri testleri: EGNN eşdeğerliği ve SchNet ile kuvvet öğrenme

PROBLEM

(a) Asetik asidin RDKit/MMFF konformerinde bir EGNN katmanının özelliklerinin döndürmeye değişmez, koordinatlarının eşdeğer olduğunu sayısal olarak göster. (b) Lennard-Jones kümesinin enerjisini mesafe tabanlı SchNet ile öğren; kuvvetleri yalnızca enerjiyle eğitilmiş modelin autograd türevinden al.

ÇÖZÜM (PYTHON / PYTORCH)

```
import torch, torch.nn as nn, numpy as np, math
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from rdkit import Chem, RDLogger
from rdkit.Chem import AllChem
RDLogger.DisableLog('rdApp.*')
torch.manual_seed(0); rng = np.random.default_rng(0)

class EGNNLayer(nn.Module):
    """E(n)-esdeger katman: ozellikler degismez (h), koordinatlar esdeger (x)."""
    def __init__(s, d):
        super().__init__()
        s.phi_e = nn.Sequential(nn.Linear(2 * d + 1, d), nn.SiLU(), nn.Linear(d, d), nn.SiLU())
        s.phi_x = nn.Sequential(nn.Linear(d, d), nn.SiLU(), nn.Linear(d, 1))
        s.phi_h = nn.Sequential(nn.Linear(2 * d, d), nn.SiLU(), nn.Linear(d, d))
    def forward(s, h, x):
        n = len(h); off = (1 - torch.eye(n))[..., None]
        diff = x[:, None] - x[None]                                    # x_i - x_j  (esdeger vektor)
        d2 = (diff ** 2).sum(-1, keepdim=True)                         # |x_i - x_j|^2  (DEGISMEZ
              skaler)
        m = s.phi_e(torch.cat([h[:, None].expand(n, n, -1), h[None].expand(n, n, -1), d2], -1)) * off
        x_new = x + (diff * s.phi_x(m) * off).sum(1) / (n - 1)         # x' = x + sum (x_i-x_j)
              phi(m_ij)
        h_new = h + s.phi_h(torch.cat([h, m.sum(1)], -1))
        return h_new, x_new

m = Chem.AddHs(Chem.MolFromSmiles("CC(=O)O")); AllChem.EmbedMolecule(m, randomSeed=1);
      AllChem.MMFFOptimizeMolecule(m)
x = torch.tensor(m.GetConformer().GetPositions(), dtype=torch.float64)       # 3B koordinatlar (Angstrom)
Z = torch.tensor([a.GetAtomicNum() for a in m.GetAtoms()])
torch.set_default_dtype(torch.float64)
h = torch.nn.functional.one_hot(Z, 10).double()[:, :10] @ torch.randn(10, 16, dtype=torch.float64)
layer = EGNNLayer(16).double()
Q, _ = torch.linalg.qr(torch.randn(3, 3, dtype=torch.float64))
if torch.det(Q) < 0: Q[:, 0] = -Q[:, 0]                                    # gercek donme: det = +1
t = torch.randn(3, dtype=torch.float64)
h1, x1 = layer(h, x); h2, x2 = layer(h, x @ Q.T + t)
print("molekul: asetik asit,", len(x), "atom (H dahil)")
print(f"mesafe matrisi degismezligi : {((torch.cdist(x, x) - torch.cdist(x @ Q.T + t, x @ Q.T +
```

```
      t)).abs().max()):.2e}")
print(f"EGNN ozellik degismezligi   : max |h(Rx+t) - h(x)|         = {(h2 - h1).abs().max():.2e}")
print(f"EGNN koordinat esdegerligi  : max |x'(Rx+t) - (Rx'(x)+t)|  = {(x2 - (x1 @ Q.T +
      t)).abs().max():.2e}")
torch.set_default_dtype(torch.float32)

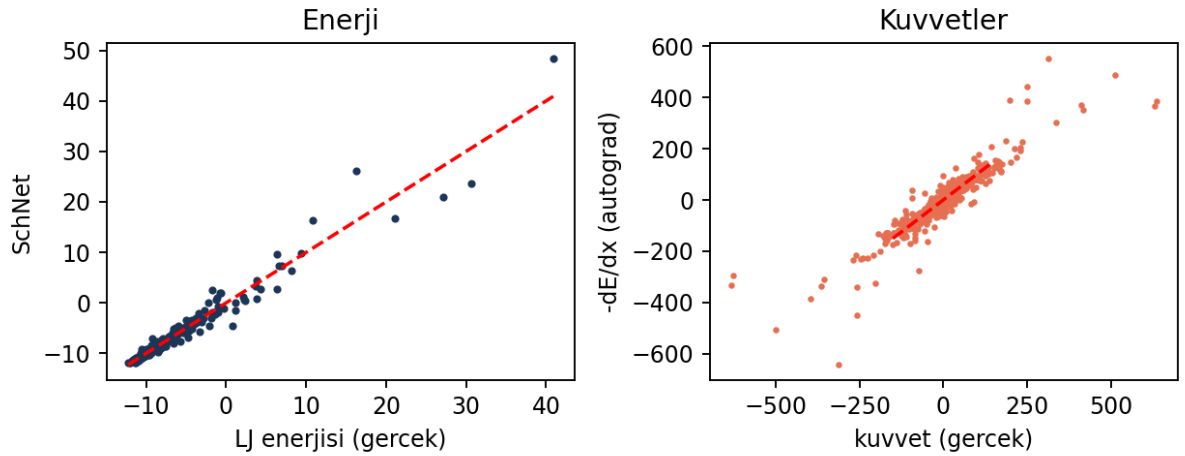
# ---- SchNet tipi mesafe-tabanli model: Lennard-Jones kumesinin enerjisini ogren, kuvveti AUTOGRAD ile al
def lj(x):                                              # E = sum_{i<j} 4 (r^-12 - r^-6); F = -dE/dx
      (analitik)
    d = x[:, None] - x[None]; r2 = (d ** 2).sum(-1) + np.eye(len(x)); i6 = r2 ** -3
    e = 4 * (i6 ** 2 - i6); np.fill_diagonal(e, 0)
    c = 4 * (-12 * i6 ** 2 + 6 * i6) / r2; np.fill_diagonal(c, 0)
    return e.sum() / 2, -(c[:, :, None] * d).sum(1)
base = np.array([[1, 0, 0], [-1, 0, 0], [0, 1, 0], [0, -1, 0], [0, 0, 1], [0, 0, -1]], float) * 0.8
def sample(n):
    out = []
    for _ in range(n):
        xx = base + rng.normal(0, 0.07, base.shape); e, f = lj(xx); out.append((xx, e, f))
    return out
tr, te = sample(1500), sample(300)
T = lambda a: torch.tensor(np.array(a), dtype=torch.float32)
X, E = T([s[0] for s in tr]), T([s[1] for s in tr]); Xt, Et, Ft = T([s[0] for s in te]), T([s[1] for s in
      te]), T([s[2] for s in te])
class SchNet(nn.Module):
    def __init__(s, d=32, nrbf=32, L=2, cutoff=4.0):
        super().__init__(); s.h0 = nn.Parameter(torch.randn(d) * 0.1); s.mu = torch.linspace(0.7, cutoff,
              nrbf)
        s.filt = nn.ModuleList(nn.Sequential(nn.Linear(nrbf, d), nn.Tanh(), nn.Linear(d, d)) for _ in
              range(L))
        s.upd = nn.ModuleList(nn.Sequential(nn.Linear(d, d), nn.Tanh(), nn.Linear(d, d)) for _ in
              range(L))
        s.out = nn.Sequential(nn.Linear(d, d), nn.Tanh(), nn.Linear(d, 1))
    def forward(s, X):
        B, n, _ = X.shape
        D = torch.sqrt(((X[:, :, None] - X[:, None]) ** 2).sum(-1) + 1e-9)    # yalnizca MESAFELER ->
              degisme garantili
        rbf = torch.exp(-6.0 * (D[..., None] - s.mu) ** 2); off = (1 - torch.eye(n))[None, :, :, None]
        h = s.h0.expand(B, n, -1)
        for f, u in zip(s.filt, s.upd): h = h + u((f(rbf) * off * h[:, None]).sum(2))   # surekli
              filtreli konvolusyon
        return s.out(h).squeeze(-1).sum(1)                                      # E = toplam atomik katkilar
net = SchNet(); opt = torch.optim.Adam(net.parameters(), lr=2e-3); sched =
      torch.optim.lr_scheduler.CosineAnnealingLR(opt, 400)
Em, Es = E.mean(), E.std()
for ep in range(1, 401):
    p = torch.randperm(len(X))
    for k in range(0, len(X), 100):
        i = p[k:k + 100]; loss = nn.functional.mse_loss(net(X[i]), (E[i] - Em) / Es)
        opt.zero_grad(); loss.backward(); opt.step()
    sched.step()
    if ep in (1, 100, 200, 400):
        with torch.no_grad(): pe = net(Xt) * Es + Em
        print(f"epoch {ep:3d}  test enerji MAE = {(pe - Et).abs().mean().item():.3f}   (E std =
              {Et.std().item():.2f})")
Xg = Xt.clone().requires_grad_(True); (net(Xg) * Es).sum().backward(); Fp = -Xg.grad     # F = -dE/dx
      (autograd)
r2f = 1 - ((Fp - Ft) ** 2).sum() / ((Ft - Ft.mean()) ** 2).sum()
print(f"kuvvetler (yalnizca enerjiyle egitildi): R2 = {r2f.item():.3f}")
q, _ = torch.linalg.qr(torch.randn(3, 3))
with torch.no_grad(): print(f"SchNet donme/oteleme degismezligi: {(net(Xt[:20]) - net(Xt[:20] @ q.T +
      torch.randn(3))).abs().max().item():.1e}")
fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
ax[0].scatter(Et, pe, s=6, c="#1d3557"); ax[0].plot([Et.min(), Et.max()], [Et.min(), Et.max()], "r--");
      ax[0].set_xlabel("LJ enerjisi (gercek)"); ax[0].set_ylabel("SchNet"); ax[0].set_title("Enerji")
ax[1].scatter(Ft.flatten(), Fp.detach().flatten(), s=3, c="#e76f51"); ax[1].plot([-150, 150], [-150,
      150], "r--"); ax[1].set_xlabel("kuvvet (gercek)"); ax[1].set_ylabel("-dE/dx (autograd)");
           ax[1].set_title("Kuvvetler")
plt.tight_layout(); plt.savefig("f_schnet_lj.png", dpi=170)
```

ÇIKTI

```
molekul: asetik asit, 8 atom (H dahil)
mesafe matrisi degismezligi : 1.33e-15
EGNN ozellik degismezligi   : max |h(Rx+t) - h(x)|         = 4.44e-16
EGNN koordinat esdegerligi  : max |x'(Rx+t) - (Rx'(x)+t)|  = 4.44e-16
epoch   1  test enerji MAE = 4.941   (E std = 6.21)
epoch 100  test enerji MAE = 1.016   (E std = 6.21)
epoch 200  test enerji MAE = 0.950   (E std = 6.21)
epoch 400  test enerji MAE = 0.622   (E std = 6.21)
kuvvetler (yalnizca enerjiyle egitildi): R2 = 0.849
SchNet donme/oteleme degismezligi: 2.5e-06
```

NASIL ÇALIŞIYOR?

(a) Mesafe matrisi dönmeyle değişmez (≈ 1e-15); EGNN katmanının özellikleri h de değişmezdir, çünkü mesajlar yalnızca $\|x_i - x_j\|^2$ görür. Koordinat güncellemesi `x' = x + Σ(x_i−x_j)φ(·)` ise $(x_i - x_j)$ vektörünü *değişmez bir skalerle* çarpar; dolayısıyla Rx + t girişi R x' + t çıkışını verir (hata ≈ 1e-15). **Mimari simetriyi garanti eder; veriden öğrenmek gerekmez.** (b) SchNet yalnızca mesafeleri kullanır. 1500 örnekle eğitilen model enerjiyi ≈ 0.6 birim hatayla öğrenir (E standart sapması ≈ 6) ve **hiç kuvvet verisi görmeden**, enerjinin türevinden alınan kuvvetlerde $R^2 \approx 0.85$ elde eder. Bu, makine öğrenmesi potansiyellerinin (MLIP) temel fikridir: DFT enerjileriyle eğit, molekül dinamiği için kuvvetleri autograd ile al. Döndürme testinde fark ≈ 1e-6 (float32 hassasiyeti).



Şekil 15.3 — SchNet'in Lennard-Jones kümesinde (solda) enerji tahmini ve (sağda) yalnızca enerjiyle eğitilip autograd ile türetilmiş kuvvet tahmini.

BÖLÜM 16

16. Molekül Üretimi, Özellik Tahmini ve İlaç Keşfi

Önceki bölümlerin her şeyi burada bir araya geliyor: Transformer ile yeni moleküller üretmek, ilaç benzerliğini ölçmek, Fortran kuantum hesaplarından makine öğrenmesi potansiyeli eğitmek ve DFT'nin fonksiyonelini sinir ağıyla öğrenmek.

Bu bölümde

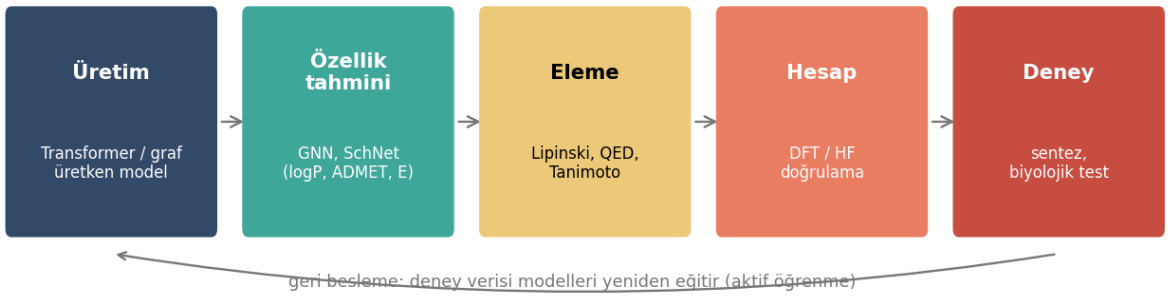
- Üretken modeller: SMILES üzerinde Transformer dil modeli; geçerlilik, benzersizlik, yenilik
- İlaç keşfi boru hattı: Lipinski kuralı, QED, Tanimoto benzerliği, sanal tarama
- Hesaplamalı kimya ile ML: Fortran HF verisinden potansiyel enerji yüzeyi öğrenmek (kuvvet = $-\nabla E$)
- DFT + ML: değişim-korelasyon fonksiyonelini sinir ağıyla temsil etmek ve türevini autograd ile almak

Hedef: Molekül verisinden üretken ve tahminci modeller kurmak, hesaplamalı kimya verisini makine öğrenmesine bağlamak.

Büyük resim: ilaç keşfi ve hesaplamalı kimyada yapay zekâ

Bir ilacı laboratuvardan hastaya götürmek ortalama 10–15 yıl ve milyarlarca dolar sürer; en büyük darboğaz, **kimyasal uzayın devasa olmasıdır**: ilaç benzeri küçük moleküllerin sayısı $\approx 10^{60}$ tahmin edilir. Makine öğrenmesi bu uzayda üç yerde yardım eder:

Aşama	Soru	Yöntem (bu kitapta)
Üretim (de novo tasarım)	Hangi moleküller var olabilir ve ilgi çekici olabilir?	Transformer dil modeli (SMILES), graf/difüzyon modeller
Özellik tahmini	Çözünürlük, toksisite, bağlanma afinitesi ne olur?	GNN, geometrik GNN (Bölüm 15)
Hesaplamalı kimya	Enerji, kuvvet, reaksiyon yolu doğru hesaplanabilir mi?	HF/DFT (Bölüm 13–14) + ML potansiyelleri
Eleme / sıralama	Milyon molekülden hangisi laboratuvara gitsin?	Benzerlik, QED, sanal tarama, aktif öğrenme



Şekil 16.1 — Yapay zekâ destekli ilaç keşfi boru hattı: üretim → özellik tahmini → eleme → deney; her aşamada hesaplamalı kimya (HF/DFT) hem veri üretir hem doğrulama yapar.

Üretken model: SMILES üzerinde Transformer

SMILES, molekülü karakter dizisi olarak yazar: `CC(=O)O` asetik asit, `c1ccccc1` benzen (küçük harf = aromatik, rakamlar halka kapanışı, parantez = dal). Bir molekül birçok farklı SMILES ile yazılabilir (atom sırasına göre); **kanonik SMILES** tekil bir yazım seçer. Bu bir dildir ve Bölüm 4 ve 11'deki mimarilerle aynı şekilde modellenebilir: sonraki karakteri tahmin et. Eğitimde aynı molekülün *rastgele SMILES yazımlarını* kullanmak (veri artırma) model genelleştirmesini ciddi biçimde artırır.

Basit matematik

Dil modeli: P(SMILES) = Π_t P(c_t | $c_{<t}$); eğitim kaybı çapraz entropi. Üretim: BOS'tan başla, sıcaklık T ile örnekle, EOS'ta dur.

Üretken model metrikleri: • Geçerlilik = RDKit'in ayrıştırabildiği (kimyasal olarak geçerli) SMILES oranı. • Benzersizlik = geçerliler içinde tekil kanonik SMILES oranı (mod çökmesi ölçüsü). • Yenilik = eğitim kümesinde bulunmayanların oranı (ezber mi, genelleme mi?). İyi bir üretken model yüksek geçerlilik, yüksek benzersizlik ve yüksek yenilik ister; üçü birbiriyle çekişir.

Örnek 16.1 — SMILES Transformer'ı ile yeni molekül üretimi

PROBLEM

Bölüm 4'teki Transformer bloğunu (causal attention, FFN, residual, weight tying) SMILES karakterleri üzerinde eğit (rastgele SMILES veri artırmasıyla). 500 molekül üret; geçerlilik, benzersizlik ve yenilik oranlarını ölç; yeni moleküllerden bir ızgara resmi çiz.

Bu örnek, Örnek 15.3'teki veri üreticisini (`make_dataset`) kullanır; yer kazanmak için burada yeniden listelenmemiştir.

ÇÖZÜM (PYTHON / PYTORCH)

```
import torch, torch.nn as nn, torch.nn.functional as F, numpy as np, random
from rdkit import Chem, RDLogger
from rdkit.Chem import Draw
RDLogger.DisableLog('rdApp.*')
torch.manual_seed(0); random.seed(0); np.random.seed(0)
smiles = make_dataset(); random.shuffle(smiles)
train_s, test_s = smiles[:550], smiles[550:]
aug = lambda s: Chem.MolToSmiles(Chem.MolFromSmiles(s), doRandom=True)      # ayni molekulun rastgele
      SMILES yazimi
chars = sorted(set("".join(smiles + [aug(s) for s in smiles for _ in range(3)])))
stoi = {c: i + 2 for i, c in enumerate(chars)}; itos = {i: c for c, i in stoi.items()}
BOS, EOS, V, TT = 0, 1, len(chars) + 2, 48
enc = lambda s: [BOS] + [stoi[c] for c in s] + [EOS]

class Block(nn.Module):
    def __init__(s, d, nh):
        super().__init__(); s.nh = nh
        s.n1, s.n2 = nn.LayerNorm(d), nn.LayerNorm(d); s.qkv = nn.Linear(d, 3 * d); s.proj = nn.Linear(d,
            d)
        s.ffn = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(), nn.Linear(4 * d, d))
    def forward(s, x):
        B, T, d = x.shape
        q, k, v = (t.view(B, T, s.nh, d // s.nh).transpose(1, 2) for t in s.qkv(s.n1(x)).chunk(3, -1))
        a = F.scaled_dot_product_attention(q, k, v, is_causal=True).transpose(1, 2).reshape(B, T, d)
        x = x + s.proj(a); return x + s.ffn(s.n2(x))
class SmilesGPT(nn.Module):                                    # Bolum 4/11'deki Transformer'in SMILES
      versiyonu
    def __init__(s, d=96, nh=4, nl=3):
        super().__init__(); s.E = nn.Embedding(V, d); s.P = nn.Embedding(TT, d)
        s.blocks = nn.ModuleList(Block(d, nh) for _ in range(nl)); s.nf = nn.LayerNorm(d)
        for p in s.parameters():
            if p.dim() > 1: nn.init.normal_(p, std=0.02)
    def forward(s, x):
        h = s.E(x) + s.P(torch.arange(x.shape[1]))
        for b in s.blocks: h = b(h)
        return F.linear(s.nf(h), s.E.weight)                    # weight tying
m = SmilesGPT(); opt = torch.optim.AdamW(m.parameters(), lr=2e-3, weight_decay=0.01)
print("parametre sayisi:", sum(p.numel() for p in m.parameters()), "  sozluk:", V, "  ln V =",
      round(float(np.log(V)), 3))
def batch(n):
    x = torch.zeros(n, TT, dtype=torch.long) + EOS; msk = torch.zeros(n, TT, dtype=torch.bool)
    for i in range(n):
```

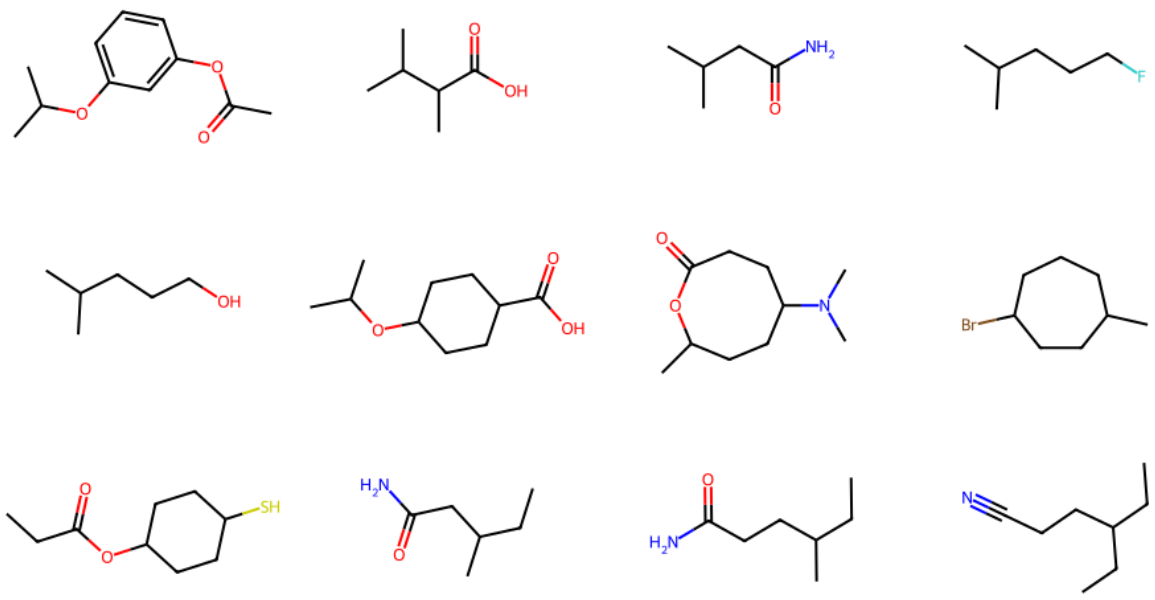
```
        q = enc(aug(random.choice(train_s)))[:TT]; x[i, :len(q)] = torch.tensor(q); msk[i, :len(q) - 1] =
            True
    return x, msk
for step in range(1, 1601):
    x, msk = batch(64)
    loss = F.cross_entropy(m(x[:, :-1])[msk[:, :-1]], x[:, 1:][msk[:, :-1]])    # sonraki karakter tahmini
    opt.zero_grad(); loss.backward(); opt.step()
    if step in (1, 400, 800, 1600): print(f"adim {step:5d}  kayip = {loss.item():.3f}")
@torch.no_grad()
def sample(n, temp=0.8):
    x = torch.zeros(n, 1, dtype=torch.long); done = torch.zeros(n, dtype=torch.bool)
    for _ in range(TT - 1):
        nx = torch.multinomial(torch.softmax(m(x)[:, -1] / temp, -1), 1)
        nx[done] = EOS; x = torch.cat([x, nx], 1); done |= nx.squeeze(1) == EOS
        if done.all(): break
    out = []
    for row in x[:, 1:].tolist():
        s = ""
        for t in row:
            if t == EOS: break
            s += itos[t]
        out.append(s)
    return out
gen = sample(500)
valid = [s for s in gen if s and Chem.MolFromSmiles(s) is not None]
can = [Chem.MolToSmiles(Chem.MolFromSmiles(s)) for s in valid]; uniq = sorted(set(can)); known =
      set(smiles)
novel = [c for c in uniq if c not in known]
print(f"uretilen: {len(gen)}   gecerli (RDKit): {len(valid)/len(gen):.1%}   benzersiz:
      {len(uniq)/len(valid):.1%}   yeni (egitimde yok): {len(novel)/len(uniq):.1%}")
print("ornek yeni moleküller:", novel[:8])
show = novel[:12]
Draw.MolsToGridImage([Chem.MolFromSmiles(s) for s in show], molsPerRow=4, subImgSize=(230,
      170)).save("f_gen_mols.png")
```

ÇIKTI

```
parametre sayisi: 342144   sozluk: 19   ln V = 2.944
adim     1  kayip = 2.899
adim   400  kayip = 0.829
adim   800  kayip = 0.713
adim  1600  kayip = 0.675
uretilen: 500   gecerli (RDKit): 99.0%   benzersiz: 69.7%   yeni (egitimde yok): 11.9%
ornek yeni moleküller: ['CC(=O)Oc1cccc(OC(C)C)c1', 'CC(C)C(C)C(=O)O', 'CC(C)CC(N)=O', 'CC(C)CCCF', 'CC(C
   )CCCO', 'CC(C)OC1CCC(C(=O)O)CC1', 'CC1CCC(N(C)C)CCC(=O)O1', 'CC1CCCC(Br)CC1']
```

NASIL ÇALIŞIYOR?

Aynı Transformer mimarisi, hiçbir değişiklik olmadan, karakterlerden kimyasal yapıya geçti: tek fark sözlüğün SMILES karakterleri olması. Eğitim kaybı başlangıçta ≈ ln V = 2.94'e çok yakın (2.90: Bölüm 4 sağlık kontrolü) ve 0.68 civarına iner. 500 üretimin yaklaşık %99'u **geçerli** bir molekül (RDKit ayrıştırır), geçerlilerin ≈ %70'i benzersiz, benzersizlerin ≈ %12'si eğitim kümesinde olmayan **yeni** moleküllerdir (örneğin eğitimde görülmemiş bir sübstitüent kombinasyonu). Geçersiz üretimler çoğunlukla açılmış ama kapanmamış halka rakamları veya parantezlerdir: dil modeli sözdizimini istatistiksel olarak öğrenir, garanti etmez (burada çok iyi öğrenmiş, çünkü veri kümesi kolay ve düzenli). Gelişmiş çözümler: grammar-tabanlı (SELFIES), graf üreten modeller, difüzyon, ve geçerliliği ödül alan pekiştirmeli öğrenme. Oyuncak veri (640 molekül) nedeniyle yenilik düşüktür; milyonlarca moleküle (ZINC, ChEMBL) eğitilen modellerin ürettiği uzay çok daha çeşitlidir.



Şekil 16.2 — Transformer'ın ürettiği, eğitim kümesinde bulunmayan 12 yeni ve geçerli molekül (RDKit çizimi).

İlaç benzerliği ve sanal tarama

Basit matematik

Lipinski'nin 5 kuralı (ağızdan alınan ilaçlar için): molekül ağırlığı ≤ 500, logP ≤ 5, hidrojen bağı vericisi ≤ 5, alıcısı ≤ 10. Birden fazla ihlal, zayıf bağırsak emilimi riskidir.

QED (Bickerton 2012): 8 özelliğin (MW, logP, HBD, HBA, TPSA, dönebilen bağ, aromatik halka, uyarıcı alt yapı) arzu edilirlik fonksiyonlarının geometrik ortalaması, 0–1 arası.

Parmak izi ve Tanimoto benzerliği: Morgan (ECFP) parmak izi, her atomun r yarıçaplı çevresini bir tamsayıya hash'ler ve sabit uzunluklu bit vektörü kurar. İki molekül için

$T(A, B) = |A \cap B| / |A \cup B| = c / (a + b - c)$ (a, b: kümelerdeki set bit sayıları, c: ortak)

Bölüm 7'deki yakın kopya bulma için kullandığımız **Jaccard** benzerliğinin aynısıdır! Tanimoto > 0.4 genellikle "benzer", > 0.85 "çok benzer" (aynı aktivite beklenir) sayılır.

Örnek 16.2 — Lipinski, QED ve Tanimoto ile sanal tarama

PROBLEM

Yedi bilinen ilaç için MW, logP, HBD, HBA, TPSA, dönebilen bağ, Lipinski ihlali ve QED hesapla. Sonra molekül kütüphanesinde aspirine en benzer molekülleri Tanimoto ile sırala.

Bu örnek, Örnek 15.3'teki veri üreticisini (`make_dataset`) kullanır; yer kazanmak için burada yeniden listelenmemiştir.

ÇÖZÜM (PYTHON / PYTORCH)

```
import numpy as np
from rdkit import Chem, DataStructs
from rdkit.Chem import Descriptors, Crippen, Lipinski, QED, AllChem

drugs = {
    "aspirin":      "CC(=O)Oc1ccccc1C(=O)O",
    "parasetamol":  "CC(=O)Nc1ccc(O)cc1",
    "ibuprofen":    "CC(C)Cc1ccc(cc1)C(C)C(=O)O",
    "kafein":       "Cn1cnc2c1c(=O)n(C)c(=O)n2C",
    "metformin":    "CN(C)C(=N)N=C(N)N",
    "diazepam":     "CN1C(=O)CN=C(c2ccccc2)c2cc(Cl)ccc12",
    "atorvastatin": "CC(C)c1c(C(=O)Nc2ccccc2)c(-c2ccccc2)c(-c2ccc(F)cc2)n1CCC(O)CC(O)CC(=O)O",
}
print(f"{'ilac':13s}{'MW':>8s}{'logP':>7s}{'HBD':>5s}{'HBA':>5s}{'TPSA':>7s}{'RotB':>6s}{'Lipinski
      ihlal':>16s}{'QED':>7s}")
```

```
for ad, smi in drugs.items():
    m = Chem.MolFromSmiles(smi)
    mw, lp, hbd, hba = Descriptors.MolWt(m), Crippen.MolLogP(m), Lipinski.NumHDonors(m),
          Lipinski.NumHAcceptors(m)
    viol = int(mw > 500) + int(lp > 5) + int(hbd > 5) + int(hba > 10)            # Lipinski "5 kurali"
    print(f"{ad:13s}{mw:8.1f}{lp:7.2f}{hbd:5d}{hba:5d}{Descriptors.TPSA(m):7.1f}{Descriptors.NumRotatableB
          onds(m):6d}{viol:16d}{QED.qed(m):7.2f}")

# --- benzerlik tabanli sanal tarama: aspirine en cok benzeyen moleküller (Tanimoto, Morgan r=2)
fp = lambda m: AllChem.GetMorganFingerprintAsBitVect(m, 2, 2048)
query = fp(Chem.MolFromSmiles(drugs["aspirin"]))
lib = make_dataset(); mols = [Chem.MolFromSmiles(s) for s in lib]
sim = DataStructs.BulkTanimotoSimilarity(query, [fp(m) for m in mols])
order = np.argsort(sim)[::-1][:6]
print("\naspirine en benzer 6 molekul (kutuphane: %d molekul):" % len(lib))
for i in order: print(f"  Tanimoto = {sim[i]:.3f}   {lib[i]:28s} QED = {QED.qed(mols[i]):.2f}")
a, b = fp(Chem.MolFromSmiles(drugs["aspirin"])), fp(Chem.MolFromSmiles(drugs["parasetamol"]))
print(f"\nTanimoto(aspirin, parasetamol) = {DataStructs.TanimotoSimilarity(a, b):.3f}")
```

ÇIKTI

```
ilac              MW   logP  HBD  HBA   TPSA  RotB  Lipinski ihlal    QED
aspirin        180.2   1.31    1    3   63.6     2               0   0.55
parasetamol    151.2   1.35    2    2   49.3     1               0   0.60
ibuprofen      206.3   3.07    1    1   37.3     4               0   0.82
kafein         194.2  -1.03    0    3   61.8     0               0   0.54
metformin      129.2  -1.24    3    1   91.5     0               0   0.28
diazepam       284.7   3.15    0    2   32.7     1               0   0.79
atorvastatin   558.7   6.31    4    4  111.8    12               2   0.16

aspirine en benzer 6 molekul (kutuphane: 637 molekul):
  Tanimoto = 0.536   CC(=O)Oc1ccccc1              QED = 0.43
  Tanimoto = 0.438   CC(=O)Oc1ccc(C(=O)O)cc1      QED = 0.55
  Tanimoto = 0.429   CC(=O)Oc1ccc(OC(C)=O)cc1     QED = 0.53
  Tanimoto = 0.429   CC(=O)Oc1cccc(C(=O)O)c1      QED = 0.55
  Tanimoto = 0.419   CC(=O)Oc1cccc(OC(C)=O)c1     QED = 0.53
  Tanimoto = 0.406   CC(=O)Oc1ccc(F)cc1           QED = 0.45

Tanimoto(aspirin, parasetamol) = 0.222
```

NASIL ÇALIŞIYOR?

Küçük, hidrofilik ilaçlar (metformin: logP ≈ −1.2, QED ≈ 0.28 çünkü çok polar) ve tipik ağızdan alınan moleküller (aspirin, ibuprofen, parasetamol, kafein, diazepam: QED 0.54–0.82) kurallara uyar. **Atorvastatin** (MW 559, logP 6.3: iki ihlal, QED 0.16) Lipinski kuralını ihlal eder ama yine de çok başarılı bir ilaçtır: kurallar sıkı bir yasak değil, *ölçüt*tür (kitleyle büyük moleküller transport proteinleri vb. ile emilebilir). Tanimoto taramasında aspirinin asetoksi-benzoik asit iskeletine en çok benzeyen kütüphane molekülleri üstte çıkar (asetoksi-benzen ve asetoksi-benzoik asit türevleri). Bu fikir, **sanal tarama**nın en basit biçimidir: bilinen bir aktif molekülün parmak izine benzeyen adayları ucuzca eleyip laboratuvara gönderilecek listeyi kısaltmak. Üretken model (önceki örnek) + özellik tahmini (GNN) + bu filtreler birleşince tam bir "tasarla–tahmin et–ele" döngüsü olur.

Kuantum kimyası verisiyle makine öğrenmesi potansiyeli

Bölüm 13'te Fortran ile H_2 için potansiyel enerji eğrisini hesapladık: her nokta bir SCF hesabıydı. Gerçek sistemlerde (yüzlerce atom, milyonlarca adım) DFT çok yavaştır. **Makine öğrenmesi potansiyelleri (MLIP)**, az sayıda kuantum hesabıyla bir sinir ağı eğitir; sonra enerjiyi ve kuvvetleri $\sim 10^3$–10^6 kat hızlı verir. Eğitim verisi ab initio hesaplardır; model `E(R)`'yi öğrenir, kuvvet $F = -\nabla E$ **autograd** ile otomatik elde edilir.

Örnek 16.3 — Fortran HF verisinden H_2 için ML potansiyeli

PROBLEM

Örnek 13.2'deki Fortran RHF programının ürettiği 54 H_2 enerjisini (R = 0.7–6.0 Bohr) bir sinir ağına öğret (44 eğitim / 10 test). Ağın autograd türeviyle kuvveti elde et; geometriyi öğrenilmiş yüzeyde optimize et ve titreşim frekansını Hessian'dan hesapla; Fortran sonuçlarıyla karşılaştır.

ÇÖZÜM (PYTHON / PYTORCH)

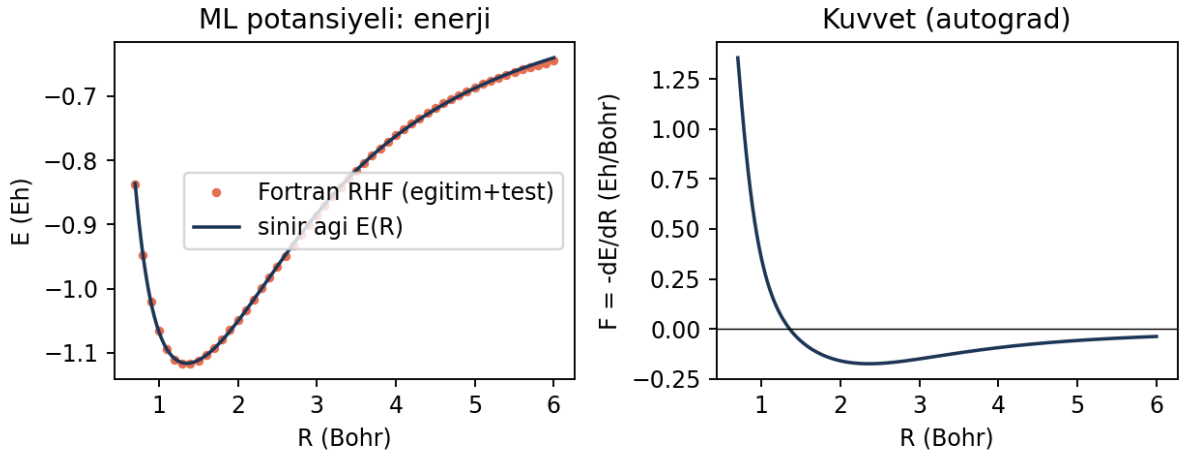
```
import torch, torch.nn as nn, numpy as np
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
torch.manual_seed(0); torch.set_default_dtype(torch.float64)
# ---- Fortran RHF/STO-3G programindan (Ornek 13.2) uretilen H2 enerjileri (R: Bohr, E: Hartree)
R_DATA = [
    0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5,
    1.6, 1.7, 1.8, 1.9, 2.0, 2.1, 2.2, 2.3, 2.4,
    2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3,
    3.4, 3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2,
    4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 5.0, 5.1,
    5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6.0,
]
E_DATA = [
    -0.8371303, -0.94730793, -1.0194971, -1.0659995, -1.0945641, -1.1103339,
    -1.1168711, -1.1167143, -1.1116959, -1.103141, -1.0920059, -1.0789822,
    -1.0645769, -1.0491709, -1.0330609, -1.0164857, -0.99964323, -0.98269933,
    -0.96579368, -0.94904315, -0.93254452, -0.91637682, -0.90060359, -0.885275,
    -0.87042979, -0.85609692, -0.84229704, -0.82904365, -0.81634416, -0.80420065,
    -0.79261066, -0.78156784, -0.7710625, -0.76108225, -0.75161246, -0.74263681,
    -0.73413773, -0.72609682, -0.71849519, -0.7113138, -0.70453368, -0.69813612,
    -0.69210281, -0.68641592, -0.6810582, -0.67601293, -0.67126398, -0.66679577,
    -0.66259329, -0.65864203, -0.654928, -0.6514377, -0.64815813, -0.64507675,
]
R = torch.tensor(R_DATA)[:, None]; E = torch.tensor(E_DATA)[:, None]
perm = torch.randperm(len(R)); te, tr = perm[:10], perm[10:]               # 44 egitim, 10 test
net = nn.Sequential(nn.Linear(1, 32), nn.Tanh(), nn.Linear(32, 32), nn.Tanh(), nn.Linear(32, 1))
energy = lambda r: net(1.0 / r)                                           # girdi ozelligi 1/R
opt = torch.optim.Adam(net.parameters(), lr=5e-3)
for ep in range(1, 6001):
    loss = ((energy(R[tr]) - E[tr]) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
    if ep in (1, 1000, 3000, 6000):
        print(f"epoch {ep:5d}  egitim MSE = {loss.item():.2e}   test max |hata| = {(energy(R[te]) -
              E[te]).abs().max().item():.4f} Eh")
r = torch.tensor([[2.0]], requires_grad=True)                             # geometri optimizasyonu: R <-
      R - lr dE/dR
for it in range(80):
    g, = torch.autograd.grad(energy(r).sum(), r)
    r = (r - 0.8 * g).detach().requires_grad_(True)
g, = torch.autograd.grad(energy(r).sum(), r, create_graph=True); k, = torch.autograd.grad(g.sum(), r)
mu = 1836.152673 / 2
print(f"ML potansiyeli:  R_denge = {r.item():.4f} Bohr   k = {k.item():.4f} Eh/Bohr^2   omega =
      {np.sqrt(k.item() / mu) * 219474.63:.0f} cm^-1")
print("Fortran RHF   :  R_denge = 1.3459 Bohr   k = 0.5729 Eh/Bohr^2   omega = 5483 cm^-1   (Ornek 13.4)")
rr = torch.linspace(0.7, 6.0, 200)[:, None].requires_grad_(True)
e = energy(rr); f, = torch.autograd.grad(e.sum(), rr)
fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
ax[0].plot(R, E, "o", ms=3, c="#e76f51", label="Fortran RHF (egitim+test)"); ax[0].plot(rr.detach(),
      e.detach(), c="#1d3557", label="sinir agi E(R)")
ax[0].set_xlabel("R (Bohr)"); ax[0].set_ylabel("E (Eh)"); ax[0].legend(); ax[0].set_title("ML
      potansiyeli: enerji")
ax[1].plot(rr.detach(), -f.detach(), c="#1d3557"); ax[1].axhline(0, c="k", lw=0.6); ax[1].set_xlabel("R
      (Bohr)"); ax[1].set_ylabel("F = -dE/dR (Eh/Bohr)")
ax[1].set_title("Kuvvet (autograd)"); plt.tight_layout(); plt.savefig("f_h2_ml.png", dpi=170)
```

ÇIKTI

```
epoch     1  egitim MSE = 5.72e-01   test max |hata| = 0.8126 Eh
epoch  1000  egitim MSE = 2.38e-05   test max |hata| = 0.0105 Eh
epoch  3000  egitim MSE = 1.33e-05   test max |hata| = 0.0104 Eh
epoch  6000  egitim MSE = 2.09e-06   test max |hata| = 0.0049 Eh
ML potansiyeli:  R_denge = 1.3550 Bohr   k = 0.5169 Eh/Bohr^2   omega = 5208 cm^-1
Fortran RHF   :  R_denge = 1.3459 Bohr   k = 0.5729 Eh/Bohr^2   omega = 5483 cm^-1   (Ornek 13.4)
```

NASIL ÇALIŞIYOR?

Ağ, 54 Fortran hesabından **tüm potansiyel enerji eğrisini** öğrenir: test hatası ≈ 0.005 Eh (≈ 3 kcal/mol), eğri hem bağlanma çukurunu hem ayrışma platosunu (HF'nin yanlış limiti dahil, Bölüm 13) yeniden üretir. **Denge uzunluğu** öğrenilmiş yüzeyde gradyan inişiyle ≈ 1.355 Bohr bulunur (Fortran: 1.346). **Titreşim frekansı**, ağın ikinci türevinden (autograd'ın autograd'ı: Hessian) ≈ 5200 cm^{-1}; Fortran'ın sayısal ikinci türevi 5483 cm^{-1}. İkinci türevler enerji yüzeyinin *eğriliğine* bağlı olduğundan enerjiden daha zor öğrenilir: pratikte kuvvet verisini de kayba katmak ve daha çok nokta kullanmak şarttır. Bu örnek, Fortran (hesap motoru) + PyTorch (öğrenme) birleşiminin küçük bir modelidir: kuantum hesabı *veriyi*, sinir ağı ise *hızlı vekili* üretir.



Şekil 16.3 — Fortran RHF noktalarına uydurulmuş sinir ağı potansiyeli (solda) ve autograd ile elde edilen kuvvet eğrisi F = −dE/dR (sağda).

DFT + makine öğrenmesi: öğrenilebilir fonksiyoneller

Kohn–Sham DFT'nin bilinmeyen tek parçası $E_{xc}[n]$'dir. Mevcut fonksiyoneller el yapımı formüllerdir (LDA, PBE, B3LYP). Son yıllarda sinir ağlarıyla E_{xc}'yi veriden öğrenen fonksiyoneller (DM21, Skala vb.) geliştirildi; hedef kimyasal doğruluk. Teknik olarak iş basit: yerel yoğunluk özelliklerinden (n, ∇n, τ) enerji yoğunluğunu veren bir ağ, ve $v_{xc} = \delta E_{xc}/\delta n$ **otomatik türevle** gelir. Aşağıdaki örnekte bu zinciri küçük ölçekte izliyoruz.

Örnek 16.4 — Autograd ile fonksiyonel türev ve öğrenilmiş korelasyon fonksiyoneli

PROBLEM

(1) H atomu yoğunluğunda LDA değişim enerjisini PyTorch'ta hesaplayıp autograd ile $v_x = \delta E_x/\delta n$ bul (Fortran Örnek 14.3 ile aynı). (2) PW92 korelasyon enerjisini bir sinir ağına öğret ve $v_c = \varepsilon_c - (r_s/3)\, d\varepsilon_c/dr_s$ türevini autograd ile elde et; analitik değerle karşılaştır.

ÇÖZÜM (PYTHON / PYTORCH)

```
import torch, torch.nn as nn, numpy as np, math
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
torch.manual_seed(0); torch.set_default_dtype(torch.float64)
pi = math.pi
def pw92(rs):                                          # Fortran Ornek 14.1'deki ayni formul
    A, a1, b1, b2, b3, b4 = 0.0310907, 0.21370, 7.5957, 3.5876, 1.6382, 0.49294
    return -2 * A * (1 + a1 * rs) * torch.log(1 + 1 / (2 * A * (b1 * rs.sqrt() + b2 * rs + b3 * rs ** 1.5
          + b4 * rs ** 2)))
# (1) degisim: E_x[n] = integral n eps_x(n) d^3r  ->  v_x = dE_x/dn, AUTOGRAD ile
h = 0.01; r = torch.arange(1, 800) * h
n = (torch.exp(-2 * r) / pi).requires_grad_(True)       # H atomu 1s yogunlugu
eps_x = -0.75 * (3 / pi) ** (1 / 3) * n ** (1 / 3)
Ex = (4 * pi * r ** 2 * n * eps_x).sum() * h
Ex.backward()
vx_autograd = n.grad / (4 * pi * r ** 2 * h)            # fonksiyonel turev = gradyan / hacim elemani
```

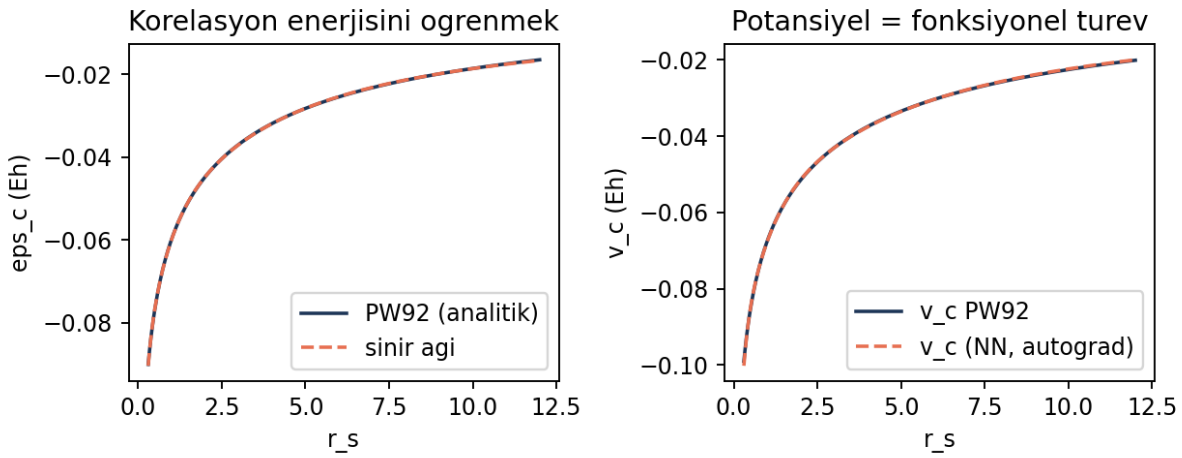
```
vx_formula = (4 / 3) * eps_x.detach()
idx = [10, 50, 100, 200]
print("E_x[n] =", round(Ex.item(), 5), "Eh   (H atomu 1s icin LDA degisimi; tam: -0.3125)")
print("   r        v_x (autograd)    (4/3) eps_x")
for i in idx: print(f"{r[i].item():6.2f}{vx_autograd[i].item():18.6f}{vx_formula[i].item():16.6f}")
# (2) korelasyon: PW92'yi bir sinir agina OGRET, sonra v_c = eps_c - (rs/3) d eps_c/d rs'yi autograd ile
      bul
rs_tr = torch.exp(torch.linspace(math.log(0.3), math.log(12), 400))
net = nn.Sequential(nn.Linear(1, 32), nn.Tanh(), nn.Linear(32, 32), nn.Tanh(), nn.Linear(32, 1))
f = lambda rs: net(torch.log(rs)[..., None]).squeeze(-1)
opt = torch.optim.Adam(net.parameters(), lr=5e-3)
for ep in range(1, 4001):
    loss = ((f(rs_tr) - pw92(rs_tr)) ** 2).mean()
    opt.zero_grad(); loss.backward(); opt.step()
    if ep in (1, 1000, 4000): print(f"epoch {ep:5d}  MSE = {loss.item():.2e}")
rs = torch.tensor([0.5, 1.0, 2.0, 4.0, 8.0], requires_grad=True)
def vc(fun, rs):
    e = fun(rs); d, = torch.autograd.grad(e.sum(), rs, create_graph=True)
    return e - rs / 3 * d
print("  rs     eps_c(NN)   eps_c(PW92)     v_c(NN, autograd)   v_c(PW92)")
for a, b, c, d_ in zip(rs, f(rs), pw92(rs), zip(vc(f, rs), vc(pw92, rs))):
    print(f"{a.item():5.1f}{b.item():12.5f}{c.item():13.5f}{d_[0].item():20.5f}{d_[1].item():13.5f}")
x = torch.linspace(0.3, 12, 300, requires_grad=True)
fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
ax[0].plot(x.detach(), pw92(x).detach(), c="#1d3557", label="PW92 (analitik)"); ax[0].plot(x.detach(),
      f(x).detach(), "--", c="#e76f51", label="sinir agi")
ax[0].set_xlabel("r_s"); ax[0].set_ylabel("eps_c (Eh)"); ax[0].legend(); ax[0].set_title("Korelasyon
      enerjisini ogrenmek")
ax[1].plot(x.detach(), vc(pw92, x).detach(), c="#1d3557", label="v_c PW92"); ax[1].plot(x.detach(), vc(f,
      x).detach(), "--", c="#e76f51", label="v_c (NN, autograd)")
ax[1].set_xlabel("r_s"); ax[1].set_ylabel("v_c (Eh)"); ax[1].legend(); ax[1].set_title("Potansiyel =
      fonksiyonel turev")
plt.tight_layout(); plt.savefig("f_nn_xc.png", dpi=170)
```

ÇIKTI

```
E_x[n] = -0.21274 Eh   (H atomu 1s icin LDA degisimi; tam: -0.3125)
   r        v_x (autograd)    (4/3) eps_x
  0.11         -0.624826       -0.624826
  0.51         -0.478572       -0.478572
  1.01         -0.342912       -0.342912
  2.01         -0.176057       -0.176057
epoch     1  MSE = 1.66e-02
epoch  1000  MSE = 6.42e-08
epoch  4000  MSE = 5.11e-09
  rs     eps_c(NN)   eps_c(PW92)     v_c(NN, autograd)   v_c(PW92)
  0.5    -0.07650     -0.07662           -0.08489     -0.08511
  1.0    -0.05984     -0.05977           -0.06760     -0.06746
  2.0    -0.04473     -0.04476           -0.05140     -0.05149
  4.0    -0.03190     -0.03187           -0.03761     -0.03751
  8.0    -0.02133     -0.02139           -0.02568     -0.02581
```

NASIL ÇALIŞIYOR?

İlk bölüm, DFT'nin en güzel köprüsünü gösterir: **fonksiyonel türev = autograd gradyanı** (hacim elemanına bölünmüş); sonuç Fortran'daki sayısal fonksiyonel türevle ve $(4/3)\varepsilon_x$ formülüyle aynıdır. İkinci bölümde ağ, PW92'yi $\varepsilon_c(r_s)$ için çok küçük hatayla taklit eder ve v_c (ε_c'nin türevi içerdiği için ağın *türevinin* doğruluğunu da sınar) yaklaşık 1e-3 hatayla doğru çıkar. Gerçek öğrenilmiş fonksiyoneller aynı fikri çok daha zengin girdilerle (∇n, τ, hibrit değişim) uygular ve fiziksel kısıtlar (doğru ölçekleme, LDA limiti) ya kayba ya mimariye gömülür. Böylece kitabın iki yarısı (yapay zekâ ve kuantum kimyası) tek bir aracın, otomatik türevin, iki yüzü olarak buluşur.



Şekil 16.4 — PW92 korelasyon enerjisini öğrenen sinir ağı (solda) ve autograd ile elde edilen korelasyon potansiyeli v_c (sağda).

Sınırlar ve sorumlu kullanım

- **Veri kalitesi**: ML modeli eğitim verisinin (HF/DFT doğruluğu, deneysel gürültü) ötesine geçemez; ekstrapolasyon (yeni kimyasal bölgeler) en büyük risktir.
- **Doğrulama**: rastgele bölme iyimser sonuç verir; scaffold split (eğitimde görülmeyen iskeletler) ve zamana göre bölme gerçek performansı gösterir.
- **Belirsizlik**: topluluk modelleri veya Bayesçi yaklaşımlar güveni tahmin etmeli; yüksek riskli kararlarda (ilaç adayı seçimi) şarttır.
- **Fiziksel kısıtlar**: simetriler (Bölüm 15), enerji korunumu, doğru uzun menzil davranışı mimariye gömülmeli.
- **Güvenlik**: üretken modeller zararlı moleküller de önerebilir; üretim ve değerlendirme sistemlerinde etik ve düzenleyici denetim gerekir.
- **Deneyle doğrulama**: hesaplamalı tahmin bir *hipotezdir,* kanıt değil; laboratuvar deneyi hâlâ son söz.

EK A

Ek A. Ortak Modüller

Kitabın birçok örneği iki küçük yardımcı modülü kullanır. Kaynakları burada.

`prec_mod` çift duyarlık türünü tanımlar. `rng_mod` ise tekrarlanabilir bir rastgele sayı üretecidir: sabit tohumla her makinede aynı sayıları üretir (doğrusal eşlenik üreteç + Box–Muller dönüşümü). Gerçek eğitimde daha kaliteli üreteçler kullanılır; burada amaç, kitaptaki çıktıların sizin makinenizde de aynı çıkmasıdır.

```
module prec_mod
  implicit none
  integer, parameter :: dp = kind(1.0d0)
end module prec_mod
```

```
module rng_mod
  use prec_mod
  implicit none
  integer(8), save :: rng_state = 12345_8
contains
  subroutine rng_seed(s)
    integer, intent(in) :: s
    rng_state = int(s, 8)
  end subroutine rng_seed

  function uniform() result(u)          ! (0,1) arasi duzgun sayi
    real(dp) :: u
    rng_state = mod(rng_state * 1103515245_8 + 12345_8, 2147483648_8)
    u = (real(rng_state, dp) + 0.5_dp) / 2147483648.0_dp
  end function uniform

  function gauss() result(g)            ! standart normal (Box-Muller)
    real(dp) :: g, u1, u2
    u1 = uniform()
    u2 = uniform()
    g = sqrt(-2.0_dp * log(u1)) * cos(6.283185307179586_dp * u2)
  end function gauss

  subroutine randn1(a, s)               ! vektoru N(0, s^2) ile doldur
    real(dp), intent(out) :: a(:)
    real(dp), intent(in) :: s
    integer :: i
    do i = 1, size(a)
      a(i) = s * gauss()
    end do
  end subroutine randn1

  subroutine randn2(a, s)               ! matrisi N(0, s^2) ile doldur
    real(dp), intent(out) :: a(:,:)
    real(dp), intent(in) :: s
    integer :: i, j
    do j = 1, size(a,2)
      do i = 1, size(a,1)
        a(i,j) = s * gauss()
      end do
    end do
  end subroutine randn2
end module rng_mod
```

Bu modülleri kullanan bir örneği derlemek için modül kaynaklarını (önce `prec_mod`, sonra `rng_mod`, sonra o örneğin kendi modülleri) programdan önce aynı dosyaya yapıştırın. Kitapla birlikte verilen *ornekler* klasöründeki dosyalarda bu işlem zaten yapılmıştır.

EK B

Ek B. Derleme, Hata Ayıklama ve Hız İpuçları

Bayrak	Ne yapar?	Ne zaman?
-O2	Standart optimizasyon	Her zaman (kitaptaki tüm örnekler)
-O3 -march=native	Agresif optimizasyon, bu işlemciye özel komutlar (SIMD)	Performans ölçümleri
-fcheck=all -g	Dizi sınırı, bellek, sıfıra bölme kontrolleri + hata ayıklama bilgisi	Geliştirme sırasında; çok daha yavaş
-Wall -Wextra	Uyarıları aç	Her zaman okumaya değer
-fopenmp	OpenMP paralelliği	Bölüm 5 örneği ve çok çekirdekli kodlar
-ffree-line-length-none	Uzun satırlara izin ver	Uzun ifadelerde
-fno-omit-frame-pointer -pg	Profil çıkar (gprof)	Yavaşlığı bulmak için
-shared -fPIC	Paylaşılan kütüphane (.so)	Python/C ile bağlamak için (Bölüm 10)

Sık karşılaşılan Fortran hataları

- **Büyük/küçük harf duyarsızdır:** `V` ve `v` aynı değişkendir. Bu kitabın 11. bölümünde de bir isim çakışması yaşandı (modül değişkeni `N` ve yerel `n`). İsimleri ayırt edilebilir seç.
- **Tamsayı bölmesi:** `1/2` sonucu `0`'dır. Gerçek sayı için `1.0_dp/2.0_dp` yaz.
- **Kesinlik:** `0.1` tek duyarlıklıdır; çift duyarlık için `0.1_dp`. Karışık kesinlik sessizce hassasiyet kaybettirir.
- **Sütun-ana düzen:** iç döngü ilk indeksi değiştirmeli (Bölüm 5).
- **Dizi sınırı aşımı** sessiz çökme veya yanlış sonuç üretir; geliştirirken `-fcheck=all` kullan.
- **Hepsi sıfırlanmış mı?** Yerel dizi başlangıçta rastgele çöp içerebilir; toplamadan önce `= 0.0_dp` yap.
- **Gradyan kontrolü ihmal etme:** elle yazılmış her geri yayılım, sayısal türevle doğrulanmalıdır (Bölüm 2).

EK C

Ek C. Formül Kartı

Kavram	Formül
Sigmoid / türev	$\sigma(x) = 1/(1+e^{-x})$, $\sigma' = \sigma(1-\sigma)$
Tanh türevi	$\tanh' = 1 - \tanh^2$
Softmax	$p_i = \exp(z_i - \max z) / \Sigma_j \exp(z_j - \max z)$
Çapraz entropi gradyanı	$\partial L/\partial z = p - \text{onehot}$
Yoğun katman	$Y = XW + b$; $dW = X^T dY$, $db = \Sigma\, dY$, $dX = dY W^T$
Geri yayılım	$\delta_l = (W_{l+1}{}^T\, \delta_{l+1}) \odot \varphi'(z_l)$
Gradyan inişi	$\theta \leftarrow \theta - \eta\, \nabla L$
Momentum	$v \leftarrow \beta v + g$; $\theta \leftarrow \theta - \eta v$
Adam	$m \leftarrow \beta_1 m+(1-\beta_1)g$; $s \leftarrow \beta_2 s+(1-\beta_2)g^2$; $\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{s}}+\varepsilon)$
AdamW	$\theta \leftarrow \theta - \eta\, (\hat{m}/(\sqrt{\hat{s}}+\varepsilon) + \lambda\theta)$
Attention	$\text{softmax}(QK^T/\sqrt{d_k} + \text{mask})\cdot V$
Softmax geri yayılımı	$ds = p \odot (dp - \Sigma\, p \odot dp)$
LayerNorm / RMSNorm	$(x-\mu)/\sqrt{(\sigma^2+\varepsilon)}$; $x/\sqrt{(\text{mean}(x^2)+\varepsilon)}$
Dil modeli kaybı	$L = -(1/T)\, \Sigma \log P(x_t \mid x_{<t})$; $PPL = e^L$
Başlangıç kaybı	$\approx \ln V$ (rastgele model)
Parametre sayısı (GPT)	$V\cdot d + T\cdot d + L(12d^2+13d) + 2d$
Eğitim FLOP	$\approx 6\cdot N\cdot D$
Eğitim belleği (Adam)	≈ 16 bayt / parametre (+ aktivasyonlar)
KV cache	$2 \cdot L \cdot T \cdot d \cdot$ bayt/eleman
Boru hattı kabarcığı	$(p-1)/(m+p-1)$
INT8 quantization	$s = \max\|w\|/127$; $q = \text{round}(w/s)$; $\hat{w} = s\cdot q$
Schrödinger (atomik birimler)	$\hat{H}\psi = [-\tfrac{1}{2}\nabla^2 + V]\psi = E\psi$; 1 Eh = 27.2114 eV
H atomu	$E_n = -Z^2/(2n^2)$
Varyasyonel ilke	$E[\psi] = \langle\psi\|\hat{H}\|\psi\rangle/\langle\psi\|\psi\rangle \geq E_0$
Hückel	$H = \alpha I + \beta A$; $E_k = \alpha + x_k\beta$, $x = \text{spec}(A)$
Roothaan (HF)	$F\, C = S\, C\, \varepsilon$; $P = 2\, C_{occ} C_{occ}{}^T$; $E = \tfrac{1}{2}\Sigma P(h+F) + E_{nuc}$
Kohn–Sham	$[-\tfrac{1}{2}\nabla^2 + v + v_H + v_{xc}]\psi_i = \varepsilon_i\psi_i$; $n = \Sigma\|\psi_i\|^2$
LDA değişim	$\varepsilon_x = -(3/4)(3/\pi)^{1/3}\, n^{1/3}$; $v_x = (4/3)\varepsilon_x$
Mesaj geçirme (GNN)	$h_i' = U(h_i, \Sigma_{j\in N(i)}\, M(h_j, e_{ij}))$
E(3) değişmezlik	$E(Rx+t) = E(x)$; $F = -\nabla E$ eşdeğer
Tanimoto	$T = \|A\cap B\| / \|A\cup B\|$

EK D

Ek D. Sonraki Adımlar ve Küçük Sözlük

Bu kitaptan gerçek bir LLM'e giden yol

Bu kitapta bir GPT'nin her parçasını gördün. Gerçek bir modele giden yolda eksik kalan başlıca adımlar:

- **Tam Transformer geri yayılımı**: Model 3'e RMSNorm, FFN ve çoklu head ekleyip bloğu N kez üst üste koymak. Her yeni parça için gradyan kontrolü ekle.
- **Daha iyi konum bilgisi**: öğrenilmiş konum embedding'i yerine RoPE (dönel konum kodlaması).
- **Verimli attention**: GQA/MQA (daha az K,V head), FlashAttention, KV cache'in sayfalanması.
- **Gerçek veri**: yüz milyonlarca-milyarlarca token, kaliteli temizleme ve tekilleştirme (Bölüm 7), alt-kelime tokenizer'ı (Bölüm 6).
- **Ölçek mühendisliği**: BLAS/GPU çekirdekleri, mixed precision, gradient checkpointing, veri/tensör/boru hattı paralelliği, ZeRO/FSDP (Bölüm 5, 9).
- **Eğitim sonrası**: talimat ince ayarı (SFT), tercih öğrenme (RLHF/DPO), güvenlik ve değerlendirme.
- **Mimari çeşitlemeleri**: Mixture-of-Experts (seyrek FFN), uzun bağlam teknikleri.

Küçük sözlük

Terim	Anlamı
Token	Modelin işlediği ayrık metin birimi (karakter, alt-kelime veya bayt dizisi)
Logit	Softmax'tan önceki ham skor
Epoch	Veri kümesinin bir kez tamamen görülmesi
Batch / mikro-batch	Bir adımda işlenen örnek grubu / onun bellek için bölünmüş parçası
Gradyan	Kaybın parametrelere göre türevi vektörü
Perplexity	$e^{\text{ortalama kayıp}}$; modelin "ortalama kaç seçenek arasında kaldığı"
Residual	y = x + f(x) atlama bağlantısı
Weight tying	Girdi embedding'i ile çıkış matrisinin paylaşılması
KV cache	Üretimde geçmiş token'ların K ve V vektörlerinin saklanması
MFU	Model FLOP kullanımı: gerçek / tepe işlem gücü
All-reduce	Tüm GPU'lardaki değerleri toplayıp sonucu hepsine dağıtan işlem
Quantization	Ağırlıkları az bitli tamsayılarla saklama
Hartree (Eh)	Atomik enerji birimi, 27.2114 eV
SCF	Öz-tutarlı alan: Fock matrisi ⇄ orbitaller yinelemesi
DFT	Yoğunluk fonksiyoneli teorisi: enerjiyi n(r)'nin fonksiyoneli olarak yazar
SMILES	Molekülü karakter dizisi olarak yazan gösterim
MLIP	Makine öğrenmesi atomlar arası potansiyeli (enerji + kuvvet)
Eşdeğerlik (equivariance)	Girdi dönerse çıktı da aynı şekilde döner

İyi eğlenceler ve iyi öğrenmeler: bir sinir ağının içindeki her sayıyı artık sen yönetiyorsun.